TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

MÔN HỌC: Các công nghệ lập trình hiện đại (841072)

ĐỀ TÀI: Nghiên cứu Spring Boot – xây dựng hệ thống chat real-time bằng WebSocket

Nhóm: 24 – Lớp: DCT1221 – Nhóm học phần: 01

SINH VIÊN THỰC HIỆN:

HỌ VÀ TÊN	MSSV
Huỳnh Thanh Hải	3123410091
Lê Quốc Cường	3123410040
Nguyễn Phú Tài	3121410435

Giảng viên: Phạm Thi Vương

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

LỜI CAM ĐOAN

Nhóm 24 xin cam đoan báo cáo học phần Các công nghệ lập trình hiện đại với đề tài "Nghiên cứu Spring Boot – xây dựng hệ thống chat real-time bằng WebSocket" là kết quả làm việc của ba thành viên trong nhóm dưới sự hướng dẫn của giảng viên Phạm Thi Vương. Các cơ chế kỹ thuật, đoạn mã, số liệu kiểm thử và nhận định trong báo cáo được trình bày trong phạm vi có minh chứng từ kho mã nguồn `github.com/QuoocsCuongwf/springboot-websocket-chat`, các tệp nhật ký kiểm thử của nhóm và tài liệu chính thức được liệt kê ở phần Tài liệu tham khảo.

Báo cáo phân biệt rõ ba loại thông tin: (1) điều đã được kiểm chứng bằng chạy thử và có log; (2) điều suy ra từ việc đọc mã nguồn nhưng chưa chạy lại; (3) điều là kế hoạch hoặc đề xuất. Số lượng bài kiểm thử đạt không được dùng để khẳng định hệ thống không còn lỗi hoặc đã sẵn sàng triển khai thực tế. Phần hỗ trợ của công cụ trí tuệ nhân tạo được khai báo đầy đủ ở mục 11.4.

Nhóm chịu trách nhiệm đối chiếu phần việc của từng thành viên với lịch sử commit, tự giải thích mã nguồn và kết quả trong báo cáo khi bảo vệ.

Thành viên thực hiện:

- Lê Quốc Cường – MSSV 3123410040 – tài khoản GitHub `QuoocsCuongwf`
- Huỳnh Thanh Hải – MSSV 3123410091 – tài khoản GitHub `ThanhHai151`
- Nguyễn Phú Tài – MSSV 3121410435 – tác giả commit `Nguyen Phu Tai`

TP. Hồ Chí Minh, ngày 06 tháng 11 năm 2026

Đại diện nhóm 24

LỜI CẢM ƠN

Nhóm xin chân thành cảm ơn giảng viên Phạm Thi Vương đã xây dựng nội dung học phần, đưa ra yêu cầu đồ án rõ ràng và góp ý trong suốt quá trình thực hiện. Yêu cầu của học phần – giải thích bản chất công nghệ thay vì chỉ liệt kê tính năng, gắn mỗi nhận định với minh chứng – đã định hướng cách nhóm làm việc với mã nguồn và cách viết báo cáo này.

Nhóm cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã tạo điều kiện về phòng học, lịch học và môi trường để sinh viên vận dụng kiến thức lập trình hiện đại vào một sản phẩm cụ thể. Nhóm cũng cảm ơn cộng đồng mã nguồn mở Spring, PostgreSQL, Flyway và Testcontainers – các tài liệu tham khảo chính thức của những dự án này là nguồn kiến thức chủ yếu mà nhóm dựa vào.

Quá trình làm đồ án giúp từng thành viên hiểu cách một framework tổ chức cấu hình, bảo mật, giao dịch và truyền thông thời gian thực; đồng thời rèn kỹ năng làm việc nhóm qua Git, Pull Request và việc đối chiếu hợp đồng giao tiếp giữa backend và client. Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm mong nhận được góp ý của thầy để hoàn thiện cách thiết kế, kiểm thử và trình bày.

TÓM TẮT

Đồ án nghiên cứu Spring Boot thông qua việc xây dựng NextChat – backend nhắn tin thời gian thực gồm xác thực JWT, quản lý người dùng, phòng chat riêng 1-1 và phòng nhóm, lịch sử tin nhắn phân trang và kênh truyền tin STOMP trên WebSocket. Dữ liệu được lưu trong PostgreSQL 16; lược đồ được quản lý bằng Flyway; truy cập dữ liệu dùng Spring Data JPA/Hibernate kết hợp hai hàm PL/pgSQL cho hai thao tác nhạy cảm về tính nguyên tử và hiệu năng. Ứng dụng dùng Spring Boot 4.1.1 trên Java 21.

So với bản báo cáo trước, bản viết lại này bổ sung bốn nhóm nội dung: (1) lịch sử hình thành Java, Spring Framework và Spring Boot từ năm 2002 đến phiên bản 4.1.1 mà nhóm sử dụng, kèm các thay đổi của thế hệ Boot 4 thể hiện trực tiếp trong mã nguồn; (2) lý do chọn đề tài và lý do chọn từng thành phần công nghệ, có bảng tiêu chí và ma trận so sánh phương án; (3) chương riêng mô tả chi tiết phần việc của từng thành viên, đối chiếu với từng commit và Pull Request; (4) cập nhật trạng thái mã nguồn cuối cùng sau khi Pull Request #8 (nhánh `phu-tai`) được hợp nhất vào `main` sáng ngày 06/11/2026 tại commit `54ab54e`.

Về kết quả kiểm chứng: lần chạy ngày 28/10/2026 trên bản sửa cục bộ của nhóm đạt 28/28 bài JUnit và 15/15 kiểm tra smoke đầu-cuối với hai client WebSocket thật; thí nghiệm SQL riêng đối chứng khoá advisory và đo hàm phân trang ở 1.000, 10.000 và 100.000 tin. Bản `main` cuối cùng chứa 23 bài kiểm thử (18 tích hợp + 5 đơn vị) và dùng cách giữ danh tính STOMP khác với bản sửa cục bộ. Việc đọc lại mã cho thấy hai kiểm soát của bản sửa cục bộ – chặn đăng ký đích dạng wildcard và chặn gửi thẳng vào broker – chưa có trên `main`; báo cáo nêu rủi ro, đề xuất bản vá và kịch bản kiểm thử cần chạy trước buổi bảo vệ.

Thông tin	Nội dung
Học phần	841072 – Các công nghệ lập trình hiện đại
Lớp / nhóm	DCT1221; nhóm học phần 01; nhóm đồ án 24
Giảng viên	Phạm Thi Vương
Lịch học	Chiều Thứ 6, tiết 6–10, phòng C.HB406
Mốc nộp / bảo vệ	Nộp tuần 9 (06/11/2026); bảo vệ tuần 10 theo thông báo

Thông tin	Nội dung
Kho mã nguồn	`github.com/QuoocsCuongwf/springboot-websocket-chat` — nhánh `main`, commit `54ab54e` (06/11/2026)
Công nghệ chính	Java 21, Spring Boot 4.1.1, Spring Security, Spring Data JPA, Spring WebSocket/STOMP, PostgreSQL 16, Flyway, Testcontainers

Từ khoá: Spring Boot, auto-configuration, dependency injection, Spring Security, JWT, WebSocket, STOMP, PostgreSQL, PL/pgSQL, advisory lock, Flyway, Testcontainers.

MỤC LỤC

LỜI CAM ĐOAN.....**i**
LỜI CẢM ƠN.....**ii**
TÓM TẮT.....**iii**
DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT.....**x**
DANH MỤC BẢNG.....**xii**
DANH MỤC HÌNH VÀ SƠ ĐỒ.....**xv**
MỞ ĐẦU.....**1**
1. Lý do chọn đề tài.....1
2. Mục tiêu nghiên cứu.....2
3. Đối tượng và phạm vi nghiên cứu.....3
4. Phương pháp nghiên cứu.....4
5. Kế hoạch và tiến độ thực hiện.....4
6. Tổ chức nhóm và phân công tổng quát.....5
7. Cấu trúc báo cáo.....6
PHẦN 1: TỔNG QUAN VÀ PHÂN TÍCH CÔNG NGHỆ.....**8**
CHƯƠNG 1. LỊCH SỬ HÌNH THÀNH VÀ PHÁT TRIỂN CỦA JAVA, SPRING FRAMEWORK VÀ SPRING BOOT.....**9**
1.1. Ngôn ngữ Java và nền tảng JVM.....9
1.2. Bối cảnh J2EE đầu những năm 2000 và sự ra đời của Spring Framework.11
1.3. Các thế hệ Spring Framework.....13
1.4. Sự ra đời của Spring Boot.....15
1.5. Các phiên bản Spring Boot đến 4.1.1.....16
1.6. Dấu ấn của Spring Boot 4 trong mã nguồn NextChat.....17
1.7. Kiến trúc cốt lõi của Spring Boot.....18
1.8. So sánh Spring Boot với các framework backend hiện đại.....20
1.9. Các trường hợp hạn chế và cần cân nhắc khi dùng Spring Boot.....21
1.10. Tiểu kết chương.....22
CHƯƠNG 2. LÝ DO LỰA CHỌN ĐỀ TÀI VÀ CÔNG NGHỆ.....**23**
2.1. Bối cảnh bài toán nhắn tin thời gian thực.....23
2.2. Lý do chọn đề tài "chat real-time bằng WebSocket".....24
2.3. Tiêu chí lựa chọn công nghệ.....26
2.4. Lý do chọn từng thành phần công nghệ.....27
2.5. Ma trận đánh giá phương án nền tảng.....31

2.6. Vị thế công nghệ và thị trường....32
2.7. Phạm vi ứng dụng thực tế của Spring Boot....32
2.8. Xu hướng phát triển....33
2.9. Tiểu kết chương....33
PHẦN 2: KIẾN THỨC BẢN CHẤT VÀ THỰC HÀNH....35
CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN....36
3.1. Công cụ và phiên bản....36
3.2. Các phụ thuộc trong pom.xml....37
3.3. Cài đặt và chạy ứng dụng....38
3.4. Cấu hình ứng dụng....39
3.5. Kiến trúc tổng thể....41
3.6. Tổ chức mã nguồn theo tính năng....43
3.7. Kiểm chứng môi trường....44
3.8. Nhận xét về kiến trúc hiện tại....45
CHƯƠNG 4. BẢN CHẤT CÁC CƠ CHẾ LÕI CỦA SPRING BOOT....46
4.1. Starter và auto-configuration....47
4.2. IoC container và Dependency Injection....50
4.3. Cấu hình ngoài mã nguồn và profile....52
4.4. Spring MVC REST, kiểm tra dữ liệu và xử lý lỗi tập trung....53
4.5. Spring Security: chuỗi filter và xác thực JWT không trạng thái....54
4.6. Spring Data JPA....57
4.7. Quản lý giao dịch với @Transactional....58
4.8. Tích hợp Flyway và ddl-auto: validate....60
4.9. Kiểm thử tích hợp với Spring Boot Test và Testcontainers....62
4.10. WebSocket/STOMP và kiểm soát truy cập trên kênh thông điệp....64
4.11. Tiểu kết chương....70
CHƯƠNG 5. TÍNH NGUYÊN TỬ KHI TẠO PHÒNG 1-1 – HÀM PL/PGSQL VÀ KHOÁ ADVISORY....71
5.1. Vấn đề: "tìm, nếu chưa có thì tạo" dưới truy cập đồng thời....71
5.2. Các phương án đã cân nhắc....71
5.3. Cơ chế khoá advisory của PostgreSQL....72
5.4. Gọi hàm từ Spring....73
5.5. Kiểm chứng....74
5.6. Giới hạn và hướng tăng cường....75
5.7. Tiểu kết....75

CHƯƠNG 6. PHÂN TRANG LỊCH SỬ TIN NHẮN – PL/PGSQL, OFFSET VÀ CHI PHÍ COUNT **77**
6.1. Vì sao chọn nghiên cứu phân trang 77
6.2. Hàm get_room_messages 77
6.3. Tầng service và trường hợp trang rỗng 78
6.4. Kết quả kiểm thử 78
6.5. Thí nghiệm phân trang ở ba mức dữ liệu 79
6.6. Đọc kế hoạch thực thi 80
6.7. OFFSET và keyset: đánh đổi và hướng cải tiến 80
6.8. Dùng ở đâu 81
CHƯƠNG 7. REFRESH TOKEN XOAY VÒNG VÀ CÁC ĐÁNH ĐỔI BẢO MẬT **82**
7.1. Vì sao cần refresh token 82
7.2. Cơ chế trong NextChat 82
7.3. Kiểm chứng 84
7.4. Phân tích tình huống tấn công 84
7.5. Giới hạn và dùng ở đâu 85
PHẦN 3: XÂY DỰNG ĐỒ ÁN TỔNG HỢP **86**
CHƯƠNG 8. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **87**
8.1. Mô tả bài toán 87
8.2. Thiết kế cơ sở dữ liệu 90
8.3. Hợp đồng REST API 93
8.4. Hợp đồng STOMP 95
8.5. Luồng xử lý dữ liệu và điều khiển 96
8.6. Tiểu kết chương 99
CHƯƠNG 9. TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ **100**
9.1. Tích hợp các thành phần theo từng luồng 100
9.2. Kết quả kiểm chứng 101
9.3. Bảng đối chiếu kỹ thuật và minh chứng 103
9.4. Client web thử nghiệm 104
9.5. Trạng thái mã nguồn cuối cùng (06/11/2026) 108
9.6. Danh sách hạn chế và rủi ro tồn đọng 109
9.7. Tiểu kết chương 110
PHẦN 4: TỔ CHỨC THỰC HIỆN, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN **111**

CHƯƠNG 10. TỔ CHỨC THỰC HIỆN VÀ PHẦN VIỆC CHI TIẾT CỦA TỪNG THÀNH VIÊN **112**
10.1. Mô hình tổ chức và quy trình làm việc 112
10.2. Tổng quan đóng góp theo dấu vết Git 115
10.3. Lê Quốc Cường – Người 1: trưởng nhóm, kiến trúc, WebSocket, tích hợp và kiểm thử 116
10.4. Huỳnh Thanh Hải – Người 2: Backend Auth, REST API và cơ sở dữ liệu 122
10.5. Nguyễn Phú Tài – Người 3: client web thử nghiệm và hạ tầng 127
10.6. Phối hợp và tích hợp giữa các thành viên 131
10.7. Tổng hợp mức độ hoàn thành theo phân công 132
10.8. Tiểu kết chương 133
CHƯƠNG 11. ĐÁNH GIÁ VÀ NHÌN LẠI **134**
11.1. So sánh với mục tiêu ban đầu 134
11.2. Phân tích lỗi (debugging case studies) 134
11.3. Bài học kinh nghiệm 137
11.4. Khai báo sử dụng công cụ AI 138
11.5. Tiểu kết chương 139
CHƯƠNG 12. HƯỚNG PHÁT TRIỂN **140**
12.1. Lộ trình cải tiến sản phẩm 140
12.2. Báo lỗi nghiệp vụ trên kênh STOMP 140
12.3. Trạng thái trực tuyến (presence) 140
12.4. Mở rộng nhiều instance với broker relay 141
12.5. Đóng gói và tích hợp liên tục 142
12.6. Hiệu năng và đo tải 142
12.7. Định hướng phát triển của các thành viên 143
KẾT LUẬN **144**
TÀI LIỆU THAM KHẢO **145**
PHỤ LỤC **148**
Phụ lục 1. Nhật ký công việc theo tuần 148
Phụ lục 2. Lịch sử commit của kho mã 148
Phụ lục 3. Minh chứng làm việc nhóm trên GitHub 150
Phụ lục 4. Bảng tự kiểm ba tầng 151
Phụ lục 5. Hands-on Lab: kiểm chứng phòng 1-1 và luồng gửi tin 151
Phụ lục 6. Hướng dẫn chạy demo và biên bản kiểm tra trước buổi bảo vệ 153

Phụ lục 7. Câu hỏi ôn tập cho buổi bảo vệ.....154
Phụ lục 8. Lộ trình từ ngày nộp đến buổi bảo vệ.....156

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu	Diễn giải
AOP	Aspect-Oriented Programming – lập trình hướng khía cạnh; Spring dùng proxy AOP để áp dụng giao dịch
API	Application Programming Interface – giao diện lập trình ứng dụng
BOM	Bill of Materials – danh sách phiên bản phụ thuộc tương thích do Spring Boot quản lý
CDN	Content Delivery Network – mạng phân phối nội dung, nơi client nạp Tailwind, SockJS, stomp.js
CI/CD	Continuous Integration / Continuous Delivery – tích hợp và chuyển giao liên tục
CORS	Cross-Origin Resource Sharing – cơ chế chia sẻ tài nguyên khác nguồn
CSRF	Cross-Site Request Forgery – tấn công giả mạo yêu cầu liên trang
DI	Dependency Injection – tiêm phụ thuộc
DTO	Data Transfer Object – đối tượng truyền dữ liệu
E2E	End-to-End – kiểm tra đầu-cuối qua các thành phần tích hợp thật
EJB	Enterprise JavaBeans – mô hình thành phần của J2EE
ERD	Entity Relationship Diagram – sơ đồ quan hệ thực thể
HMAC	Hash-based Message Authentication Code – mã xác thực thông điệp dựa trên băm, dùng ký JWT
HTTP	Hypertext Transfer Protocol – giao thức truyền tải siêu văn bản
IoC	Inversion of Control – đảo ngược điều khiển
J2EE / Java EE / Jakarta EE	Các tên gọi qua từng thời kỳ của nền tảng Java doanh nghiệp
JDK / JVM	Java Development Kit / Java Virtual Machine
JPA	Jakarta Persistence API – đặc tả ánh xạ đối tượng – quan hệ
JPQL	Jakarta Persistence Query Language – ngôn ngữ truy vấn trên entity
JSON	JavaScript Object Notation – định dạng dữ liệu văn bản
JWT	JSON Web Token (RFC 7519) – chuỗi xác thực có chữ ký
LTS	Long-Term Support – phiên bản được hỗ trợ dài hạn
MVC	Model – View – Controller
N+1	Hiện tượng phát sinh một truy vấn cho danh sách và N truy vấn cho từng phần tử
ORM	Object-Relational Mapping – ánh xạ đối tượng – quan hệ
PL/pgSQL	Ngôn ngữ thủ tục của PostgreSQL
PR	Pull Request – yêu cầu hợp nhất mã trên GitHub
REST	Representational State Transfer – kiểu kiến trúc dịch vụ dựa trên tài nguyên
SQL	Structured Query Language – ngôn ngữ truy vấn có cấu trúc
STOMP	Simple (Streaming) Text Oriented Messaging Protocol – giao thức thông điệp hướng văn bản
SockJS	Thư viện mô phỏng WebSocket bằng các cơ chế dự phòng khi WebSocket bị chặn
SPR-9888	Mã yêu cầu trên hệ thống quản lý vấn đề của Spring, khởi nguồn của Spring Boot
TTL	Time To Live – thời gian sống của token

Ký hiệu	Diễn giải
US	User Story – câu chuyện người dùng mô tả yêu cầu
XSS	Cross-Site Scripting – tấn công chèn mã kịch bản

DANH MỤC BẢNG

Bảng MĐ.1. Kế hoạch và tiến độ theo lịch học phần.....5
Bảng MĐ.2. Phân công công việc Nhóm 24.....6
Bảng 1.1. Các tính năng Java hiện đại được dùng trong NextChat.....11
Bảng 1.2. Các thế hệ Spring Framework và điểm nổi bật.....13
Bảng 1.3. Các mốc phát hành chính của Spring Boot.....16
Bảng 1.4. Dấu hiệu của thế hệ Spring Boot 4 trong mã nguồn.....18
Bảng 1.5. So sánh định tính Spring Boot với một số framework backend.....20
Bảng 2.1. So sánh các kỹ thuật truyền thông thời gian thực trên web.....24
Bảng 2.2. Tiêu chí lựa chọn công nghệ và trọng số.....26
Bảng 2.3. So sánh phiên máy chủ và JWT không trạng thái.....27
Bảng 2.4. Các phương án cho kênh thời gian thực.....29
Bảng 2.5. Ma trận đánh giá có trọng số các nền tảng backend (điểm 1–5).....31
Bảng 3.1. Công cụ và phiên bản triển khai.....36
Bảng 3.2. Phụ thuộc Maven và vai trò.....37
Bảng 3.3. Trách nhiệm của từng package.....43
Bảng 4.1. Các cơ chế Spring Boot được sử dụng trong đồ án.....46
Bảng 4.2. Các bean do nhóm định nghĩa và phụ thuộc của chúng.....50
Bảng 4.3. Ánh xạ ngoại lệ sang phản hồi HTTP trong GlobalExceptionHandler. .54
Bảng 4.4. Các phương thức repository tiêu biểu và truy vấn tương ứng.....57
Bảng 4.5. Các phương thức có giao dịch trong NextChat.....59
Bảng 4.6. Hai mươi bảng trong migration V1.....61
Bảng 4.7. Các kịch bản trong AuthAndRoomIntegrationTest.....62
Bảng 4.8. Bộ kiểm thử của bản sửa cục bộ 28/10 và của nhánh main 06/11.....63
Bảng 4.9. Hai cách khắc phục lỗi mất Principal.....67
Bảng 5.1. Các phương án chống tạo trùng phòng 1-1.....71
Bảng 5.2. Kết quả đối chứng có/không khoá trong lab SQL ngày 28/10/2026.....75
Bảng 6.1. Thời gian thực thi hàm lịch sử trong lab SQL (ms).....79
Bảng 6.2. So sánh phân trang OFFSET và keyset.....80
Bảng 7.1. Đặc tính và giới hạn của refresh token.....84
Bảng 7.2. Một số tình huống lộ token và hành vi của hệ thống.....84
Bảng 8.1. Yêu cầu chức năng theo user story.....88
Bảng 8.2. Yêu cầu phi chức năng và cách đáp ứng.....89
Bảng 8.3. Bảng users.....90

Bảng 8.4. Bảng refresh_tokens, roles và user_roles.....91
Bảng 8.5. Bảng conversations và conversation_participants.....91
Bảng 8.6. Bảng messages.....92
Bảng 8.7. Đặc tả chín endpoint REST.....93
Bảng 8.8. Endpoint kết nối và destination STOMP trên main.....95
Bảng 9.1. Bảng đối chiếu kỹ thuật và minh chứng.....103
Bảng 9.2. Tệp và thành phần của client thử nghiệm.....104
Bảng 9.3. Ánh xạ chức năng giao diện với REST API.....106
Bảng 9.4. Kịch bản kiểm thử thủ công client web.....107
Bảng 9.5. Ba commit được hợp nhất qua Pull Request #8.....108
Bảng 9.6. Hạn chế và rủi ro tồn đọng trên main, mức độ và đề xuất xử lý.....109
Bảng 10.1. Các nhánh trong kho mã.....113
Bảng 10.2. Tám Pull Request được hợp nhất vào main.....113
Bảng 10.3. Thống kê commit và dòng mã theo tác giả (không tính commit merge; trừ mvnw, Maven wrapper và tệp .docx).....115
Bảng 10.4. Các commit không phải merge của Lê Quốc Cường.....116
Bảng 10.5. Mức độ hoàn thành nhiệm vụ của Lê Quốc Cường.....120
Bảng 10.6. Các REST endpoint được giao cho Huỳnh Thanh Hải (theo task.md) và trạng thái.....122
Bảng 10.7. Các commit của Huỳnh Thanh Hải.....123
Bảng 10.8. Nội dung commit a2626e6 theo nhóm tệp.....123
Bảng 10.9. Mức độ hoàn thành nhiệm vụ của Huỳnh Thanh Hải.....126
Bảng 10.10. Nhiệm vụ của Nguyễn Phú Tài và trạng thái thực hiện.....127
Bảng 10.11. Các điểm tích hợp giữa thành viên và vấn đề phát sinh.....132
Bảng 10.12. Tổng hợp mức độ hoàn thành nhiệm vụ của ba thành viên.....132
Bảng 11.1. Đối chiếu kết quả với các mục tiêu cụ thể.....134
Bảng 11.2. Các công cụ AI đã sử dụng và phạm vi.....138
Bảng 12.1. Lộ trình cải tiến NextChat.....140
Bảng PL.1. Nhật ký tổng hợp công việc.....148
Bảng PL.2. Toàn bộ commit của kho mã (mọi nhánh, theo thời gian).....148
Bảng PL.3. Bảng tự kiểm kỹ thuật.....151
Bảng PL.4. Bài tập và đáp án đối chiếu.....152
Bảng PL.5. Lỗi thường gặp khi chạy demo và cách xử lý.....153
Bảng PL.6. Biên bản kiểm tra trước buổi bảo vệ (điền khi chạy).....154
Bảng PL.7. Câu hỏi ôn tập và gợi ý trả lời.....155

Bảng PL.8. Việc còn phải thực hiện sau ngày nộp.....156

DANH MỤC HÌNH VÀ SƠ ĐỒ

Hình 1.1. Dòng thời gian Spring Framework, Spring Boot và các đơn vị bảo trợ (tổng hợp từ [14], [16]–[24], [29]).....14
Hình 1.2. Ngăn xếp công nghệ Spring mà NextChat sử dụng.....19
Hình 3.1. Kiến trúc và luồng request của NextChat trên main (tổng hợp từ mã nguồn commit 54ab54e).....42
Hình 4.1. Cơ chế auto-configuration: starter đưa thư viện vào classpath, điều kiện quyết định bean hạ tầng nào được tạo.....48
Hình 4.2. Luồng một request REST qua chuỗi filter bảo mật của NextChat.....55
Hình 4.3. Luồng thông điệp STOMP trong NextChat: kênh inbound, interceptor, controller, broker và kênh outbound.....65
Hình 5.1. Hai giao dịch đảo chiều được tuần tự hoá bởi cùng một khoá advisory. 73
Hình 6.1. Thời gian thực thi trang đầu và trang sâu theo số lượng tin (thang logarit) 79
Hình 7.1. Đăng nhập, xoay vòng và dùng lại refresh token.....83
Hình 8.1. Sơ đồ ca sử dụng của NextChat.....88
Hình 8.2. ERD cốt lõi rút gọn; reply_to_message_id và forwarded_from_id tự tham chiếu messages, last_message_id thuộc conversations (nguồn: migration V1).....90
Hình 8.3. Luồng xác thực, gửi, lưu và nhận tin trên main (nguồn: mã nguồn commit 54ab54e).....97
Hình 8.4. Sơ đồ tuần tự gửi và nhận một tin nhắn giữa hai client qua STOMP.….98
Hình 9.1. Bố cục giao diện client web: cột phòng, khung chat và công cụ trình diễn giao thức (vẽ lại theo index.html).....104
Hình 10.1. Commit theo thời gian của từng thành viên và thời điểm hợp nhất Pull Request (dữ liệu: git log toàn bộ nhánh).....114
Hình 10.2. Số commit và số dòng thêm theo thành viên.....115
Hình 10.3. Các điểm bàn giao và phản hồi giữa ba thành viên.....132
Hình PL.1. GitHub Insights/Contributors của kho mã, chụp ngày 28/10/2026.…150

MỞ ĐẦU

1. Lý do chọn đề tài

Nhắn tin tức thời đã trở thành chức năng mặc định của hầu hết ứng dụng hiện đại: từ mạng xã hội, sàn thương mại điện tử, hệ thống chăm sóc khách hàng cho tới công cụ cộng tác trong doanh nghiệp và lớp học trực tuyến. Người dùng kỳ vọng tin nhắn xuất hiện ngay khi đối phương bấm gửi, lịch sử được lưu đầy đủ, và chỉ người trong cuộc trò chuyện mới đọc được nội dung. Ba kỳ vọng này tương ứng với ba bài toán kỹ thuật cốt lõi của backend: truyền thông hai chiều thời gian thực, lưu trữ nhất quán và phân quyền theo phòng.

Học phần Các công nghệ lập trình hiện đại yêu cầu nhóm nghiên cứu một công nghệ, giải thích bản chất cơ chế của nó và chứng minh bằng một đồ án có thể kiểm chứng. Nhóm chọn Spring Boot làm công nghệ nghiên cứu và chọn bài toán chat real-time làm đồ án vì bài toán này buộc phải vận dụng gần như toàn bộ các mô-đun quan trọng của hệ sinh thái Spring trong cùng một ứng dụng: starter và auto-configuration để dựng hạ tầng; IoC/DI để nối các thành phần; Spring Security để xác thực không trạng thái bằng JWT; Spring Data JPA và quản lý giao dịch để lưu dữ liệu nhất quán; Flyway để quản lý lược đồ; Spring WebSocket và Spring Messaging để truyền tin theo giao thức STOMP; Spring Boot Test và Testcontainers để kiểm thử với cơ sở dữ liệu thật.

Bài toán chat còn chứa những tình huống đồng thời có thật, không phải ví dụ giả lập: hai người cùng bấm "nhắn tin" cho nhau có thể khiến hệ thống tạo trùng phòng; hai yêu cầu làm mới token cùng lúc có thể phá vỡ tính dùng-một-lần; lịch sử tin nhắn tăng nhanh làm phân trang bằng OFFSET chậm dần. Đây là cơ hội để nhóm giải thích các khái niệm như giao dịch, khoá, mức cô lập và kế hoạch thực thi truy vấn bằng thí nghiệm cụ thể. Phân tích chi tiết lý do chọn đề tài và lý do chọn từng công nghệ được trình bày riêng ở Chương 2.

2. Mục tiêu nghiên cứu

2.1. Mục tiêu tổng quát

Hiểu và giải thích được bản chất các cơ chế cốt lõi của Spring Boot, vận dụng chúng để xây dựng một backend nhắn tin thời gian thực có xác thực, phân quyền, lưu trữ nhất quán và kiểm thử tự động; đồng thời trình bày được giới hạn của từng kết luận dựa trên minh chứng thực tế.

2.2. Mục tiêu cụ thể

1) Trình bày lịch sử hình thành và phát triển của Java, Spring Framework và Spring Boot; chỉ ra những thay đổi của thế hệ Spring Boot 4 thể hiện trong mã nguồn đồ án.

2) Phân tích lý do chọn đề tài và căn cứ chọn từng thành phần công nghệ bằng tiêu chí rõ ràng, có so sánh với phương án thay thế.

3) Giải thích và vận dụng các cơ chế: starter/auto-configuration, IoC/DI, cấu hình ngoài và profile, Spring MVC và xử lý lỗi tập trung, Spring Security với JWT không trạng thái, Spring Data JPA, `@Transactional`, Flyway, Spring Boot Test và WebSocket/STOMP; đối chiếu từng cơ chế với mã nguồn và kết quả thực thi.

4) Xây dựng backend gồm đăng ký, đăng nhập, làm mới token, hồ sơ và tìm kiếm người dùng, tạo phòng riêng và phòng nhóm, phân trang lịch sử, gửi và nhận tin qua STOMP; xây dựng client web thử nghiệm để trình diễn luồng đầu-cuối.

5) Kiểm chứng bằng kiểm thử đơn vị, kiểm thử tích hợp với PostgreSQL thật và kiểm tra đầu-cuối với hai client WebSocket; thực hiện thí nghiệm SQL có kiểm soát cho khoá advisory và phân trang.

6) Ghi nhận chi tiết phần việc của từng thành viên, đối chiếu với lịch sử commit và Pull Request; đánh giá mức độ hoàn thành so với phân công ban đầu.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Spring Boot 4.1.1 và các mô-đun Spring được dùng trong đồ án: Spring Framework 7 (core, AOP, transaction), Spring MVC, Spring Security, Spring Data JPA, Spring WebSocket và Spring Messaging.
- Các giao thức và chuẩn liên quan: WebSocket (RFC 6455), STOMP 1.2, JSON Web Token (RFC 7519).
- Hệ quản trị cơ sở dữ liệu PostgreSQL 16 ở các khía cạnh: ràng buộc, hàm PL/pgSQL, khoá advisory mức giao dịch, kế hoạch thực thi truy vấn.
- Quy trình phát triển nhóm trên GitHub: nhánh tính năng, Pull Request, quy ước commit.

3.2. Phạm vi nghiên cứu

- Về công nghệ: tập trung vào backend Java Spring Boot một tiến trình, broker STOMP đơn giản trong bộ nhớ (simple broker), cơ sở dữ liệu PostgreSQL. Client web chỉ đóng vai trò công cụ trình diễn và kiểm thử thủ công.
- Về chức năng: tài khoản, phòng 1-1, phòng nhóm, lịch sử và nhắn tin văn bản. Lược đồ V1 có sẵn bảng cho tệp đính kèm, trạng thái đã đọc, cảm xúc, thông báo, cuộc gọi; các bảng này chưa có API và nằm ngoài phạm vi hoàn thành.
- Về thời gian: theo lịch học phần, nộp hồ sơ ở tuần 9 (06/11/2026) và bảo vệ ở tuần 10. Mốc mã nguồn cuối cùng được báo cáo là commit `54ab54e` lúc 08:27 ngày 06/11/2026.
- Về kiểm chứng: chưa thực hiện đánh giá tải API, đo độ trễ p50/p95/p99, triển khai nhiều instance, kiểm thử SockJS đầu-cuối, hay đánh giá bảo mật toàn diện. Các nội dung này được ghi là hướng phát triển.

4. Phương pháp nghiên cứu

Nghiên cứu tài liệu

Nhóm ưu tiên tài liệu gốc: Spring Boot Reference, Spring Framework Reference, Spring Security Reference, tài liệu PostgreSQL 16, đặc tả RFC 6455, STOMP 1.2 và RFC 7519; các bài công bố phát hành trên blog chính thức `spring.io` để xác định mốc lịch sử. Số liệu khảo sát thị trường chỉ được dùng khi có nguồn và năm cụ thể.

Phân tích mã nguồn

Mọi mô tả về hệ thống được đối chiếu với mã nguồn tại commit xác định. Khi báo cáo nói "theo mã nguồn", đó là kết luận từ việc đọc code, chưa phải kết quả chạy thử; khi nói "đã kiểm chứng", luôn kèm tên bài kiểm thử, lệnh chạy và ngày chạy.

Thực nghiệm và đối chứng

Mỗi cơ chế quan trọng có một phép thử tương ứng: kiểm thử tích hợp trên ApplicationContext thật với PostgreSQL trong Testcontainers; kiểm thử đầu-cuối bằng hai client WebSocket thật; thí nghiệm SQL có biến thể đối chứng (có khoá và bỏ khoá) để thấy rõ tác dụng của cơ chế thay vì chỉ thấy kết quả "đạt".

Phân tích lịch sử phiên bản

Phần việc của từng thành viên được xác định từ lịch sử Git (tác giả, thời điểm, tệp thay đổi, số dòng), Pull Request và tài liệu phân công của nhóm. Số commit và số dòng chỉ là một nguồn đối chiếu, không được quy đổi thành tỷ lệ phần trăm đóng góp.

5. Kế hoạch và tiến độ thực hiện

Tuần được tính từ buổi học đầu tiên ngày 11/09/2026 (Thứ 6). Bảng MĐ.1 tổng hợp nội dung từng tuần và mốc có minh chứng; các tuần sau ngày nộp là kế hoạch.

Bảng MĐ.1. *Kế hoạch và tiến độ theo lịch học phần*

Tuần	Nội dung chính	Trạng thái và minh chứng
1–2 (11–24/09)	Khảo sát Spring Boot, WebSocket, STOMP, JWT; thống nhất đề tài, phạm vi và phân công; khởi tạo kho mã.	Hoàn thành. Commit đầu `15b9513`, `6ab071a` ngày 22/09 (Cường).
3 (25/09–01/10)	Dựng entity User, cấu hình Security, JWT, seed dữ liệu, schema SQL, Docker Compose, tái cấu trúc package; hoàn thiện auth, phòng, lịch sử, Flyway, test tích hợp.	Hoàn thành. PR #1–#3 ngày 25/09; `7c56340` (Cường); `a2626e6` ngày 01/10 (Hải).
4 (02–08/10)	Hợp nhất nhánh API; xử lý tệp `.env`; cấu hình WebSocket và controller STOMP đầu tiên.	Hoàn thành. PR #4–#6, `deabd07`, `c3acb71` ngày 02/10.
5 (09–15/10)	Đối chiếu JPA, giao dịch, hàm PL/pgSQL; chuẩn bị tài liệu nền.	Không có commit; tổng hợp theo tài liệu nhóm.
6 (16–22/10)	Interceptor JWT cho STOMP, lưu tin qua service, test interceptor và test #18; đồng bộ tài liệu với mã.	Hoàn thành. PR #7; `d20e734`, `3eaa91a`, `39b4d75` ngày 20/10.
7 (23–29/10)	Rà soát báo cáo, chạy kiểm chứng; phát hiện lỗi mất Principal; bổ sung test, smoke E2E, thí nghiệm SQL.	25/10: 25 JUnit + 11 smoke; 28/10: 28 JUnit + 15 smoke trên bản sửa cục bộ (chưa push).
8 (30/10–05/11)	Client web thử nghiệm, cấu hình truy cập tài nguyên tĩnh, chỉnh Compose; đồng bộ endpoint STOMP giữa client và server.	`4699986` (Tài) và `332d069` (Cường) ngày 30/10 trên nhánh `phu-tai`.
9 (06/11)	Sửa giữ danh tính STOMP và origin; hợp nhất `phu-tai` vào `main`; nộp báo cáo, nhật ký, mã nguồn.	`1046984` lúc 08:26 và PR #8 (`54ab54e`) lúc 08:27 ngày 06/11.
10	Bảo vệ đồ án; chạy lại kiểm thử trên `main` và demo theo kịch bản ở Phụ lục.	Kế hoạch; theo lịch của giảng viên.

6. Tổ chức nhóm và phân công tổng quát

Nhóm có ba thành viên, chia theo ba vai trò bổ trợ nhau: một người giữ khung kiến trúc và kênh thời gian thực, một người phụ trách nghiệp vụ REST và cơ sở dữ liệu, một người phụ trách phía client và hạ tầng. Bảng MĐ.2 tóm tắt phân công; Chương 10 trình bày chi tiết từng phần việc, từng commit, kết quả, hạn chế và tự đánh giá.

Bảng MĐ.2. *Phân công công việc Nhóm 24*

Thành viên	Vai trò và công việc chính	Minh chứng chính
Lê Quốc Cường – 3123410040 (Người 1, trưởng nhóm)	Khởi tạo dự án, cấu hình Security và JWT ban đầu, schema SQL 20 bảng, Docker Compose, tái cấu trúc package-by-feature, cấu hình WebSocket/STOMP, interceptor JWT, kiểm thử STOMP; quản lý nhánh, duyệt và hợp nhất 8 Pull Request; đồng bộ endpoint với client và sửa giữ Principal.	21 commit không phải merge; `6ab071a`, `5dae9ab`, `dfa4a3b`, `67fb244`, `7c56340`, `deabd07`, `c3acb71`, `d20e734`, `3eaa91a`, `332d069`, `1046984`.
Huỳnh Thanh Hải – 3123410091 (Người 2)	Backend Auth, REST API và CSDL: hoàn thiện đăng ký/đăng nhập/làm mới token xoay vòng, 9 endpoint REST, entity và repository phòng/tin nhắn, hai hàm PL/pgSQL, Flyway V1/V2, xử lý lỗi tập trung, 17 bài kiểm thử tích hợp, README hợp đồng API.	`a2626e6` ngày 01/10 (47 tệp); `8bb8f4f`.
Nguyễn Phú Tài – 3121410435 (Người 3)	Client web thử nghiệm: đăng ký/đăng nhập, quản lý JWT, danh sách phòng, lịch sử, tạo phòng, kết nối SockJS/STOMP, nhật ký giao thức; cấu hình truy cập tài nguyên tĩnh; chỉnh Docker Compose. Dockerfile, CI/CD, load test chưa thực hiện.	`4699986` ngày 30/10 (4 tệp, 874 dòng thêm), hợp nhất qua PR #8 ngày 06/11.

7. Cấu trúc báo cáo

Ngoài phần mở đầu, kết luận, tài liệu tham khảo và phụ lục, báo cáo gồm bốn phần với mười hai chương:

- Phần 1 – Tổng quan và phân tích công nghệ (Chương 1–2): lịch sử Java, Spring Framework và Spring Boot; kiến trúc cốt lõi; so sánh với các framework khác; lý do chọn đề tài và lý do chọn từng thành phần công nghệ.
- Phần 2 – Kiến thức bản chất và thực hành (Chương 3–7): môi trường và kiến trúc dự án; chín cơ chế lõi của Spring Boot cùng WebSocket/STOMP; ba kỹ thuật dữ liệu chuyên sâu (khoá advisory, phân trang PL/pgSQL, refresh token xoay vòng).
- Phần 3 – Xây dựng đồ án tổng hợp (Chương 8–9): phân tích yêu cầu, thiết kế cơ sở dữ liệu, hợp đồng REST và STOMP, triển khai, kiểm thử, minh chứng và trạng thái mã nguồn cuối cùng.

- Phần 4 – Tổ chức thực hiện, đánh giá và hướng phát triển (Chương 10–12): phần việc chi tiết của từng thành viên, đánh giá so với mục tiêu, phân tích lỗi, khai báo sử dụng AI và hướng phát triển.
- Phụ lục: nhật ký công việc, lịch sử commit, hướng dẫn cài đặt và demo, bài lab có đáp án, bộ câu hỏi ôn tập bảo vệ và mã nguồn tiêu biểu.

PHẦN 1: TỔNG QUAN VÀ PHÂN TÍCH CÔNG NGHỆ

Phần này trả lời hai câu hỏi: Spring Boot từ đâu mà có và đã thay đổi thế nào cho tới phiên bản nhóm sử dụng (Chương 1); vì sao nhóm chọn đề tài chat thời gian thực và chọn từng thành phần công nghệ trong đồ án (Chương 2).

CHƯƠNG 1. LỊCH SỬ HÌNH THÀNH VÀ PHÁT TRIỂN CỦA JAVA, SPRING FRAMEWORK VÀ SPRING BOOT

Để hiểu vì sao Spring Boot được thiết kế với starter, auto-configuration và cấu hình ngoài, cần nhìn lại bối cảnh mà nó ra đời: ngôn ngữ Java và nền tảng Java doanh nghiệp đầu những năm 2000, sự xuất hiện của Spring Framework như một phản ứng trước sự phức tạp của EJB, và yêu cầu "ứng dụng không cần container ngoài" của cộng đồng năm 2012. Chương này trình bày các mốc đó theo trình tự thời gian, rồi chỉ ra những thay đổi của thế hệ Spring Boot 4 thể hiện trực tiếp trong mã nguồn NextChat.

1.1. Ngôn ngữ Java và nền tảng JVM

1.1.1. Sự ra đời và triết lý thiết kế

Java được Sun Microsystems phát triển từ đầu thập niên 1990 dưới sự dẫn dắt của James Gosling, công bố rộng rãi năm 1995 và phát hành JDK 1.0 vào tháng 01/1996. Ý tưởng trung tâm của Java là "viết một lần, chạy mọi nơi": mã nguồn được biên dịch thành bytecode, sau đó máy ảo Java (JVM) trên từng hệ điều hành thông dịch hoặc biên dịch tức thời (JIT) bytecode đó thành mã máy. Nhờ lớp trung gian này, cùng một tệp `.jar` có thể chạy trên Windows, Linux hay macOS mà không phải biên dịch lại [1].

Bên cạnh tính di động, Java chọn một số quyết định thiết kế ảnh hưởng lâu dài tới các framework sau này: bộ thu gom rác tự động, hệ thống kiểu tĩnh và chặt chẽ, cơ chế phản chiếu (reflection) cho phép chương trình đọc cấu trúc lớp lúc chạy, và về sau là chú thích (annotation) từ Java 5 (2004). Reflection và annotation chính là hai công cụ mà Spring dùng để tìm các thành phần, đọc metadata cấu hình và tạo proxy – tức là nền móng kỹ thuật của IoC container và auto-configuration được phân tích ở Chương 4.

1.1.2. Từ J2EE, Java EE đến Jakarta EE

Năm 1999, Sun tách nền tảng Java thành ba phiên bản, trong đó J2EE (Java 2 Platform, Enterprise Edition) dành cho ứng dụng doanh nghiệp: Servlet, JSP, JDBC, JTA và đặc biệt là Enterprise JavaBeans (EJB). Năm 2006 tên gọi được đổi thành Java EE. Sau khi Oracle mua lại Sun (hoàn tất năm 2010), Java EE tiếp tục phát triển cho tới khi Oracle chuyển giao nền tảng cho Eclipse Foundation (công bố năm 2017) và nền tảng được đổi tên thành Jakarta EE (2018). Thay đổi lớn nhất đối với lập trình viên là ở Jakarta EE 9 (2020): toàn bộ không gian tên `javax.*` của các đặc tả doanh nghiệp được đổi thành `jakarta.*`.

Đổi tên không gian tên nghe như chi tiết hành chính nhưng có hệ quả trực tiếp cho NextChat: mọi chú thích ánh xạ và kiểm tra dữ liệu trong mã nguồn đều dùng `jakarta.persistence.*` và `jakarta.validation.*` (ví dụ `@Entity`, `@Table`, `@NotBlank`, `@Valid`), bởi vì từ Spring Boot 3.0 trở đi, nền tảng đã chuyển hẳn sang Jakarta EE 9+ [18].

1.1.3. Nhịp phát hành và các phiên bản LTS

Từ Java 9 (2017), nền tảng chuyển sang nhịp phát hành sáu tháng một phiên bản, và cứ một số phiên bản lại có một bản hỗ trợ dài hạn (LTS). Các bản LTS được doanh nghiệp dùng phổ biến là Java 8, 11, 17, 21 và 25. Java 21 (phát hành tháng 09/2023) đưa luồng ảo (virtual threads) từ trạng thái thử nghiệm thành tính năng chính thức, đồng thời hoàn thiện record, pattern matching và text block. Spring Boot 3.x yêu cầu tối thiểu Java 17; Spring Boot 4 giữ mức tối thiểu Java 17 và hỗ trợ đầy đủ Java 25 [19].

1.1.4. Những tính năng Java 21 xuất hiện trong mã nguồn NextChat

Tệp `pom.xml` khai báo `java.version = 21`. Việc chọn Java 21 không chỉ là con số phiên bản: nhiều cấu trúc ngôn ngữ mới được dùng thật trong mã nguồn, giúp mã ngắn và an toàn hơn. Bảng 1.1 liệt kê các tính năng và vị trí cụ thể.

Bảng 1.1. *Các tính năng Java hiện đại được dùng trong NextChat*

Tính năng	Vị trí trong mã nguồn	Tác dụng
Record (Java 16)	`RegisterRequest`, `LoginRequest`, `JwtResponse`, `ApiResponse<T>`, `MessageDTO`, `RoomDTO`, `SendMessageRequest`, `RefreshTokenService.RefreshTokenIssue`	DTO bất biến, tự sinh constructor, `equals`, `hashCode`; chú thích kiểm tra đặt trực tiếp lên thành phần record.
Text block `"""` (Java 15)	`UserRepository.searchUsers`, `ConversationRepository.findRoomsByUserId`, `MessageRepository`	Viết câu JPQL nhiều dòng dễ đọc, không cần nối chuỗi.
Pattern matching cho `instanceof` (Java 16)	`UserController.currentUser`, `RoomController.getCurrentUserId`: `authentication.getPrincipal() instanceof UserDetails details`; `RoomService.toInstant`	Kiểm tra kiểu và ép kiểu trong một biểu thức, tránh lỗi ép sai.
`var` (Java 10)	`JwtAuthenticationFilter`, `DatabaseUserDetailsService`, `RoomDTO.fromEntity`	Suy luận kiểu biến cục bộ, giảm lặp tên kiểu dài.
`List.of`, `Set.of`, `Stream.toList()` (Java 9–16)	`RoomService.sendMessage` (tập loại tin hợp lệ), `UserController.searchUsers`	Tạo tập hợp bất biến và thu kết quả stream gọn gàng.
Luồng ảo (Java 21)	Chưa bật (không có `spring.threads.virtual.enabled`)	Là hướng nghiên cứu; việc dùng Java 21 không tự chứng minh ứng dụng đã chạy trên luồng ảo.

1.2. Bối cảnh J2EE đầu những năm 2000 và sự ra đời của Spring Framework

1.2.1. Sự phức tạp của EJB 2.x

Vào đầu những năm 2000, cách "chuẩn mực" để viết ứng dụng Java doanh nghiệp là dùng EJB 2.x chạy trong một application server nặng như WebLogic hay WebSphere. Một thành phần nghiệp vụ đơn giản đòi hỏi nhiều giao diện (home, remote, local), lớp cài đặt phải kế thừa các giao diện của container, và hành vi giao dịch, bảo mật được khai báo trong các tệp mô tả triển khai XML dài. Hệ quả thực tế là mã nghiệp vụ gắn chặt vào container, rất khó kiểm thử đơn vị vì không thể khởi tạo đối tượng bên ngoài server, và chu kỳ "sửa – đóng gói – triển khai – kiểm tra" kéo dài.

1.2.2. Cuốn sách của Rod Johnson và mã nguồn đi kèm

Rod Johnson, một chuyên gia tư vấn người Úc, xuất bản cuốn "Expert One-on-One J2EE Design and Development" vào cuối năm 2002 [13]. Cuốn sách phê phán sự phức tạp không cần thiết của EJB và đề xuất cách tiếp cận dựa trên đối tượng Java thuần (POJO), cấu hình bên ngoài và tiêm phụ thuộc. Theo chính tác giả, cuốn sách đi kèm khoảng 30.000 dòng mã framework – sản phẩm của gần một năm làm việc toàn thời gian [14]. Bộ mã này là hạt giống của Spring Framework.

Đầu năm 2003, Juergen Hoeller và Yann Caroff cùng Rod Johnson biến bộ mã thành một dự án mã nguồn mở. Cái tên "Spring" (mùa xuân) được chọn để nói về một khởi đầu mới sau "mùa đông" của J2EE truyền thống. Phiên bản 0.9 được phát hành theo giấy phép Apache 2.0 vào tháng 06/2003 và bản phát hành chính thức đầu tiên, Spring Framework 1.0, ra mắt ngày 24/03/2004.

1.2.3. Những nguyên lý nền tảng của Spring

- Đối tượng Java thuần (POJO): lớp nghiệp vụ không cần kế thừa lớp hay giao diện của framework, nhờ đó có thể khởi tạo và kiểm thử bằng `new` như mọi lớp Java khác.
- Đảo ngược điều khiển và tiêm phụ thuộc (IoC/DI): container chịu trách nhiệm tạo đối tượng và truyền phụ thuộc vào, thay vì đối tượng tự đi tìm hoặc tự tạo phụ thuộc của mình.
- Lập trình hướng khía cạnh (AOP): các mối quan tâm cắt ngang như giao dịch, bảo mật, ghi log được áp dụng bằng proxy bao quanh đối tượng, tách khỏi mã nghiệp vụ.
- Trừu tượng hoá hạ tầng: lớp trừu tượng thống nhất cho JDBC, quản lý giao dịch, ORM, giúp đổi công nghệ bên dưới mà không viết lại nghiệp vụ.
- Không xâm lấn: framework được dùng như thư viện, có thể áp dụng từng phần.

Hai mươi năm sau, chính các nguyên lý này vẫn hiện diện trong NextChat: `RoomService` là một POJO nhận bốn repository qua constructor (DI); chú thích

`@Transactional` trên `RoomService.sendMessage` được thực thi bởi một proxy AOP (mục 4.7); Spring Data JPA trừu tượng hoá việc truy cập dữ liệu (mục 4.6).

1.3. Các thế hệ Spring Framework

Spring Framework phát triển qua nhiều thế hệ, mỗi thế hệ gắn với một bước tiến của ngôn ngữ Java và của cách xây dựng ứng dụng. Bảng 1.2 tóm tắt các mốc chính.

Bảng 1.2. *Các thế hệ Spring Framework và điểm nổi bật*

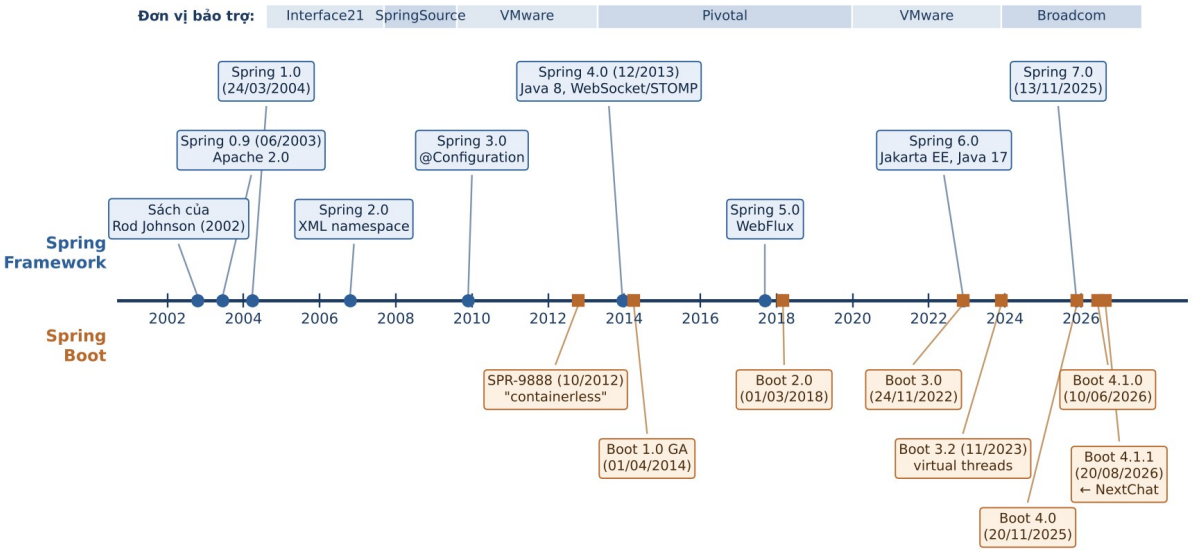
Phiên bản	Thời điểm	Điểm nổi bật
1.0	03/2004	IoC container cấu hình bằng XML, AOP, trừu tượng JDBC và giao dịch, Spring MVC đầu tiên.
2.0 / 2.5	2006 / 2007	Namespace XML mở rộng, tích hợp AspectJ; 2.5 mở ra cấu hình bằng chú thích (`@Autowired`, `@Component`, `@Controller`).
3.0	12/2009	Nền tảng Java 5, cấu hình bằng lớp Java (`@Configuration`, `@Bean`), ngôn ngữ biểu thức SpEL, hỗ trợ REST trong MVC.
4.0	12/2013	Hỗ trợ Java 8; mô-đun mới `spring-websocket` và `spring-messaging` với WebSocket, SockJS, STOMP; cơ chế `@Conditional` – nền tảng của auto-configuration trong Spring Boot [22].
5.0	09/2017	Lập trình phản ứng với Spring WebFlux, hỗ trợ Kotlin, nền tảng Java 8.
6.0	11/2022	Nền tảng Java 17, chuyển sang Jakarta EE 9+, biên dịch trước (AOT) và ảnh native GraalVM, khả năng quan sát với Micrometer.
7.0	13/11/2025	Thế hệ mới tập trung vào Java 25 và hệ sinh thái mã nguồn mở mới nhất, an toàn null với JSpecify, nền tảng cho Spring Boot 4.0 [24].

Mốc Spring 4.0 (12/2013) đặc biệt quan trọng với đề tài của nhóm vì hai lý do. Thứ nhất, đây là phiên bản đưa hỗ trợ WebSocket, SockJS và STOMP vào lõi framework thông qua hai mô-đun `spring-websocket` và `spring-messaging`. Nhóm phát triển Spring khi đó chọn STOMP làm giao thức con vì định dạng khung của nó mô phỏng HTTP và đã được nhiều thư viện client hỗ trợ, đồng thời mô hình lập trình cho phép ánh xạ thông điệp vào phương thức controller tương tự cách Spring MVC ánh xạ request HTTP [23]. Toàn bộ `WebSocketConfig`, `RoomWebSocketController` và `JwtStompChannelInterceptor` của NextChat được xây trên nền này. Thứ hai, Spring 4.0 giới thiệu cơ chế bean có điều kiện

`@Conditional`, thứ mà Spring Boot dùng để quyết định tạo hay không tạo một bean hạ tầng.

1.3.1. Các tổ chức bảo trợ Spring qua thời gian

Song song với sự phát triển kỹ thuật, đơn vị đứng sau Spring cũng thay đổi nhiều lần. Năm 2004, Rod Johnson cùng các cộng sự thành lập Interface21 để cung cấp tư vấn và hỗ trợ cho Spring; công ty đổi tên thành SpringSource năm 2007 và được VMware mua lại năm 2009. Năm 2013, VMware và EMC tách nhóm Spring sang công ty liên doanh Pivotal; đây cũng là nơi Spring Boot ra đời. Cuối năm 2019 VMware mua lại Pivotal, và ngày 22/11/2023 Broadcom hoàn tất thương vụ mua VMware [29]. Qua tất cả các lần chuyển giao, Spring vẫn là dự án mã nguồn mở theo giấy phép Apache 2.0 và các phiên bản vẫn được phát hành công khai trên Maven Central.



Hình 1.1. *Dòng thời gian Spring Framework, Spring Boot và các đơn vị bảo trợ (tổng hợp từ [14], [16]–[24], [29])*

1.4. Sự ra đời của Spring Boot

1.4.1. Vấn đề cần giải quyết

Đến đầu những năm 2010, Spring đã thay thế EJB trong rất nhiều dự án, nhưng chính Spring lại trở nên nặng nề theo một cách khác. Để có một ứng dụng web cơ bản, lập trình viên phải tự chọn và căn chỉnh phiên bản của hàng chục thư viện (Spring, Hibernate, Jackson, driver cơ sở dữ liệu, thư viện log...), viết cấu hình XML hoặc Java cho DataSource, EntityManagerFactory, TransactionManager, DispatcherServlet, rồi đóng gói thành tệp WAR và triển khai vào một servlet container cài sẵn như Tomcat. Phần lớn cấu hình này lặp lại gần như giống hệt giữa các dự án.

1.4.2. Yêu cầu SPR-9888: ứng dụng "không cần container"

Tháng 10/2012, Mike Youngstrom tạo yêu cầu SPR-9888 trên hệ thống quản lý vấn đề của Spring với tiêu đề "Improved support for 'containerless' web application architectures" [15]. Ông lập luận rằng servlet container đi kèm đường cong học tập riêng và mô hình cấu hình riêng, và kiến trúc ứng dụng web của Spring có thể đơn giản hơn nhiều nếu dùng mô hình thành phần và cấu hình của Spring từ trên xuống dưới, nhúng luôn các dịch vụ web vào trong container của Spring.

Phil Webb và Dave Syer tại Pivotal nhận yêu cầu này và phát triển thành một dự án mới. Spring Boot được giới thiệu tại hội nghị SpringOne2GX năm 2013 và phát hành bản chính thức 1.0 vào ngày 01/04/2014. Trong bài công bố, Phil Webb nhắc lại rằng đã 18 tháng kể từ yêu cầu ban đầu "cải thiện kiến trúc ứng dụng web không cần container" – yêu cầu đã khai sinh ra Spring Boot [16].

1.4.3. Những ý tưởng cốt lõi của Spring Boot

- Mặc định có chủ kiến (opinionated defaults): framework chọn sẵn cấu hình hợp lý cho đa số trường hợp, nhưng lùi lại ngay khi lập trình viên khai báo cấu hình riêng.

- Starter: mỗi starter là một gói phụ thuộc đã được kiểm tra tương thích cho một nhóm chức năng (web, JPA, security, websocket...). Phiên bản do BOM của Spring Boot quản lý tập trung.
- Auto-configuration: dựa trên những gì có trong classpath, các thuộc tính cấu hình và các bean đã có, Spring Boot tự tạo bean hạ tầng tương ứng.
- Máy chủ nhúng và tệp jar thực thi: ứng dụng chạy bằng phương thức `main`, Tomcat được nhúng sẵn; không cần cài và cấu hình container ngoài.
- Cấu hình ngoài và profile: cùng một bản build chạy ở nhiều môi trường, khác nhau ở tệp cấu hình, biến môi trường hoặc tham số dòng lệnh.
- Sẵn sàng vận hành: Actuator cung cấp health check, metrics; về sau là tích hợp Micrometer và tracing.

1.5. Các phiên bản Spring Boot đến 4.1.1

Spring Boot duy trì nhịp phát hành đều đặn: mỗi phiên bản phụ (minor) ra mắt khoảng sáu tháng một lần, thường vào tháng 5 và tháng 11, và mỗi nhánh được hỗ trợ mã nguồn mở khoảng 12 tháng [33]. Bảng 1.3 liệt kê các mốc chính, tập trung vào những thay đổi có liên quan tới đồ án.

Bảng 1.3. *Các mốc phát hành chính của Spring Boot*

Phiên bản	Ngày phát hành	Thay đổi chính
1.0	01/04/2014	Bản GA đầu tiên: starter, auto-configuration, Tomcat nhúng, Actuator, CLI Groovy; xây trên Spring Framework 4.0 [16].
1.5	02/2017	Bản cuối của dòng 1.x, được dùng rất rộng rãi trước khi lên 2.0.
2.0	01/03/2018	Nền tảng Java 8, hỗ trợ ứng dụng phản ứng (WebFlux, Netty nhúng), kiến trúc Actuator mới, metrics Micrometer, HTTP/2; kết quả của 17 tháng làm việc với hơn 6.800 commit từ 215 người đóng góp [17].
2.1 – 2.7	11/2018 – 05/2022	Hỗ trợ Java 11 rồi Java 17, tắt máy an toàn (graceful shutdown), cơ chế nạp cấu hình mới `spring.config.import` (2.4) – chính cơ chế NextChat dùng để nạp `database.yml` và `.env`.
3.0	24/11/2022	Nền tảng Java 17, chuyển sang Jakarta EE 9+ (`javax` → `jakarta`), ảnh native GraalVM, quan sát bằng Micrometer Tracing; xây trên Spring Framework 6.0 [18].
3.2	11/2023	Hỗ trợ luồng ảo Java 21, `RestClient`, khởi động nhanh bằng CRaC.

Phiên bản	Ngày phát hành	Thay đổi chính
3.5	05/2025	Bản cuối dòng 3.x; hết hỗ trợ mã nguồn mở ngày 30/06/2026 [33].
4.0	20/11/2025	Thế hệ mới trên Spring Framework 7: mô-đun hoá toàn bộ mã nguồn Spring Boot thành các jar nhỏ và tập trung hơn, an toàn null với JSpecify, hỗ trợ hạng nhất cho Java 25 (vẫn giữ tương thích Java 17), hỗ trợ API versioning và HTTP Service Client [19].
4.1.0	10/06/2026	Bản phụ đầu tiên của dòng 4.x: tự cấu hình Spring gRPC cho server và client, giảm thiểu SSRF cho HTTP client, đọc biến môi trường OpenTelemetry, nâng cấp Kotlin 2.3 [20].
4.1.1	20/08/2026	Bản vá của 4.1 – phiên bản được khai báo trong `pom.xml` của NextChat [21].

Một số nhận xét rút ra từ lịch sử phiên bản có ý nghĩa thực tế cho nhóm:

- Mỗi phiên bản chính (1.0, 2.0, 3.0, 4.0) đi kèm với một bước nâng nền tảng Java và một thế hệ Spring Framework. Vì vậy khi đọc tài liệu hay hỏi đáp trên mạng, cần xác định câu trả lời viết cho thế hệ nào; ví dụ mã dùng `javax.persistence` chỉ đúng cho Spring Boot 2.x.
- Nhánh 4.0 được hỗ trợ mã nguồn mở tới 31/12/2026 và nhánh 4.2 dự kiến phát hành tháng 11/2026 [33]. Việc nhóm chọn 4.1.1 – bản vá mới nhất của nhánh mới nhất tại thời điểm làm đồ án – giúp nhận các bản sửa lỗi bảo mật trong suốt thời gian học phần.
- Tài liệu tham khảo trên web luôn hiển thị phiên bản mới nhất; báo cáo ghi rõ phiên bản lấy từ `pom.xml` để tránh nhầm lẫn khi một tính năng thay đổi giữa các bản.

1.6. Dấu ấn của Spring Boot 4 trong mã nguồn NextChat

Spring Boot 4.0 là thay đổi kiến trúc lớn nhất kể từ 3.0. Điều đáng chú ý là những thay đổi này không chỉ nằm trong ghi chú phát hành mà có thể thấy trực tiếp khi đọc `pom.xml` và các câu lệnh `import` của NextChat. Bảng 1.4 đối chiếu từng dấu hiệu với thay đổi tương ứng.

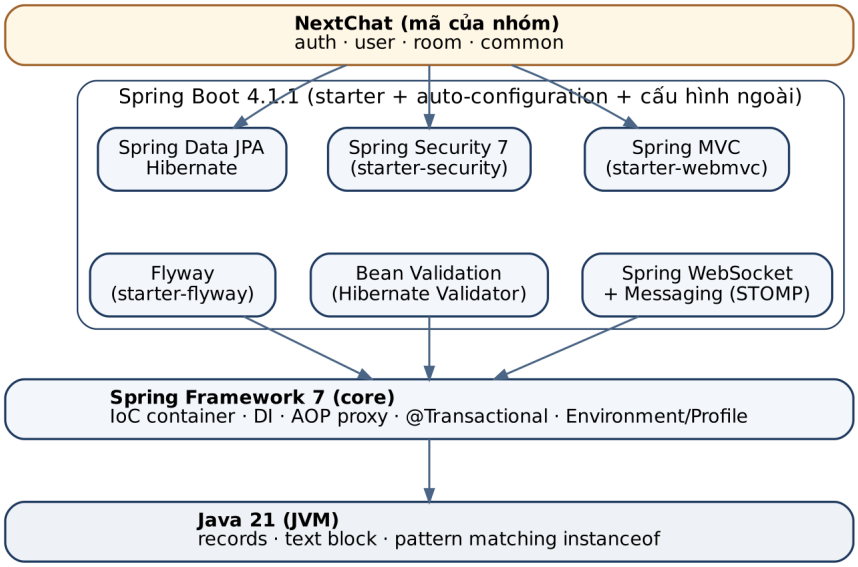
Bảng 1.4. *Dấu hiệu của thế hệ Spring Boot 4 trong mã nguồn*

Dấu hiệu trong mã nguồn	Thay đổi của Spring Boot 4	Ý nghĩa đối với nhóm
`spring-boot-starter-webmvc` thay cho `spring-boot-starter-web` quen thuộc	Starter được tái tổ chức theo mô-đun; mỗi công nghệ có starter và mô-đun auto-configuration riêng	Tài liệu cũ hướng dẫn `starter-web` vẫn chạy nhờ cầu nối nhưng cần hiểu tên mới.
`spring-boot-starter-flyway`, `spring-boot-starter-jackson`	Flyway và Jackson có starter riêng thay vì chỉ thêm thư viện	Auto-configuration của Flyway nằm trong mô-đun riêng, chỉ kích hoạt khi có starter.
Các starter kiểm thử theo mô-đun: `spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-security-test`, `-validation-test`, `-websocket-test`	Hạ tầng kiểm thử cũng được tách theo công nghệ	Test chỉ kéo những gì cần cho từng lát cắt.
`import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc` trong test tích hợp	Gói Java đổi theo mô-đun `org.springframework.boot.<module>`	Đoạn mã sao chép từ tài liệu Boot 3 sẽ không biên dịch được nếu giữ gói cũ.
Lớp điều kiện `org.springframework.boot.jdbc.autoconfigure.DataSourceConfiguration$Hikari` trong log kiểm chứng ngày 28/10	Auto-configuration DataSource thuộc mô-đun `jdbc`	Minh chứng mục 4.1 phải tìm điều kiện theo tên gói mới.
`import tools.jackson.databind.ObjectMapper` trong `JsonAuthenticationEntryPoint` và `JsonAccessDeniedHandler`	Spring Boot 4 dùng Jackson 3 với không gian tên mới `tools.jackson`	Lớp test vẫn dùng `com.fasterxml.jackson.databind.ObjectMapper` để đọc chuỗi JSON; nên thống nhất một thế hệ Jackson khi mở rộng.
`@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)` trên lớp `NextchatApplication`	Spring Data tuần tự hoá `Page` qua DTO ổn định gồm `content` và `page{size, number, totalElements, totalPages}`	Hợp đồng JSON của API lịch sử dùng `data.page.totalElements`; test #14 khẳng định đúng đường dẫn này.
`new DaoAuthenticationProvider(users)` và cấu hình Security dạng lambda	Spring Security 7 đi cùng Boot 4, khuyến khích DSL lambda và constructor nhận `UserDetailsService`	Cấu hình bảo mật ngắn và rõ hơn so với phong cách `WebSecurityConfigurerAdapter` đã bị loại bỏ.

1.7. Kiến trúc cốt lõi của Spring Boot

Hình 1.2 trình bày vị trí của mã nguồn NextChat trong ngăn xếp công nghệ. Ở dưới cùng là JVM chạy Java 21; phía trên là Spring Framework 7 cung cấp IoC

container, AOP và quản lý giao dịch; các dự án Spring chuyên biệt (MVC, Security, Data JPA, WebSocket/Messaging) xây trên lõi đó; Spring Boot nằm ở lớp trên cùng, chịu trách nhiệm chọn phiên bản tương thích, tự cấu hình và nạp cấu hình ngoài. Mã của nhóm chỉ cần khai báo starter và viết các thành phần nghiệp vụ.



Hình 1.2. *Ngăn xếp công nghệ Spring mà NextChat sử dụng*

Có thể tóm tắt kiến trúc Spring Boot thành năm trụ cột; mỗi trụ cột được phân tích sâu kèm minh chứng ở Chương 4:

- IoC container và DI: container đọc các lớp có `@Component`, `@Service`, `@Configuration`, `@Bean`, tạo đối tượng và nối phụ thuộc qua constructor (mục 4.2).
- Starter và BOM: `spring-boot-starter-parent` 4.1.1 cố định phiên bản của mọi thư viện đi kèm; nhóm không phải ghi phiên bản cho Hibernate, Jackson, Tomcat hay driver PostgreSQL (mục 3.1).
- Auto-configuration có điều kiện: các lớp AutoConfiguration chỉ tạo bean khi điều kiện `@ConditionalOnClass`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean` thoả mãn (mục 4.1).

- Cấu hình ngoài và profile: thuộc tính được gộp từ tệp YAML, tệp nhập thêm, biến môi trường theo thứ tự ưu tiên; profile chọn bean theo môi trường (mục 4.3).
- Máy chủ nhúng: Tomcat chạy bên trong ứng dụng, cùng một tiến trình phục vụ REST, WebSocket và tài nguyên tĩnh của client web (mục 3.3).

1.8. So sánh Spring Boot với các framework backend hiện đại

Để đặt Spring Boot vào bối cảnh, Bảng 1.5 so sánh định tính với các lựa chọn phổ biến khác cho backend. Đây là so sánh về mô hình lập trình và hệ sinh thái, không phải phép đo hiệu năng: đồ án không thực hiện benchmark giữa các nền tảng, nên báo cáo không kết luận nền tảng nào nhanh hơn hay tốn ít bộ nhớ hơn.

Bảng 1.5. *So sánh định tính Spring Boot với một số framework backend*

Tiêu chí	Spring Boot (Java)	NestJS (Node.js)	FastAPI / Django (Python)	ASP.NET Core (C#)
Mô hình thực thi chính	Servlet đa luồng (MVC) hoặc phản ứng (WebFlux); luồng ảo từ Java 21	Vòng lặp sự kiện đơn luồng, I/O không chặn	ASGI bất đồng bộ (FastAPI) hoặc WSGI/ASGI (Django)	Đa luồng, async/await
Tiêm phụ thuộc	Lõi của framework (IoC container)	Có, lấy cảm hứng từ Angular	FastAPI có `Depends`; Django không có DI container đầy đủ	Có sẵn trong framework
Bảo mật	Spring Security: filter chain, nhiều cơ chế xác thực	Passport, Guard	Thư viện bổ sung; Django có auth sẵn	ASP.NET Identity, middleware
Truy cập dữ liệu	JPA/Hibernate, Spring Data, JDBC	TypeORM, Prisma	SQLAlchemy, Django ORM	Entity Framework Core
Thời gian thực	WebSocket + STOMP + broker có sẵn trong Spring	Gateway với Socket.IO hoặc ws	WebSocket (Starlette), Django Channels	SignalR
Quản lý lược đồ	Flyway, Liquibase tích hợp tự cấu hình	Migration của ORM	Alembic, Django migrations	EF Core migrations
Kiểm thử tích hợp	Spring Boot Test, MockMvc, Testcontainers	Jest + Supertest	pytest + TestClient	xUnit + WebApplicationFactory

Tiêu chí	Spring Boot (Java)	NestJS (Node.js)	FastAPI / Django (Python)	ASP.NET Core (C#)
Độ dốc học tập	Cao: nhiều khái niệm (proxy, context, auto-config)	Trung bình	Thấp đến trung bình	Trung bình

Nhận định của nhóm sau khi so sánh: điểm mạnh nổi bật của Spring Boot đối với bài toán chat là tính thống nhất – xác thực, giao dịch, truy cập dữ liệu, quản lý lược đồ, WebSocket/STOMP và kiểm thử tích hợp đều nằm trong cùng một hệ sinh thái, dùng chung mô hình bean và cấu hình. Đây là lợi thế về độ gắn kết trong phạm vi đồ án, không phải bằng chứng rằng các nền tảng khác kém an toàn hoặc kém hiệu quả. Điểm yếu tương ứng là lượng khái niệm cần nắm lớn: khi có lỗi ở tầng proxy giao dịch hay ở kênh thông điệp STOMP, người học phải hiểu cơ chế bên trong mới gỡ được (xem các tình huống ở mục 11.2).

1.9. Các trường hợp hạn chế và cần cân nhắc khi dùng Spring Boot

Khi chọn Spring Boot cần cân nhắc quy mô ứng dụng, kinh nghiệm đội ngũ và tài nguyên vận hành. Một số trường hợp cần thử nghiệm kỹ trước khi quyết định:

- Hàm serverless hoặc dịch vụ có ngân sách thời gian khởi động rất nhỏ: ứng dụng JVM truyền thống cần thời gian khởi động và làm nóng; có thể cân nhắc ảnh native GraalVM nhưng phải đo trên bản build cụ thể. NextChat chưa xây dựng ảnh native.
- Thiết bị hoặc môi trường có bộ nhớ hạn chế: cần đo dung lượng heap và thư viện thực tế của bản build thay vì áp một ngưỡng RAM chung.
- Nhóm chưa quen Java và Spring: cần dành thời gian cho IoC, Security và giao dịch; lỗi do hiểu sai proxy (gọi nội bộ `this.method()` làm mất `@Transactional`) hoặc sai thứ tự filter rất khó nhận ra nếu chỉ đọc mã bề mặt.
- Ứng dụng rất nhỏ, chỉ vài endpoint, không cần bảo mật hay giao dịch phức tạp: chi phí khái niệm của Spring có thể lớn hơn lợi ích.

1.10. Tiểu kết chương

Spring Framework ra đời năm 2002–2004 như một phản ứng trước sự phức tạp của EJB, đặt nền móng bằng POJO, IoC/DI và AOP. Spring Boot ra đời năm 2014 từ yêu cầu SPR-9888, giải quyết một vấn đề khác: chính cấu hình Spring đã trở nên lặp lại và nặng nề. Qua bốn thế hệ, Spring Boot liên tục nâng nền tảng Java và Jakarta EE; thế hệ 4 (11/2025) mô-đun hoá toàn bộ mã nguồn, và phiên bản 4.1.1 (08/2026) là nền tảng của NextChat. Những thay đổi đó hiện diện trực tiếp trong mã của nhóm: tên starter, gói `import`, Jackson 3, định dạng JSON của `Page`. Chương tiếp theo trình bày vì sao nhóm chọn đề tài chat thời gian thực và chọn từng thành phần công nghệ trên nền tảng này.

CHƯƠNG 2. LÝ DO LỰA CHỌN ĐỀ TÀI VÀ CÔNG NGHỆ

Chương 1 trả lời câu hỏi Spring Boot là gì và đến từ đâu. Chương này trả lời câu hỏi vì sao: vì sao nhóm chọn bài toán chat thời gian thực để nghiên cứu Spring Boot, và vì sao trong mỗi lớp của hệ thống nhóm chọn công nghệ này mà không chọn phương án khác. Mỗi lựa chọn được trình bày theo cùng một khuôn: nhu cầu của bài toán → các phương án → căn cứ chọn → đánh đổi chấp nhận.

2.1. Bối cảnh bài toán nhắn tin thời gian thực

2.1.1. Nhu cầu thực tế

Ứng dụng nhắn tin là loại phần mềm mà hầu hết sinh viên dùng hằng ngày, nên yêu cầu nghiệp vụ dễ hiểu và dễ đánh giá: đăng ký, đăng nhập, tìm bạn, mở cuộc trò chuyện riêng, tạo nhóm, gửi tin và thấy tin của người khác xuất hiện ngay. Tuy nhiên phía sau những thao tác quen thuộc đó là các yêu cầu kỹ thuật không đơn giản:

- Độ trễ thấp: tin nhắn phải được đẩy tới người nhận ngay khi được lưu, không đợi người nhận hỏi lại máy chủ.
- Nhất quán: tin nhắn nào đã hiển thị cho người nhận thì phải tồn tại trong cơ sở dữ liệu với cùng mã định danh, để tải lại trang vẫn thấy.
- Phân quyền theo phòng: người ngoài phòng không được đọc lịch sử, không được đăng ký nhận tin, không được gửi tin vào phòng.
- Xác thực trên hai kênh: cả request HTTP lẫn kết nối WebSocket dài hạn đều phải biết người gọi là ai.
- Dữ liệu tăng nhanh: số tin nhắn tăng theo thời gian sử dụng, nên truy vấn lịch sử phải phân trang và cần được phân tích chi phí.

2.1.2. Các kỹ thuật truyền thông thời gian thực trên web

HTTP truyền thống theo mô hình yêu cầu – phản hồi: máy chủ chỉ trả lời khi client hỏi. Để đẩy dữ liệu từ máy chủ xuống client, có bốn kỹ thuật thường gặp; Bảng 2.1 so sánh chúng theo các tiêu chí liên quan tới bài toán chat.

Bảng 2.1. *So sánh các kỹ thuật truyền thông thời gian thực trên web*

Tiêu chí	Short polling	Long polling	Server-Sent Events (SSE)	WebSocket
Nguyên lý	Client hỏi định kỳ (ví dụ 2 giây/lần)	Client hỏi, máy chủ giữ request đến khi có dữ liệu	Một kết nối HTTP dài, máy chủ đẩy sự kiện văn bản	Nâng cấp HTTP thành kết nối TCP hai chiều toàn song công (RFC 6455)
Chiều dữ liệu	Client → máy chủ	Client → máy chủ	Chỉ máy chủ → client	Hai chiều
Độ trễ	Phụ thuộc chu kỳ hỏi	Thấp	Thấp	Thấp
Chi phí	Nhiều request rỗng, tốn băng thông	Phải mở lại request sau mỗi lần trả	Thấp; gửi từ client vẫn phải dùng HTTP riêng	Thấp sau khi bắt tay; cần quản lý phiên dài
Phù hợp với chat	Kém	Chấp nhận được	Khá (cho chiều nhận)	Tốt nhất
Hỗ trợ trong Spring	MVC thông thường	`DeferredResult`	`SseEmitter`, WebFlux	`spring-websocket` + STOMP + SockJS

WebSocket phù hợp nhất cho chat vì cả hai chiều gửi và nhận đều đi trên một kết nối duy trì [3]. Tuy nhiên WebSocket chỉ là "đường ống" chuyển các khung byte; nó không định nghĩa thông điệp gửi cho ai, ai đăng ký nhận gì. Vì thế ứng dụng cần một giao thức con ở phía trên – STOMP – để diễn đạt đích đến (destination), đăng ký (SUBSCRIBE) và gửi (SEND) [4]. Đây chính là mô hình mà Spring hỗ trợ từ phiên bản 4.0 (mục 1.3).

2.2. Lý do chọn đề tài "chat real-time bằng WebSocket"

Nhóm cân nhắc ba hướng đề tài trước khi chốt: (a) hệ thống quản lý thư viện bằng REST thuần, (b) cửa hàng trực tuyến có giỏ hàng và thanh toán giả lập, (c) hệ thống chat thời gian thực. Đề tài (c) được chọn vì sáu lý do dưới đây.

Lý do 1 – Phủ được nhiều cơ chế cốt lõi của Spring Boot nhất

Một backend REST thuần chủ yếu luyện MVC và JPA. Bài toán chat bắt buộc dùng thêm Spring Security cho hai kênh (HTTP và STOMP), quản lý giao dịch cho thao tác nhiều bước (lưu tin và cập nhật tin cuối cùng của phòng), Spring Messaging cho kênh thời gian thực, cùng với Flyway và Testcontainers để kiểm thử với cơ sở dữ liệu thật. Bảng 4.1 ở Chương 4 cho thấy cả chín yêu cầu lõi của học phần đều có vị trí sử dụng cụ thể trong NextChat.

Lý do 2 – Có bài toán đồng thời thật để nghiên cứu sâu

Học phần yêu cầu giải thích bản chất chứ không chỉ liệt kê tính năng. Bài toán chat tự nhiên sinh ra các tình huống tranh chấp: hai người cùng mở phòng 1-1 cho nhau, hai yêu cầu làm mới token đến cùng lúc, nhiều tin được gửi vào cùng một phòng. Những tình huống này cho phép nhóm giải thích giao dịch, mức cô lập, khoá advisory bằng thí nghiệm có đối chứng (Chương 5) thay vì chỉ trích định nghĩa.

Lý do 3 – Dễ kiểm chứng đầu-cuối và dễ trình diễn

Kết quả của hệ thống chat có thể quan sát trực tiếp: mở hai cửa sổ trình duyệt với hai tài khoản, gửi tin ở cửa sổ này và thấy tin xuất hiện ở cửa sổ kia. Ở mức tự động, một kịch bản hai client WebSocket có thể khẳng định cả hai nhận cùng mã tin nhắn và cơ sở dữ liệu chứa đúng bản ghi đó (mục 9.2). Điều này giúp buổi bảo vệ có minh chứng trực quan và có minh chứng định lượng.

Lý do 4 – Phù hợp quy mô ba người trong mười tuần

Bài toán tách được thành ba mảng tương đối độc lập, khớp với ba thành viên: kênh thời gian thực và hạ tầng chung; REST và cơ sở dữ liệu; client và triển khai. Các mảng giao tiếp với nhau qua hợp đồng rõ ràng (9 endpoint REST, các destination STOMP), nên có thể làm song song và hợp nhất qua Pull Request. Chương 10 cho thấy cách chia này vận hành ra sao trong thực tế, kể cả những điểm lệch hợp đồng phải sửa khi tích hợp.

Lý do 5 – Gắn với nhu cầu nghề nghiệp

Kiến thức về xác thực bằng token, WebSocket, giao dịch và quản lý lược đồ có giá trị lâu dài cho công việc backend, bất kể sau này dùng framework nào. Spring Boot cũng là framework Java được dùng phổ biến nhất theo nhiều khảo sát (mục 2.6), nên kỹ năng thu được có tính ứng dụng cao.

Lý do 6 – Có không gian mở rộng rõ ràng

Lược đồ V1 đã thiết kế sẵn bảng cho tệp đính kèm, trạng thái đã đọc, cảm xúc, ghim tin, thông báo và cuộc gọi. Ngoài ra còn các hướng như trạng thái trực tuyến, phân trang keyset, broker ngoài cho nhiều instance. Nhờ vậy đề tài có thể tiếp tục phát triển cho các học phần sau mà không phải làm lại từ đầu (Chương 12).

2.3. Tiêu chí lựa chọn công nghệ

Trước khi chọn từng thành phần, nhóm thống nhất bộ tiêu chí ở Bảng 2.2. Trọng số phản ánh ưu tiên của một đồ án học phần: phải giải thích được bản chất và kiểm chứng được, quan trọng hơn hiệu năng tuyệt đối.

Bảng 2.2. *Tiêu chí lựa chọn công nghệ và trọng số*

Tiêu chí	Câu hỏi đánh giá	Trọng số
T1. Phù hợp mục tiêu học phần	Công nghệ có cho phép giải thích các cơ chế bản chất (IoC, giao dịch, bảo mật...) và gắn với nội dung môn học không?	25%
T2. Hệ sinh thái tích hợp	Xác thực, dữ liệu, thời gian thực, quản lý lược đồ, kiểm thử có nằm trong cùng hệ sinh thái, cấu hình thống nhất không?	20%
T3. Khả năng kiểm chứng	Có công cụ kiểm thử tích hợp với hạ tầng thật, tái lập được trên máy khác không?	20%
T4. Tài liệu và cộng đồng	Có tài liệu chính thức đầy đủ, cập nhật theo phiên bản không?	15%
T5. Năng lực hiện có của nhóm	Thành viên đã học ngôn ngữ và công cụ liên quan ở các học phần trước chưa?	10%
T6. Giá trị nghề nghiệp	Kỹ năng có được sử dụng rộng rãi trong tuyển dụng không?	10%

2.4. Lý do chọn từng thành phần công nghệ

2.4.1. Java 21 và Spring Boot 4.1.1

Các thành viên đã học Java và lập trình hướng đối tượng ở các học phần trước, nên chọn nền tảng Java giúp nhóm dành thời gian cho framework thay vì cho ngôn ngữ (T5). Java 21 là bản LTS mới nhất mà cả ba máy của nhóm đều cài sẵn và được Spring Boot 4 hỗ trợ đầy đủ. Spring Boot 4.1.1 (phát hành 20/08/2026) là bản vá mới nhất của nhánh mới nhất tại thời điểm Lê Quốc Cường khởi tạo dự án ngày 22/09/2026 (commit `6ab071a`); chọn bản mới giúp nhóm học đúng API hiện hành (Jackson 3, Spring Security 7, starter mô-đun) và nhận bản sửa lỗi trong suốt học phần. Đánh đổi chấp nhận: nhiều bài viết hướng dẫn trên mạng vẫn viết cho Boot 2.x/3.x, nhóm phải đối chiếu với tài liệu chính thức khi sao chép ví dụ (xem Bảng 1.4).

2.4.2. Spring Security và JWT không trạng thái

Có hai cách phổ biến để duy trì đăng nhập: phiên trên máy chủ (session cookie) và token tự chứa (JWT). Bảng 2.3 so sánh trong bối cảnh NextChat.

Bảng 2.3. *So sánh phiên máy chủ và JWT không trạng thái*

Tiêu chí	Session cookie	JWT trong header Authorization
Trạng thái phía máy chủ	Máy chủ lưu phiên (bộ nhớ hoặc Redis)	Không lưu phiên; mỗi request tự mang token (`SessionCreationPolicy.STATELESS`)
Dùng cho WebSocket	Trình duyệt tự gửi cookie khi bắt tay	Gửi token trong native header của khung STOMP CONNECT
Rủi ro CSRF	Có, cần token CSRF	Thấp vì token không tự động gửi kèm; nhóm tắt CSRF có chủ đích
Thu hồi tức thì	Dễ: xoá phiên	Khó: token hợp lệ tới khi hết hạn; NextChat giảm rủi ro bằng hạn 15 phút và nạp lại người dùng ở mỗi request
Mở rộng nhiều instance	Cần chia sẻ kho phiên	Mọi instance chỉ cần cùng khoá ký

Nhóm chọn JWT vì cùng một cơ chế phục vụ được cả REST và STOMP, phù hợp với client JavaScript và với hướng mở rộng nhiều instance. Spring Security

được chọn vì nó là cơ chế bảo mật chuẩn của hệ sinh thái Spring, cho phép chèn `JwtAuthenticationFilter` vào chuỗi filter và tuỳ biến phản hồi 401/403 dạng JSON. Thư viện JJWT 0.12.6 được dùng để ký và xác minh token bằng HMAC. Để bù cho nhược điểm khó thu hồi của JWT, nhóm kết hợp access token ngắn hạn (15 phút) với refresh token dài hạn được lưu dưới dạng băm và xoay vòng sau mỗi lần dùng (Chương 7).

2.4.3. PostgreSQL 16

Dữ liệu chat có cấu trúc quan hệ rõ ràng (người dùng – phòng – thành viên – tin nhắn) và cần ràng buộc toàn vẹn chặt, nên cơ sở dữ liệu quan hệ là lựa chọn tự nhiên. Giữa PostgreSQL và MySQL, nhóm chọn PostgreSQL vì các tính năng được dùng trực tiếp trong đồ án:

- Khoá advisory mức giao dịch `pg_advisory_xact_lock` kết hợp hàm băm `hashtextextended`: nền tảng của cơ chế chống tạo trùng phòng 1-1 (Chương 5) [6].
- Ngôn ngữ thủ tục PL/pgSQL với hàm trả về bảng (`RETURNS TABLE`): dùng cho `get_room_messages` (Chương 6).
- Cột định danh chuẩn SQL `GENERATED ALWAYS AS IDENTITY`, ràng buộc `CHECK` cho các giá trị liệt kê (trạng thái, loại phòng, loại tin).
- `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` cho phép đo chi phí truy vấn ở thí nghiệm phân trang [12].
- Có ảnh Docker chính thức `postgres:16-alpine` dùng chung cho môi trường phát triển (Docker Compose) và kiểm thử (Testcontainers).

Cơ sở dữ liệu tài liệu như MongoDB được cân nhắc cho tin nhắn nhưng bị loại vì đồ án cần giao dịch trải trên nhiều bảng và cần minh hoạ JPA – nội dung trọng tâm của học phần.

2.4.4. Spring Data JPA kết hợp hàm PL/pgSQL

Spring Data JPA giúp viết repository bằng giao diện, sinh truy vấn từ tên phương thức và JPQL, phù hợp cho phần lớn thao tác CRUD. Tuy nhiên hai thao tác nhạy cảm được nhóm chủ động đưa vào hàm cơ sở dữ liệu: tạo-hoặc-lấy phòng 1-1 (cần khoá để nguyên tử) và lấy lịch sử phân trang kèm thông tin người gửi (cần kiểm soát câu SQL để phân tích kế hoạch thực thi). Đây là cách tiếp cận lai: JPA cho mô hình miền, SQL cho chỗ cần kiểm soát chính xác. Đánh đổi là logic bị chia ở hai nơi; nhóm giảm rủi ro bằng cách quản lý hàm SQL trong migration Flyway và kiểm tra sự tồn tại của hàm trong test #1.

2.4.5. Flyway thay vì để Hibernate tự sinh bảng

Hibernate có chế độ `ddl-auto: update` tự sinh và sửa bảng từ entity. Cách này tiện khi thử nghiệm nhưng không có lịch sử phiên bản, không tạo được hàm PL/pgSQL hay trigger, và có thể thay đổi lược đồ ngoài ý muốn. Nhóm chọn Flyway: mỗi thay đổi lược đồ là một tệp `V<số>__<mô tả>.sql` được đánh phiên bản và ghi vào bảng `flyway_schema_history`; Hibernate chỉ chạy ở chế độ `validate` để phát hiện lệch giữa entity và bảng. Liquibase là phương án tương đương nhưng dùng định dạng changelog riêng; Flyway được chọn vì viết SQL thuần, dễ đọc với cả ba thành viên.

2.4.6. WebSocket, STOMP, simple broker và SockJS

Với kênh thời gian thực, nhóm cân nhắc bốn phương án ở Bảng 2.4.

Bảng 2.4. *Các phương án cho kênh thời gian thực*

Phương án	Ưu điểm	Nhược điểm trong bối cảnh đồ án
`WebSocketHandler` thuần của Spring	Kiểm soát hoàn toàn khung dữ liệu	Phải tự định nghĩa định dạng thông điệp, tự quản lý danh sách phiên theo phòng
STOMP trên WebSocket + simple broker (được chọn)	Có sẵn khái niệm destination, SUBSCRIBE, SEND; ánh xạ vào `@MessageMapping`; xác thực bằng `ChannelInterceptor`; client stomp.js phổ biến	Broker trong bộ nhớ chỉ chạy trong một tiến trình; muốn nhiều instance phải chuyển sang broker relay

Phương án	Ưu điểm	Nhược điểm trong bối cảnh đồ án
Socket.IO (qua thư viện Java bên thứ ba)	Client phổ biến trong cộng đồng Node.js	Không thuộc hệ sinh thái Spring, khó tích hợp Security và giao dịch
Server-Sent Events + REST POST	Đơn giản, dùng HTTP thuần	Hai kênh tách rời; không có mô hình đăng ký theo phòng

STOMP được chọn vì nó cho phép tổ chức thông điệp theo phòng bằng destination (`/topic/room/{roomId}`), tách phần xử lý của ứng dụng (tiền tố `/app`) khỏi phần phát tán của broker (tiền tố `/topic`), và cho phép kiểm tra quyền ngay trên kênh vào bằng interceptor – tất cả đều là tính năng có sẵn của Spring [25]. Simple broker được chọn cho phạm vi một tiến trình; khi cần nhiều instance có thể đổi sang RabbitMQ hoặc ActiveMQ qua broker relay mà không đổi controller. Endpoint `/ws-chat` bật SockJS để dự phòng cho mạng chặn WebSocket; endpoint `/ws` giữ WebSocket thuần cho client kiểm thử.

2.4.7. Testcontainers thay vì cơ sở dữ liệu nhúng H2

Cách nhanh nhất để kiểm thử tích hợp là dùng H2 trong bộ nhớ. Tuy nhiên H2 không có PL/pgSQL, không có `pg_advisory_xact_lock`, và cú pháp `GENERATED ALWAYS AS IDENTITY` hay `CHECK` có thể hành xử khác. Một bài test "đạt" trên H2 không chứng minh được hàm tạo phòng chạy đúng trên PostgreSQL. Testcontainers dựng một container `postgres:16-alpine` tạm thời cho mỗi lần chạy test, Flyway áp dụng migration thật, nên kết quả test phản ánh đúng môi trường chạy. Đánh đổi là máy chạy test phải có Docker và thời gian chạy dài hơn vài giây.

2.4.8. Docker Compose, Maven Wrapper và GitHub

- Docker Compose dựng PostgreSQL cho môi trường phát triển chỉ với một lệnh, tránh việc mỗi thành viên cài và cấu hình PostgreSQL khác nhau.
- Maven Wrapper (`mvnw`) đi kèm kho mã, đảm bảo cả nhóm dùng cùng một phiên bản Maven mà không cần cài riêng.

- Git và GitHub với quy trình nhánh tính năng và Pull Request giúp mỗi thành viên làm việc độc lập, còn trưởng nhóm duyệt và hợp nhất; lịch sử commit đồng thời là minh chứng phân công (Chương 10).

2.4.9. Client thử nghiệm bằng HTML và JavaScript thuần

Client web không phải trọng tâm của học phần, nên nhóm chọn cách đơn giản nhất có thể trình diễn đầy đủ luồng: một trang `index.html` và một tệp `app.js` đặt trong thư mục `static` của Spring Boot, nạp Tailwind CSS, sockjs-client và stomp.js qua CDN. Cách này không cần bước build, không cần máy chủ web riêng, và vì client chạy cùng nguồn (origin) với backend nên không gặp vấn đề CORS cho REST. Một framework như React sẽ cho giao diện có cấu trúc tốt hơn nhưng tăng khối lượng việc ngoài trọng tâm môn học.

2.5. Ma trận đánh giá phương án nền tảng

Bảng 2.5 tổng hợp đánh giá của nhóm cho bốn nền tảng backend theo các tiêu chí ở Bảng 2.2, thang điểm 1–5. Đây là đánh giá chủ quan dựa trên mục tiêu học phần và năng lực nhóm tại thời điểm chọn đề tài, không phải kết quả đo lường; với một nhóm hoặc một mục tiêu khác, kết quả có thể khác.

Bảng 2.5. *Ma trận đánh giá có trọng số các nền tảng backend (điểm 1–5)*

Tiêu chí	Trọng số	Spring Boot	NestJS	FastAPI	ASP.NET Core
T1. Phù hợp mục tiêu học phần	25%	5	4	3	4
T2. Hệ sinh thái tích hợp	20%	5	4	3	5
T3. Khả năng kiểm chứng	20%	5	4	4	4
T4. Tài liệu và cộng đồng	15%	5	4	4	4
T5. Năng lực hiện có của nhóm	10%	4	3	3	2
T6. Giá trị nghề nghiệp	10%	5	4	4	4
Điểm có trọng số	100%	**4,90**	**3,90**	**3,45**	**4,00**

Cách tính: điểm có trọng số bằng tổng của (điểm × trọng số). Ví dụ với Spring Boot: 5×0,25 + 5×0,20 + 5×0,20 + 5×0,15 + 4×0,10 + 5×0,10 = 4,90. Khoảng cách lớn nhất nằm ở T1 và T2: với Spring Boot, mọi cơ chế mà học phần yêu cầu giải

thích (IoC, auto-configuration, proxy giao dịch) đều là lõi của framework và được tài liệu hoá chi tiết.

2.6. Vị thế công nghệ và thị trường

Nhóm chỉ dùng số liệu thị trường khi có nguồn, năm và cách lấy mẫu cụ thể, và không suy từ độ phổ biến ra độ phù hợp kỹ thuật.

- Theo khảo sát nhà phát triển của Stack Overflow năm 2025, trong nhóm "web frameworks and technologies", Spring Boot là đại diện duy nhất của Java trong nhóm đầu với khoảng 14,7% người trả lời sử dụng, và khoảng 53,7% người đã dùng muốn tiếp tục dùng [27]. Tỷ lệ này tính trên toàn bộ người trả lời thuộc mọi ngôn ngữ, nên thấp hơn nhiều so với các công nghệ JavaScript như Node.js (khoảng 48,7%) hay React (khoảng 44,7%).
- Các báo cáo hệ sinh thái nhà phát triển của JetBrains qua nhiều năm liên tục ghi nhận Spring Boot và Spring MVC là hai framework web được dùng nhiều nhất trong nhóm lập trình viên chọn Java làm ngôn ngữ chính; báo cáo năm 2023 nhận xét khoảng cách dẫn đầu của Spring và Spring Boot là rất lớn [28].
- Giới hạn của các số liệu trên: mẫu khảo sát là người tự nguyện trả lời, nghiêng về một số khu vực và cộng đồng; con số phản ánh mức độ sử dụng, không phản ánh chất lượng hay mức phù hợp với từng bài toán.

2.7. Phạm vi ứng dụng thực tế của Spring Boot

- Hệ thống nghiệp vụ dữ liệu quan hệ: ngân hàng, bảo hiểm, thương mại điện tử, quản lý nội bộ – nơi cần giao dịch, ràng buộc và bảo mật chặt chẽ. NextChat vận dụng các đặc tính này ở Chương 5–7.
- Dịch vụ theo mô-đun và microservices: kết hợp với Spring Cloud cho cấu hình tập trung, gateway, service discovery. Bản NextChat hiện tại là một ứng dụng backend đơn khối, không phải hệ microservices.

- Ứng dụng thời gian thực: thông báo, bảng điều khiển trực tiếp, chat, cộng tác – dùng WebSocket/STOMP hoặc SSE. NextChat dùng simple broker trong bộ nhớ và chưa đo giới hạn số kết nối.
- Xử lý theo lô và tích hợp hệ thống: Spring Batch, Spring Integration, các kết nối hàng đợi thông điệp.

2.8. Xu hướng phát triển

Một số hướng kỹ thuật đáng nghiên cứu tiếp, chưa được áp dụng hoặc đánh giá trong đồ án:

- Ảnh native và AOT: biên dịch trước ứng dụng thành tệp thực thi native để giảm thời gian khởi động và bộ nhớ; cần thử nghiệm trên bản build cụ thể, nhất là với các thành phần dùng reflection.
- Luồng ảo: từ Spring Boot 3.2 có thể bật luồng ảo cho máy chủ nhúng; phù hợp với ứng dụng MVC/JPA nhiều thao tác I/O chặn như NextChat. Cần đo trước và sau khi bật.
- Quan sát hệ thống: Micrometer, tracing và tích hợp OpenTelemetry ngày càng được tự cấu hình sâu hơn (Spring Boot 4.1 đọc trực tiếp biến môi trường OpenTelemetry) [20].
- Giao tiếp giữa dịch vụ: Spring Boot 4.1 bổ sung tự cấu hình cho Spring gRPC, mở ra lựa chọn ngoài REST cho giao tiếp nội bộ [20].
- Tích hợp mô hình AI: hệ sinh thái Spring có dự án Spring AI cung cấp lớp trừu tượng gọi các mô hình ngôn ngữ; với ứng dụng chat, đây là hướng mở rộng như tóm tắt hội thoại hoặc gợi ý trả lời.

2.9. Tiểu kết chương

Đề tài chat thời gian thực được chọn vì nó phủ gần như toàn bộ cơ chế cốt lõi của Spring Boot, chứa bài toán đồng thời thật để giải thích bản chất, dễ kiểm chứng đầu-cuối và chia việc tốt cho ba người. Mỗi thành phần công nghệ được chọn theo bộ tiêu chí có trọng số: Spring Boot 4.1.1 trên Java 21 cho hệ sinh thái thống nhất; JWT không trạng thái cho cả hai kênh; PostgreSQL vì khoá advisory và PL/pgSQL;

Flyway cho lược đồ có phiên bản; STOMP với simple broker cho kênh thời gian thực; Testcontainers để kiểm thử trên cơ sở dữ liệu thật. Các đánh đổi đi kèm – broker một tiến trình, JWT khó thu hồi, logic chia giữa Java và SQL – được ghi nhận và xử lý ở các chương sau.

PHẦN 2: KIẾN THỨC BẢN CHẤT VÀ THỰC HÀNH

Công nghệ chính: Spring Boot 4.1.1 trên Java 21. Đồ án: NextChat – backend nhắn tin gồm xác thực JWT, tìm người dùng, phòng chat 1-1 và nhóm, lịch sử phân trang, kênh STOMP thời gian thực; dữ liệu trên PostgreSQL 16. Mã nguồn: `github.com/QuoocsCuongwf/springboot-websocket-chat`.

CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN

3.1. Công cụ và phiên bản

Công cụ và phiên bản trong Bảng 3.1 được lấy trực tiếp từ `pom.xml`, `docker-compose.yml`, `README.md` và môi trường chạy kiểm chứng của nhóm, không lấy theo tài liệu nháp.

Bảng 3.1. *Công cụ và phiên bản triển khai*

Công cụ	Phiên bản / nguồn	Vai trò trong dự án
JDK	Java 21 (`java.version` trong `pom.xml`); kiểm chứng trên OpenJDK 21.0.12.1	Biên dịch và chạy ứng dụng.
Spring Boot	4.1.1 (`spring-boot-starter-parent`)	Framework nền; quản lý phiên bản của toàn bộ thư viện đi kèm qua BOM.
Maven Wrapper	`./mvnw`, `mvnw.cmd` có trong kho	Build, chạy `spring-boot:run`, chạy test mà không cần cài Maven riêng.
PostgreSQL	`postgres:16-alpine` qua Docker Compose; kiểm chứng trên 16.15	Cơ sở dữ liệu chính; chứa hai hàm PL/pgSQL.
Docker / Docker Compose	`nextchat/docker-compose.yml`	Dựng PostgreSQL cho môi trường phát triển; Testcontainers dùng Docker cho test. Chưa có Dockerfile đóng gói ứng dụng.
Flyway	`spring-boot-starter-flyway`, `flyway-database-postgresql`	Quản lý phiên bản lược đồ (V1, V2).
JJWT	0.12.6 (`jjwt-api`, `jjwt-impl`, `jjwt-jackson`)	Tạo và xác minh JWT access token.
Testcontainers	1.21.4 (`junit-jupiter`, `postgresql`)	Dựng PostgreSQL 16 tạm thời cho test tích hợp.
Lombok	Khai báo trong `pom.xml` (optional)	Có cấu hình annotation processor nhưng mã hiện tại viết getter/setter thủ công, chưa dùng chú thích Lombok.
Client web	`static/index.html`, `static/app.js`; Tailwind CSS, sockjs-client 1.x, stomp.js 2.3.3 qua CDN	Giao diện thử nghiệm, chạy cùng cổng 8080.
Công cụ gọi API	`curl` hoặc công cụ tương đương	Kho chưa tích hợp Swagger/OpenAPI; hợp đồng API nằm trong `README.md` và Chương 8.

3.2. Các phụ thuộc trong pom.xml

Mỗi phụ thuộc trong `pom.xml` kéo theo một nhóm khả năng và có thể kích hoạt các lớp auto-configuration tương ứng. Bảng 3.2 giải thích vai trò từng phụ thuộc và người đưa phụ thuộc đó vào dự án theo lịch sử Git.

Bảng 3.2. *Phụ thuộc Maven và vai trò*

Phụ thuộc	Vai trò	Thêm bởi
`spring-boot-starter-webmvc`	Spring MVC, Tomcat nhúng, `DispatcherServlet`, chuyển đổi JSON cho REST	Cường (khởi tạo)
`spring-boot-starter-security`	Chuỗi filter bảo mật, `AuthenticationManager`, mã hoá mật khẩu	Cường (khởi tạo)
`spring-boot-starter-data-jpa`	Hibernate, Spring Data JPA, `EntityManagerFactory`, `JpaTransactionManager`, HikariCP	Cường (khởi tạo)
`spring-boot-starter-validation`	Bean Validation (Hibernate Validator) cho `@Valid`, `@NotBlank`, `@Size`	Cường (khởi tạo)
`spring-boot-starter-websocket`	`spring-websocket`, `spring-messaging`, hỗ trợ STOMP và SockJS	Cường (khởi tạo)
`spring-boot-starter-jackson`	Jackson 3 cho tuần tự hoá JSON, `ObjectMapper` được inject vào handler lỗi	Hải (`a2626e6`)
`postgresql` (runtime)	Driver JDBC PostgreSQL	Cường (khởi tạo)
`spring-boot-starter-flyway`, `flyway-database-postgresql`	Chạy migration lúc khởi động	Hải (`a2626e6`)
`jjwt-api`, `jjwt-impl`, `jjwt-jackson` 0.12.6	Ký và xác minh JWT bằng HMAC	Cường (`5dae9ab`)
`lombok` (optional)	Sinh mã getter/setter (chưa dùng thực tế)	Cường (khởi tạo)
Các starter test: `data-jpa-test`, `security-test`, `validation-test`, `webmvc-test`, `websocket-test`	Hạ tầng kiểm thử theo từng lát cắt, MockMvc, công cụ test bảo mật	Cường (khởi tạo)
`testcontainers` `junit-jupiter`, `postgresql` 1.21.4	Dựng PostgreSQL thật trong Docker cho test tích hợp	Hải (`a2626e6`)

Điểm cần lưu ý: chỉ có JJWT và Testcontainers ghi phiên bản tường minh. Mọi thư viện khác lấy phiên bản từ `spring-boot-starter-parent` 4.1.1. Nhờ cơ chế BOM này, nâng Spring Boot lên một bản vá mới sẽ đồng loạt nâng Hibernate, Jackson, Tomcat, driver PostgreSQL lên các phiên bản đã được kiểm tra tương thích.

3.3. Cài đặt và chạy ứng dụng

3.3.1. Các bước chạy cục bộ

Các bước dưới đây theo `README.md` của kho mã; hướng dẫn chi tiết hơn kèm cách xử lý lỗi thường gặp nằm ở Phụ lục 6.

1) Sao chép `nextchat/.env.example` thành `nextchat/.env` và thay các giá trị mẫu: mật khẩu cơ sở dữ liệu và `NEXTCHAT_JWT_SECRET` dài tối thiểu 32 byte. Tệp `.env` bị `.gitignore` loại khỏi Git.

2) Từ thư mục `nextchat`, khởi động PostgreSQL: `docker compose --env-file .env up -d`.

3) Nạp biến môi trường cho phiên terminal: `set -a && . ./.env && set +a`.

4) Chạy ứng dụng: `./mvnw spring-boot:run`. Flyway áp dụng V1, V2 trên cơ sở dữ liệu trống; Hibernate chỉ kiểm tra lược đồ (`ddl-auto: validate`).

5) Mở `http://localhost:8080` để dùng client web; hoặc gọi API bằng `curl`.

Đoạn mã 3.1. *Tệp .env.example trong kho mã*

```
POSTGRES_DB=messenger_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=change-this-database-password
POSTGRES_PORT=5432
NEXTCHAT_JWT_SECRET=replace-with-a-random-secret-of-at-least-32-bytes
```

3.3.2. "Hello World" của dự án

Kho mã không có endpoint `/hello`. Điểm kiểm chứng tương đương mà nhóm dùng gồm ba điều: ứng dụng khởi động thành công, lược đồ được Flyway tạo, và một endpoint được bảo vệ trả JSON 401 khi chưa đăng nhập. Điều cuối được khẳng định tự động bởi test thứ tự 1 của `AuthAndRoomIntegrationTest` (khẳng định `status().isUnauthorized()` và `$.success == false`).

Đoạn mã 3.2. *Kiểm tra nhanh sau khi khởi động*

```
curl -i http://localhost:8080/api/v1/users/me
# Kỳ vọng: HTTP/1.1 401
# {"success":false,"message":"Authentication
required","data":null,"timestamp":"..."}
# Thông điệp do JsonAuthenticationEntryPoint sinh ra.
```

3.4. Cấu hình ứng dụng

3.4.1. Tệp application.yaml

Đoạn mã 3.3. *nextchat/src/main/resources/application.yaml (bản trên main)*

```
spring:
  application:
    name: nextchat
  profiles:
    default: dev
  config:
    import:
      - optional:file:../database.yml
      - optional:file:.env
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:${POSTGRES_PORT:5432}/$
{POSTGRES_DB:messenger_db}}
    username: ${SPRING_DATASOURCE_USERNAME:${POSTGRES_USER:postgres}}
    password: ${SPRING_DATASOURCE_PASSWORD:${POSTGRES_PASSWORD:change-this-database-password}}
  jpa:
    hibernate:
      ddl-auto: validate
app:
  security:
    jwt:
      secret: ${NEXTCHAT_JWT_SECRET:development-only-secret-change-me-before-production-12345}
      expiration: ${NEXTCHAT_JWT_EXPIRATION:PT15M}
      refresh-token-expiration: ${NEXTCHAT_JWT_REFRESH_EXPIRATION:P7D}
  seed:
    password: ${NEXTCHAT_SEED_PASSWORD:ChangeMe123!}
```

Các điểm đáng chú ý của tệp cấu hình:

- `spring.profiles.default: dev`: khi không đặt profile, ứng dụng chạy profile `dev`, nhờ đó `SeedDataConfig` tạo sẵn hai tài khoản `alice` và `bob` để thử đăng nhập. Dùng `default` thay vì `active` cho phép môi trường khác ghi đè profile dễ dàng.
- `spring.config.import` nạp thêm `../database.yml` (cấu hình HikariCP và JPA dùng chung) và tệp `.env`. Tiền tố `optional:` giúp ứng dụng không lỗi khi tệp không tồn tại, ví dụ trong môi trường test.

- Cú pháp `${BIEN:mac_dinh}` lồng nhau: URL kết nối ưu tiên `SPRING_DATASOURCE_URL`, nếu không có thì ghép từ `POSTGRES_PORT` và `POSTGRES_DB`, cuối cùng mới dùng giá trị mặc định.
- `baseline-on-migrate: true` cho phép một cơ sở dữ liệu đã có bảng (tạo thủ công trước khi có Flyway) được đánh dấu là phiên bản 1 rồi nhận các migration sau. Bộ test đặt lại `false` để luôn chạy từ cơ sở dữ liệu trống.
- Thời hạn token dùng định dạng ISO-8601 của `java.time.Duration`: `PT15M` là 15 phút, `P7D` là 7 ngày. Spring tự chuyển chuỗi thành `Duration` khi inject bằng `@Value`.

3.4.2. Tệp database.yml và docker-compose.yml

Đoạn mã 3.4. *database.yml ở thư mục gốc kho mã (trích)*

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      idle-timeout: 30000
      connection-timeout: 20000
      max-lifetime: 1800000
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
```

Thiết lập `open-in-view: false` tắt mẫu "Open Session in View": phiên Hibernate không còn mở tới khi trả phản hồi HTTP. Hệ quả là mọi quan hệ LAZY phải được nạp trong phạm vi giao dịch của service; đây là lý do `RoomService.getUserRooms` dựng DTO ngay bên trong phương thức `@Transactional(readOnly = true)` (mục 4.6). Nhóm chọn tắt để tránh truy vấn ngầm phát sinh ở tầng controller và khiến lỗi nạp dữ liệu lộ ra sớm trong test.

Đoạn mã 3.5. *nextchat/docker-compose.yml (bản trên main sau commit 4699986)*

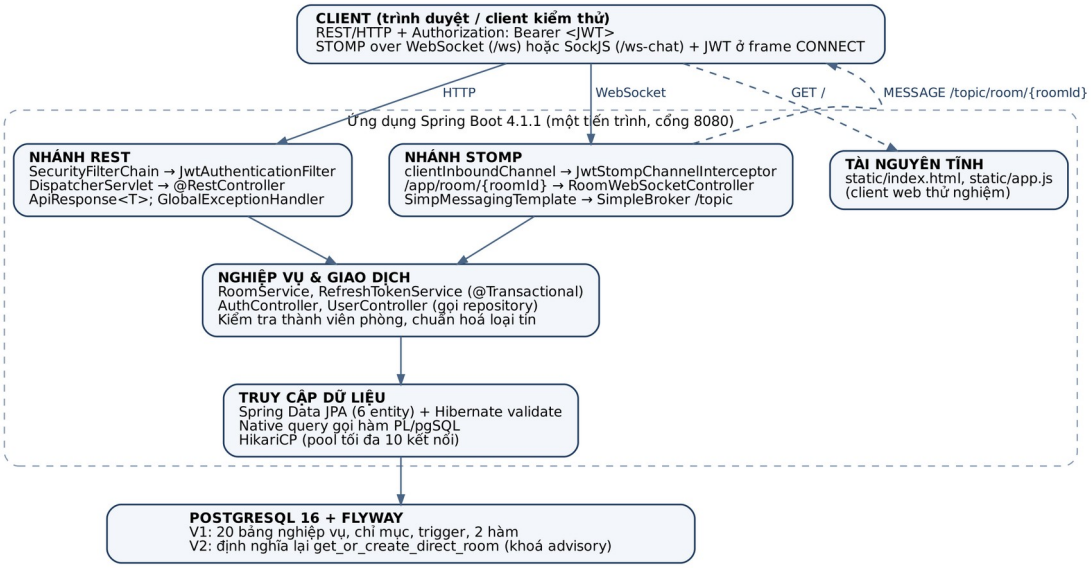
```
services:
  postgres:
    image: postgres:16-alpine
    container_name: nextchat-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-nextchat}
```

```
      POSTGRES_USER: ${POSTGRES_USER:-nextchat}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-nextchat123}
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-nextchat} -d ${POSTGRES_DB:-
nextchat}"]
      interval: 10s
      timeout: 5s
      retries: 5
volumes:
  postgres_data:
```

Lưu ý về tính nhất quán: giá trị mặc định trong Compose (`nextchat` / `nextchat` / `nextchat123`) khác giá trị mặc định trong `application.yaml` và `.env.example` (`messenger_db` / `postgres`). Khi chạy đúng hướng dẫn với tệp `.env`, cả hai bên cùng đọc một bộ giá trị nên không có vấn đề; nhưng nếu bỏ qua `.env`, ứng dụng sẽ không kết nối được cơ sở dữ liệu. Điểm này được ghi ở danh sách hạn chế (mục 9.6) và trong phần việc của Nguyễn Phú Tài (mục 10.5).

3.5. Kiến trúc tổng thể

Hình 3.1 mô tả kiến trúc của NextChat trên nhánh `main` tại commit `54ab54e`. Toàn bộ hệ thống chạy trong một tiến trình Spring Boot, phục vụ ba loại lưu lượng trên cùng cổng 8080: request REST, kết nối WebSocket/SockJS và tài nguyên tĩnh của client web.



Hình 3.1. *Kiến trúc và luồng request của NextChat trên main (tổng hợp từ mã nguồn commit 54ab54e)*

- Nhánh REST: request đi qua `SecurityFilterChain`, trong đó `JwtAuthenticationFilter` xác thực Bearer token; sau đó `DispatcherServlet` định tuyến tới `@RestController`. Phản hồi thành công được bọc trong `ApiResponse<T>`; lỗi thuộc luồng MVC do `GlobalExceptionHandler` xử lý, lỗi ở tầng Security do `JsonAuthenticationEntryPoint` và `JsonAccessDeniedHandler` xử lý.
- Nhánh STOMP: khung từ client vào `clientInboundChannel`, đi qua `JwtStompChannelInterceptor`; khung có đích bắt đầu bằng `/app` được định tuyến tới `RoomWebSocketController`, khung có đích `/topic` đi tới simple broker. Controller gọi `RoomService` để lưu tin, rồi dùng `SimpMessagingTemplate` phát `MessageDTO` tới `/topic/room/{roomId}`.
- Tầng nghiệp vụ: `RoomService` và `RefreshTokenService` có `@Transactional`; `AuthController` và `UserController` hiện gọi repository trực tiếp.

- Tầng dữ liệu: sáu entity JPA (`User`, `Role`, `RefreshToken`, `Conversation`, `ConversationParticipant`, `Message`), các repository Spring Data, và hai hàm PL/pgSQL gọi qua native query.

3.6. Tổ chức mã nguồn theo tính năng

Commit `67fb244` ngày 25/09/2026 của Lê Quốc Cường tái cấu trúc dự án từ cách chia theo tầng kỹ thuật (`config`, `repository`, `security`) sang cách chia theo tính năng kèm một gói dùng chung (package-by-feature + common). Mỗi tính năng tự chứa controller, service, repository, entity và DTO của nó.

Đoạn mã 3.6. *Cây thư mục mã nguồn chính (rút gọn)*

```
nextchat/src/main/java/com/quoocscuongwf/nextchat/
├── NextchatApplication.java
├── auth/             AuthController, RefreshToken, RefreshTokenRepository,
│   └── dto/          RefreshTokenService, JwtResponse, LoginRequest, ...
├── user/             User, Role, UserRepository, RoleRepository, UserController
│   └── dto/          UserDTO, UserProfileDTO, UserProfileResponse
├── room/             Conversation, ConversationParticipant, Message,
│   │                 ConversationRepository, ConversationParticipantRepository,
│   │                 MessageRepository, RoomController, RoomService,
│   │                 RoomWebSocketController
│   └── dto/          CreateGroupRoomRequest, DirectRoomRequest, MessageDTO,
│                     ParticipantDTO, RoomDTO, SendMessageRequest
└── common/
    ├── config/       WebSecurityConfig, WebSocketConfig, PasswordConfig,
SeedDataConfig
    ├── security/     JwtService, JwtAuthenticationFilter,
DatabaseUserDetailsService,
    │                 JsonAuthenticationEntryPoint, JsonAccessDeniedHandler,
    │                 JwtStompChannelInterceptor
    ├── exception/    GlobalExceptionHandler
    └── dto/          ApiResponse
nextchat/src/main/resources/
├── application.yaml
├── db/migration/     V1__messenger_schema.sql, V2__lock_direct_room_creation.sql
└── static/           index.html, app.js
nextchat/src/test/java/com/quoocscuongwf/nextchat/
├── api/              AuthAndRoomIntegrationTest (18 test)
└── security/         JwtStompChannelInterceptorTest (5 test)
```

Bảng 3.3. *Trách nhiệm của từng package*

Package	Thành phần chính	Trách nhiệm
`auth`	`AuthController`, `RefreshTokenService`, `RefreshToken` (+ repository), `dto/`	Đăng ký, đăng nhập, làm mới token xoay vòng.
`user`	`User`, `Role`, `UserRepository`, `RoleRepository`, `UserController`, `dto/`	Hồ sơ người dùng hiện tại, tìm kiếm người dùng, vai trò.

Package	Thành phần chính	Trách nhiệm
`room`	`Conversation`, `ConversationParticipant`, `Message`, các repository, `RoomController`, `RoomService`, `RoomWebSocketController`, `dto/`	REST phòng và lịch sử; nhận khung SEND, gọi service lưu tin rồi phát tới topic.
`common/security`	`JwtService`, `JwtAuthenticationFilter`, `DatabaseUserDetailsService`, `JsonAuthenticationEntryPoint`, `JsonAccessDeniedHandler`, `JwtStompChannelInterceptor`	Xác thực JWT trên HTTP; xác thực CONNECT, khôi phục danh tính và kiểm tra SUBSCRIBE trên STOMP.
`common/config`	`WebSecurityConfig`, `PasswordConfig`, `SeedDataConfig`, `WebSocketConfig`	Bean bảo mật, mã hoá mật khẩu, dữ liệu mẫu theo profile, endpoint và broker STOMP.
`common/exception`, `common/dto`	`GlobalExceptionHandler`, `ApiResponse`	Định dạng phản hồi và lỗi thống nhất cho REST.

Ưu điểm của cách tổ chức theo tính năng là khi làm một chức năng, thành viên chỉ cần mở một thư mục; ranh giới giữa các phần việc cũng rõ, thuận lợi cho chia việc (Hải làm chủ yếu trong `auth`, `user`, `room`; Cường làm `common/config`, `common/security` và phần WebSocket của `room`). Nhược điểm là một số lớp có tính chất dùng chung nhưng nằm trong một tính năng, ví dụ `UserRepository` được `auth`, `room` và `common/security` cùng sử dụng.

3.7. Kiểm chứng môi trường

Lần kiểm chứng ngày 28/10/2026 của nhóm cho thấy Maven tạo PostgreSQL 16.15 bằng Testcontainers, chạy Flyway và khởi tạo Hibernate với `ddl-auto: validate` thành công. Các khẳng định cụ thể:

- Lược đồ V1 gồm 20 bảng nghiệp vụ (không tính `flyway_schema_history`); Flyway áp dụng V1 và V2 trong ApplicationContext của test.
- Hai hàm nghiệp vụ `get_room_messages(bigint, integer, integer)` và `get_or_create_direct_room(bigint, bigint)` tồn tại sau migration – được test #1 khẳng định bằng `to_regprocedure`.

- Hibernate tạo được `EntityManagerFactory` ở chế độ validate, tức sáu entity khớp với bảng tương ứng.
- Chỉ mục `idx_messages_conversation_created` trên `messages(conversation_id, created_at)`, chỉ mục `idx_messages_sender`; ràng buộc `uq_conversation_user` UNIQUE (`conversation_id`, `user_id`).

Các minh chứng gốc gồm Surefire XML, log Maven và JSON của kịch bản đầu-cuối được lưu trong thư mục minh chứng của nhóm [10]. Những minh chứng này xác nhận chức năng ở các trường hợp đã nêu, không xác nhận độ bao phủ mọi nhánh hay hiệu năng dưới tải.

3.8. Nhận xét về kiến trúc hiện tại

- Điểm mạnh: ranh giới REST và STOMP tách bạch nhưng dùng chung tầng nghiệp vụ; danh tính người gửi luôn lấy từ Principal ở phía máy chủ, không tin `senderId` do client gửi; mọi thay đổi lược đồ đi qua Flyway.
- Điểm cần cải thiện: `AuthController` và `UserController` gọi repository trực tiếp, chưa có `AuthService`/`UserService`; các đường dẫn ngoài `/api/**` được `permitAll`, nên bắt tay WebSocket không được bảo vệ ở tầng HTTP – xác thực thực sự diễn ra ở khung CONNECT; chưa có tài liệu OpenAPI.
- Rủi ro trên kênh STOMP của bản `main` (đích wildcard, gửi thẳng vào broker) được phân tích chi tiết ở mục 4.10.5 và 9.6 kèm đề xuất bản vá.

CHƯƠNG 4. BẢN CHẤT CÁC CƠ CHẾ LÕI CỦA SPRING BOOT

Mỗi mục trong chương đi theo chuỗi: cơ chế → mã nguồn → kết quả kiểm chứng → ý nghĩa trong đồ án. Bảng 4.1 tóm tắt chín cơ chế lõi và kênh WebSocket/STOMP, kèm vị trí sử dụng thật trong mã nguồn. Trừ khi ghi chú khác, mã nguồn trích dẫn là bản trên nhánh `main` tại commit `54ab54e`; kết quả kiểm chứng có ghi ngày chạy và phiên bản mã tương ứng.

Bảng 4.1. *Các cơ chế Spring Boot được sử dụng trong đồ án*

TT	Cơ chế	Bản chất cần giải thích	Minh chứng	Vị trí và ý nghĩa
1	Starter và auto-configuration	Starter gom phụ thuộc; lớp AutoConfiguration tạo bean khi điều kiện đúng	Test bean `dataSource` là HikariDataSource, báo cáo điều kiện khớp `DataSourceConfiguration$Hikari` (28/10)	Hạ tầng DB, JPA, Flyway, MVC, WebSocket có sẵn nhờ cấu hình
2	IoC container và DI	Container tạo, nối, quản lý vòng đời bean	`RoomService` singleton và là proxy AOP; `SeedDataConfig` vắng mặt ở profile test	Chuỗi bean bảo mật, bean theo profile
3	Cấu hình ngoài và profile	Nhiều nguồn thuộc tính, `${ENV:default}`, `@Profile`	`application.yaml`, `database.yml`, kiểm tra khoá trong `JwtService`	Khoá JWT không ghi cứng; chặn khoá mặc định ngoài dev/test
4	Spring MVC, Validation, xử lý lỗi	`DispatcherServlet`, `@Valid`, `@RestControllerAdvice`	Test 201/400/401/403/404/409	Hợp đồng lỗi REST thống nhất
5	Spring Security và JWT	`SecurityFilterChain`, `SecurityContext`, `AuthenticationProvider`	Test #1 (401), #16 (403)	Bảo vệ `/api/**`, không dùng session
6	Spring Data JPA	Repository sinh từ interface, JPQL, `@EntityGraph`	`UserRepository`, `ConversationRepository`	Truy cập người dùng, phòng, thành viên
7	`@Transactional`	Proxy AOP, commit/rollback, `readOnly`	Test rollback qua proxy (28/10)	Xoay token nguyên tử; lưu tin và cập nhật tin cuối
8	Flyway và `ddl-auto: validate`	Migration có phiên bản; Hibernate đối chiếu lược đồ	Khởi tạo `EntityManagerFactory` thành công	Lược đồ kiểm soát bằng mã
9	Kiểm thử tích hợp	ApplicationContext thật, MockMvc, Testcontainers	18 test tích hợp + 5 test đơn vị trên `main`	Bằng chứng tự động cho API và PL/pgSQL

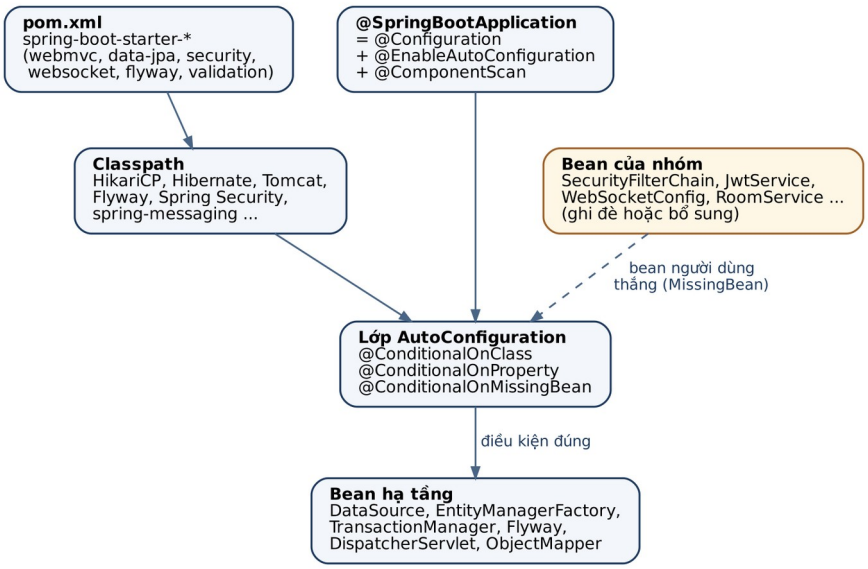
TT	Cơ chế	Bản chất cần giải thích	Minh chứng	Vị trí và ý nghĩa
10	WebSocket/ STOMP	Endpoint, kênh inbound, interceptor, broker	2 client thật, 15 smoke (28/10, bản sửa cục bộ)	Gửi nhận tin theo phòng

4.1. Starter và auto-configuration

4.1.1. Cơ chế

Chú thích `@SpringBootApplication` trên lớp `NextchatApplication` là chú thích tổng hợp của ba chú thích: `@Configuration` (lớp là nguồn định nghĩa bean), `@ComponentScan` (quét các lớp trong gói `com.quoocscuongwf.nextchat` và gói con) và `@EnableAutoConfiguration` (bật cơ chế tự cấu hình). Khi `SpringApplication.run` chạy, Spring Boot đọc danh sách các lớp auto-configuration được khai báo trong tệp `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` của từng mô-đun trên classpath. Mỗi lớp chỉ đóng góp bean khi các điều kiện của nó đúng [2]:

- `@ConditionalOnClass`: một lớp nhất định có trên classpath (ví dụ `HikariDataSource`, `Flyway`, `DispatcherServlet`).
- `@ConditionalOnProperty`: một thuộc tính cấu hình có giá trị phù hợp (ví dụ `spring.flyway.enabled`).
- `@ConditionalOnMissingBean`: người dùng chưa tự định nghĩa bean cùng kiểu. Đây là quy tắc "bean của người dùng thắng".



Hình 4.1. *Cơ chế auto-configuration: starter đưa thư viện vào classpath, điều kiện quyết định bean hạ tầng nào được tạo*

Đoạn mã 4.1. *Lớp khởi động NextchatApplication*

```
@SpringBootApplication
@EnableSpringDataWebSupport(pageSerializationMode =
        EnableSpringDataWebSupport.PageSerializationMode.VIA_DTO)
public class NextchatApplication {
    public static void main(String[] args) {
        SpringApplication.run(NextchatApplication.class, args);
    }
}
```

Trong NextChat, chỉ với các starter ở Bảng 3.2 và các thuộc tính `spring.datasource.*`, Spring Boot tạo sẵn: `HikariDataSource` (pool kết nối), `LocalContainerEntityManagerFactoryBean` và `JpaTransactionManager` (JPA và giao dịch), `Flyway` cùng `FlywayMigrationInitializer` (chạy migration trước khi JPA khởi tạo), `DispatcherServlet` và Tomcat nhúng (MVC), `ObjectMapper` (JSON), hạ tầng message broker cho STOMP khi có `@EnableWebSocketMessageBroker`. Không có dòng mã nào của nhóm khai báo các bean này.

Ngược lại, ở những chỗ nhóm cần hành vi riêng, nhóm tự khai báo bean và auto-configuration lùi lại: `SecurityFilterChain` trong `WebSecurityConfig` thay cho chuỗi filter mặc định (vốn yêu cầu đăng nhập form cho mọi đường dẫn);

`PasswordEncoder` là `BCryptPasswordEncoder` trong `PasswordConfig`; `AuthenticationManager` được tạo tường minh từ `DaoAuthenticationProvider`.

4.1.2. Thực hành trên ApplicationContext thật

Để không chỉ suy luận từ việc "không thấy `@Bean` trong mã", ngày 28/10/2026 nhóm bổ sung lớp `SpringMechanismIntegrationTest` trong bản sửa cục bộ. Test lấy bean `dataSource` hai lần, kiểm tra phạm vi singleton, truy cập `ConditionEvaluationReport` – báo cáo mà Spring Boot ghi lại cho từng điều kiện – và khẳng định điều kiện của cấu hình Hikari đã khớp hoàn toàn.

Đoạn mã 4.2. *Trích SpringMechanismIntegrationTest (bản kiểm chứng 28/10)*

```
Object ds = context.getBean("dataSource");
assertThat(ds).isSameAs(context.getBean("dataSource"));
assertThat(context.getBeanFactory().isSingleton("dataSource")).isTrue();
var report = ConditionEvaluationReport.get(context.getBeanFactory());
// Lọc nguồn DataSourceConfiguration$Hikari và assert isFullMatch().
```

Kết quả trong `maven-review-verify.log` ngày 28/10: bean `dataSource` là `com.zaxxer.hikari.HikariDataSource`; lấy hai lần nhận cùng một đối tượng; điều kiện `org.springframework.boot.jdbc.autoconfigure.DataSourceConfiguration$Hikari` khớp. Khi chạy ứng dụng bình thường, có thể xem cùng báo cáo này bằng cờ `--debug` (Spring Boot in mục "CONDITIONS EVALUATION REPORT" với hai danh sách "Positive matches" và "Negative matches").

4.1.3. Ý nghĩa

Starter kết hợp cấu hình kết nối cho phép Spring Boot dựng toàn bộ hạ tầng dữ liệu của dự án. Cần phân biệt: auto-configuration là đóng góp riêng của Spring Boot, còn IoC, MVC, JPA và giao dịch là các mô-đun của Spring Framework – chúng vẫn dùng được khi cấu hình Spring thủ công không qua Boot. Lớp test này cũng là ví dụ cho việc Spring Boot 4 đổi gói theo mô-đun: điều kiện Hikari nằm

trong gói `org.springframework.boot.jdbc.autoconfigure`, không còn ở gói `org.springframework.boot.autoconfigure.jdbc` như Boot 3.

4.2. IoC container và Dependency Injection

4.2.1. Cơ chế

Spring tạo các đối tượng (bean) từ những lớp có `@Component` và các chú thích chuyên biệt của nó (`@Service`, `@Controller`, `@RestController`, `@Configuration`), cùng các phương thức `@Bean`. Container phân tích phụ thuộc của từng bean – trong NextChat luôn là tham số constructor – rồi tạo bean theo thứ tự phụ thuộc và truyền vào. Mặc định mỗi bean là singleton trong phạm vi một ApplicationContext [11]. Nếu một bean cần được bao bởi khía cạnh (ví dụ giao dịch), container tạo proxy và chính proxy là thứ được inject vào nơi khác.

4.2.2. Bean trong mã nguồn

Bảng 4.2. *Các bean do nhóm định nghĩa và phụ thuộc của chúng*

Bean	Cách khai báo	Phụ thuộc nhận qua constructor / tham số
`JwtService`	`@Service`	Chuỗi khoá, thời hạn (`@Value`), `Environment`
`DatabaseUserDetailsService`	`@Service`	`UserRepository`
`JwtAuthenticationFilter`	`@Component`	`JwtService`, `UserDetailsService`
`JsonAuthenticationEntryPoint`, `JsonAccessDeniedHandler`	`@Component`	`ObjectMapper` (do auto-configuration tạo)
`JwtStompChannelInterceptor`	`@Component`	`JwtService`, `UserDetailsService`, `UserRepository`, `ConversationParticipantRepository`
`passwordEncoder`	`@Bean` trong `PasswordConfig`	–
`authenticationProvider`, `authenticationManager`, `securityFilterChain`	`@Bean` trong `WebSecurityConfig`	`UserDetailsService`, `PasswordEncoder`, filter, entry point, handler
`WebSocketConfig`	`@Configuration` + `@EnableWebSocketMessageBroker`	`JwtStompChannelInterceptor`, danh sách origin (`@Value`)

Bean	Cách khai báo	Phụ thuộc nhận qua constructor / tham số
`seedUsers` (`CommandLineRunner`)	`@Bean` trong `SeedDataConfig` với `@Profile("dev")`	`UserRepository`, `RoleRepository`, `PasswordEncoder`, mật khẩu mẫu
`RefreshTokenService`	`@Service`	`RefreshTokenRepository`, thời hạn refresh token
`RoomService`	`@Service`	4 repository; `EntityManager` qua `@PersistenceContext`
`AuthController`, `UserController`, `RoomController`	`@RestController`	Service, repository, `AuthenticationManager`, `JwtService`
`RoomWebSocketController`	`@Controller`	`RoomService`, `UserRepository`, `SimpMessagingTemplate`

Toàn bộ dự án dùng tiêm qua constructor, không dùng `@Autowired` trên trường trong mã chính. Cách này giúp phụ thuộc hiện rõ trong chữ ký constructor, cho phép khai báo trường `final`, và cho phép unit test khởi tạo lớp bằng `new` với mock – đúng như `JwtStompChannelInterceptorTest` khởi tạo interceptor bằng `new` với bốn mock. Trường hợp duy nhất khác là `EntityManager` trong `RoomService` được inject bằng `@PersistenceContext`, chú thích chuẩn của JPA để nhận một proxy `EntityManager` gắn với giao dịch hiện hành.

Một ví dụ về "chuỗi phụ thuộc" mà container tự giải: để tạo `securityFilterChain`, container cần `JwtAuthenticationFilter`; filter cần `JwtService` và `UserDetailsService`; `UserDetailsService` (tức `DatabaseUserDetailsService`) cần `UserRepository`; `UserRepository` là proxy do Spring Data sinh, cần `EntityManagerFactory`; `EntityManagerFactory` cần `DataSource` – bean do auto-configuration tạo ở mục 4.1. Nhóm chỉ viết từng mắt xích; container tìm ra thứ tự khởi tạo đúng.

4.2.3. Kiểm chứng phạm vi và profile

Bài test ở mục 4.1.2 còn lấy `RoomService` hai lần và khẳng định cùng một đối tượng, đồng thời `AopUtils.isAopProxy(roomService)` trả về `true` vì lớp có phương thức `@Transactional`; và khẳng định không có bean `SeedDataConfig` khi profile `test` hoạt động. Dòng log minh chứng: `EVIDENCE beans:`

dataSource=com.zaxxer.hikari.HikariDataSource; singleton=true; RoomService singleton/proxy=true; SeedDataConfig(test)=absent. Đây là kiểm chứng phạm vi và profile của bean thật, chưa phải khảo sát đầy đủ mọi callback vòng đời. Trong mã hiện tại không có tình huống hai bean cùng kiểu cần @Primary hoặc @Qualifier.

4.3. Cấu hình ngoài mã nguồn và profile

4.3.1. Cơ chế

Spring gộp thuộc tính từ nhiều nguồn theo thứ tự ưu tiên: thuộc tính của test (@DynamicPropertySource), tham số dòng lệnh, biến môi trường hệ điều hành, các tệp được nhập qua spring.config.import, rồi application.yaml. Nguồn ưu tiên cao ghi đè nguồn thấp. Cú pháp ${BIEN:mac_dinh} lấy giá trị thuộc tính hoặc biến môi trường BIEN nếu có, nếu không thì dùng giá trị mặc định. Profile cho phép đổi tập bean và tập thuộc tính theo môi trường: bean có @Profile("dev") chỉ được tạo khi profile dev hoạt động.

4.3.2. Mã nguồn: chặn khoá phát triển ở môi trường thật

application.yaml dùng ${NEXTCHAT_JWT_SECRET:development-only-secret-...}. Để giá trị mặc định này không bao giờ lọt vào môi trường thật, JwtService kiểm tra ngay trong constructor – tức là ngay lúc container tạo bean.

Đoạn mã 4.3. *Kiểm tra khoá JWT trong constructor của JwtService*

```
public JwtService(
        @Value("${app.security.jwt.secret}") String secret,
        @Value("${app.security.jwt.expiration:PT15M}") Duration expiration,
        Environment environment) {
    if (secret.getBytes(StandardCharsets.UTF_8).length < 32) {
        throw new IllegalArgumentException("JWT secret must be at least 32 bytes");
    }
    if (DEVELOPMENT_SECRET.equals(secret)
            && !environment.acceptsProfiles(Profiles.of("dev", "test"))) {
        throw new IllegalStateException(
            "NEXTCHAT_JWT_SECRET must be configured outside dev and test profiles");
    }
    this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    this.expiration = expiration;
}
```

4.3.3. Kết quả theo mã nguồn

- Profile `dev` hoặc `test` (kể cả profile mặc định khi không đặt gì, vì `spring.profiles.default: dev`): chấp nhận khoá mặc định để thuận tiện phát triển.
- Profile khác (ví dụ `prod`) mà chưa đặt `NEXTCHAT_JWT_SECRET`: bean `JwtService` không tạo được, ApplicationContext thất bại và ứng dụng dừng ngay khi khởi động – lỗi lộ ra sớm thay vì chạy với khoá yếu.
- Khoá ngắn hơn 32 byte bị từ chối ở mọi profile, vì thuật toán HMAC-SHA256 cần khoá tối thiểu 256 bit.

Bộ test tích hợp ghi đè URL cơ sở dữ liệu, tài khoản và khoá JWT bằng `@DynamicPropertySource`, cho thấy cùng một mã chạy được với cấu hình khác hoàn toàn. Bí mật và tham số môi trường không nằm trong Git: `.env` bị `.gitignore` loại; commit `87fe186` của Lê Quốc Cường ngày 02/10 đã gỡ tệp `.env` từng bị theo dõi nhầm khỏi kho.

4.4. Spring MVC REST, kiểm tra dữ liệu và xử lý lỗi tập trung

4.4.1. Cơ chế

`DispatcherServlet` nhận request, tìm phương thức controller khớp với đường dẫn và phương thức HTTP, dùng Jackson chuyển JSON thành đối tượng, áp dụng Bean Validation khi tham số có `@Valid`, gọi phương thức rồi chuyển giá trị trả về thành JSON. `@RestControllerAdvice` định nghĩa các phương thức bắt ngoại lệ dùng chung cho mọi controller trong luồng MVC. Lỗi phát sinh trong filter bảo mật (trước khi tới `DispatcherServlet`) và lỗi trong luồng STOMP không đi qua advice này, nên cần cơ chế riêng.

4.4.2. Mã nguồn

`AuthController.register(@Valid @RequestBody RegisterRequest request)` nhận một record có ràng buộc: `username` 3–50 ký tự, `email` đúng định dạng và tối đa 150 ký tự, `password` 8–128 ký tự, `fullName` tối đa 150 ký tự, tất cả

`@NotBlank`. Bảng 4.3 là ánh xạ ngoại lệ sang mã HTTP trong `GlobalExceptionHandler`.

Bảng 4.3. *Ánh xạ ngoại lệ sang phản hồi HTTP trong GlobalExceptionHandler*

Ngoại lệ	HTTP	Thông điệp / ghi chú
`ResponseStatusException`	do service chọn	`ex.getReason()`; dùng cho 400, 401, 403, 404, 409 nghiệp vụ
`MethodArgumentNotValidException`	400	"Validation error", `data` là map lỗi theo trường
`AuthenticationException`	401	"Invalid credentials" – không lộ việc tài khoản có tồn tại hay không
`MethodArgumentTypeMismatchException`	400	"Invalid request parameter" (ví dụ `roomId` không phải số)
`DataIntegrityViolationException`	409	"The requested resource conflicts with existing data"
`AccessDeniedException`	403	Thông điệp của ngoại lệ
`Exception` (còn lại)	500	"Internal server error"; chi tiết chỉ ghi log phía máy chủ

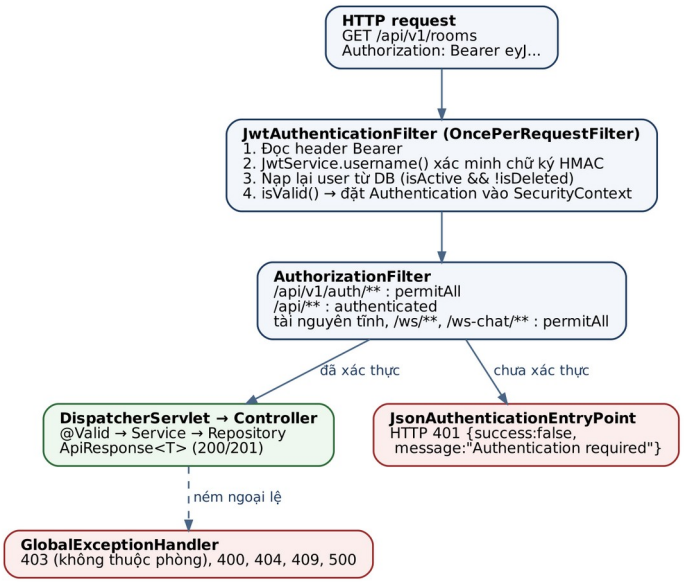
4.4.3. Kết quả kiểm thử

Các khẳng định của bộ test tích hợp: đăng ký hợp lệ trả 201 với `success=true` và đúng `username`, `email`, `fullName` (test #2); đăng ký trùng trả 409 (test #3); đăng nhập sai mật khẩu trả 401 với thông điệp đúng bằng "Invalid credentials" (test #5); tham số `size=101` khi lấy lịch sử trả 400 (test #14); tạo nhóm với thành viên không tồn tại trả 404 (test #15). Mọi endpoint REST trả về vỏ `ApiResponse<T>`; `MessageDTO` gửi trên kênh STOMP không dùng vỏ này.

4.5. Spring Security: chuỗi filter và xác thực JWT không trạng thái

4.5.1. Cơ chế

Spring Security chèn một chuỗi filter (`SecurityFilterChain`) đứng trước `DispatcherServlet`. Mỗi filter xử lý một khía cạnh; filter xác thực đặt đối tượng `Authentication` vào `SecurityContext` nếu request hợp lệ; filter ủy quyền ở cuối chuỗi quyết định cho request đi tiếp hay trả 401/403 dựa trên quy tắc `authorizeHttpRequests`. Với `SessionCreationPolicy.STATELESS`, máy chủ không tạo session HTTP; mỗi request phải tự mang token [32].



Hình 4.2. *Luồng một request REST qua chuỗi filter bảo mật của NextChat*

4.5.2. Mã nguồn

Đoạn mã 4.4. *Cấu hình chuỗi filter trong WebSecurityConfig (bản trên main)*

```
http
    .csrf(csrf -> csrf.disable())
    .authenticationProvider(authenticationProvider)
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/", "/index.html", "/app.js", "/favicon.ico",
                         "/ws/**", "/ws-chat/**", "/api/v1/auth/**").permitAll()
        .requestMatchers("/api/**").authenticated()
        .anyRequest().permitAll())
    .exceptionHandling(e -> e
        .authenticationEntryPoint(authenticationEntryPoint)
        .accessDeniedHandler(accessDeniedHandler))
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

- Tắt CSRF: chấp nhận được vì xác thực bằng header `Authorization: Bearer`, trình duyệt không tự gửi kèm như cookie phiên.
- `addFilterBefore(..., UsernamePasswordAuthenticationFilter.class)`: filter JWT chạy trước filter đăng nhập form mặc định.
- Danh sách `permitAll` tường minh cho tài nguyên tĩnh và hai endpoint bắt tay WebSocket được Nguyễn Phú Tài bổ sung trong commit `4699986`; trước đó các đường dẫn này vẫn được cho qua nhờ

`anyRequest().permitAll()`, thay đổi giúp danh sách công khai hiện rõ để rà soát.

`JwtAuthenticationFilter` (kế thừa `OncePerRequestFilter`) đọc header, gọi `JwtService.username(token)` để xác minh chữ ký HMAC và đọc chủ thể, nạp lại người dùng qua `DatabaseUserDetailsService` – lớp này chỉ chấp nhận tài khoản `isActive && !isDeleted` – rồi gọi `isValid` để kiểm tên và hạn. Token sai, hết hạn hoặc người dùng không còn hợp lệ làm phát sinh `JwtException` hoặc `UsernameNotFoundException`; ngoại lệ bị bắt và bỏ qua, request đi tiếp ở trạng thái chưa xác thực nên bị filter ủy quyền chặn. Mật khẩu được băm bằng BCrypt.

Đoạn mã 4.5. *Phần xử lý chính của JwtAuthenticationFilter*

```
String header = request.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
    String token = header.substring(7).trim();
    try {
        String username = jwtService.username(token);
        if (SecurityContextHolder.getContext().getAuthentication() == null) {
            var user = userDetailsService.loadUserByUsername(username);
            if (jwtService.isValid(token, user)) {
                var authentication = new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                authentication.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }
    } catch (JwtException | IllegalArgumentException
             | UsernameNotFoundException ignored) {
        // Quy tắc ủy quyền sẽ từ chối request mang token không hợp lệ.
    }
}
chain.doFilter(request, response);
```

4.5.3. Kết quả và ý nghĩa

Test #1 khẳng định `GET /api/v1/users/me` không token trả 401 và `success=false`; test #16 khẳng định người ngoài phòng đọc lịch sử bị 403. Từ mã nguồn suy ra thêm: khi một tài khoản bị đặt `is_active=false`, mọi access token còn hạn của tài khoản đó bị từ chối ngay ở request kế tiếp vì filter tra cơ sở dữ liệu ở mỗi request; đổi lại, mỗi request phát sinh thêm một truy vấn người dùng.

Cần phân biệt hai lớp kiểm soát: lỗi 403 khi người ngoài đọc lịch sử không đến từ Spring Security mà từ kiểm tra thành viên trong `RoomService.getRoomMessages`. Spring Security chỉ đảm bảo người gọi đã đăng nhập; quyền theo từng phòng là nghiệp vụ của service.

4.6. Spring Data JPA

4.6.1. Cơ chế

Chỉ cần khai báo interface kế thừa `JpaRepository<Entity, Id>`, Spring Data sinh lớp cài đặt lúc chạy dưới dạng proxy. Tên phương thức như `findByUsername` hay `existsByConversationIdAndUserIdAndLeftAtIsNull` được phân tích thành truy vấn; `@Query` cho phép viết JPQL hoặc SQL gốc; `@EntityGraph` chỉ định các quan hệ cần nạp ngay trong cùng truy vấn. Quan hệ `LAZY` chỉ được nạp khi truy cập lần đầu và cần phiên Hibernate còn mở.

Bảng 4.4. *Các phương thức repository tiêu biểu và truy vấn tương ứng*

Phương thức	Cách sinh truy vấn	Nơi dùng
`UserRepository.findByUsername`, `existsByUsername`, `existsByEmail`	Suy ra từ tên phương thức	Đăng ký, đăng nhập, nạp người dùng
`UserRepository.searchUsers(query)`	JPQL: `LOWER(...) LIKE LOWER(CONCAT('%', :query, '%'))` trên username, email, fullName, chỉ tài khoản hoạt động	`GET /users/search`
`ConversationParticipantRepository.existsByConversationIdAndUserIdAndLeftAtIsNull`	Suy ra từ tên; trả `boolean`	Kiểm tra thành viên ở REST, SUBSCRIBE và gửi tin
`ConversationRepository.findRoomsByUserId`	JPQL `SELECT DISTINCT c ... JOIN c.participants p` + `@EntityGraph` 5 đường quan hệ	`GET /rooms`
`ConversationRepository.findUnreadCountsByUserId`	JPQL tổng hợp `COUNT(m)` theo phòng với điều kiện `lastReadMessageId`	`GET /rooms` (số tin chưa đọc)
`ConversationRepository.callGetOrCreateDirectRoom`	SQL gốc: `SELECT get_or_create_direct_room(:userA, :userB)`	`POST /rooms/direct`
`MessageRepository.countByConversationIdAndDeletedFalse`	Suy ra từ tên	Đếm dự phòng khi trang lịch sử rỗng

4.6.2. Tránh N+1 khi lấy danh sách phòng

Danh sách phòng cần, với mỗi phòng: các thành viên và thông tin người dùng của họ, người tạo, tin nhắn cuối và người gửi tin đó. Nếu nạp lười từng quan hệ, số truy vấn tăng theo số phòng – hiện tượng N+1. `findRoomsByUserId` khai báo `@EntityGraph(attributePaths = {"participants", "participants.user", "createdBy", "lastMessage", "lastMessage.sender"})` để Hibernate nạp các quan hệ này bằng JOIN trong cùng truy vấn; số tin chưa đọc được tính bằng một truy vấn tổng hợp riêng rồi ghép theo `roomId` trong bộ nhớ. Vì `open-in-view=false`, DTO được dựng ngay trong `RoomService.getUserRooms` khi giao dịch chỉ-đọc còn mở.

Nhóm chưa đo số câu SQL thực tế của request này (ví dụ bằng log Hibernate hoặc thống kê), nên không khẳng định mọi request luôn chỉ có hai câu SQL; cần tính cả truy vấn xác thực người dùng ở filter và truy vấn lấy `userId` ở controller.

4.6.3. Khi nào chuyển sang SQL

Toàn bộ truy cập người dùng, phòng và thành viên đi qua repository. Hai thao tác nhạy cảm được chủ động chuyển sang hàm PL/pgSQL gọi bằng native query: tạo phòng 1-1 (cần khoá để nguyên tử, Chương 5) và lấy lịch sử phân trang (cần kiểm soát câu truy vấn và đo kế hoạch thực thi, Chương 6). Kết quả native query của lịch sử là `List<Object[]>`; `RoomService` ánh xạ từng cột sang `MessageDTO` và chuyển kiểu thời gian bằng `toInstant` để xử lý nhiều kiểu mà driver có thể trả về (`Timestamp`, `LocalDateTime`, `OffsetDateTime`, `Instant`).

4.7. Quản lý giao dịch với @Transactional

4.7.1. Cơ chế

Spring bọc bean có phương thức `@Transactional` bằng proxy AOP. Khi phương thức được gọi từ bên ngoài bean, proxy mở giao dịch (mặc định lan truyền `REQUIRED`: tham gia giao dịch đang có hoặc mở mới), gọi phương thức thật, rồi commit nếu kết thúc bình thường hoặc rollback nếu phát sinh `RuntimeException`

hay `Error` [7]. `readOnly = true` báo ý định chỉ đọc để tầng dưới tối ưu. Điểm quan trọng: lời gọi nội bộ trong cùng lớp (`this.method()`) không đi qua proxy nên không có giao dịch mới.

Bảng 4.5. *Các phương thức có giao dịch trong NextChat*

Phương thức	Kiểu	Lý do cần giao dịch
`RefreshTokenService.createRefreshToken`	`@Transactional`	Xoá token cũ của người dùng rồi phát token mới như một khối
`RefreshTokenService.rotateRefreshToken`	`@Transactional`	Xoá token đã dùng và chèn token thay thế; lỗi giữa chừng không được làm mất phiên
`RoomService.getUserRooms`	`readOnly = true`	Nạp quan hệ và dựng DTO trong phiên còn mở
`RoomService.getOrCreateDirectRoom`	`@Transactional`	Khoá advisory của hàm SQL gắn với giao dịch; đọc lại phòng vừa tạo
`RoomService.createGroupRoom`	`@Transactional`	Tạo phòng và mọi bản ghi thành viên cùng thành công hoặc cùng huỷ
`RoomService.sendMessage`	`@Transactional`	Lưu `Message` và cập nhật `Conversation.lastMessage` nhất quán
`RoomService.getRoomMessages`	`readOnly = true`	Kiểm thành viên, gọi hàm lịch sử và đếm dự phòng trong cùng ảnh dữ liệu

4.7.2. Minh hoạ tính nguyên tử khi xoay token

Minh hoạ dưới đây giải thích vì sao thao tác "xoá rồi chèn" cần giao dịch; đây là mô tả, không phải log của Spring.

Đoạn mã 4.6. *Minh hoạ trạng thái bảng refresh_tokens khi chèn token thay thế bị lỗi*

```
Trước:  HASH_OLD, HASH_TAKEN
A) Không giao dịch: DELETE HASH_OLD -> INSERT lỗi UniqueViolation
   Sau: HASH_TAKEN                (mất HASH_OLD: người dùng mất phiên)
B) Một giao dịch:   DELETE HASH_OLD -> INSERT lỗi -> ROLLBACK
   Sau: HASH_OLD, HASH_TAKEN      (dữ liệu nguyên vẹn)
```

4.7.3. Kiểm thử rollback qua proxy thật (28/10)

Bài test trong `SpringMechanismIntegrationTest` không đặt `@Transactional` bao ngoài (nếu có, chính test sẽ rollback mọi thứ và che mất hành vi cần kiểm). Test tạo người dùng và token cũ, đặt một ràng buộc `CHECK` tạm

thời chỉ cho phép hash cũ của người dùng đó, rồi gọi `rotateRefreshToken` qua bean được inject (khẳng định là proxy AOP). Lệnh chèn token thay thế vi phạm ràng buộc và ném `DataIntegrityViolationException`. Sau lời gọi, `JdbcTemplate` đọc cơ sở dữ liệu ngoài giao dịch thất bại: còn đúng một hàng với hash cũ, không có hàng thay thế. Gỡ ràng buộc và xoay lại thì thành công – đối chứng cho thấy lỗi đến từ ràng buộc, không phải từ dữ liệu test.

Đoạn mã 4.7. *Trích phần khẳng định của test rollback*

```
assertThat(AopUtils.isAopProxy(tokens)).isTrue();
assertThatThrownBy(() -> tokens.rotateRefreshToken(raw))
        .isInstanceOf(DataIntegrityViolationException.class);
assertThat(jdbc.queryForList(
        "SELECT token FROM refresh_tokens WHERE user_id=?",
        String.class, user.getId())).containsExactly(oldHash);
// Gỡ CHECK; rotate lại: token cũ mất, token mới tồn tại.
```

Giới hạn: test chứng minh trạng thái cơ sở dữ liệu sau lỗi trong luồng service/proxy. Thứ tự câu DELETE vật lý có thể do Hibernate trì hoãn tới lúc flush; không dùng lời gọi `repository.delete` để khẳng định DELETE đã thực thi trước INSERT. Test này cũng không chứng minh an toàn khi hai yêu cầu refresh đến đồng thời (Chương 7).

Một hệ quả ít được để ý của quy tắc rollback: trong `rotateRefreshToken`, nhánh token hết hạn gọi `delete(token)` rồi ném `ResponseStatusException` – đây là `RuntimeException`, nên thao tác xoá vừa thực hiện cũng bị rollback. Token hết hạn vì vậy không được dọn khỏi bảng; nó vẫn vô hại vì lần sau sẽ lại bị từ chối, nhưng nếu muốn dọn thật cần tách thao tác xoá sang giao dịch riêng hoặc khai báo `noRollbackFor`.

4.8. Tích hợp Flyway và ddl-auto: validate

4.8.1. Cơ chế

Spring Boot chạy Flyway lúc khởi động, trước khi Hibernate khởi tạo `EntityManagerFactory`. Flyway áp dụng các tệp `V<số>__<mô tả>.sql` theo thứ tự phiên bản, ghi mỗi lần áp dụng vào bảng `flyway_schema_history` kèm checksum; tệp đã áp dụng mà bị sửa sẽ làm checksum lệch và Flyway báo lỗi. Với

`ddl-auto: validate`, Hibernate chỉ đối chiếu entity với lược đồ và dừng khởi động nếu thiếu bảng hoặc cột.

4.8.2. Nội dung migration

V1 (`V1__messenger_schema.sql`, 404 dòng) tạo 20 bảng nghiệp vụ, chỉ mục, trigger cập nhật `updated_at` và hai hàm nghiệp vụ. Bảng 4.6 nhóm 20 bảng theo chức năng và cho biết bảng nào đã có entity JPA.

Bảng 4.6. *Hai mươi bảng trong migration V1*

Nhóm	Bảng	Có entity JPA
Người dùng và xác thực	`users`, `roles`, `user_roles`, `refresh_tokens`, `user_settings`	4/5 (trừ `user_settings`; `user_roles` là bảng nối)
Bạn bè và chặn	`friendships`, `user_blocks`	Chưa
Hội thoại	`conversations`, `conversation_participants`, `group_invite_links`	2/3
Tin nhắn	`messages`, `message_attachments`, `message_status`, `message_deletions`, `message_reactions`, `message_mentions`, `pinned_messages`	1/7
Thông báo	`notifications`	Chưa
Gọi thoại / video	`calls`, `call_participants`	Chưa

V2 (`V2__lock_direct_room_creation.sql`) định nghĩa lại hàm `get_or_create_direct_room` bằng `CREATE OR REPLACE`. Đối chiếu Git cho thấy cả V1 và V2 cùng xuất hiện ở commit `a2626e6` của Huỳnh Thanh Hải ngày 01/10/2026, và V1 tại commit đó đã có khoá advisory; không có bằng chứng test #17 từng thất bại ở một phiên bản không khoá. Vì vậy V2 không được trình bày như bản vá sau một lỗi đã quan sát. Lịch sử của V1 cũng cho thấy sự phối hợp giữa hai thành viên: lược đồ 20 bảng ban đầu do Lê Quốc Cường viết trong tệp `sql/messenger_schema_postgres.sql` (commit `5dae9ab`, 292 dòng); Huỳnh Thanh Hải chuyển tệp này thành migration Flyway và bổ sung hai hàm PL/pgSQL (thêm 112 dòng).

4.8.3. Đối chiếu lược đồ và kết quả

Tên cột được đọc từ sáu entity và migration để đối chiếu tĩnh: `refresh_tokens` 5 cột, `conversations` 8 cột, `messages` 12 cột, `roles` 2 cột, `users` 14 cột, `conversation_participants` 11 cột – không thiếu cột nào trong cơ sở dữ liệu. Bằng chứng động là ApplicationContext khởi tạo thành công trong lần Maven verify, tức Hibernate validate không báo lỗi. Những bảng mở rộng chưa có entity cần kiểm tra riêng khi phát triển chức năng tương ứng.

4.9. Kiểm thử tích hợp với Spring Boot Test và Testcontainers

4.9.1. Cơ chế

`@SpringBootTest` nạp toàn bộ ApplicationContext thật như khi chạy ứng dụng. `@AutoConfigureMockMvc` cung cấp `MockMvc` để gọi controller mà không mở cổng mạng nhưng vẫn đi qua chuỗi filter bảo mật. `@Testcontainers` cùng `@Container` dựng `PostgreSQLContainer("postgres:16-alpine")` trong Docker; `@DynamicPropertySource` trỏ ứng dụng tới container đó. `@TestMethodOrder(OrderAnnotation)` cho phép các test chạy theo thứ tự và dùng chung dữ liệu (đăng ký → đăng nhập → tạo phòng → gửi tin).

4.9.2. Bộ kiểm thử trên nhánh main

Bảng 4.7. *Các kịch bản trong AuthAndRoomIntegrationTest*

Thứ tự	Nội dung kiểm tra (theo khẳng định trong mã test)
1	Hai hàm PL/pgSQL tồn tại sau migration; `/users/me` không token → 401
2, 3, 4	Đăng ký hợp lệ (201); đăng ký trùng (409); đăng ký người dùng thứ hai
5, 6, 7	Đăng nhập sai (401, "Invalid credentials"); đăng nhập hai người dùng, nhận access token
8	Refresh: nhận cặp token mới, refresh token mới khác token cũ; dùng lại token cũ → 401
9, 10	Hồ sơ người dùng hiện tại (id, username, email, roles); tìm kiếm người dùng
11, 12, 13	Phòng 1-1 (gọi hai lần trả cùng id); phòng nhóm (201, `type=GROUP`); danh sách phòng (≥ 2 phòng)
14	Lịch sử qua PL/pgSQL: tổng 2, trang 1 rỗng nhưng `totalElements` vẫn 2, `size=101` → 400
15	Thành viên nhóm không tồn tại bị từ chối (404)
16	Người không thuộc phòng đọc lịch sử → 403
17	Hai luồng tạo phòng 1-1 cùng lúc cho một cặp → cùng một id phòng

Thứ tự	Nội dung kiểm tra (theo khẳng định trong mã test)
18	`testWebSocketMessageIsPersistedAndUpdatesLastMessage`: gọi `RoomService.sendMessage` trực tiếp, kiểm `Message` và `lastMessage` trong DB. Tên có "WebSocket" nhưng test không mở socket, không có khung STOMP.

Ngoài 18 bài trên, `JwtStompChannelInterceptorTest` trên `main` có 5 bài kiểm thử đơn vị dùng Mockito: CONNECT với Bearer token hợp lệ đặt Principal; CONNECT thiếu token bị từ chối; SUBSCRIBE phòng khi là thành viên được chấp nhận; SUBSCRIBE khi không là thành viên bị từ chối; và bài mới trong commit `1046984` – khung tiếp theo khôi phục danh tính từ thuộc tính phiên `stompUsername`. Tổng cộng 23 bài kiểm thử trên `main`.

Đoạn mã 4.8. *Bài test mới của commit 1046984*

```
@Test
void subsequentFrameRestoresAuthenticationFromSession() {
    StompHeaderAccessor accessor = StompHeaderAccessor.create(StompCommand.SEND);
    accessor.setSessionAttributes(new java.util.HashMap<>());
    accessor.getSessionAttributes().put("stompUsername", "alice");
    Message<?> result = interceptor.preSend(message(accessor), mock());
    assertThat(StompHeaderAccessor.wrap(result).getUser()).isNotNull();
    assertThat(StompHeaderAccessor.wrap(result).getUser().getName()).isEqualTo("alice");
}
```

4.9.3. Phân biệt các mức kiểm thử và hai phiên bản mã

Kết quả 28/28 ngày 28/10 được chạy trên bản sửa cục bộ (dựa trên `39b4d75`, chưa push), không phải trên `main` hiện tại. Bảng 4.8 so sánh hai bộ kiểm thử để tránh nhầm lẫn khi bảo vệ.

Bảng 4.8. *Bộ kiểm thử của bản sửa cục bộ 28/10 và của nhánh main 06/11*

Lớp / kịch bản	28/10 (cục bộ)	main (`54ab54e`)	Ghi chú
`AuthAndRoomIntegrationTest`	18	18	Spring + MockMvc + PostgreSQL; #17–18 gọi service; bản 28/10 có thêm dữ liệu xoá mềm ở #14
`JwtStompChannelInterceptorTest`	7	5	Bản cục bộ có 3 test bổ sung (giữ callback Principal, chặn wildcard, chặn SEND vào broker); `main` có 1 test mới khôi phục danh tính từ phiên
`SpringMechanismIntegrationTest`	2	–	Bean/điều kiện và rollback qua proxy; chưa có trên `main`

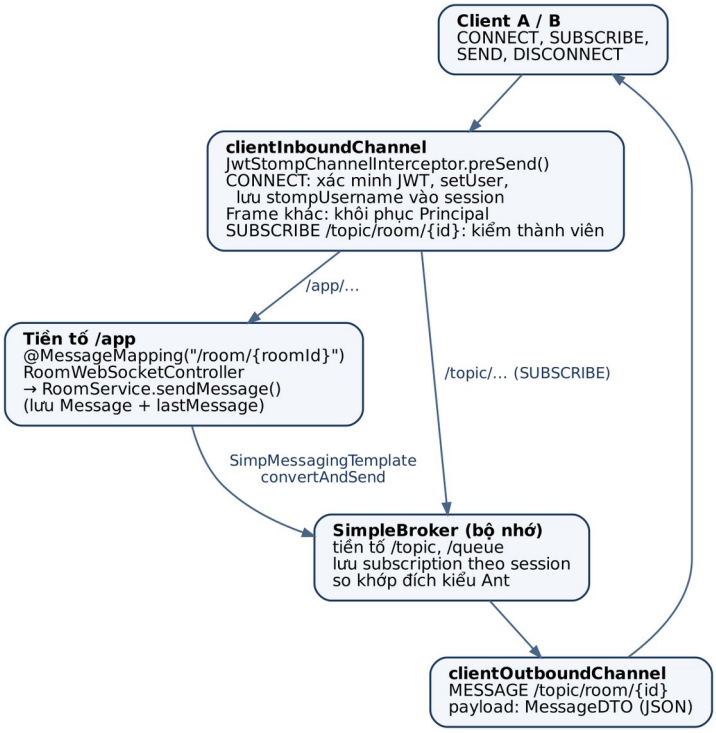
Lớp / kịch bản	28/10 (cục bộ)	main (54ab54e)	Ghi chú
`RoomServiceUnitTest`	1	–	Mockito: người ngoài gửi tin → 403, không ghi repository; chưa có trên `main`
Tổng JUnit	28	23	`main` chưa được chạy lại trong quá trình viết báo cáo (mục 9.5)
`tools/e2e_check.py` (smoke)	15 kiểm tra	–	Jar + HTTP + WebSocket thật + đọc DB; script dùng đích dạng dấu chấm của bản cục bộ, cần cập nhật sang `/room/{id}`

Mỗi khẳng định chứng minh một hành vi cụ thể. Số bài test đạt không thay cho tỷ lệ bao phủ dòng/nhánh, đánh giá bảo mật đầy đủ hay khả năng chịu tải.

4.10. WebSocket/STOMP và kiểm soát truy cập trên kênh thông điệp

4.10.1. Cơ chế luồng thông điệp

Khi bật `@EnableWebSocketMessageBroker`, ứng dụng Spring trở thành một STOMP broker đối với client. Khung từ client được giải mã thành `Message` và gửi vào `clientInboundChannel`; khung có đích bắt đầu bằng tiền tố ứng dụng (`/app`) được định tuyến tới các phương thức `@MessageMapping`, còn khung có đích `/topic` hoặc `/queue` có thể được định tuyến thẳng tới broker. Controller gửi thông điệp tới broker qua `brokerChannel`; broker tìm mọi subscriber khớp và gửi khung MESSAGE qua `clientOutboundChannel` [25]. Simple broker lưu subscription trong bộ nhớ và hỗ trợ cả đích dạng mẫu kiểu Ant [26] – chi tiết này quan trọng cho phân tích bảo mật ở mục 4.10.5.



Hình 4.3. *Luồng thông điệp STOMP trong NextChat: kênh inbound, interceptor, controller, broker và kênh outbound*

4.10.2. Cấu hình endpoint và broker

Đoạn mã 4.9. *WebSocketConfig trên main (lược bỏ import)*

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    public WebSocketConfig(JwtStompChannelInterceptor jwtStompChannelInterceptor,
            @Value("${app.websocket.allowed-origins:http://localhost:8080,"
                 + "http://127.0.0.1:8080,http://localhost:3000,http://localhost:5173}")
            String[] allowedOrigins) { ... }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.setApplicationDestinationPrefixes("/app");
        registry.enableSimpleBroker("/topic", "/queue");
        registry.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns(allowedOrigins);
        registry.addEndpoint("/ws-chat").setAllowedOriginPatterns(allowedOrigins).withSockJS();
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(jwtStompChannelInterceptor);
    }
}
```

`/ws` là endpoint WebSocket thuần, `/ws-chat` bật SockJS. Commit `1046984` ngày 06/11 bổ sung `http://localhost:8080` và `http://127.0.0.1:8080` vào danh sách origin mặc định – cần thiết vì client web giờ được phục vụ ngay từ cổng 8080 của backend; trước đó danh sách chỉ có cổng 3000 và 5173 dành cho máy chủ phát triển frontend. Khai báo tiền tố `/queue` và `/user` chưa có nghĩa tính năng nhắn riêng theo người dùng đã được xây dựng.

4.10.3. Interceptor: CONNECT, khôi phục danh tính và SUBSCRIBE

Đoạn mã 4.10. *preSend của JwtStompChannelInterceptor trên main (rút gọn phần đọc token)*

```
StompHeaderAccessor accessor = StompHeaderAccessor.wrap(message);
if (StompCommand.CONNECT.equals(accessor.getCommand())) {
    String token = token(accessor.getFirstNativeHeader("Authorization"));
    if (token == null) token = token(accessor.getFirstNativeHeader("access-token"));
    if (token == null) throw new IllegalArgumentException("A valid JWT is required to connect");
    String username = jwtService.username(token);
    UserDetails user = userDetailsService.loadUserByUsername(username);
    if (!jwtService.isValid(token, user)) throw new IllegalArgumentException("...");
    accessor.setUser(new UsernamePasswordAuthenticationToken(user, null,
user.getAuthorities()));
    if (accessor.getSessionAttributes() != null) {
        accessor.getSessionAttributes().put("stompUsername", username);   // 1046984
    }
} else {
    restoreAuthentication(accessor);                                      // 1046984
}
if (StompCommand.SUBSCRIBE.equals(accessor.getCommand())) {
    Matcher matcher = ROOM_DESTINATION.matcher(       // ^/topic/room/(\d+)$
            accessor.getDestination() == null ? "" : accessor.getDestination());
    if (matcher.matches() && !isRoomMember(matcher.group(1), accessor)) {
        throw new IllegalArgumentException("You are not a participant in this room");
    }
}
accessor.setLeaveMutable(true);
return MessageBuilder.createMessage(message.getPayload(), accessor.getMessageHeaders());
```

- CONNECT: đọc JWT từ native header `Authorization` (dạng `Bearer ...`) hoặc `access-token`; không đọc tham số truy vấn `access_token` trên URL. Token hợp lệ thì gán `Authentication` làm user của khung và lưu tên người dùng vào thuộc tính phiên WebSocket `stompUsername`.
- Khung khác (SUBSCRIBE, SEND, DISCONNECT): `restoreAuthentication` – nếu khung chưa mang user mà phiên có `stompUsername`, nạp lại `UserDetails` từ cơ sở dữ liệu và gán lại user cho khung. Nhờ đó `RoomWebSocketController` nhận được `Principal` hợp lệ.

- SUBSCRIBE: nếu đích khớp chính xác mẫu `/topic/room/{số}`, kiểm tra người dùng còn hoạt động và là thành viên chưa rời phòng (`left_at IS NULL`).

Vì sao cần khôi phục danh tính? Ngày 25/10 nhóm tái lập được lỗi: mã gốc tại `39b4d75` vượt qua 22 bài JUnit nhưng hai client thật nhận CONNECTED rồi bị ERROR khi SUBSCRIBE. Nguyên nhân là interceptor tạo accessor mới bằng `wrap(message)` và trả về một message mới; transport STOMP gắn một callback vào accessor gốc để ghi nhớ user cho các khung sau của phiên, và callback này không được gọi khi user được đặt trên accessor mới. Hệ quả: CONNECT nhìn như có user, nhưng khung sau mất Principal (Tình huống 3, mục 11.2). Có hai cách khắc phục, và nhóm đã dùng cả hai ở hai thời điểm:

Bảng 4.9. *Hai cách khắc phục lỗi mất Principal*

Tiêu chí	Bản sửa cục bộ 25/10 (chưa push)	Commit `1046984` trên main (06/11)
Ý tưởng	Lấy accessor sẵn có của transport bằng `MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class)`, gọi `setUser` để kích hoạt callback, trả lại message gốc	Vẫn dùng `wrap`, nhưng lưu `stompUsername` vào thuộc tính phiên ở CONNECT và gán lại user ở mỗi khung sau
Truy vấn thêm	Không	Nạp lại `UserDetails` từ DB ở mỗi khung sau CONNECT
Hành vi khi tài khoản bị khoá giữa phiên	Phiên tiếp tục với Principal cũ; quyền phòng vẫn được kiểm ở SUBSCRIBE và ở service khi gửi	`loadUserByUsername` ném `UsernameNotFoundException` ở khung kế tiếp, khung bị từ chối
Kiểm thử	Test hồi quy gắn `userChangeCallback`; 15 smoke với hai client thật	1 test đơn vị khôi phục từ phiên; chưa có smoke trên `main`

4.10.4. Controller gửi tin và hợp đồng khung

Đoạn mã 4.11. *RoomWebSocketController trên main (sau commit 332d069)*

```
@MessageMapping("/room/{roomId}")
public void sendMessage(@DestinationVariable Long roomId,
                        @Valid @Payload SendMessageRequest request,
                        Principal principal) {
    if (principal == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Authentication required");
    }
    Long userId = userRepository.findByUsername(principal.getName())
            .filter(user -> user.isActive() && !user.isDeleted())
            .map(User::getId)
```

```
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.UNAUTHORIZED, "Authentication required"));
    MessageDTO message = roomService.sendMessage(
            userId, roomId, request.content(), request.messageType(),
            request.replyToMessageId());
    messagingTemplate.convertAndSend("/topic/room/" + roomId, message);
}
```

Client gửi JSON tới `/app/room/{roomId}`; tiền tố `/app` bị bỏ, phần `/room/{roomId}` khớp `@MessageMapping`. Danh tính người gửi lấy từ `Principal`, không lấy `senderId` do client tự khai. `RoomService.sendMessage` kiểm tra phòng tồn tại, người gửi là thành viên còn hoạt động, tin trả lời (nếu có) thuộc cùng phòng và chưa bị xoá, chuẩn hoá loại tin về một trong bảy giá trị hợp lệ, rồi lưu tin và cập nhật tin cuối của phòng trong một giao dịch. Chỉ sau khi service trả về thành công, controller mới gọi `SimpMessagingTemplate.convertAndSend` để phát `MessageDTO` tới `/topic/room/{roomId}`. Commit `332d069` của Lê Quốc Cường ngày 30/10 đổi từ `@SendTo` sang `SimpMessagingTemplate` đồng thời với việc đổi đích sang dạng dấu gạch chéo; về hành vi, cả hai cách đều phát tới broker sau khi phương thức chạy xong.

Đoạn mã 4.12. *Hợp đồng khung STOMP minh hoạ cho phòng 7 trên main*

```
CONNECT
accept-version:1.2
host:localhost
Authorization:Bearer <accessToken>

^@
SUBSCRIBE
id:room-7
destination:/topic/room/7

^@
SEND
destination:/app/room/7
content-type:application/json

{"content":"Xin chào","messageType":"TEXT","replyToMessageId":null}^@
```

Ký hiệu `^@` biểu thị byte NUL kết thúc khung STOMP, không phải chuỗi ký tự gửi nguyên văn. `Authorization` là native header của khung CONNECT; không giả định trình duyệt đặt được header tuỳ ý trong yêu cầu bắt tay WebSocket. `MessageDTO` nhận được mang `id` do máy chủ cấp, dùng để đối chiếu với lịch sử REST.

4.10.5. Phân tích rủi ro kiểm soát truy cập trên main

Đọc lại mã `main` tại `54ab54e` và đối chiếu với tài liệu Spring, nhóm xác định hai điểm mà bản sửa cục bộ ngày 25/10 đã chặn nhưng bản `main` chưa chặn. Đây là kết luận từ phân tích mã nguồn; nhóm chưa chạy kịch bản tấn công trên `main` trong lúc viết báo cáo, nên các điểm này được ghi là rủi ro cần xác nhận bằng test.

Rủi ro R1 – Đăng ký đích dạng mẫu bỏ qua kiểm tra thành viên

Điều kiện chặn SUBSCRIBE là `matcher.matches() && !isRoomMember(...)`. Với đích không khớp chính xác `^/topic/room/(\d+)$` – ví dụ `/topic/room/*` hoặc `/topic/**` – `matches()` trả `false`, biểu thức đánh giá là `false`, và khung SUBSCRIBE được cho qua mà không kiểm tra gì. Simple broker hỗ trợ subscription theo mẫu kiểu Ant [26], nên về nguyên tắc một người dùng đã đăng nhập bất kỳ có thể đăng ký `/topic/room/*` và nhận tin của mọi phòng. Bản sửa cục bộ xử lý bằng cách từ chối mọi SUBSCRIBE không khớp chính xác mẫu phòng ("Only exact room subscriptions are supported").

Rủi ro R2 – Gửi thẳng vào đích của broker

Interceptor trên `main` không kiểm tra khung SEND. Theo luồng thông điệp của Spring, khung có đích `/topic/...` từ client có thể được định tuyến thẳng tới broker [25]. Một client đã đăng nhập có thể gửi `SEND destination:/topic/room/7` với nội dung JSON tuỳ ý (kể cả `senderId`, `senderUsername` giả), và broker phát nguyên văn tới mọi subscriber của phòng 7 – bỏ qua controller, không lưu vào cơ sở dữ liệu, không kiểm tra thành viên. Bản sửa cục bộ chặn bằng mẫu `^/app/room\.(\d+)$` cho SEND; khi chuyển sang đích dạng gạch chéo, mẫu tương ứng là `^/app/room/(\d+)$`.

Đề xuất bản vá

Đoạn mã 4.13. *Đề xuất bổ sung vào JwtStompChannelInterceptor (chưa có trên main)*

```
private static final Pattern ROOM_SEND_DESTINATION = Pattern.compile("^/app/room/(\\d+)
$");

if (StompCommand.SUBSCRIBE.equals(accessor.getCommand())) {
```

```
    Matcher matcher = ROOM_DESTINATION.matcher(
            accessor.getDestination() == null ? "" : accessor.getDestination());
    if (!matcher.matches()) {
        throw new IllegalArgumentException("Only exact room subscriptions are supported");
    }
    if (!isRoomMember(matcher.group(1), accessor)) {
        throw new IllegalArgumentException("You are not a participant in this room");
    }
} else if (StompCommand.SEND.equals(accessor.getCommand())) {
    String destination = accessor.getDestination();
    if (destination == null || !ROOM_SEND_DESTINATION.matcher(destination).matches()) {
        throw new IllegalArgumentException(
            "Messages must be sent through the room application destination");
    }
}
```

Kèm bản vá cần hai bài test đơn vị (SUBSCRIBE `/topic/room/*` bị từ chối; SEND `/topic/room/7` bị từ chối) và cập nhật kịch bản smoke sang đích dạng gạch chéo để kiểm bằng socket thật. Bản vá không thay đổi hợp đồng với client web hiện tại vì client chỉ dùng `SUBSCRIBE /topic/room/{id}` và `SEND /app/room/{id}`.

4.11. Tiểu kết chương

Mười cơ chế trong chương đều có vị trí sử dụng thật và có minh chứng: auto-configuration dựng hạ tầng dữ liệu; IoC nối chuỗi bean bảo mật; cấu hình ngoài chặn khoá phát triển; MVC chuẩn hoá lỗi; Security xác thực JWT không trạng thái; JPA nạp quan hệ có kiểm soát; proxy giao dịch giữ dữ liệu nguyên vẹn khi lỗi; Flyway quản lý lược đồ; Testcontainers cho kiểm thử trên PostgreSQL thật; Spring Messaging cho kênh STOMP có xác thực. Kết luận cần phân biệt theo từng lớp: tính chất khoá hay hiệu năng truy vấn thuộc về PostgreSQL, không phải của riêng Spring Boot. Kênh STOMP trên `main` đã giải quyết được lỗi mất Principal bằng cách khôi phục danh tính từ phiên, nhưng còn hai rủi ro kiểm soát truy cập (R1, R2) cần áp dụng bản vá và kiểm thử trước khi coi là hoàn chỉnh.

CHƯƠNG 5. TÍNH NGUYÊN TỬ KHI TẠO PHÒNG 1-1 – HÀM PL/PGSQL VÀ KHOÁ ADVISORY

5.1. Vấn đề: "tìm, nếu chưa có thì tạo" dưới truy cập đồng thời

Khi người dùng A bấm "Nhắn tin" cho người dùng B, hệ thống phải trả về phòng PRIVATE đã có giữa hai người, hoặc tạo phòng mới nếu chưa có. Thao tác này gồm hai bước: tìm rồi tạo. Nếu hai yêu cầu đến gần như đồng thời – A nhắn cho B và B nhắn cho A cùng lúc, hoặc A bấm hai lần – cả hai giao dịch có thể cùng thấy "chưa có phòng" ở bước tìm, rồi cùng chèn ở bước tạo. Kết quả là hai phòng cho một cặp người dùng; tin nhắn bị chia vào hai phòng và mỗi người có thể đang xem một phòng khác nhau.

Lược đồ không tự chặn được tình huống này. Ràng buộc `uq_conversation_user` UNIQUE (`conversation_id`, `user_id`) chỉ ngăn một người bị thêm hai lần vào cùng một phòng; nó không ngăn hai phòng khác nhau có cùng tập hai thành viên. Giao dịch ở mức cô lập mặc định READ COMMITTED của PostgreSQL cũng không giúp: mỗi giao dịch chỉ thấy dữ liệu đã commit, nên hai giao dịch chưa commit không thấy phòng của nhau.

5.2. Các phương án đã cân nhắc

Bảng 5.1. *Các phương án chống tạo trùng phòng 1-1*

Phương án	Cách làm	Đánh giá
Mức cô lập SERIALIZABLE	Chạy giao dịch tạo phòng ở mức cao nhất; PostgreSQL phát hiện xung đột và huỷ một giao dịch	Đúng nhưng giao dịch bị huỷ phải được thử lại ở tầng ứng dụng; tăng độ phức tạp
Cột khoá cặp + UNIQUE	Thêm cột `direct_key = 'min:max'` cho phòng PRIVATE với ràng buộc UNIQUE; bắt lỗi trùng khi chèn	Mạnh nhất về toàn vẹn; cần sửa lược đồ và xử lý ngoại lệ trùng khoá để đọc lại phòng
`SELECT ... FOR UPDATE` trên hai hàng `users`	Khoá hai hàng người dùng theo thứ tự id trước khi tìm và tạo	Hiệu quả nhưng khoá cả hàng người dùng, ảnh hưởng thao tác khác trên `users`
Khoá advisory mức giao dịch (được chọn)	Lấy khoá theo cặp đã chuẩn hoá trước khi tìm; giao dịch thứ hai chờ rồi thấy phòng đã tạo	Không đổi lược đồ, không khoá hàng nghiệp vụ, tự nhả khi giao dịch kết thúc; chỉ bảo vệ các đường đi qua hàm

5.3. Cơ chế khoá advisory của PostgreSQL

Khoá advisory là khoá do ứng dụng định nghĩa ý nghĩa: PostgreSQL chỉ quản lý việc ai đang giữ khoá có mã số nào, còn khoá đó bảo vệ cái gì là quy ước của ứng dụng [6]. Hàm `pg_advisory_xact_lock(bigint)` lấy khoá độc quyền mức giao dịch: nếu giao dịch khác đang giữ cùng mã, lời gọi sẽ chờ; khoá tự nhả khi giao dịch commit hoặc rollback, không cần lệnh mở khoá tường minh và không bị "quên nhả" khi có lỗi.

Hàm `get_or_create_direct_room` tạo mã khoá từ cặp người dùng như sau:

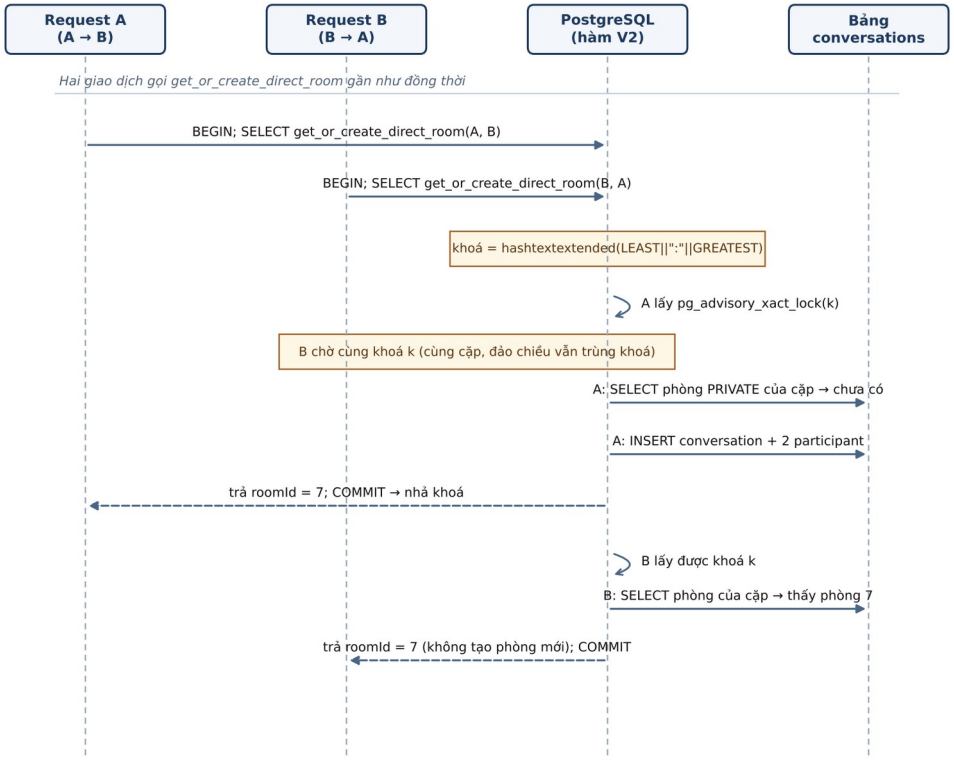
- Chuẩn hoá thứ tự bằng `LEAST(a, b)` và `GREATEST(a, b)`, nên yêu cầu (A, B) và (B, A) cho cùng chuỗi `"min:max"`, ví dụ `"3:8"`.
- Băm chuỗi thành số 64 bit bằng `hashtextextended(chuỗi, 0)`.
- Hai cặp khác nhau có thể trùng mã băm (va chạm); khi đó hai giao dịch không liên quan phải chờ nhau một chút, nhưng tính đúng đắn không bị ảnh hưởng vì sau khi lấy khoá hàm vẫn kiểm tra lại theo đúng cặp người dùng.

Đoạn mã 5.1. *Hàm get_or_create_direct_room (V1/V2, rút gọn chú thích)*

```
CREATE OR REPLACE FUNCTION get_or_create_direct_room(p_user_a BIGINT, p_user_b BIGINT)
RETURNS BIGINT AS $$
DECLARE
    v_room_id BIGINT;
BEGIN
    IF p_user_a = p_user_b THEN
        RAISE EXCEPTION 'A private room requires two different users';
    END IF;
    PERFORM pg_advisory_xact_lock(
        hashtextextended(
            LEAST(p_user_a, p_user_b)::TEXT || ':' || GREATEST(p_user_a, p_user_b)::TEXT, 0));
    SELECT cp1.conversation_id INTO v_room_id
    FROM conversation_participants cp1
    JOIN conversation_participants cp2 ON cp1.conversation_id = cp2.conversation_id
    JOIN conversations c ON c.id = cp1.conversation_id
    WHERE c.type = 'PRIVATE' AND cp1.user_id = p_user_a AND cp2.user_id = p_user_b
      AND cp1.left_at IS NULL AND cp2.left_at IS NULL
    ORDER BY c.id LIMIT 1;
    IF v_room_id IS NOT NULL THEN
        RETURN v_room_id;
    END IF;
    INSERT INTO conversations (type, created_by, created_at, updated_at)
    VALUES ('PRIVATE', p_user_a, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
    RETURNING id INTO v_room_id;
    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_a, 'OWNER', CURRENT_TIMESTAMP);
    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_b, 'MEMBER', CURRENT_TIMESTAMP);
    RETURN v_room_id;
END;
```

```
$$ LANGUAGE plpgsql;
```

Hình 5.1 mô tả diễn biến khi hai yêu cầu đảo chiều đến cùng lúc. Điểm mấu chốt nằm ở chỗ: giao dịch B chỉ được chạy câu SELECT sau khi giao dịch A đã commit và nhả khoá. Ở mức READ COMMITTED, mỗi câu lệnh thấy dữ liệu đã commit tại thời điểm câu lệnh bắt đầu, nên câu SELECT của B thấy phòng do A vừa tạo.



Hình 5.1. *Hai giao dịch đảo chiều được tuần tự hoá bởi cùng một khoá advisory*

5.4. Gọi hàm từ Spring

`POST /api/v1/rooms/direct` → `RoomController.getOrCreateDirectRoom` → `RoomService.getOrCreateDirectRoom` (có `@Transactional`) → `ConversationRepository.callGetOrCreateDirectRoom`, một native query `SELECT get_or_create_direct_room(:userA, :userB)`. Trước khi gọi hàm, service kiểm tra hai điều: không tạo phòng với chính mình (400 "Cannot create a

direct chat with yourself") và cả hai người dùng đều tồn tại, đang hoạt động (404). Sau khi hàm trả `roomId`, service đọc lại `Conversation` bằng `findById` và dựng `RoomDTO` trong cùng giao dịch. Vì khoá là khoá mức giao dịch, nó được giữ tới khi giao dịch của Spring commit – tức cũng bao phủ cả bước đọc lại.

5.5. Kiểm chứng

5.5.1. Kiểm thử tích hợp và đầu-cuối

`testConcurrentDirectRoomCreationIsIdempotent` (test #17) đăng ký hai người dùng mới, dùng hai `CountDownLatch` (`ready` và `start`) để hai luồng sẵn sàng rồi cùng bắt đầu gọi `roomService.getOrCreateDirectRoom(A, B)`, và khẳng định hai kết quả có cùng id phòng. Đây là một lượt cho một cặp theo cùng chiều; không suy ra tỷ lệ tạo trùng hay độ trễ. Test #11 kiểm tính lặp lại: gọi API hai lần tuần tự trả cùng id. Kịch bản đầu-cuối ngày 28/10 bổ sung trường hợp đảo chiều: hai request HTTP (B, C) và (C, B) gửi gần như đồng thời trả cùng id.

5.5.2. Thí nghiệm đối chứng có khoá và bỏ khoá

Một bài test "đạt" chưa chứng minh khoá có tác dụng: có thể không có khoá test vẫn đạt vì hai luồng không thực sự chồng lấn. Để thấy rõ tác dụng, ngày 28/10 nhóm làm thí nghiệm đối chứng trong một PostgreSQL 16.15 riêng (container giới hạn 1 vCPU, 512 MiB RAM, kết nối loopback, mức READ COMMITTED). Script `tools/sql_lab.py` sao chép hàm V2 thành hai biến thể `lab_locked_direct_room` và `lab_unlocked_direct_room`; cả hai thêm `pg_sleep(0.15)` sau câu SELECT chưa thấy phòng để mở rộng cửa sổ tranh chấp, và biến thể unlocked chỉ khác ở chỗ bỏ khối khoá advisory. Độ trễ nhân tạo chỉ để tạo chồng lấn, không dùng để đo hiệu năng.

Mỗi lượt dùng một cặp người dùng mới, hai kết nối độc lập và một `Barrier(2)` để hai worker cùng gọi hàm, một theo chiều (A, B), một theo chiều (B, A). Mỗi worker commit trước khi trả kết quả. Sau khi cả hai kết thúc, script đếm số phòng và số bản ghi thành viên của cặp đó trong cơ sở dữ liệu.

Bảng 5.2. *Kết quả đối chứng có/không khoá trong lab SQL ngày 28/10/2026*

Biến thể / lượt	ID hai worker nhận được	Số phòng / số thành viên của cặp
locked / 1	[2, 2]	1 / 2
locked / 2	[3, 3]	1 / 2
locked / 3	[4, 4]	1 / 2
unlocked / 1	[5, 6]	2 / 4
unlocked / 2	[7, 8]	2 / 4
unlocked / 3	[10, 9]	2 / 4

Quan sát: biến thể có khoá luôn trả cùng một id và chỉ còn một phòng với hai thành viên; biến thể bỏ khoá sinh hai id, hai phòng và bốn bản ghi thành viên trong cả ba lượt. Thí nghiệm có điều kiện làm chậm nhân tạo, chỉ ba lượt mỗi biến thể, nên được diễn giải như minh chứng cơ chế, không phải xác suất lỗi khi vận hành. Đây cũng không phải log thất bại lịch sử của test #17 trong Git. Các tệp `lab_locked.sql`, `lab_unlocked.sql`, `sql-lab-results.json` và lệnh tái lập được lưu trong hồ sơ minh chứng [10].

5.6. Giới hạn và hướng tăng cường

- Khoá chỉ bảo vệ các đường ghi đi qua hàm. Một đoạn mã khác chèn trực tiếp vào `conversations` sẽ không lấy khoá; nếu sau này có thêm đường tạo phòng (ví dụ nhập dữ liệu hàng loạt), cần bổ sung ràng buộc ở mức lược đồ.
- Phương án phòng thủ nhiều lớp được đề xuất: thêm cột `direct_key` (chuỗi `min:max`) cho phòng PRIVATE với chỉ mục UNIQUE có điều kiện, giữ khoá advisory để tránh lỗi trùng trong luồng thường và để ràng buộc làm chốt chặn cuối.
- Va chạm băm có thể tuần tự hoá thêm các cặp không liên quan; với số người dùng của đồ án, ảnh hưởng không đáng kể nhưng chưa được đo.

5.7. Tiểu kết

Thao tác "tìm hoặc tạo" phòng 1-1 được làm nguyên tử bằng khoá advisory theo cặp đã chuẩn hoá, đặt trong hàm PL/pgSQL và quản lý bằng migration. Kiểm

thử tích hợp cho thấy kết quả đúng trong luồng thường; thí nghiệm đối chứng cho thấy chính khoá là yếu tố ngăn tạo trùng khi có chồng lấn.

CHƯƠNG 6. PHÂN TRANG LỊCH SỬ TIN NHẮN – PL/PGSQL, OFFSET VÀ CHI PHÍ COUNT

6.1. Vì sao chọn nghiên cứu phân trang

Lịch sử tin nhắn là phần dữ liệu tăng nhanh nhất của một ứng dụng chat. Mỗi lần mở phòng, client cần trang tin mới nhất; khi cuộn lên, client cần các trang cũ hơn. Độ trễ của trang đầu và của các trang sâu quyết định trải nghiệm, và cách phân trang ảnh hưởng trực tiếp tới số dòng cơ sở dữ liệu phải duyệt.

6.2. Hàm get_room_messages

Huỳnh Thanh Hải viết hàm `get_room_messages(p_room_id, p_page, p_size)` trong V1. Hàm đếm tổng số tin chưa xoá của phòng vào biến `v_total`, rồi trả về một trang tin đã JOIN thông tin người gửi, sắp xếp mới nhất trước.

Đoạn mã 6.1. *Phần thân hàm get_room_messages*

```
DECLARE
    v_offset INT := GREATEST(p_page, 0) * GREATEST(p_size, 1);
    v_total BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_total
    FROM messages m
    WHERE m.conversation_id = p_room_id AND m.is_deleted = FALSE;

    RETURN QUERY
    SELECT m.id, m.conversation_id, m.sender_id,
           u.username, u.full_name, u.avatar_url,
           m.message_type, m.content, m.reply_to_message_id,
           m.is_edited, m.is_deleted, m.created_at, m.updated_at,
           COALESCE(v_total, 0) AS total_count
    FROM messages m
    JOIN users u ON m.sender_id = u.id
    WHERE m.conversation_id = p_room_id AND m.is_deleted = FALSE
    ORDER BY m.created_at DESC, m.id DESC
    LIMIT GREATEST(p_size, 1) OFFSET v_offset;
END;
```

Một số chi tiết thiết kế:

- Sắp xếp theo `created_at DESC, id DESC`: thêm `id` để thứ tự ổn định khi hai tin có cùng thời điểm, tránh một tin xuất hiện ở hai trang hoặc bị bỏ sót.

- Tổng số được tính bằng `SELECT COUNT(*) INTO v_total` trước `RETURN QUERY`, rồi lặp lại trên mỗi dòng ở cột `total_count`. Hàm không dùng hàm cửa sổ `COUNT(*) OVER()`; một tài liệu nháp từng mô tả sai điểm này và đã được sửa theo mã nguồn (mục 11.4).
- `GREATEST(p_page, 0)` và `GREATEST(p_size, 1)` phòng tham số âm ngay trong SQL, dù service đã kiểm tra trước.

6.3. Tầng service và trường hợp trang rỗng

`RoomService.getRoomMessages` kiểm tra lần lượt: phòng tồn tại (404), người gọi là thành viên còn hoạt động (403), `page ≥ 0` và `1 ≤ size ≤ 100` (400). Sau đó gọi hàm qua native query, ánh xạ 14 cột sang `MessageDTO` và trả về `PageImpl` với `PageRequest.of(page, size)` cùng tổng số.

Có một trường hợp tinh tế: khi trang yêu cầu nằm ngoài phạm vi (ví dụ trang 1 với 2 tin và `size=10`), hàm trả về 0 dòng, nên không có dòng nào mang `total_count`. Nếu service lấy tổng từ dòng đầu tiên, tổng sẽ thành 0 – sai. Mã hiện tại có nhánh đếm dự phòng:

Đoạn mã 6.2. *Đếm dự phòng khi trang rỗng trong RoomService*

```
long totalCount = rows.isEmpty()
        ? messageRepository.countByConversationIdAndDeletedFalse(roomId)
        : 0L;
for (Object[] row : rows) {
    // ... ánh xạ 13 cột đầu sang MessageDTO
    totalCount = row[13] != null ? ((Number) row[13]).longValue() : 0L;
}
return new PageImpl<>(dtoList, PageRequest.of(pageNum, pageSize), totalCount);
```

Với cấu hình `PageSerializationMode.VIA_DTO` (mục 1.6), JSON trả về có dạng `data.content[...]` và `data.page{size, number, totalElements, totalPages}`.

6.4. Kết quả kiểm thử

Test #14 chèn hai tin vào phòng 1-1 rồi khẳng định: trang 0 với `size=10` trả mảng `content` có `senderUsername` và `content` là chuỗi, `data.page.totalElements = 2`; trang 1 trả `content` rỗng nhưng

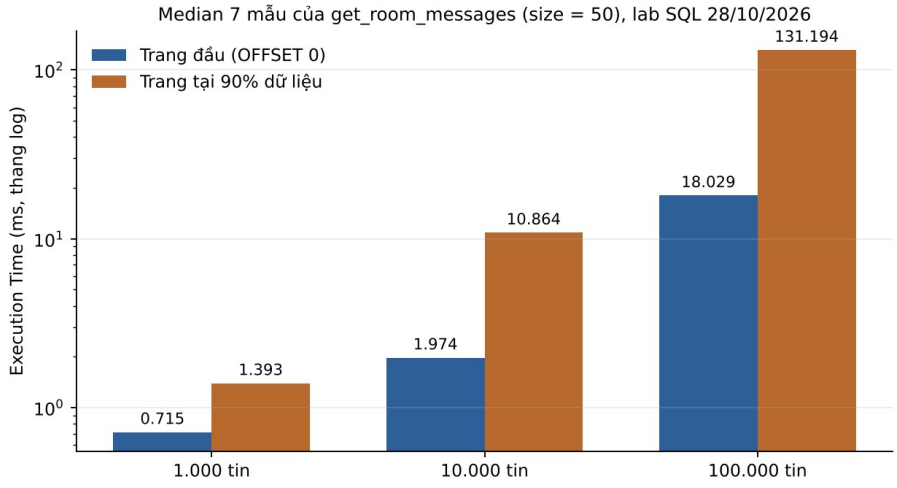
`totalElements` vẫn bằng 2; `size=101` bị từ chối 400. Bản kiểm chứng 28/10 bổ sung một tin đã xoá mềm và khẳng định tổng vẫn bằng 2, tức tin xoá mềm không được đếm và không được trả về.

6.5. Thí nghiệm phân trang ở ba mức dữ liệu

Cùng môi trường SQL ở mục 5.5.2, nhóm tạo một phòng, một người gửi và lần lượt nạp 1.000, 10.000, 100.000 tin chưa xoá có thời gian tăng dần từng giây, sau mỗi lần nạp chạy `ANALYZE`. Với `size=50`, đo trang 0 và trang ở vị trí 90% tập dữ liệu. Mỗi tổ hợp chạy 3 lần làm nóng rồi lấy 7 mẫu "Execution Time" phía máy chủ của `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT * FROM get_room_messages(...)`.

Bảng 6.1. *Thời gian thực thi hàm lịch sử trong lab SQL (ms)*

Số tin / trang / OFFSET	Median (7 mẫu)	Min – max
1.000 / 0 / 0	0,715	0,639 – 0,969
1.000 / 18 / 900	1,393	1,371 – 1,448
10.000 / 0 / 0	1,974	1,940 – 2,047
10.000 / 180 / 9.000	10,864	10,595 – 11,293
100.000 / 0 / 0	18,029	15,590 – 23,538
100.000 / 1800 / 90.000	131,194	119,061 – 143,305



Hình 6.1. *Thời gian thực thi trang đầu và trang sâu theo số lượng tin (thang logarit)*

Hai xu hướng rõ ràng: (1) ngay cả trang đầu cũng chậm dần khi dữ liệu tăng, vì hàm luôn đếm toàn bộ tin của phòng trước khi trả trang; (2) trang sâu chậm hơn trang đầu khoảng 2 lần ở 1.000 tin, 5,5 lần ở 10.000 tin và 7,3 lần ở 100.000 tin, do OFFSET buộc cơ sở dữ liệu phải duyệt rồi bỏ qua các dòng đứng trước.

6.6. Đọc kế hoạch thực thi

Kế hoạch ngoài của lời gọi hàm chỉ hiện một nút "Function Scan", không cho biết bên trong hàm làm gì. Vì vậy script lưu riêng kế hoạch của câu truy vấn trang (rút gọn cột) và của câu COUNT. Ở 100.000 tin, OFFSET 90.000:

Đoạn mã 6.3. *Kế hoạch rút gọn của truy vấn trang sâu và truy vấn đếm*

```
Limit: Actual Rows=50
  Incremental Sort: Actual Rows=90050
    Nested Loop: Actual Rows=90051
      Index Scan idx_messages_conversation_created: Actual Rows=90051
-- COUNT riêng: Seq Scan messages, Actual Rows=100000
```

Để trả 50 dòng, cơ sở dữ liệu đã đi qua khoảng 90.051 dòng qua chỉ mục, sắp xếp tăng dần (incremental sort vì chỉ mục chưa có `id`) rồi mới cắt; câu đếm quét tuần tự toàn bộ 100.000 dòng [12]. Giới hạn của phép đo: cache ấm, một cơ sở dữ liệu một worker, có chi phí của chính `EXPLAIN`; chưa đo client HTTP, JWT, ánh xạ JPA, tuần tự hoá JSON, WebSocket hay tải đồng thời. Các con số chỉ mô tả lần đo ngày 28/10, không phải cam kết thời gian đáp ứng.

6.7. OFFSET và keyset: đánh đổi và hướng cải tiến

Bảng 6.2. *So sánh phân trang OFFSET và keyset*

Tiêu chí	OFFSET (hiện tại)	Keyset / cursor (đề xuất)
Tham số	`page`, `size`	`before=(created_at, id)` của tin cũ nhất đang có, `size`
Chi phí trang sâu	Tăng tuyến tính theo OFFSET	Gần như hằng số nhờ chỉ mục
Nhảy tới trang bất kỳ	Có	Không (chỉ trang kế tiếp)
Ổn định khi có tin mới	Tin mới đẩy các trang lệch đi một vị trí	Ổn định vì neo theo khoá
Tổng số phần tử	Cần COUNT	Không cần cho cuộn vô hạn

Với giao diện chat cuộn vô hạn, keyset phù hợp hơn. Bản nháp truy vấn và chỉ mục đề xuất (chưa triển khai, cần đo đối chứng trước khi áp dụng):

Đoạn mã 6.4. *Đề xuất phân trang keyset (chưa có trong mã nguồn)*

```
CREATE INDEX idx_messages_conv_created_id
    ON messages (conversation_id, created_at DESC, id DESC) WHERE is_deleted =
FALSE;

SELECT m.*, u.username, u.full_name, u.avatar_url
FROM messages m JOIN users u ON u.id = m.sender_id
WHERE m.conversation_id = :roomId AND m.is_deleted = FALSE
  AND (m.created_at, m.id) < (:beforeCreatedAt, :beforeId)
ORDER BY m.created_at DESC, m.id DESC
LIMIT :size;
```

Ngoài ra, có thể bỏ COUNT khỏi mỗi lần lấy trang và chỉ trả cờ "còn trang cũ hơn" bằng cách lấy `size + 1` dòng. Về tính nhất quán: COUNT và truy vấn trang chạy ở hai câu lệnh khác nhau trong mức READ COMMITTED, nên nếu có tin mới chèn xen giữa, tổng và trang có thể lệch nhau một chút; với chat, sai lệch này chấp nhận được.

6.8. Dùng ở đâu

`GET /api/v1/rooms/{roomId}/messages?page=&size=` là nguồn lịch sử cho màn hình chat. Client web hiện tải trang 0 với `size=50` khi chọn phòng và sắp xếp lại tăng dần để hiển thị theo chiều đọc (mục 10.5.3); chưa có thao tác cuộn để tải trang cũ hơn.

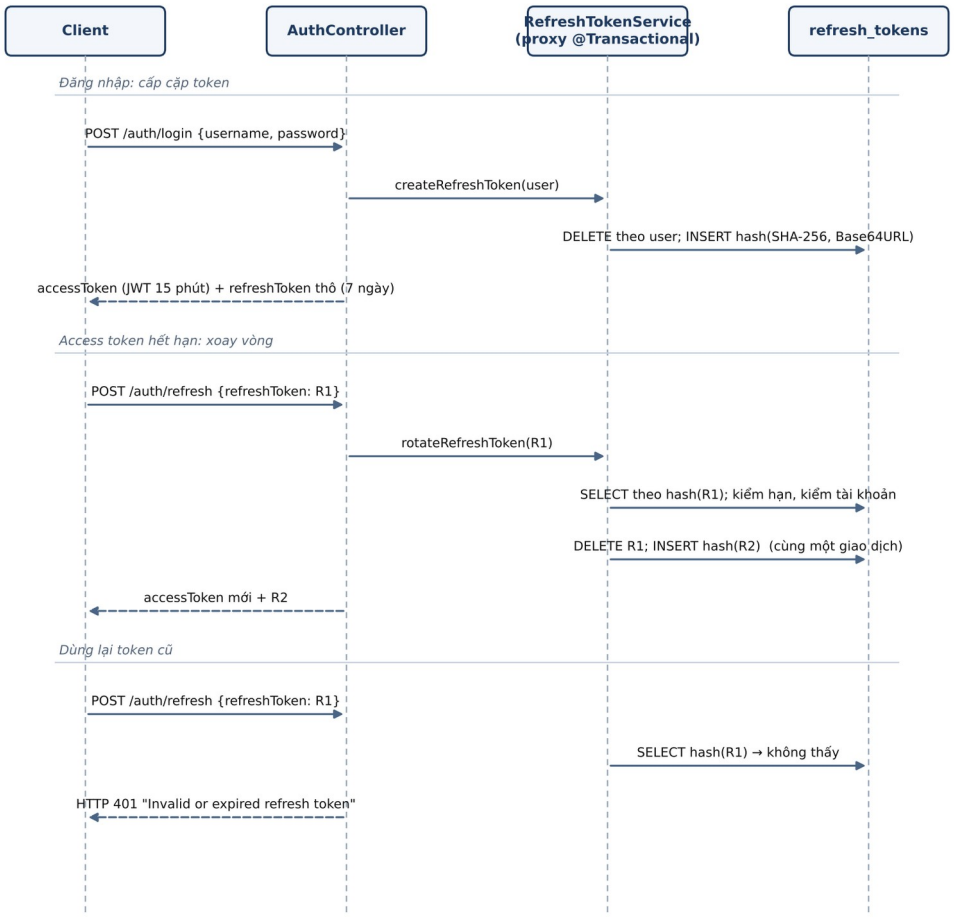
CHƯƠNG 7. REFRESH TOKEN XOAY VÒNG VÀ CÁC ĐÁNH ĐỔI BẢO MẬT

7.1. Vì sao cần refresh token

Access token ngắn hạn (15 phút) giới hạn thiệt hại khi token bị lộ: kẻ lấy được token chỉ dùng được trong thời gian ngắn. Nhưng nếu chỉ có access token, người dùng phải đăng nhập lại mỗi 15 phút. Refresh token giải quyết mâu thuẫn này: một chuỗi bí mật dài hạn (7 ngày) chỉ dùng để xin access token mới. Cách lưu và cách dùng refresh token quyết định độ an toàn của cả hệ thống xác thực.

7.2. Cơ chế trong NextChat

- Sinh token: 32 byte ngẫu nhiên từ `SecureRandom`, mã hoá Base64URL không đệm (43 ký tự). Chuỗi thô này chỉ được trả cho client một lần.
- Lưu trữ: máy chủ băm SHA-256 chuỗi thô rồi lưu digest dạng Base64URL không đệm (43 ký tự) vào cột `refresh_tokens.token`, không phải chuỗi hex 64 ký tự. Nếu cơ sở dữ liệu bị lộ, kẻ tấn công không có chuỗi thô để dùng.
- Đăng nhập: `createRefreshToken` xoá mọi refresh token cũ của người dùng rồi phát token mới, trong một giao dịch. Hệ quả: mỗi người dùng chỉ có một phiên làm mới tại một thời điểm.
- Làm mới: `rotateRefreshToken` tìm theo hash, kiểm hạn, kiểm tài khoản còn hoạt động, xoá token đã dùng và phát token thay thế – token dùng một lần (xoay vòng).



Hình 7.1. *Đăng nhập, xoay vòng và dùng lại refresh token*

Đoạn mã 7.1. *rotateRefreshToken và hàm băm trong RefreshTokenService*

```
@Transactional
public RefreshTokenIssue rotateRefreshToken(String rawToken) {
    RefreshToken token = findByToken(rawToken).orElseThrow(() ->
        new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid or expired refresh
token"));
    if (token.getExpiryDate().isBefore(Instant.now())) {
        refreshTokenRepository.delete(token);
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Refresh token has expired");
    }
    User user = token.getUser();
    if (!user.isActive() || user.isDeleted()) {
        refreshTokenRepository.delete(token);
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, "User account is inactive or
deleted");
    }
    refreshTokenRepository.delete(token);
    return issue(user);
}

private String hash(String rawToken) {
    byte[] digest = MessageDigest.getInstance("SHA-256")
            .digest(rawToken.getBytes(StandardCharsets.UTF_8));
    return Base64.getUrlEncoder().withoutPadding().encodeToString(digest);
}
```

7.3. Kiểm chứng

Test #8 xác nhận làm mới thành công, refresh token mới khác token cũ, và dùng lại token cũ theo thứ tự nhận 401 với `success=false`. Bài test rollback ở mục 4.7.3 kiểm trạng thái cơ sở dữ liệu sau lỗi chèn token thay thế qua proxy Spring: token cũ còn nguyên, người dùng không mất phiên. Như đã phân tích ở mục 4.7.3, nhánh token hết hạn hoặc tài khoản không hợp lệ gọi `delete` rồi ném `RuntimeException`, nên thao tác xoá bị rollback cùng giao dịch; báo cáo không khẳng định token hết hạn được dọn bền vững.

Bảng 7.1. *Đặc tính và giới hạn của refresh token*

Tính chất	Có / Không	Căn cứ
Lưu token dưới dạng băm	Có	`RefreshTokenService.hash`, `issue`
Token dùng một lần (xoay vòng)	Có	Test #8: dùng lại tuần tự bị từ chối; chưa kiểm hai yêu cầu đồng thời
Xoay token nguyên tử khi lỗi	Có	Test rollback qua proxy (28/10): DB giữ hash cũ; đối chứng thành công sau khi gỡ ràng buộc
Nhiều thiết bị đăng nhập song song	Không	`createRefreshToken` gọi `deleteByUser` trước khi phát token mới
Phát hiện và thu hồi khi token bị dùng lại	Không	Token đã xoá chỉ trả 401, không thu hồi các token khác của người dùng
Thu hồi access token trước hạn	Chỉ khi khoá tài khoản	Filter nạp lại người dùng ở mỗi request và từ chối tài khoản không hoạt động

7.4. Phân tích tình huống tấn công

Bảng 7.2. *Một số tình huống lộ token và hành vi của hệ thống*

Tình huống	Hành vi hiện tại	Cải tiến đề xuất
Lộ access token	Kẻ tấn công dùng được tối đa 15 phút hoặc tới khi tài khoản bị khoá	Giữ hạn ngắn; cân nhắc danh sách thu hồi theo `jti` nếu cần thu hồi tức thì
Lộ refresh token, kẻ tấn công dùng trước	Kẻ tấn công nhận cặp token mới; lần làm mới tiếp theo của người dùng thật bị 401 và bị đăng xuất	Token family: khi token đã dùng bị dùng lại, thu hồi cả họ token và buộc đăng nhập lại
Lộ refresh token, người dùng thật dùng trước	Token bị lộ trở nên vô hiệu	Đã đạt nhờ xoay vòng

Tình huống	Hành vi hiện tại	Cải tiến đề xuất
Hai yêu cầu làm mới đồng thời với cùng token	Chưa kiểm chứng; repository không khoá hàng, cả hai có thể cùng đọc thấy token trước khi một bên xoá	Xoá có điều kiện và kiểm số dòng bị xoá (`DELETE ... WHERE token = ? RETURNING`), hoặc `SELECT ... FOR UPDATE`
Lộ cơ sở dữ liệu	Chỉ có hash SHA-256, không dùng được trực tiếp	Đã đạt
Lộ token qua XSS trên client	Client lưu cả hai token trong `localStorage`, có thể bị đọc nếu trang dính XSS; client đã thoát ký tự mọi dữ liệu người dùng	Giữ refresh token trong cookie `HttpOnly`, `Secure`, `SameSite`

7.5. Giới hạn và dùng ở đâu

Thiết kế hiện tại chủ động thay các refresh token cũ khi đăng nhập mới; chưa có phiên theo thiết bị hay token family. Test hiện có kiểm dùng lại token theo thứ tự, chưa chứng minh tính dùng một lần khi hai yêu cầu làm mới đến cùng lúc. Access token hết hạn theo cấu hình; tài khoản bị khoá bị chặn ngay khi filter nạp lại người dùng. Cơ chế được dùng ở `POST /api/v1/auth/login` và `POST /api/v1/auth/refresh`; phía client, hàm `refreshAccessToken` tự gọi làm mới khi gặp 401 và kết nối lại STOMP bằng access token mới (mục 10.5.3).

PHẦN 3: XÂY DỰNG ĐỒ ÁN TỔNG HỢP

Phần này trình bày NextChat như một sản phẩm: yêu cầu, thiết kế cơ sở dữ liệu, hợp đồng giao tiếp (Chương 8); triển khai, kiểm thử, client thử nghiệm và trạng thái mã nguồn cuối cùng (Chương 9).

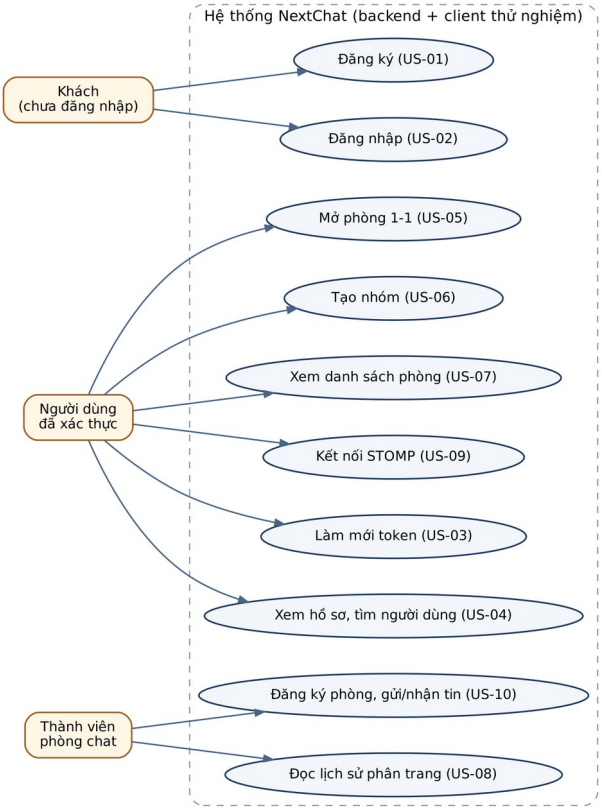
CHƯƠNG 8. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

8.1. Mô tả bài toán

NextChat là backend cho ứng dụng nhắn tin: xác định người dùng, kiểm soát quyền theo phòng, lưu dữ liệu nhất quán và truyền tin giữa các thành viên trong thời gian thực. Thiết kế phải xử lý ba điểm khó đã nêu ở Chương 2: tranh chấp khi tạo phòng 1-1, chi phí phân trang khi lịch sử lớn, và xác thực trên kênh WebSocket dài hạn. Hiệu năng chỉ được kết luận trong các phép đo có điều kiện nêu rõ.

8.1.1. Tác nhân

- Khách: người chưa đăng nhập; chỉ được đăng ký, đăng nhập và tải trang client.
- Người dùng đã xác thực: có access token hợp lệ; được xem hồ sơ, tìm người dùng, mở phòng, tạo nhóm, xem danh sách phòng, kết nối STOMP.
- Thành viên phòng: người dùng đã xác thực và có bản ghi trong `conversation_participants` của phòng với `left_at IS NULL`; được đọc lịch sử, đăng ký nhận tin và gửi tin vào phòng.



Hình 8.1. *Sơ đồ ca sử dụng của NextChat*

8.1.2. Yêu cầu chức năng theo user story

Bảng 8.1. *Yêu cầu chức năng theo user story*

Mã	Vai trò	Mong muốn	Mục đích và tiêu chí chấp nhận
US-01	Khách	Đăng ký với username, email, password, fullName	Có định danh hợp lệ; mật khẩu băm BCrypt; trùng username/email → 409; dữ liệu sai → 400 kèm lỗi theo trường.
US-02	Khách có tài khoản	Đăng nhập bằng username và password	Nhận accessToken (15 phút) và refreshToken (7 ngày); sai thông tin → 401 "Invalid credentials".
US-03	Người dùng	Làm mới phiên bằng refreshToken	Nhận cặp token mới; token cũ dùng lại → 401. Chưa bảo đảm cho hai yêu cầu đồng thời.
US-04	Người dùng	Xem hồ sơ (`/users/me`) và tìm người dùng (`/users/search?q=`)	Thấy thông tin và vai trò của mình; tìm theo username, email hoặc họ tên, chỉ tài khoản hoạt động; q quá 100 ký tự → 400.
US-05	Người dùng	Mở phòng 1-1 với người khác	Gọi nhiều lần hoặc hai người cùng mở cho nhau đều nhận cùng một phòng; tự mở với chính mình → 400.

Mã	Vai trò	Mong muốn	Mục đích và tiêu chí chấp nhận
US-06	Người dùng	Tạo nhóm với tên và danh sách thành viên	Người tạo là OWNER, thành viên là MEMBER; thành viên không tồn tại hoặc không hoạt động → 404, không tạo nhóm thiếu.
US-07	Người dùng	Xem danh sách phòng đang tham gia	Mỗi phòng có tên hiển thị, tin cuối và số tin chưa đọc; sắp xếp theo cập nhật gần nhất.
US-08	Thành viên	Đọc lịch sử theo trang	Tin xoá mềm bị loại; người ngoài → 403; `size` ngoài 1–100 → 400.
US-09	Người dùng	Kết nối STOMP	Gửi JWT ở khung CONNECT; thiếu hoặc sai → kết nối bị từ chối.
US-10	Thành viên	Đăng ký `/topic/room/{id}` và gửi tới `/app/room/{id}`	Mọi subscriber nhận cùng `MessageDTO`; tin được lưu trước khi phát.
US-11	Người ngoài phòng	(Không được) đọc, đăng ký, gửi vào phòng khác	Bị từ chối ở REST, SUBSCRIBE và service gửi tin. Trên `main` còn rủi ro R1, R2 (mục 4.10.5).

8.1.3. Yêu cầu phi chức năng

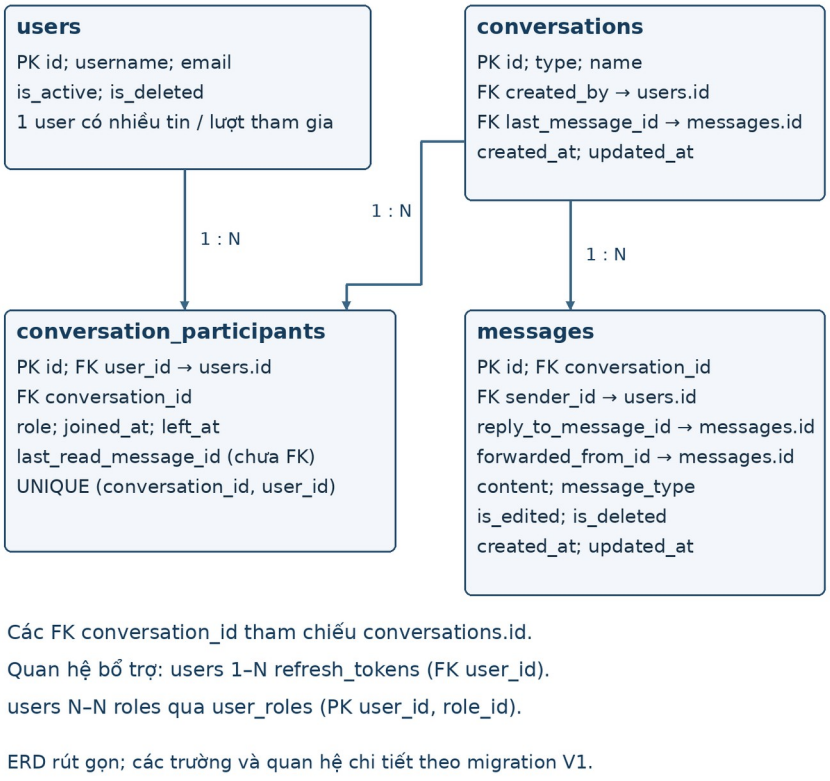
Bảng 8.2. *Yêu cầu phi chức năng và cách đáp ứng*

Nhóm yêu cầu	Yêu cầu	Cách đáp ứng / trạng thái
Bảo mật	Không lưu mật khẩu và refresh token dạng thô; không lộ thông tin tồn tại tài khoản khi đăng nhập sai	BCrypt; SHA-256 cho refresh token; thông điệp lỗi chung "Invalid credentials"
Bảo mật	Khoá ký JWT không nằm trong mã nguồn	Biến môi trường; `JwtService` từ chối khoá mặc định ngoài dev/test
Nhất quán	Không tạo trùng phòng 1-1; lưu tin và cập nhật tin cuối cùng nhau	Khoá advisory; `@Transactional`
Khả năng bảo trì	Lược đồ có phiên bản; mã chia theo tính năng	Flyway; package-by-feature
Khả năng kiểm thử	Test chạy trên PostgreSQL thật, không phụ thuộc cơ sở dữ liệu cục bộ	Testcontainers
Hiệu năng	Phân trang lịch sử; tránh N+1 ở danh sách phòng	Hàm PL/pgSQL; `@EntityGraph`. Chưa đo tải API
Khả năng mở rộng	Có thể chạy nhiều instance	Chưa đáp ứng: simple broker trong bộ nhớ; cần broker relay

8.2. Thiết kế cơ sở dữ liệu

8.2.1. Sơ đồ quan hệ thực thể

Migration V1 khai báo 20 bảng nghiệp vụ, trong khi backend hiện ánh xạ sáu entity cốt lõi: `User`, `Role`, `RefreshToken`, `Conversation`, `ConversationParticipant` và `Message`. Các bảng cho tệp, trạng thái đã đọc, cuộc gọi, thông báo chưa có API tương ứng. Hình 8.2 là ERD rút gọn của phần cốt lõi, bám theo khoá trong SQL.



Hình 8.2. *ERD cốt lõi rút gọn; reply_to_message_id và forwarded_from_id tự tham chiếu messages, last_message_id thuộc conversations (nguồn: migration V1)*

8.2.2. Từ điển dữ liệu các bảng cốt lõi

Bảng 8.3. *Bảng users*

Cột	Kiểu	Ràng buộc / ý nghĩa
`id`	BIGINT IDENTITY	Khoá chính
`username`	VARCHAR(50)	NOT NULL, UNIQUE; chỉ mục `idx_users_username`

Cột	Kiểu	Ràng buộc / ý nghĩa
email	VARCHAR(150)	NOT NULL, UNIQUE; chỉ mục idx_users_email
password_hash	VARCHAR(255)	NOT NULL; chuỗi BCrypt
full_name	VARCHAR(150)	NOT NULL
avatar_url, bio, phone_number	VARCHAR(500/255/20)	Có thể NULL
status	VARCHAR(10)	Mặc định OFFLINE; CHECK ∈ {ONLINE, OFFLINE, AWAY, BUSY}
last_seen_at	TIMESTAMP	Có thể NULL; chưa được cập nhật vì chưa có presence
is_active, is_deleted	BOOLEAN	Mặc định TRUE / FALSE; dùng ở mọi bước nạp người dùng
created_at, updated_at	TIMESTAMP	Trigger trg_users_updated_at cập nhật updated_at

Bảng 8.4. *Bảng refresh_tokens, roles và user_roles*

Cột	Kiểu	Ràng buộc / ý nghĩa
refresh_tokens.id	BIGINT IDENTITY	Khoá chính
refresh_tokens.user_id	BIGINT	FK → users(id) ON DELETE CASCADE
refresh_tokens.token	VARCHAR(255)	NOT NULL, UNIQUE; lưu hash SHA-256 Base64URL (43 ký tự), ánh xạ vào trường tokenHash
refresh_tokens.expiry_date, created_at	TIMESTAMP	Hạn và thời điểm phát
roles.name	VARCHAR(30)	UNIQUE; ROLE_USER, ROLE_ADMIN
user_roles(user_id, role_id)	BIGINT, BIGINT	Khoá chính kép; FK CASCADE; ánh xạ @ManyToMany(fetch = EAGER)

Bảng 8.5. *Bảng conversations và conversation_participants*

Cột	Kiểu	Ràng buộc / ý nghĩa
conversations.type	VARCHAR(10)	CHECK ∈ {PRIVATE, GROUP}
conversations.name, avatar_url	VARCHAR	NULL với phòng PRIVATE (tên hiển thị lấy từ người còn lại)
conversations.created_by	BIGINT	FK → users(id)
conversations.last_message_id	BIGINT	FK → messages(id) ON DELETE SET NULL, thêm bằng ALTER TABLE sau khi tạo messages để tránh phụ thuộc vòng
participants.conversation_id, user_id	BIGINT	FK CASCADE; UNIQUE (conversation_id, user_id) tên uq_conversation_user
participants.role	VARCHAR(10)	CHECK ∈ {OWNER, ADMIN, MEMBER}

Cột	Kiểu	Ràng buộc / ý nghĩa
participants.nickname, is_muted, mute_until, is_pinned	–	Thiết kế sẵn cho tuỳ chọn cá nhân, chưa có API
participants.last_read_message_id	BIGINT	Chưa có FK; dùng để tính số tin chưa đọc
participants.joined_at, left_at	TIMESTAMP	left_at IS NULL nghĩa là còn trong phòng

Bảng 8.6. *Bảng messages*

Cột	Kiểu	Ràng buộc / ý nghĩa
conversation_id	BIGINT	FK → conversations(id) CASCADE
sender_id	BIGINT	FK → users(id); chỉ mục idx_messages_sender
message_type	VARCHAR(10)	Mặc định TEXT; CHECK ∈ {TEXT, IMAGE, VIDEO, AUDIO, FILE, STICKER, SYSTEM}
content	TEXT	Nội dung; SendMessageRequest giới hạn 10.000 ký tự
reply_to_message_id, forwarded_from_id	BIGINT	FK tự tham chiếu messages(id) ON DELETE SET NULL
is_edited, is_deleted, deleted_at	BOOLEAN, TIMESTAMP	Xoá mềm; MessageDTO thay nội dung tin đã xoá bằng "Tin nhắn đã bị thu hồi"
created_at, updated_at	TIMESTAMP	Chỉ mục idx_messages_conversation_created (conversation_id, created_at)

8.2.3. Ràng buộc, chỉ mục và hàm nội tại quan trọng

- uq_conversation_user: một người không thể được thêm hai lần vào cùng một phòng.
- idx_messages_conversation_created (conversation_id, created_at): phục vụ truy vấn lịch sử; truy vấn sắp xếp thêm id DESC nên kế hoạch cần một bước sắp xếp tăng dần (mục 6.6).
- idx_participants_user: phục vụ tìm các phòng của một người dùng.
- Trigger set_updated_at: thay cho ON UPDATE CURRENT_TIMESTAMP của MySQL mà PostgreSQL không có.

- Hàm `get_or_create_direct_room`: khoá advisory theo cặp id chuẩn hoá (Chương 5).
- Hàm `get_room_messages`: lịch sử chưa xoá mềm kèm thông tin người gửi và tổng số (Chương 6).

8.3. Hợp đồng REST API

8.3.1. Vỏ phản hồi thống nhất

Mọi endpoint REST trả về `ApiResponse<T>` – một record gồm `success`, `message`, `data` và `timestamp`. Lỗi từ MVC (`GlobalExceptionHandler`) và từ Security (`JsonAuthenticationEntryPoint`, `JsonAccessDeniedHandler`) cũng dùng đúng vỏ này với `success=false`, nên client chỉ cần một cách đọc phản hồi.

Đoạn mã 8.1. *Vỏ phản hồi ApiResponse thành công và lỗi kiểm tra dữ liệu*

```
{ "success": true,  "message": "Login successful", "data": { ... },
  "timestamp": "2026-10-28T03:14:28.983Z" }

{ "success": false, "message": "Validation error",
  "data": { "username": "size must be between 3 and 50",
            "email": "must be a well-formed email address" },
  "timestamp": "2026-10-28T03:15:02.110Z" }
```

8.3.2. Danh sách endpoint

Bảng 8.7. *Đặc tả chín endpoint REST*

Phương thức / đường dẫn	Đầu vào chính	`data` trong ApiResponse	HTTP thường gặp
POST `/api/v1/auth/register`	`RegisterRequest`: username, email, password, fullName	`UserDTO`	201; 400; 409
POST `/api/v1/auth/login`	`LoginRequest`: username, password	`JwtResponse`	200; 400; 401
POST `/api/v1/auth/refresh`	`RefreshTokenRequest`: refreshToken	`JwtResponse`	200; 400; 401; 403
GET `/api/v1/users/me`	Bearer JWT	`UserProfileDTO`	200; 401; 404
GET `/api/v1/users/search?q=`	Bearer JWT; q tối đa 100 ký tự	`List<UserDTO>`	200; 400; 401
GET `/api/v1/rooms`	Bearer JWT	`List<RoomDTO>`	200; 401

Phương thức / đường dẫn	Đầu vào chính	`data` trong ApiResponse	HTTP thường gặp
POST `/api/v1/rooms/direct`	Bearer JWT; `targetUserId`	`RoomDTO`	200; 400; 401; 404
POST `/api/v1/rooms/group`	Bearer JWT; `groupName`, `memberIds`	`RoomDTO`	201; 400; 401; 404
GET `/api/v1/rooms/{roomId}/messages`	Bearer JWT; `page ≥ 0`; `size` 1..100 (mặc định 0 và 20)	`Page<MessageDTO>`	200; 400; 401; 403; 404

8.3.3. Ví dụ yêu cầu và phản hồi

Các ví dụ dưới đây minh hoạ hợp đồng theo các record DTO trong mã nguồn; giá trị id, thời gian và token là minh hoạ.

Đoạn mã 8.2. *Đăng ký – POST /api/v1/auth/register*

```
// Yêu cầu
{ "username": "alice01", "email": "alice01@example.com",
  "password": "StrongPassword123!", "fullName": "Alice Nguyen" }
// Phản hồi 201
{ "success": true, "message": "User registered successfully",
  "data": { "id": 3, "username": "alice01", "email": "alice01@example.com",
            "fullName": "Alice Nguyen", "avatarUrl": null, "bio": null,
            "status": "OFFLINE", "lastSeenAt": null, "createdAt": "2026-10-
28T03:10:00Z" },
  "timestamp": "..." }
```

Đoạn mã 8.3. *Đăng nhập – POST /api/v1/auth/login*

```
// Yêu cầu
{ "username": "alice01", "password": "StrongPassword123!" }
// Phản hồi 200
{ "success": true, "message": "Login successful",
  "data": { "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZTAxIi...",
            "refreshToken": "q8Zk3...43 ký tự Base64URL...",
            "tokenType": "Bearer", "expiresIn": 900,
            "user": { "id": 3, "username": "alice01", ... } },
  "timestamp": "..." }
// Sai mật khẩu → 401
{ "success": false, "message": "Invalid credentials", "data": null, "timestamp": "..." }
```

Đoạn mã 8.4. *Hồ sơ hiện tại – GET /api/v1/users/me*

```
{ "success": true, "message": "User profile loaded",
  "data": { "id": 3, "username": "alice01", "email": "alice01@example.com",
            "fullName": "Alice Nguyen", "avatarUrl": null, "bio": null,
            "phoneNumber": null, "status": "OFFLINE", "lastSeenAt": null,
            "roles": ["ROLE_USER"], "createdAt": "...", "updatedAt": "..." },
  "timestamp": "..." }
```

Đoạn mã 8.5. *Mở phòng 1-1 – POST /api/v1/rooms/direct*

```
// Yêu cầu
{ "targetUserId": 4 }
// Phản hồi 200 (gọi lại nhiều lần vẫn trả cùng id)
{ "success": true, "message": "Direct room retrieved or created",
  "data": { "id": 7, "type": "PRIVATE", "name": "Bob Tran", "avatarUrl": null,
            "createdById": 3, "lastMessage": null, "unreadCount": 0,
            "participants": [
              { "id": 13, "userId": 3, "username": "alice01", "fullName": "Alice Nguyen",
                "avatarUrl": null, "role": "OWNER", "nickname": null, "joinedAt": "..." },
              { "id": 14, "userId": 4, "username": "bob01", "fullName": "Bob Tran",
                "avatarUrl": null, "role": "MEMBER", "nickname": null, "joinedAt": "..." } ],
            "createdAt": "...", "updatedAt": "..." },
  "timestamp": "..." }
```

Đoạn mã 8.6. *Tạo nhóm – POST /api/v1/rooms/group*

```
// Yêu cầu: tên trường là groupName (không phải name)
{ "groupName": "Nhóm 24 - Đồ án", "memberIds": [4, 5] }
// Phản hồi 201: data là RoomDTO với "type": "GROUP", người tạo có role OWNER
// Thành viên không tồn tại → 404
{ "success": false, "message": "One or more group members were not found or are inactive", ... }
```

Đoạn mã 8.7. *Lịch sử – GET /api/v1/rooms/7/messages?page=0&size=10*

```
{ "success": true, "message": "Messages retrieved successfully",
  "data": {
    "content": [
      { "id": 2, "roomId": 7, "senderId": 4, "senderUsername": "bob01",
        "senderFullName": "Bob Tran", "senderAvatarUrl": null, "messageType":
"TEXT",
        "content": "Chào Alice", "replyToMessageId": 1, "isEdited": false,
        "isDeleted": false, "createdAt": "...", "updatedAt": "..." },
      { "id": 1, "roomId": 7, "senderId": 3, ... "content": "Chào Bob" } ],
    "page": { "size": 10, "number": 0, "totalElements": 2, "totalPages": 1 } },
  "timestamp": "..." }
// Người ngoài phòng → 403 "You are not a participant in this room"
```

8.4. Hợp đồng STOMP

Hợp đồng STOMP tách khỏi chín endpoint HTTP. Bảng 8.8 mô tả hợp đồng trên nhánh `main` sau commit `332d069`; dạng đích dùng dấu gạch chéo. Các đích `/app/chat.sendMessage` và `/topic/rooms.{roomId}` từng xuất hiện trong tài liệu nháp không thuộc hợp đồng; dạng dấu chấm `/app/room.{roomId}`, `/topic/room.{roomId}` là hợp đồng của bản 39b4d75 và bản sửa cục bộ 28/10.

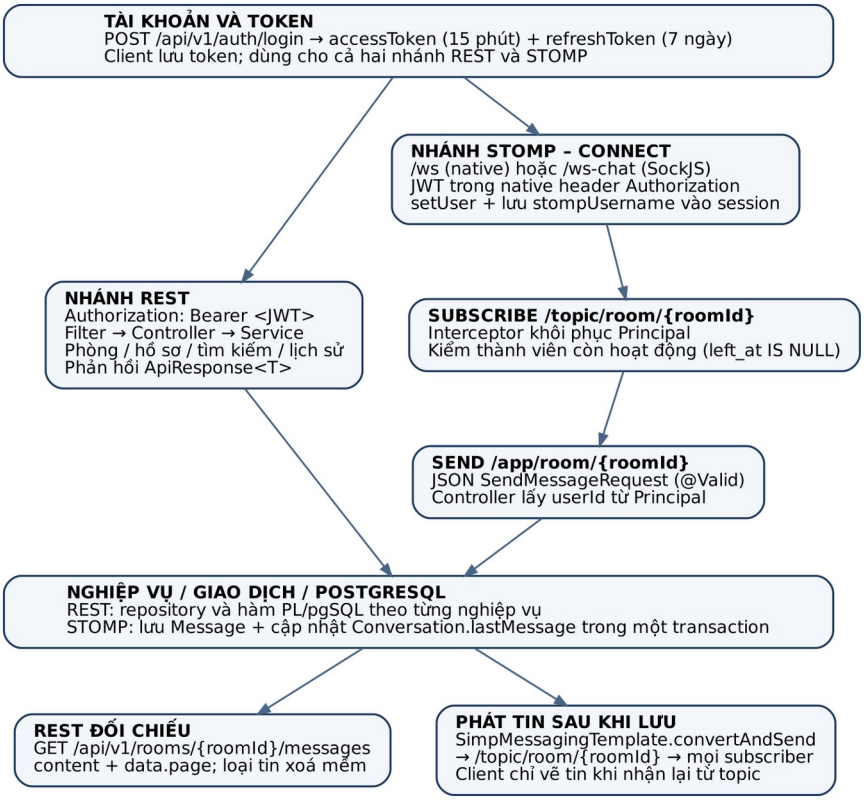
Bảng 8.8. *Endpoint kết nối và destination STOMP trên main*

Thao tác / địa chỉ	Đầu vào / đầu ra	Kiểm soát
Kết nối `/ws` (native), `/ws-chat` (SockJS)	Bắt tay HTTP được `permitAll`	JWT được xác minh tại khung CONNECT, không ở bắt tay

Thao tác / địa chỉ	Đầu vào / đầu ra	Kiểm soát
CONNECT	Native header `Authorization: Bearer <JWT>`; dự phòng `access-token`	Xác minh JWT và người dùng; lưu `stompUsername` vào phiên. Thiếu/sai → lỗi, kết nối bị đóng
SUBSCRIBE `/topic/room/{roomId}`	Đăng ký nhận tin của phòng	Khôi phục Principal; kiểm thành viên còn hoạt động. Đích không khớp mẫu chưa bị chặn (R1)
SEND `/app/room/{roomId}`	`SendMessageRequest`: `content` (bắt buộc, ≤ 10.000 ký tự), `messageType` (≤ 10 ký tự), `replyToMessageId`	Service kiểm thành viên, tin trả lời cùng phòng, loại tin; lưu `Message` và `lastMessage`
MESSAGE `/topic/room/{roomId}`	`MessageDTO` (không bọc ApiResponse)	Phát bằng `SimpMessagingTemplate` sau khi service trả thành công
SEND trực tiếp `/topic/...`	Không thuộc hợp đồng	Chưa bị interceptor chặn trên `main` (R2); bản sửa cục bộ đã chặn

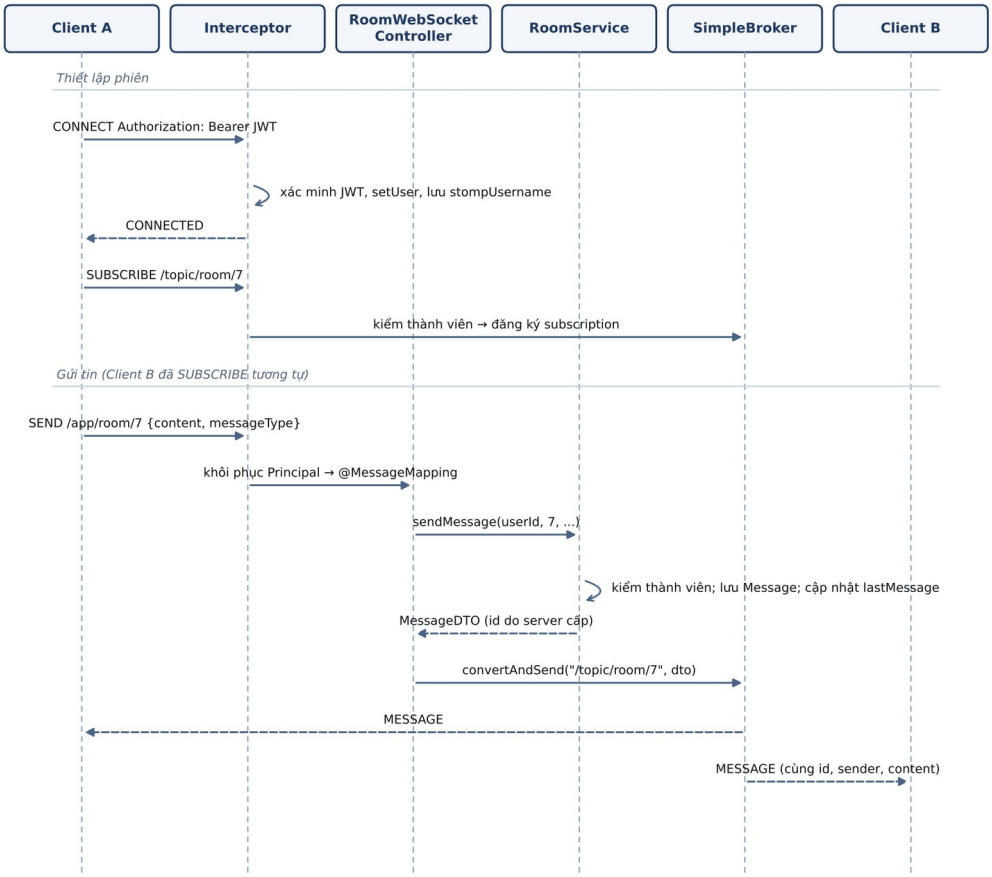
8.5. Luồng xử lý dữ liệu và điều khiển

Hình 8.3 tách hai đường vào: REST đi qua filter bảo mật và MVC; STOMP đi qua interceptor và message controller. Cả hai dùng chung access token và chung tầng nghiệp vụ.



Hình 8.3. *Luồng xác thực, gửi, lưu và nhận tin trên main (nguồn: mã nguồn commit 54ab54e)*

Với nhánh REST, sau khi vượt qua lớp bảo mật, request tới `DispatcherServlet` và Bean Validation; dữ liệu sai định dạng rẽ sang `GlobalExceptionHandler` để trả 400 kèm lỗi theo trường. Controller gọi `RoomService` hoặc `RefreshTokenService` qua proxy khi cần giao dịch; `AuthController` và `UserController` hiện gọi một số repository trực tiếp. Với nhánh STOMP, controller lấy người dùng từ `Principal`, service kiểm quyền phòng và lưu dữ liệu trước khi controller phát kết quả. Hình 8.4 là sơ đồ tuần tự của luồng gửi tin hoàn chỉnh giữa hai client.



Hình 8.4. *Sơ đồ tuần tự gửi và nhận một tin nhắn giữa hai client qua STOMP*

Ba điểm thiết kế đáng chú ý trong luồng gửi tin:

- Máy chủ là nguồn sự thật duy nhất: client gửi không tự vẽ tin của mình lên màn hình mà đợi nhận lại từ topic, nên người gửi và người nhận luôn thấy cùng `id` và `createdAt` do máy chủ cấp.
- Lưu trước, phát sau: nếu service ném lỗi (không phải thành viên, tin trả lời sai phòng, loại tin không hợp lệ), giao dịch rollback và không có gì được phát.
- Lỗi nghiệp vụ trên kênh STOMP chưa có cơ chế trả lỗi tuỳ chỉnh: ngoại lệ trong `@MessageMapping` không đi qua `GlobalExceptionHandler`. Kịch bản đầu-cuối 28/10 kiểm trường hợp người ngoài gửi tin bằng cách xác nhận cơ sở dữ liệu không đổi và không có MESSAGE được phát, thay vì giả định luôn có khung ERROR. Bổ sung

`@MessageExceptionHandler` gửi lỗi về hàng đợi riêng của người dùng là hướng cải tiến.

8.6. Tiểu kết chương

Thiết kế của NextChat gồm 11 user story, 9 endpoint REST và hợp đồng STOMP 4 loại khung; dữ liệu cốt lõi nằm trong 6 bảng có entity trên tổng số 20 bảng được thiết kế sẵn. Các quyết định thiết kế quan trọng – vỏ phản hồi thống nhất, danh tính lấy từ Principal, lưu trước phát sau, khoá advisory, phân trang trong hàm SQL – đều có lý do và minh chứng ở các chương trước. Điểm thiết kế còn mở là kiểm soát đích trên kênh STOMP và cơ chế trả lỗi nghiệp vụ cho client WebSocket.

CHƯƠNG 9. TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ

9.1. Tích hợp các thành phần theo từng luồng

Chương 4–7 phân tích từng cơ chế riêng lẻ. Mục này ghép chúng lại theo các luồng nghiệp vụ, mỗi nhận định gắn với phạm vi đã kiểm.

Luồng khởi động

Trên cơ sở dữ liệu trống, Flyway tạo lược đồ V1, V2 trước khi JPA kiểm tra các entity. Cấu hình ngoài điều khiển kết nối và HikariCP khi `database.yml` được nạp. Profile `dev` chạy `SeedDataConfig` tạo vai trò `ROLE_USER`, `ROLE_ADMIN` và hai tài khoản `alice`, `bob` với mật khẩu lấy từ `NEXTCHAT_SEED_PASSWORD`; profile `test` của bộ kiểm thử không nạp dữ liệu mẫu này. Tomcat nhúng mở cổng 8080, phục vụ REST, hai endpoint STOMP và trang client.

Luồng tài khoản và token

Đăng ký kiểm tra trùng username và email trước khi lưu, gán vai trò `ROLE_USER` (tạo vai trò nếu chưa có), băm mật khẩu bằng BCrypt. Đăng nhập dùng `AuthenticationManager` với `DaoAuthenticationProvider` để so mật khẩu, rồi cấp access token và refresh token. Làm mới thay bản băm cũ bằng bản băm mới trong giao dịch; test tuần tự xác nhận token cũ bị từ chối.

Luồng danh sách phòng và tạo phòng

`GET /api/v1/rooms` dùng `@Transactional(readOnly = true)` và `@EntityGraph` để dựng DTO trong phạm vi giao dịch, kèm số tin chưa đọc từ truy vấn tổng hợp. `POST /api/v1/rooms/direct` gọi hàm có khoá advisory; phép kiểm đầu-cuối đảo chiều trả cùng id. `POST /api/v1/rooms/group` tạo phòng và mọi thành viên trong một giao dịch, từ chối toàn bộ nếu có thành viên không hợp lệ.

Luồng lịch sử có phân quyền

Tại `GET /api/v1/rooms/{roomId}/messages`, sau khi filter xác nhận người dùng đã đăng nhập, `RoomService.getRoomMessages` kiểm tra người gọi có trong danh sách thành viên còn hoạt động. Nếu không, service ném

`ResponseStatusException(FORBIDDEN)` và `GlobalExceptionHandler` trả JSON 403. Nếu hợp lệ, service gọi `get_room_messages` và ánh xạ sang `MessageDTO`.

Luồng gửi và nhận tin thời gian thực

Client kết nối, gửi JWT ở CONNECT, đăng ký topic của phòng, gửi tin tới `/app/room/{id}`; interceptor khôi phục danh tính, controller gọi service lưu tin và cập nhật tin cuối, rồi phát `MessageDTO` cho mọi subscriber (Hình 8.4).

9.2. Kết quả kiểm chứng

9.2.1. Kiểm thử tự động ngày 28/10/2026

Lần chạy `./mvnw clean verify` ngày 28/10 trên bản sửa cục bộ có 18 + 7 + 2 + 1 = 28 bài JUnit. Test #17–18 gọi service; test REST dùng MockMvc; kịch bản đầu-cuối là script riêng. Các dòng dưới đây trích nguyên văn từ `maven-review-verify.log`; tên gói và số bài khớp với Surefire XML.

Đoạn mã 9.1. *Trích log Maven ngày 28/10/2026 (bản sửa cục bộ)*

```
[INFO] Running
com.quoocscuongwf.nextchat.security.JwtStompChannelInterceptorTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.049 s
[INFO] Running com.quoocscuongwf.nextchat.api.AuthAndRoomIntegrationTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.70 s
[INFO] Running
com.quoocscuongwf.nextchat.evidence.SpringMechanismIntegrationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.630 s
[INFO] Running com.quoocscuongwf.nextchat.room.RoomServiceUnitTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.356 s
[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Môi trường của lần chạy: Ubuntu Linux, OpenJDK 21.0.12.1, Maven Wrapper, Docker 28.3.3, PostgreSQL 16.15; profile `test`, cổng tạm và cơ sở dữ liệu riêng. Bản nền `39b4d75` từng đạt 22 bài JUnit ngày 25/10 nhưng kiểm tra bằng client thật phát hiện lỗi mất Principal; bản sửa được giữ trong hồ sơ. Lần 28/10 giữ nguyên bản sửa sản phẩm, bổ sung phép thử để đạt 28 JUnit và 15 kiểm tra smoke.

9.2.2. Phản hồi lỗi khi truy cập API được bảo vệ không có token

Đoạn mã 9.2. *GET /api/v1/users/me không có JWT (test #1)*

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{ "success": false, "message": "Authentication required", "data": null,
  "timestamp": "2026-10-28T01:15:22.410Z" }
```

9.2.3. Trao đổi giữa hai client, kiểm quyền và đối chiếu cơ sở dữ liệu

Script `tools/e2e_check.py` dựng cơ sở dữ liệu riêng, chạy jar ứng dụng và dùng ba tài khoản thử: A, B và một người ngoài. A và B mở hai kết nối `/ws`, cùng đăng ký một phòng. A gửi tin; B gửi tin trả lời tham chiếu id tin của A. Cả hai subscriber nhận cùng id, người gửi và nội dung cho từng tin; lịch sử REST và tin cuối của phòng đối chiếu được dữ liệu; sau đó script đọc trực tiếp PostgreSQL.

Kết quả ngày 28/10: 15/15 kiểm tra đạt, gồm: truyền tin hai chiều; cơ sở dữ liệu chứa hai id đã nhận; người ngoài bị chặn SUBSCRIBE và bị chặn đọc lịch sử REST; JWT thiếu hoặc sai bị từ chối; SUBSCRIBE wildcard, SEND thẳng vào broker và SEND tới đích không khớp đều bị từ chối; hai request HTTP tạo phòng ngược chiều trả cùng id. Người ngoài cũng thử SEND vào đúng `/app/room.{roomId}` (đích dạng dấu chấm của bản cục bộ): service từ chối vì không là thành viên, không có MESSAGE phát tới A, số tin vẫn là 2 và tin cuối vẫn là tin trả lời của B.

Đoạn mã 9.3. *Bản ghi messages sau trao đổi (database-messages.json, 28/10)*

```
[ { "id": 1, "sender_id": 1, "reply_to_message_id": null },
  { "id": 2, "sender_id": 2, "reply_to_message_id": 1 } ]
```

JSON lịch sử thật lấy từ lần chạy (tệp `history-response.json`), tại thời điểm vừa gửi tin đầu tiên, có `senderUsername` dạng `alice_8ea908ee` (hậu tố ngẫu nhiên để không trùng giữa các lần chạy), nội dung "Nhom 24: hai client WebSocket that 8ea908ee" và `page.totalElements = 1`. Các id chỉ có ý nghĩa trong cơ sở dữ liệu tạm của lần chạy.

Giới hạn: smoke test chỉ dùng vài tài khoản và một tiến trình ứng dụng; không đo p50/p95/p99, thông lượng, tỷ lệ lỗi dưới tải hay khả năng phục hồi. SockJS và trường hợp token hết hạn giữa một kết nối dài chưa được kiểm đầu-cuối. Quan trọng hơn, kịch bản này chạy trên bản sửa cục bộ với đích dạng dấu chấm, chưa chạy trên `main` hiện tại (mục 9.5).

9.3. Bảng đối chiếu kỹ thuật và minh chứng

Bảng 9.1. *Bảng đối chiếu kỹ thuật và minh chứng*

Nội dung	Vị trí mã nguồn	Minh chứng và giới hạn
Starter / auto-config	`pom.xml`, `application.yaml`	Bean thật: HikariDataSource singleton; điều kiện Hikari khớp (28/10). Không suy từ việc không thấy `@Bean`.
IoC / DI / Profile	Constructor, config, `SeedDataConfig`	`RoomService` singleton và là proxy; `SeedDataConfig` vắng ở test. Chưa thử xung đột `@Primary`/`@Qualifier`.
Cấu hình ngoài	`database.yml`, `JwtService`	Biến môi trường; kiểm khoá tối thiểu 32 byte và chặn khoá mặc định.
MVC / Validation / lỗi	Controller, DTO, `GlobalExceptionHandler`	201/400/401/403/404/409 theo từng trường hợp REST.
Security / JWT	Filter, `UserDetailsService`	`/api/**` yêu cầu đăng nhập; kiểm `active`/`deleted` mỗi request.
JPA / nạp quan hệ	`ConversationRepository`, `RoomService`	`@EntityGraph` và đếm chưa đọc tổng hợp; chưa đếm SQL toàn request.
Giao dịch	`RefreshTokenService`, `RoomService`	Rollback qua proxy khi chèn thay thế lỗi (28/10).
Flyway / validate	V1, V2	Migration và khởi tạo JPA thành công trên PostgreSQL 16.15.
Kiểm thử	18 tích hợp + 5 đơn vị trên `main`	28/28 JUnit và 15 smoke trên bản cục bộ (28/10); `main` cần chạy lại.
Khoá tạo phòng	`get_or_create_direct_room`	#17 cùng chiều; E2E ngược chiều; lab đối chứng có/bỏ khoá.
Phân trang	`get_room_messages`, `PageImpl`	#14 (tổng, trang rỗng, `size=101`); lab 1k/10k/100k; không đo API.
Refresh rotation	SHA-256 + Base64URL	#8 dùng lại tuần tự → 401; chưa khoá hàng khi đồng thời.
WebSocket / STOMP	Config, interceptor, controller	Hai client thật, outsider SEND và kiểm DB (28/10, bản cục bộ); trên `main`: 5 test đơn vị, rủi ro R1, R2.
Client web	`static/index.html`, `static/app.js`	Kịch bản kiểm thử thủ công (Bảng 9.4); chưa có test tự động.

9.4. Client web thử nghiệm

9.4.1. Kiến trúc và tổ chức mã

Client là ứng dụng một trang gồm hai tệp tĩnh trong `nextchat/src/main/resources/static`, do Nguyễn Phú Tài xây dựng trong commit `4699986`. Spring Boot tự phục vụ thư mục `static`, nên client chạy cùng cổng với backend: không cần máy chủ web riêng và lời gọi REST không gặp giới hạn CORS. Thư viện được nạp qua CDN: Tailwind CSS cho giao diện, sockjs-client 1.x và stomp.js 2.3.3 cho kết nối WebSocket/STOMP.



Hình 9.1. *Bố cục giao diện client web: cột phòng, khung chat và công cụ trình diễn giao thức (vẽ lại theo index.html)*

Bảng 9.2. *Tệp và thành phần của client thử nghiệm*

Tệp / vùng	Nội dung	Ghi chú
`index.html` (262 dòng)	Thanh tiêu đề (trạng thái WebSocket, người dùng hiện tại); vùng xác thực (tab Đăng nhập/Đăng ký, ô API base URL); cột trái (tìm người dùng, tạo nhóm, danh sách phòng); cột giữa (tiêu đề phòng, luồng tin, ô nhập); cột phải (JWT inspector, nhật ký REST/STOMP); hộp thoại tạo nhóm	Cột phải phục vụ demo và bảo vệ: hiển thị token đang dùng và từng khung gửi/nhận

Tệp / vùng	Nội dung	Ghi chú
app.js (592 dòng)	Trạng thái toàn cục (stompClient, activeRoom, currentUser); hàm apiFetch; nhóm hàm xác thực; nhóm hàm WebSocket; nhóm hàm phòng và tin nhắn; escapeHtml	Mọi lệnh REST và khung STOMP được ghi qua logProtocol
localStorage	nextchat_access_token, nextchat_refresh_token, nextchat_user, nextchat_api_base	Khôi phục phiên khi tải lại trang

Luồng khởi động: khi trang tải xong (DOMContentLoaded), client đọc token và thông tin người dùng đã lưu. Nếu có, onAuthenticatedSuccess gọi GET /api/v1/users/me để lấy hồ sơ mới nhất, tải danh sách phòng rồi mở kết nối STOMP. Nếu dữ liệu lưu bị hỏng, client đăng xuất và quay về màn hình xác thực.

9.4.2. Luồng xác thực và quản lý JWT

Mọi lệnh REST đi qua apiFetch: hàm ghép API base URL, gắn Authorization: Bearer <accessToken> (trừ các lệnh xác thực được đánh dấu skipAuth), đọc vỏ ApiResponse và trả về trường data. Khi một lệnh cần xác thực nhận 401, hàm gọi refreshAccessToken rồi thử lại đúng một lần; tham số retry = false ở lần thử lại ngăn vòng lặp vô hạn. Nếu làm mới thất bại, client đăng xuất.

Đoạn mã 9.4. *Trích hàm apiFetch trong app.js*

```
const accessToken = localStorage.getItem(STORAGE_KEYS.ACCESS_TOKEN);
if (accessToken && !options.skipAuth) {
    headers['Authorization'] = `Bearer ${accessToken}`;
}
const response = await fetch(url, { ...options, headers });
// ... đọc payload JSON
if (response.status === 401 && !options.skipAuth && retry) {
    const refreshed = await refreshAccessToken(true);
    if (refreshed) {
        return apiFetch(path, options, false);
    }
}
```

- Đăng ký: handleRegister gửi username, email, fullName, password; thành công thì chuyển sang tab đăng nhập và điền sẵn username; lỗi 400/409 hiển thị thông điệp từ ApiResponse.

- Đăng nhập: `handleLogin` gọi `/auth/login`; `saveAuthSession` lưu hai token và người dùng vào `localStorage`, hiển thị `expiresIn` trên JWT inspector.
- Làm mới: `refreshAccessToken` gửi refresh token, lưu cặp mới (vì máy chủ đã thu hồi token cũ), rồi mở lại kết nối STOMP bằng access token mới – cần thiết vì interceptor chỉ xác minh JWT ở khung CONNECT.
- Đăng xuất: `logout` ngắt STOMP và xoá token, người dùng khỏi `localStorage`.

9.4.3. Danh sách phòng, lịch sử và tạo phòng

Bảng 9.3. *Ánh xạ chức năng giao diện với REST API*

Chức năng	Hàm trong app.js	Endpoint
Đăng ký tài khoản	`handleRegister`	POST `/api/v1/auth/register`
Đăng nhập	`handleLogin`	POST `/api/v1/auth/login`
Làm mới token	`refreshAccessToken`	POST `/api/v1/auth/refresh`
Hiển thị người dùng hiện tại	`onAuthenticatedSuccess`	GET `/api/v1/users/me`
Tìm người dùng	`searchUsers`	GET `/api/v1/users/search?q=`
Nhắn tin 1-1	`openDirectRoom`	POST `/api/v1/rooms/direct`
Tạo phòng nhóm	`createGroupRoom`	POST `/api/v1/rooms/group`
Danh sách phòng	`loadRooms`, `renderRoomList`	GET `/api/v1/rooms`
Lịch sử tin nhắn	`loadRoomHistory`	GET `/api/v1/rooms/{roomId}/messages?page=0&size=50`

`renderRoomList` hiển thị tên phòng, loại phòng, tin cuối và số tin chưa đọc. Khi chọn phòng, `selectRoom` ghi nhận phòng đang mở, đăng ký topic của phòng rồi tải lịch sử. Vì API trả lịch sử mới nhất trước, `loadRoomHistory` sắp xếp lại tăng dần để hiển thị đúng chiều đọc; tin của chính người dùng nằm bên phải. Client hiện chỉ tải trang đầu 50 tin.

9.4.4. Kết nối STOMP thời gian thực

Đoạn mã 9.5. *Kết nối, đăng ký và gửi tin trong app.js trên main*

```
const socket = new SockJS(`${getApiBase()}/ws-chat?access_token=$
{encodeURIComponent(accessToken)}`);
stompClient = Stomp.over(socket);
stompClient.connect({ Authorization: `Bearer ${accessToken}`, access_token: accessToken },
                    onConnected, onError);

currentRoomSubscription = stompClient.subscribe(`/topic/room/${roomId}`, (frame) => {
    const body = JSON.parse(frame.body);
    appendMessageBubble(body?.data !== undefined ? body.data : body);
    loadRooms();   // cập nhật tin cuối ở danh sách phòng
});

stompClient.send(`/app/room/${activeRoom.id}`, {},
    JSON.stringify({ content, messageType, replyToMessageId: null }));
```

Khi đổi phòng, subscription cũ được huỷ trước khi đăng ký phòng mới để không nhận tin của phòng khác. Khi gửi, client không tự thêm tin vào màn hình; tin chỉ xuất hiện khi máy chủ đã lưu và phát lại qua topic. Header `Authorization` ở khung CONNECT là thứ máy chủ thực sự đọc; tham số `access_token` trên URL SockJS và header `access_token` (dấu gạch dưới) không được interceptor sử dụng.

9.4.5. Kịch bản kiểm thử thủ công

Client chưa có test tự động. Bảng 9.4 là kịch bản kiểm tra trên máy demo với hai cửa sổ trình duyệt (một cửa sổ ẩn danh) đăng nhập hai tài khoản khác nhau; nhóm đánh dấu kết quả trong biên bản chạy demo ở Phụ lục 6.

Bảng 9.4. *Kịch bản kiểm thử thủ công client web*

TT	Thao tác	Kết quả mong đợi
1	Mở `http://localhost:8080`, đăng ký tài khoản A	Báo đăng ký thành công, chuyển sang tab đăng nhập
2	Đăng nhập A	Hiện tên người dùng và TTL access token; trạng thái WebSocket "Connected"
3	Ở cửa sổ ẩn danh, đăng ký và đăng nhập B	Như bước 2
4	A tìm B, bấm "Nhắn tin"	Phòng PRIVATE xuất hiện; nhật ký ghi `SUBSCRIBE /topic/room/{id}`
5	B tìm A, bấm "Nhắn tin"	Mở đúng id phòng của bước 4
6	A gửi một tin	Tin hiện ở cả hai cửa sổ với cùng giờ; tin cuối ở danh sách phòng được cập nhật
7	Tải lại trang của B	Phiên được khôi phục; lịch sử hiển thị đúng thứ tự cũ đến mới
8	Xoá `nextchat_access_token` trong `localStorage` rồi tải lại danh sách phòng	Nhận 401, client tự gọi `/auth/refresh`, nhật ký AUTH ghi rotation, thao tác tiếp tục

TT	Thao tác	Kết quả mong đợi
9	Mở hộp thoại tạo nhóm, nhập tên và id thành viên	Hiện tại: lỗi 400 do client gửi `name` thay vì `groupName` (mục 9.6); sau khi sửa: nhóm được tạo

9.5. Trạng thái mã nguồn cuối cùng (06/11/2026)

9.5.1. Những gì Pull Request #8 đưa vào main

Sáng 06/11/2026, Lê Quốc Cường đẩy commit `1046984` (08:26) lên nhánh `phu-tai` và hợp nhất Pull Request #8 vào `main` (08:27, commit `54ab54e`). Nhờ đó toàn bộ phần việc của Nguyễn Phú Tài – vốn nằm riêng trên nhánh `phu-tai` từ 30/10 – đã có trên nhánh mặc định. Bảng 9.5 tổng hợp ba commit mà PR #8 mang vào.

Bảng 9.5. *Ba commit được hợp nhất qua Pull Request #8*

Commit	Tác giả, thời điểm	Tệp thay đổi	Nội dung
`4699986`	Nguyễn Phú Tài, 30/10 14:06	`app.js` (+592), `index.html` (+262), `docker-compose.yml`, `WebSecurityConfig.java`	Client web thử nghiệm; danh sách `permitAll` cho tài nguyên tĩnh và bắt tay WebSocket; đổi giá trị mặc định, tên volume và healthcheck của Compose
`332d069`	Lê Quốc Cường, 30/10 15:04	Interceptor, `RoomWebSocketController`, `app.js`, `index.html`, test interceptor	Đồng bộ đích STOMP sang dạng `/app/room/{id}`, `/topic/room/{id}`; controller dùng `SimpMessagingTemplate`; client bỏ `roomId` khỏi payload
`1046984`	Lê Quốc Cường, 06/11 08:26	`WebSocketConfig`, interceptor, test interceptor	Thêm origin `localhost:8080`, `127.0.0.1:8080`; lưu `stompUsername` vào phiên ở CONNECT và khôi phục Principal ở khung sau; 1 test mới

9.5.2. Điều đã và chưa được kiểm chứng trên main

- Đã có: 23 bài kiểm thử trong mã nguồn (18 tích hợp, 5 đơn vị). Commit `1046984` có test đơn vị riêng cho việc khôi phục danh tính.
- Chưa có minh chứng chạy: báo cáo này được viết lại trong cùng ngày PR #8 được hợp nhất. Trong môi trường dùng để viết lại báo cáo, kho

Maven Central và Docker Hub không truy cập được, nên không thể build và chạy lại bộ kiểm thử trên `54ab54e`. Báo cáo không ghi kết quả "đạt" cho `main` khi chưa có log.

- Việc cần làm trước buổi bảo vệ: chạy `./mvnw clean verify` trên `main` và lưu log; cập nhật `tools/e2e_check.py` sang đích dạng gạch chéo và chạy lại; chạy kịch bản thủ công Bảng 9.4 bằng client web; áp dụng bản vá R1, R2 kèm hai bài test (mục 4.10.5).

9.6. Danh sách hạn chế và rủi ro tồn đọng

Bảng 9.6. *Hạn chế và rủi ro tồn đọng trên main, mức độ và đề xuất xử lý*

Mã	Mô tả	Mức độ	Đề xuất	Phụ trách đề xuất
R1	SUBSCRIBE đích không khớp mẫu (ví dụ `/topic/room/*`) không bị kiểm tra thành viên	Cao	Từ chối mọi SUBSCRIBE không khớp chính xác mẫu phòng	Cường
R2	SEND thẳng vào `/topic/...` không bị chặn, có thể phát tin giả không qua lưu trữ	Cao	Chỉ chấp nhận SEND khớp `^/app/room/(\d+)$`	Cường
R3	`createGroupRoom` trong client gửi `name` thay vì `groupName` → 400	Trung bình	Đổi khoá JSON thành `groupName`	Tài
R4	Giá trị mặc định của Compose khác `application.yaml` và `.env.example`	Trung bình	Thống nhất một bộ giá trị; không ghi cứng mật khẩu	Tài
R5	Client nối access token vào URL SockJS (`?access_token=`); máy chủ không đọc, token có thể vào log truy cập	Trung bình	Bỏ tham số URL, chỉ giữ header CONNECT	Tài
R6	`formatRoomDisplayName` so sánh cả `p.id` (id bản ghi thành viên) với id người dùng	Thấp	Chỉ so sánh `p.userId`	Tài
R7	Khôi phục Principal nạp lại người dùng từ DB ở mỗi khung STOMP	Thấp	Đo chi phí; cân nhắc cách giữ accessor của transport như bản cục bộ	Cường
R8	Lỗi nghiệp vụ trên kênh STOMP không được báo về client	Thấp	`@MessageExceptionHandler` gửi lỗi tới `/user/queue/errors`	Cường, Hải
R9	`AuthController`, `UserController` gọi repository trực tiếp	Thấp	Tách `AuthService`, `UserService`	Hải
R10	Hai yêu cầu làm mới token đồng thời chưa được bảo vệ	Trung bình	Xoá có điều kiện và kiểm số dòng, hoặc khoá hàng	Hải

Mã	Mô tả	Mức độ	Đề xuất	Phụ trách đề xuất
R11	Chưa có Dockerfile, CI/CD, load test, presence	–	Hướng phát triển (Chương 12)	Tài, cả nhóm

9.7. Tiểu kết chương

Các cơ chế được tích hợp thành năm luồng nghiệp vụ hoàn chỉnh. Minh chứng mạnh nhất là lần chạy 28/10: 28/28 JUnit, 15/15 smoke với hai client thật và đối chiếu cơ sở dữ liệu. Client web hoàn thiện việc trình diễn đầu-cuối và đã được hợp nhất vào `main` qua PR #8. Trạng thái `main` cuối cùng cần được chạy lại kiểm thử, và hai rủi ro kiểm soát truy cập trên kênh STOMP cần được vá trước khi coi kênh thời gian thực là hoàn chỉnh.

PHẦN 4: TỔ CHỨC THỰC HIỆN, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

Phần này ghi nhận chi tiết ai đã làm gì, khi nào và với kết quả ra sao (Chương 10); đánh giá đồ án so với mục tiêu, phân tích các lỗi đã gặp và khai báo việc sử dụng AI (Chương 11); và đề ra hướng phát triển (Chương 12).

CHƯƠNG 10. TỔ CHỨC THỰC HIỆN VÀ PHẦN VIỆC CHI TIẾT CỦA TỪNG THÀNH VIÊN

Chương này mô tả phần việc của từng thành viên dựa trên ba nguồn đối chiếu: tài liệu phân công của nhóm (bao gồm tệp `task.md` từng có trong kho và bảng nhiệm vụ của từng vai trò), lịch sử Git của kho mã (tác giả, thời điểm, tệp và số dòng thay đổi của từng commit) và các Pull Request trên GitHub. Số commit và số dòng là dữ liệu đối chiếu, không phải thước đo công sức; một commit lớn có thể chứa nhiều ngày làm việc, còn một commit sửa vài dòng có thể là kết quả của nhiều giờ gỡ lỗi.

10.1. Mô hình tổ chức và quy trình làm việc

10.1.1. Vai trò

- Người 1 – Lê Quốc Cường (trưởng nhóm): kiến trúc tổng thể, môi trường và cấu hình dùng chung, kênh WebSocket/STOMP, tích hợp, kiểm thử và tổng hợp báo cáo; chủ kho mã, duyệt và hợp nhất Pull Request.
- Người 2 – Huỳnh Thanh Hải: Backend Auth, REST API và cơ sở dữ liệu PostgreSQL/PL/pgSQL.
- Người 3 – Nguyễn Phú Tài: ứng dụng client kết nối backend và phần hạ tầng/kiểm thử (Frontend Client App, DevOps/QA).

10.1.2. Quy trình Git và Pull Request

Kho mã do Lê Quốc Cường tạo trên GitHub ngày 22/09/2026 dưới tài khoản `QuoocsCuongwf`. Nhóm làm việc theo mô hình nhánh tính năng: mỗi mảng việc trên một nhánh riêng, hoàn thành thì mở Pull Request vào `main`. Bảng 10.1 liệt kê các nhánh và Bảng 10.2 liệt kê toàn bộ tám Pull Request.

Bảng 10.1. *Các nhánh trong kho mã*

Nhánh	Người làm chính	Nội dung
`feat/auth-register-login-jwt`	Cường; Hải tiếp tục	Entity người dùng, Security, JWT, seed dữ liệu, kết nối DB, tái cấu trúc package, vai trò
`feat/DevOps`	Cường	Docker Compose, `.env.example`
`feat/complete-task-api`	Hải; Cường hợp nhất	Hoàn thiện auth, refresh token, REST phòng và lịch sử, Flyway, test tích hợp
`feat/auth-websocket`	Cường	Xử lý tệp `.env`, `WebSocketConfig`, controller STOMP đầu tiên
`phu-tai`	Tài; Cường đồng bộ	Client web, cấu hình truy cập, Compose; đồng bộ endpoint và sửa Principal
`main`	–	Nhánh mặc định; nhận mọi thay đổi qua PR

Bảng 10.2. *Tám Pull Request được hợp nhất vào main*

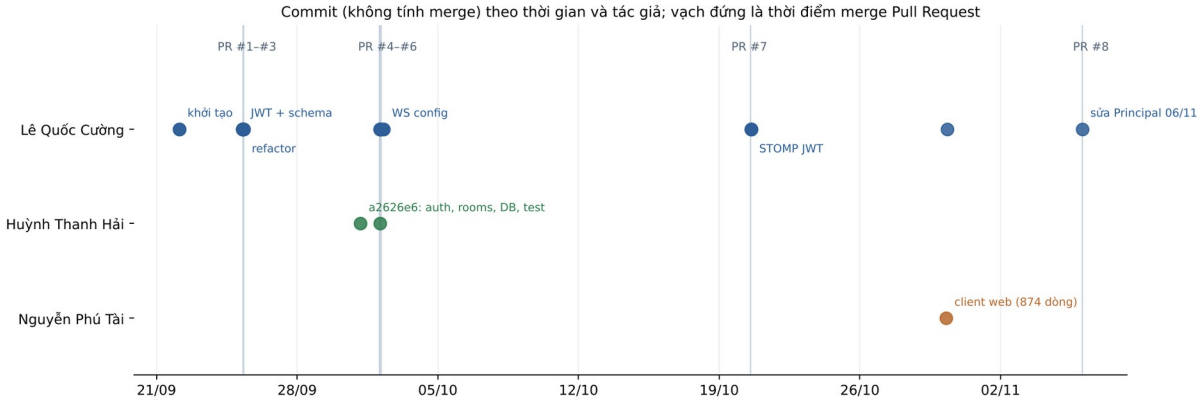
PR	Thời điểm hợp nhất	Nhánh nguồn	Nội dung chính
#1	25/09 14:10	`feat/auth-register-login-jwt`	User entity, Security, seed `dev`, kết nối DB (`0c94c66`, `993ada7`, `5dae9ab`, `296e342`)
#2	25/09 14:10	`feat/DevOps`	Docker Compose và `.env.example` (`dfa4a3b`)
#3	25/09 15:01	`feat/auth-register-login-jwt`	Tái cấu trúc package-by-feature + common (`67fb244`)
#4	02/10 09:23	`feat/auth-register-login-jwt`	JWT và vai trò người dùng (`7c56340`)
#5	02/10 10:16	`feat/complete-task-api`	Phần việc của Hải (`a2626e6`, `8bb8f4f`), sau khi Cường kéo `main` vào nhánh (`16352f1`)
#6	02/10 11:06	`feat/auth-websocket`	Xử lý tệp `.env` và `.gitignore` (`1a441b5`, `87717e5`, `87fe186`, `acf0844`)
#7	20/10 20:16	`feat/auth-websocket`	`WebSocketConfig`, `RoomWebSocketController` (`deabd07`, `c3acb71`)
#8	06/11 08:27	`phu-tai`	Client web của Tài (`4699986`), đồng bộ endpoint (`332d069`), sửa Principal và origin (`1046984`)

Ngoài các PR, Lê Quốc Cường có hai commit "merge main into branch" (`16352f1`, `5ea2fc2`) để kéo thay đổi mới của `main` vào nhánh tính năng trước khi mở PR, giải quyết xung đột tại nhánh thay vì tại `main`. Huỳnh Thanh Hải có một commit hợp nhất cục bộ `9aeb4aa` (28/09) để đưa nhánh `feat/auth-register-`

`login-jwt` vào bản `main` trên máy mình trước khi làm tiếp. Một số commit được đẩy thẳng lên `main` không qua PR: `85eb2a5` (02/10) và ba commit `d20e734`, `3eaa91a`, `39b4d75` (20/10, sau PR #7).

10.1.3. Quy ước commit

Phần lớn commit theo quy ước Conventional Commits với tiền tố loại và phạm vi: `feat(auth)`, `feat(security)`, `feat(socket)`, `feat(chat)`, `feat(api)`, `feat(DevOps)`, `refactor(project)`, `chore(application)`, `test(chat)`, `docs(report)`, `fix(socket)`. Quy ước này giúp đọc lịch sử nhanh và là cơ sở để báo cáo nhóm các commit theo hạng mục ở các mục sau. Hai commit của Cường (`7c56340`, `85eb2a5`) có dòng `Co-authored-by: Copilot`, cho thấy có dùng GitHub Copilot hỗ trợ khi viết; đây là lý do trang Contributors của GitHub hiển thị tài khoản Copilot (Phụ lục 3). Copilot là công cụ, không phải thành viên nhóm.

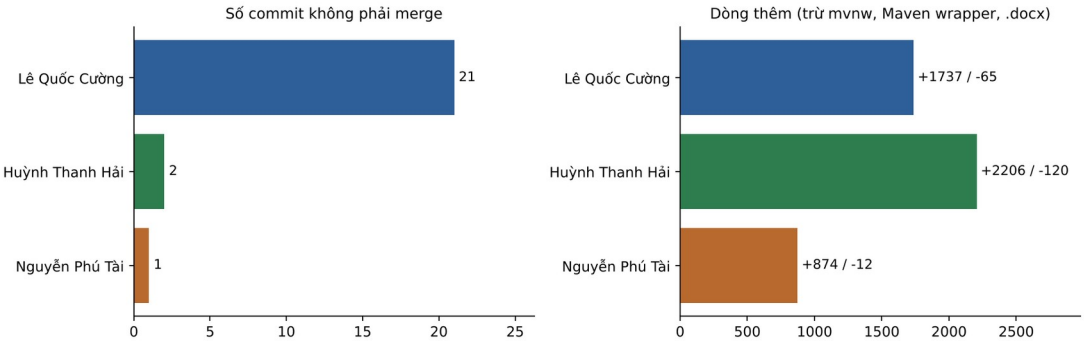


Hình 10.1. *Commit theo thời gian của từng thành viên và thời điểm hợp nhất Pull Request (dữ liệu: git log toàn bộ nhánh)*

10.2. Tổng quan đóng góp theo dấu vết Git

Bảng 10.3. *Thống kê commit và dòng mã theo tác giả (không tính commit merge; trừ mvnw, Maven wrapper và tệp .docx)*

Thành viên	Commit (không merge)	Dòng thêm	Dòng xoá	Ghi chú
Lê Quốc Cường	21	1.737	65	Thêm 10 commit merge (8 PR, 2 merge main vào nhánh); tài khoản `Le Quoc Cuong`, `lequoccuong`, `QuoocsCuongwf`
Huỳnh Thanh Hải	2	2.206	120	`a2626e6` gồm 47 tệp, trong đó 479 dòng test tích hợp; thêm 1 commit merge cục bộ
Nguyễn Phú Tài	1	874	12	`4699986` gồm 4 tệp, trong đó 854 dòng client web



Hình 10.2. *Số commit và số dòng thêm theo thành viên*

Cách đọc đúng các con số:

- Số dòng của Hải lớn nhất vì toàn bộ phần REST, phòng chat, migration và bộ test được gom trong một commit; số commit của Cường lớn nhất vì anh làm nhiều thay đổi nhỏ về cấu hình, tích hợp và quản lý nhánh.
- Một phần dòng mã được tính cho người di chuyển tệp: lược đồ 20 bảng do Cường viết (292 dòng, commit `5dae9ab`) được Hải chuyển thành `V1__messenger_schema.sql`; Git ghi nhận đây là đổi tên kèm 112 dòng Hải thêm (hai hàm PL/pgSQL).

- Các con số không phản ánh khảo sát, thiết kế, kiểm chứng và viết báo cáo – những việc không để lại commit tương ứng (tuần 5 không có commit nào).
- Nhóm không quy đổi các con số thành phần trăm đóng góp.

10.3. Lê Quốc Cường – Người 1: trưởng nhóm, kiến trúc, WebSocket, tích hợp và kiểm thử

10.3.1. Vai trò và nhiệm vụ được giao

- Khảo sát công nghệ, đề xuất đề tài và lập kế hoạch; phân công công việc (tuần 1–2).
- Khởi tạo dự án, chọn phiên bản nền tảng, cấu hình bảo mật và JWT ban đầu, cấu hình dùng chung và môi trường phát triển.
- Thiết kế và xây dựng kênh thời gian thực WebSocket/STOMP có xác thực JWT.
- Quản lý kho mã: tạo nhánh, duyệt và hợp nhất Pull Request, giải quyết xung đột.
- Tích hợp phần việc của các thành viên, kiểm thử tích hợp, kiểm chứng đầu-cuối và tổng hợp báo cáo.

10.3.2. Nhật ký commit

Bảng 10.4. *Các commit không phải merge của Lê Quốc Cường*

Ngày	Commit	Nội dung	Tệp / dòng
22/09	`15b9513`	Initial commit trên GitHub (LICENSE, `.gitignore`)	Khởi tạo kho
22/09	`6ab071a`	Dự án Spring Boot 4.1.1 / Java 21: `pom.xml` với các starter, Maven Wrapper, `NextchatApplication`, `application.yaml`	10 tệp; `pom.xml` 136 dòng
25/09	`0c94c66`	Entity `User` và `UserRepository`	2 tệp, +56
25/09	`993ada7`	`PasswordConfig` (BCrypt), `WebSecurityConfig` ban đầu	2 tệp, +64

Ngày	Commit	Nội dung	Tệp / dòng
25/09	`5dae9ab`	`JwtService`, `JwtAuthenticationFilter`, `DatabaseUserDetailsService`, `AuthController` v1, `SeedDataConfig` (profile `dev`), JJWT trong pom, lược đồ SQL 20 bảng	8 tệp, +556
25/09	`296e342`	Cấu hình kết nối cơ sở dữ liệu	2 tệp, +8 / −1
25/09	`dfa4a3b`	`docker-compose.yml` PostgreSQL 16 và `.env.example`	2 tệp, +26
25/09	`67fb244`	Tái cấu trúc package-by-feature + common; `UserController` đầu tiên	10 tệp, +39 / −1
25/09	`7c56340`	Vai trò `Role`/`RoleRepository`, `ApiResponse`, DTO đăng ký/đăng nhập, `UserProfileResponse` (có Copilot đồng tác giả)	13 tệp, +128 / −32
02/10	`1a441b5`, `87717e5`, `87fe186`, `acf0844`	Đường dẫn tệp `.env`; `.gitignore` cho IDE và `target`; gỡ `.env` bị theo dõi khỏi kho	4 commit nhỏ
02/10	`85eb2a5`	Nạp cấu hình môi trường cục bộ (`optional:file:.env`)	1 tệp, +1
02/10	`deabd07`	`WebSocketConfig`: endpoint `/ws`, tiền tố `/app`, broker `/topic`, `/queue`, tiền tố `/user`, đăng ký interceptor vào kênh inbound	1 tệp, +43
02/10	`c3acb71`	`RoomWebSocketController` với `@MessageMapping("/room.{roomId}")`	1 tệp, +40
20/10	`d20e734`	`JwtStompChannelInterceptor` (92 dòng), `RoomService.sendMessage`, `SendMessageRequest`, thêm endpoint `/ws-chat` có SockJS	5 tệp, +141
20/10	`3eaa91a`	4 test đơn vị interceptor và test tích hợp #18	2 tệp, +127
20/10	`39b4d75`	Đồng bộ tài liệu báo cáo nháp với tính năng đã cài đặt	1 tệp .docx
30/10	`332d069`	Đồng bộ đích STOMP với client: `/app/room/{id}`, `/topic/room/{id}`, `SimpMessagingTemplate`; sửa `app.js`, `index.html`, test	5 tệp, +24 / −19
06/11	`1046984`	Khôi phục Principal từ thuộc tính phiên; thêm origin cổng 8080; 1 test mới	3 tệp, +43 / −6

10.3.3. Phân tích các hạng mục chính

a) Khởi tạo dự án và chọn nền tảng

Commit `6ab071a` tạo dự án từ Spring Initializr với Spring Boot 4.1.1 và Java 21 – bản mới nhất tại thời điểm đó. Danh sách starter ban đầu đã bao gồm đủ năm

nhóm khả năng cần cho đề tài (webmvc, security, data-jpa, validation, websocket) cùng các starter kiểm thử tương ứng, cho thấy phạm vi kỹ thuật được xác định từ đầu. Quyết định chọn Boot 4 thay vì 3.5 khiến nhóm phải làm quen với các thay đổi ở Bảng 1.4, nhưng đổi lại học đúng thế hệ hiện hành.

b) Nền móng bảo mật và JWT

Trong ngày 25/09, Cường dựng toàn bộ nền móng bảo mật: entity `User`, `PasswordConfig` dùng BCrypt, `WebSecurityConfig` với chuỗi filter không trạng thái, `JwtService` ký token bằng HMAC qua JJWT, `JwtAuthenticationFilter` đọc Bearer token, `DatabaseUserDetailsService` nạp người dùng hoạt động. Bản `AuthController` đầu tiên trả về một token sau khi đăng ký hoặc đăng nhập; `SeedDataConfig` gắn `@Profile("dev")` tạo hai tài khoản mẫu để thử. Đây là khung mà Huỳnh Thanh Hải tiếp tục phát triển thành luồng xác thực hoàn chỉnh có refresh token (mục 10.4).

c) Lược đồ cơ sở dữ liệu ban đầu

Cũng trong commit `5dae9ab`, Cường viết tệp `sql/messenger_schema_postgres.sql` (292 dòng) định nghĩa 20 bảng cho toàn bộ miền nghiệp vụ nhắn tin: người dùng và xác thực, bạn bè và chặn, hội thoại, tin nhắn và các bảng phụ, thông báo, cuộc gọi. Một số quyết định thiết kế được ghi ngay trong chú thích SQL: dùng `VARCHAR` kèm `CHECK` thay cho kiểu `ENUM` gốc của PostgreSQL để ánh xạ thẳng sang JPA không cần thư viện phụ; dùng trigger `set_updated_at` thay cho `ON UPDATE CURRENT_TIMESTAMP` của MySQL; thêm khoá ngoại `last_message_id` sau khi tạo bảng `messages` để tránh phụ thuộc vòng. Lược đồ này được Hải chuyển thành migration V1.

d) Hạ tầng phát triển và quản lý bí mật

`dfa4a3b` tạo Docker Compose cho PostgreSQL 16 với healthcheck và `.env.example` chứa giá trị mẫu. Ngày 02/10, chuỗi bốn commit nhỏ xử lý việc tệp `.env` từng bị đưa vào kho: chỉnh đường dẫn nạp, mở rộng `.gitignore` (IDE, `target`, mọi biến thể `.env` trừ `.env.example`), và gỡ `.env` khỏi chỉ mục Git

(87fe186). Đây là thao tác bảo mật quan trọng: bí mật không được nằm trong lịch sử mã nguồn.

e) Tái cấu trúc package-by-feature

Commit 67fb244 chuyển các gói config, repository, security thành cấu trúc auth, user, room, common/... (mục 3.6). Việc tái cấu trúc sớm – trước khi Hải bắt đầu phần lớn công việc – giúp tránh xung đột khi hai người cùng sửa một gói, và tạo ranh giới rõ cho việc chia việc.

f) Vai trò người dùng và vỏ phản hồi chung

Commit 7c56340 thêm entity Role với quan hệ nhiều-nhiều qua user_roles, RoleRepository, record ApiResponse<T> – vỏ phản hồi mà về sau mọi endpoint REST dùng chung – và tách DTO đăng ký, đăng nhập ra thư mục dto. Commit có Copilot đồng tác giả.

g) Kênh WebSocket/STOMP

Kênh thời gian thực được xây trong ba bước. Ngày 02/10: WebSocketConfig khai báo endpoint /ws, tiền tố ứng dụng, simple broker và đăng ký interceptor vào kênh inbound; RoomWebSocketController đầu tiên nhận khung tại /room.{roomId} và phát bằng @SendTo. Ngày 20/10 (d20e734): thêm endpoint /ws-chat có SockJS; JwtStompChannelInterceptor xác thực JWT ở CONNECT và kiểm thành viên ở SUBSCRIBE; RoomService.sendMessage kiểm quyền, kiểm tin trả lời cùng phòng, chuẩn hoá loại tin, lưu tin và cập nhật tin cuối trong một giao dịch; SendMessageRequest với ràng buộc độ dài; controller lấy người gửi từ Principal. Ngày 30/10 và 06/11: đồng bộ đích với client và sửa giữ danh tính (mục g, i dưới đây).

h) Kiểm thử kênh STOMP

3eaa91a thêm JwtStompChannelInterceptorTest với 4 bài kiểm thử đơn vị dùng Mockito (CONNECT hợp lệ, thiếu token, SUBSCRIBE khi là thành viên và khi không là thành viên) và test #18 trong bộ tích hợp của Hải để khẳng định tin được lưu và lastMessage được cập nhật.

i) Tích hợp với client web

Khi client của Tài được đẩy lên nhánh `phu-tai` ngày 30/10, client dùng đích theo tài liệu nháp (`/app/chat.sendMessage`, `/topic/rooms.{roomId}`) và gửi kèm `roomId` trong payload, không khớp mapping của backend. Chưa tới một giờ sau (`332d069`, 15:04), Cường đồng bộ hai phía: đổi mapping controller sang `/room/{roomId}`, đổi cách phát sang `SimpMessagingTemplate`, sửa mẫu đích của interceptor, sửa test, và sửa `app.js`, `index.html` cho đúng hợp đồng mới. Sáng 06/11 (`1046984`), Cường xử lý tiếp hai trở ngại để demo chạy được trên client web: thêm origin cổng 8080 và khôi phục Principal ở các khung sau CONNECT; sau đó hợp nhất PR #8.

j) Quản lý kho và hợp nhất

Cường hợp nhất cả tám Pull Request trên GitHub và hai lần kéo `main` vào nhánh tính năng để giải quyết xung đột trước. Việc này giúp `main` luôn ở trạng thái đã hợp nhất sạch.

k) Rà soát, kiểm chứng và tổng hợp

Theo phân công, tích hợp, kiểm thử và tổng hợp báo cáo thuộc vai trò của Cường. Đợt rà soát tuần 7 của nhóm đối chiếu báo cáo với mã nguồn và chạy kiểm chứng: ngày 25/10 tái lập lỗi mất Principal bằng client thật, viết bản sửa cục bộ và test hồi quy (25 JUnit, 11 smoke); ngày 28/10 bổ sung test bean/điều kiện, test rollback qua proxy, test Mockito cho nhánh lỗi gửi tin, kịch bản đầu-cuối hai chiều và thí nghiệm SQL (28 JUnit, 15 smoke). Các bước này có hỗ trợ của công cụ AI và được khai báo ở mục 11.4.

10.3.4. Kết quả so với nhiệm vụ

Bảng 10.5. *Mức độ hoàn thành nhiệm vụ của Lê Quốc Cường*

Nhiệm vụ	Trạng thái	Minh chứng
Khởi tạo dự án, nền tảng, cấu hình dùng chung	Hoàn thành	`6ab071a`, `296e342`, `85eb2a5`
Bảo mật và JWT ban đầu	Hoàn thành	`993ada7`, `5dae9ab`, `7c56340`
Lược đồ cơ sở dữ liệu 20 bảng	Hoàn thành	`5dae9ab` (được chuyển thành V1)
Môi trường phát triển, quản lý bí mật	Hoàn thành	`dfa4a3b`, `87fe186`, `.gitignore`

Nhiệm vụ	Trạng thái	Minh chứng
Kiến trúc package-by-feature	Hoàn thành	`67fb244`
Kênh WebSocket/STOMP có xác thực	Hoàn thành, còn rủi ro R1, R2 trên `main`	`deabd07`, `c3acb71`, `d20e734`, `332d069`, `1046984`
Kiểm thử STOMP	Hoàn thành mức đơn vị trên `main`; đầu-cuối trên bản cục bộ	`3eaa91a`, `1046984`; log 28/10
Quản lý PR, tích hợp	Hoàn thành	PR #1–#8
Kiểm chứng và tổng hợp báo cáo	Hoàn thành; bản sửa cục bộ 28/10 chưa được đẩy lên kho	Hồ sơ minh chứng [10]

10.3.5. Hạn chế và bài học

- Bản sửa cục bộ ngày 25/10 và 28/10 (giữ accessor của transport, chặn SUBSCRIBE wildcard, chặn SEND vào broker, các test bổ sung) không được đẩy lên kho. Khi sửa lỗi Principal trên `main` ngày 06/11, Cường chọn một cách khác (khôi phục từ phiên) và hai kiểm soát còn lại chưa được đưa vào. Bài học: bản sửa đã kiểm chứng cần được đưa vào kho ngay dưới dạng một nhánh và PR, kể cả khi chưa hoàn chỉnh, để không bị "rơi" khi tích hợp.
- Hợp đồng STOMP thay đổi hai lần (dấu chấm sang gạch chéo) khi tích hợp với client, vì hợp đồng ban đầu chỉ nằm trong mã và trong tài liệu nháp đã lỗi thời. Bài học: hợp đồng giao tiếp nên được ghi trong README như hợp đồng REST, và client nên đọc từ đó.
- Hai commit ngày 02/10 (`deabd07`, `c3acb71`) đã tham chiếu `JwtStompChannelInterceptor` và `RoomService.sendMessage` – hai thành phần chỉ được commit ngày 20/10 (`d20e734`). Vì vậy nhánh `feat/auth-websocket` từ 02/10, và `main` trong khoảng 40 phút giữa lúc hợp nhất PR #7 (20:16) và lúc đẩy `d20e734` (20:55), không biên dịch được độc lập. Bài học: mỗi commit nên tự biên dịch và qua test; đây cũng là lý do cần CI chạy cho mỗi PR (mục 12.5).

- Tệp `.docx` của báo cáo nháp nằm trong kho mã (`39b4d75`); tài liệu nhị phân nên để ở nơi lưu trữ tài liệu riêng.

10.3.6. Kiến thức và kỹ năng rút ra

Hiểu cách Spring Boot tự cấu hình và cách ghi đè bằng bean riêng; hiểu chuỗi filter bảo mật và kênh thông điệp STOMP ở mức đủ để tìm ra lỗi mất Principal nằm ở việc tạo accessor mới; kỹ năng quản lý nhánh, giải quyết xung đột và tích hợp phần việc của người khác; kỹ năng thiết kế phép thử có đối chứng thay vì chỉ chạy cho "đạt".

10.4. Huỳnh Thanh Hải – Người 2: Backend Auth, REST API và cơ sở dữ liệu

10.4.1. Vai trò và nhiệm vụ được giao

Phân công của Hải được ghi trong tệp `task.md` mà Hải đưa vào kho cùng commit `a2626e6` rồi gỡ ra ở commit `8bb8f4f` sau khi hoàn thành. Nội dung nguyên văn phần nhiệm vụ chính:

Hoàn thiện luồng xác thực JWT (Đăng ký, Đăng nhập, Làm mới Token) dựa trên nhánh feat/auth-register-login-jwt.

Thiết kế và phát triển các REST API quản lý tài khoản, danh sách phòng chat và truy xuất lịch sử tin nhắn.

Quản lý kết nối cơ sở dữ liệu qua tệp database.yml và tối ưu truy vấn bằng các Stored Procedures/Functions bằng PLpgSQL.

Bảng 10.6. *Các REST endpoint được giao cho Huỳnh Thanh Hải (theo task.md) và trạng thái*

Phương thức	Endpoint	Chức năng	Đầu vào / đầu ra	Trạng thái
POST	`/api/v1/auth/register`	Đăng ký tài khoản	`RegisterRequest` / `UserDTO`	Hoàn thành
POST	`/api/v1/auth/login`	Đăng nhập, trả JWT	`LoginRequest` / `JwtResponse`	Hoàn thành
POST	`/api/v1/auth/refresh`	Làm mới access token	`RefreshTokenRequest` / `JwtResponse`	Hoàn thành, có xoay vòng

Phương thức	Endpoint	Chức năng	Đầu vào / đầu ra	Trạng thái
GET	`/api/v1/users/me`	Thông tin người dùng đang đăng nhập	Header Authorization / `UserProfileDTO`	Hoàn thành
GET	`/api/v1/users/search?q=`	Tìm theo tên, email	`List<UserDTO>`	Hoàn thành (thêm tìm theo username)
GET	`/api/v1/rooms`	Danh sách cuộc trò chuyện	`List<RoomDTO>`	Hoàn thành, có số tin chưa đọc
POST	`/api/v1/rooms/direct`	Tạo hoặc lấy phòng 1-1	`targetUserId` / `RoomDTO`	Hoàn thành, có khoá advisory
POST	`/api/v1/rooms/group`	Tạo nhóm	`groupName`, `memberIds` / `RoomDTO`	Hoàn thành
GET	`/api/v1/rooms/{roomId}/messages`	Lịch sử phân trang	`Page<MessageDTO>` (PL/pgSQL)	Hoàn thành

10.4.2. Nhật ký commit

Bảng 10.7. *Các commit của Huỳnh Thanh Hải*

Ngày	Commit	Nội dung	Tệp / dòng
28/09	`9aeb4aa`	Hợp nhất cục bộ `feat/auth-register-login-jwt` vào `main` trên máy để làm tiếp trên nền mới nhất	Commit merge
01/10	`a2626e6`	"feat(api): complete auth, rooms, and database task": toàn bộ luồng xác thực, refresh token, xử lý lỗi, mô hình phòng chat, REST, Flyway V1/V2, test tích hợp, README	47 tệp, khoảng +2.200 / −74
02/10	`8bb8f4f`	Gỡ `task.md` sau khi hoàn thành nhiệm vụ	−46

10.4.3. Phân tích commit a2626e6 theo nhóm tệp

Bảng 10.8. *Nội dung commit a2626e6 theo nhóm tệp*

Nhóm	Tệp	Dòng thêm
Xác thực và token	`AuthController` (viết lại), `RefreshToken`, `RefreshTokenRepository`, `RefreshTokenService`, `JwtResponse`, `RefreshTokenRequest`	≈ 262

Nhóm	Tệp	Dòng thêm
Bảo mật và lỗi	`GlobalExceptionHandler`, `JsonAuthenticationEntryPoint`, `JsonAccessDeniedHandler`; chỉnh `WebSecurityConfig`, `JwtService`	≈ 166
Phòng chat	`Conversation`, `ConversationParticipant`, `Message`, ba repository, `RoomController`, `RoomService` (200 dòng), 5 DTO	≈ 776
Người dùng	Mở rộng `User` lên 14 cột, `UserController` (`/me`, `/search`), `UserRepository.searchUsers`, `UserDTO`, `UserProfileDTO`	≈ 195
Cơ sở dữ liệu	Chuyển lược đồ thành `V1__messenger_schema.sql` (+112 dòng hai hàm), `V2__lock_direct_room_creation.sql`, `database.yml`, cấu hình Flyway, Testcontainers trong pom	≈ 222
Kiểm thử	`AuthAndRoomIntegrationTest` (17 bài), xoá `NextchatApplicationTests` mặc định	479
Tài liệu	`README.md` (hợp đồng API, cách chạy, cách test), `task.md`	91

10.4.4. Phân tích các hạng mục chính

a) Hoàn thiện luồng xác thực

Hải viết lại `AuthController` từ khung của Cường: đăng ký kiểm tra trùng username và trùng email riêng (thông điệp 409 rõ ràng hơn), gán vai trò `ROLE_USER` (tự tạo vai trò nếu chưa có) và trả `UserDTO` thay vì đăng nhập luôn; đăng nhập kiểm tài khoản còn hoạt động, phát access token kèm refresh token và thời hạn `expiresIn`; thêm endpoint `/refresh`. Thiết kế này tách rõ "tạo tài khoản" và "mở phiên", phù hợp với client có màn hình đăng ký riêng.

b) Refresh token lưu băm và xoay vòng

`RefreshTokenService` là phần có giá trị bảo mật cao nhất trong commit: token thô 32 byte ngẫu nhiên, chỉ lưu hash SHA-256, mỗi người dùng một token tại một thời điểm, xoay vòng sau mỗi lần dùng, kiểm hạn và kiểm tài khoản, đặt trong giao dịch (Chương 7). Trường `token` trong bảng được ánh xạ sang thuộc tính `tokenHash` trong entity để tên trong mã nói đúng bản chất dữ liệu.

c) Xử lý lỗi tập trung và phản hồi JSON cho Security

Hải thêm `GlobalExceptionHandler` với bảy loại ngoại lệ (Bảng 4.3) và hai lớp `JsonAuthenticationEntryPoint`, `JsonAccessDeniedHandler` để lỗi 401/403 ở tầng Security cũng trả đúng vỏ `ApiResponse` thay vì trang lỗi mặc định. Đồng thời chỉnh `WebSecurityConfig`: đổi đường dẫn mở thành `/api/v1/auth/**` cho khớp hợp đồng mới và đăng ký hai handler. Trong `JwtService`, Hải thêm kiểm tra chặn khoá phát triển ngoài profile dev/test (mục 4.3) và phương thức `getExpiration` để controller trả `expiresIn`.

d) Mô hình phòng chat và RoomService

Ba entity `Conversation`, `ConversationParticipant`, `Message` ánh xạ đủ cột của bảng tương ứng (8, 11 và 12 cột), kể cả các cột chưa dùng như `nickname`, `is_muted`, `forwarded_from_id`, giúp Hibernate validate không báo lỗi và sẵn sàng cho tính năng sau. `ConversationRepository` có truy vấn danh sách phòng với `@EntityGraph`, truy vấn tổng hợp số tin chưa đọc và lời gọi hàm tạo phòng. `RoomService` gồm bốn nghiệp vụ: danh sách phòng, mở phòng 1-1, tạo nhóm (loại trùng id, bỏ chính mình, từ chối toàn bộ nếu có thành viên không hợp lệ), lịch sử phân trang (kiểm thành viên, kiểm tham số, gọi hàm PL/pgSQL, đếm dự phòng khi trang rỗng). `RoomDTO.fromEntity` tự lấy tên và ảnh của người còn lại làm tên phòng PRIVATE.

e) Người dùng

Entity `User` được mở rộng từ vài cột ban đầu lên 14 cột khớp lược đồ (avatar, bio, số điện thoại, trạng thái, lần cuối trực tuyến, cờ hoạt động/xoá, thời điểm tạo/cập nhật với `@PrePersist`/`@PreUpdate`). `UserController` có `/me` trả `UserProfileDTO` kèm vai trò và `/search` với giới hạn 100 ký tự, truy vấn JPQL không phân biệt hoa thường trên ba trường và chỉ trả tài khoản hoạt động.

f) Cơ sở dữ liệu, Flyway và PL/pgSQL

Hải chuyển lược đồ của Cường thành migration Flyway V1, bổ sung hai hàm `get_room_messages` và `get_or_create_direct_room` (có khoá advisory ngay

từ V1), thêm V2 định nghĩa lại hàm tạo phòng, cấu hình Flyway trong `application.yaml` với `baseline-on-migrate`, và tạo `database.yml` dùng chung cho pool HikariCP và JPA (`open-in-view: false`). Hải cũng thêm `@EnableSpringDataWebSupport(VIA_DTO)` để định dạng JSON của `Page` ổn định.

g) Bộ kiểm thử tích hợp

`AuthAndRoomIntegrationTest` (479 dòng, 17 bài theo thứ tự) là xương sống kiểm chứng của đồ án: chạy trên PostgreSQL 16 trong Testcontainers, đi qua chuỗi filter thật bằng MockMvc, phủ toàn bộ chín endpoint, các nhánh lỗi chính (401, 403, 404, 409, 400) và cả tình huống đồng thời tạo phòng (test #17). Cường bổ sung test #18 về sau. Bộ test thay thế lớp test mặc định do Spring Initializr sinh ra.

h) Tài liệu hợp đồng API

`README.md` mô tả cách chạy cục bộ, vỏ phản hồi chung và bảng chín endpoint với đầu vào, đầu ra. Đây là tài liệu mà client web của Tài dựa vào cho phần REST; phần REST tích hợp không gặp lệch hợp đồng nào, trừ tên trường `groupName` bị client gửi nhầm (R3).

10.4.5. Kết quả so với nhiệm vụ

Bảng 10.9. *Mức độ hoàn thành nhiệm vụ của Huỳnh Thanh Hải*

Nhiệm vụ	Trạng thái	Minh chứng
Hoàn thiện đăng ký, đăng nhập, làm mới token	Hoàn thành	Test #2–#8
REST tài khoản, phòng, lịch sử (9 endpoint)	Hoàn thành	Test #9–#16; README
Quản lý kết nối qua `database.yml`	Hoàn thành	`database.yml`, `spring.config.import`
Tối ưu truy vấn bằng hàm PL/pgSQL	Hoàn thành 2 hàm	V1, V2; test #1, #14, #17; lab SQL 28/10
Kiểm thử tích hợp (ngoài phân công ban đầu)	Hoàn thành 17 bài	`AuthAndRoomIntegrationTest`

10.4.6. Hạn chế và bài học

- Toàn bộ phần việc nằm trong một commit 47 tệp, khiến việc duyệt mã và truy vết từng thay đổi khó hơn; chia thành các commit theo hạng mục (auth, room, DB, test) sẽ giúp PR #5 dễ review và dễ quay lui khi có lỗi.
- `AuthController` và `UserController` gọi repository trực tiếp, chưa tách service; logic đăng ký nằm trong controller (R9).
- Làm mới token đồng thời chưa được bảo vệ; nhánh token hết hạn bị rollback cả thao tác xoá (R10, mục 4.7.3).
- Hàm lịch sử luôn đếm toàn bộ tin của phòng, và chỉ mục chưa có `id`; thí nghiệm 28/10 cho thấy chi phí tăng rõ ở 100.000 tin (Chương 6).

10.4.7. Kiến thức và kỹ năng rút ra

Thiết kế hợp đồng REST với vỏ phản hồi và mã lỗi nhất quán; xây luồng xác thực có refresh token an toàn; ánh xạ JPA đầy đủ với lược đồ có sẵn và kiểm bằng `validate`; viết hàm PL/pgSQL và gọi từ Spring; viết kiểm thử tích hợp với Testcontainers, kể cả kiểm thử đồng thời bằng `CountDownLatch`.

10.5. Nguyễn Phú Tài – Người 3: client web thử nghiệm và hạ tầng

10.5.1. Vai trò và nhiệm vụ được giao

Bảng 10.10 ghi nguyên văn từng nhiệm vụ trong bảng phân công vai trò của nhóm cho Nguyễn Phú Tài theo thứ tự gốc: dòng 1–4 là nhiệm vụ chính, dòng 5–8 là công việc cụ thể và điểm kết nối API. Cột kết quả và trạng thái được đối chiếu với mã nguồn trên `main` sau PR #8. Nhiệm vụ chưa có minh chứng được ghi rõ là chưa thực hiện. Dòng 9 là việc phát sinh khi tích hợp, nằm ngoài bảng phân công.

Bảng 10.10. *Nhiệm vụ của Nguyễn Phú Tài và trạng thái thực hiện*

TT	Nhiệm vụ được phân công (nguyên văn)	Kết quả thực hiện	Trạng thái / minh chứng
1	Xây dựng ứng dụng giao diện Client kết nối với backend nextchat.	Client web một trang gồm `index.html` và `app.js` trong thư mục `static`, chạy cùng cổng 8080; dùng đủ 9 endpoint REST (mục 9.4)	Hoàn thành. `4699986`

TT	Nhiệm vụ được phân công (nguyên văn)	Kết quả thực hiện	Trạng thái / minh chứng
2	Tích hợp @stomp/stompjs hoặc sockjs-client trên Frontend để trao đổi tin nhắn qua WebSocket.	Dùng sockjs-client 1.x kết hợp stomp.js 2.3.3 qua CDN; gửi và nhận tin qua STOMP	Hoàn thành. `4699986`, `332d069`
3	Đóng gói dự án bằng Docker (Dockerfile cho nextchat và docker-compose.yml kết hợp dịch vụ PostgreSQL).	Chỉnh `docker-compose.yml`: giá trị mặc định, tên volume, healthcheck. Dockerfile: chưa có	Hoàn thành một phần. Kho không có Dockerfile
4	Thiết lập pipeline CI/CD với GitHub Actions để tự động build và kiểm thử mã nguồn.	–	Chưa thực hiện. Kho không có `.github/workflows`
5	Xây dựng UI Auth: Màn hình Đăng ký/Đăng nhập kết nối API /api/v1/auth/* từ Người 2 (Huỳnh Thanh Hải) và quản lý JWT token.	Form đăng ký, đăng nhập; `handleRegister`, `handleLogin`, `refreshAccessToken`, `saveAuthSession`; token trong `localStorage`; tự làm mới khi gặp 401	Hoàn thành. `4699986`
6	Xây dựng Khung Chat UI: Gọi /api/v1/rooms để hiển thị danh sách cuộc hội thoại. Truy xuất lịch sử tin nhắn qua /api/v1/rooms/{roomId}/messages.	`loadRooms`, `renderRoomList` (tin cuối, số chưa đọc); `loadRoomHistory`; bổ sung tìm người dùng, tạo phòng 1-1 và nhóm	Hoàn thành; tạo nhóm còn lỗi tên trường (R3)
7	Tích hợp WebSocket Real-time: Kết nối tới ws://localhost:8080/ws, subscribe các kênh /topic/room.{roomId} và /topic/presence.	Kết nối SockJS tới `/ws-chat` thay vì `/ws`; CONNECT kèm `Authorization: Bearer`; subscribe `/topic/room/{roomId}` sau khi đồng bộ; gửi `/app/room/{roomId}`	Hoàn thành một phần. Chưa có `/topic/presence` vì backend chưa phát trạng thái trực tuyến
8	Hạ tầng & Test: Viết tệp docker-compose.yml tích hợp cấu hình CSDL từ database.yml và chạy kiểm thử tải (Load Test) kết nối WebSocket đồng thời.	Compose có chỉnh nhưng giá trị mặc định `nextchat`/`nextchat123` lệch với `database.yml` (`messenger_db`/`postgres`). Load test: chưa có	Hoàn thành một phần (Compose); load test chưa thực hiện
9	Ngoài phân công: cấu hình truy cập cho client, phát sinh khi tích hợp.	`WebSecurityConfig` cho phép công khai `/`, `/index.html`, `/app.js`, `/favicon.ico`, `/ws/**`, `/ws-chat/**`	Hoàn thành. `4699986`

10.5.2. Nhật ký commit

Toàn bộ phần mã của Tài nằm trong commit `4699986` lúc 14:06 ngày 30/10/2026 trên nhánh `phu-tai`: `app.js` (+592), `index.html` (+262), `docker-`

`compose.yml` (+8/−8), `WebSecurityConfig.java` (+12/−4). Nhánh được hợp nhất vào `main` qua PR #8 sáng 06/11. Trước khi hợp nhất, trang Contributors của GitHub chỉ thống kê nhánh mặc định nên chưa hiển thị commit của Tài; ảnh chụp ngày 28/10 ở Phụ lục 3 phản ánh thời điểm đó.

10.5.3. Phân tích các hạng mục chính

a) Kiến trúc client và trải nghiệm xác thực

Client được thiết kế như một bảng điều khiển trình diễn (Hình 9.1): ngoài khung chat, cột phải có JWT inspector hiển thị access token, refresh token và TTL, cùng nhật ký giao thức ghi lại mọi lệnh REST và khung STOMP với màu theo loại (REST, STOMP, AUTH, ERROR). Đây là lựa chọn hợp lý cho đồ án học phần: khi bảo vệ, có thể chỉ cho giảng viên thấy từng khung CONNECT, SUBSCRIBE, SEND, MESSAGE và lần xoay vòng token đang diễn ra. Hàm `apiFetch` gom toàn bộ logic gắn token, đọc vỏ `ApiResponse` và tự làm mới khi gặp 401 với cơ chế chống lặp vô hạn (mục 9.4.2).

b) Danh sách phòng, lịch sử và tạo phòng

`renderRoomList` hiển thị tên phòng, loại, id, tin cuối và huy hiệu số tin chưa đọc; `formatRoomDisplayName` lấy tên người còn lại cho phòng PRIVATE. `loadRoomHistory` tải 50 tin mới nhất rồi sắp xếp lại tăng dần. Tìm người dùng hiển thị kết quả kèm nút "Nhắn tin" gọi `openDirectRoom`; nhờ hàm tạo phòng có khoá, A nhắn cho B và B nhắn cho A đều mở cùng một phòng. Hộp thoại tạo nhóm nhận tên và danh sách id cách nhau bằng dấu phẩy.

c) Kết nối thời gian thực

`connectWebSocket` mở SockJS tới `/ws-chat`, gửi CONNECT kèm header `Authorization`, cập nhật chấm trạng thái "Connected" và đăng ký lại topic của phòng đang mở khi kết nối lại. `subscribeToRoomTopic` huỷ subscription cũ trước khi đăng ký phòng mới. `sendRealtimeMessage` kiểm tra kết nối trước khi gửi và thử kết nối lại nếu đã rớt. Tin chỉ được vẽ khi nhận lại từ topic, nên giao diện luôn phản ánh dữ liệu máy chủ đã lưu.

d) An toàn phía client

Mọi dữ liệu do người dùng nhập (tên, nội dung tin, email) đều đi qua `escapeHtml` trước khi chèn vào DOM, giảm nguy cơ XSS – điều quan trọng vì token được lưu trong `localStorage`. Phương án chặt chẽ hơn là giữ refresh token trong cookie `HttpOnly` (mục 7.4).

e) Cấu hình truy cập và Docker Compose

Tài bổ sung danh sách `permitAll` tường minh trong `WebSecurityConfig` cho tài nguyên tĩnh và hai endpoint bắt tay WebSocket. Trong `docker-compose.yml`, Tài đổi giá trị mặc định, đổi tên volume thành `postgres_data`, đổi healthcheck sang nội suy biến với giá trị mặc định, chạy mỗi 10 giây và thử tối đa 5 lần.

10.5.4. Quá trình tích hợp với backend

Tích hợp client với backend là nơi bộc lộ rõ nhất các điểm lệch hợp đồng giữa tài liệu và mã:

- Đích STOMP: `4699986` dùng `/app/chat.sendMessage` và `/topic/rooms.{roomId}` theo tài liệu nháp, gửi `roomId` trong payload; backend lúc đó dùng `/app/room.{roomId}`. Cường đồng bộ trong `332d069` cùng ngày, chuyển cả hai phía sang dạng gạch chéo.
- Danh tính STOMP: `main` lúc đó chưa có bản sửa giữ Principal, nên client nhận CONNECTED nhưng SUBSCRIBE bị từ chối. Cường sửa trong `1046984` ngày 06/11.
- Origin: client được phục vụ từ cổng 8080, nhưng danh sách origin mặc định chỉ có cổng 3000 và 5173; `1046984` bổ sung.
- Hợp đồng REST khớp với README, trừ trường `groupName` (R3).

Bài học chung của cả nhóm: đọc hợp đồng từ mã nguồn và README, không từ tài liệu nháp; và có một kịch bản đầu-cuối chạy được trước khi coi tích hợp là xong.

10.5.5. Kết quả, hạn chế và việc chưa hoàn thành

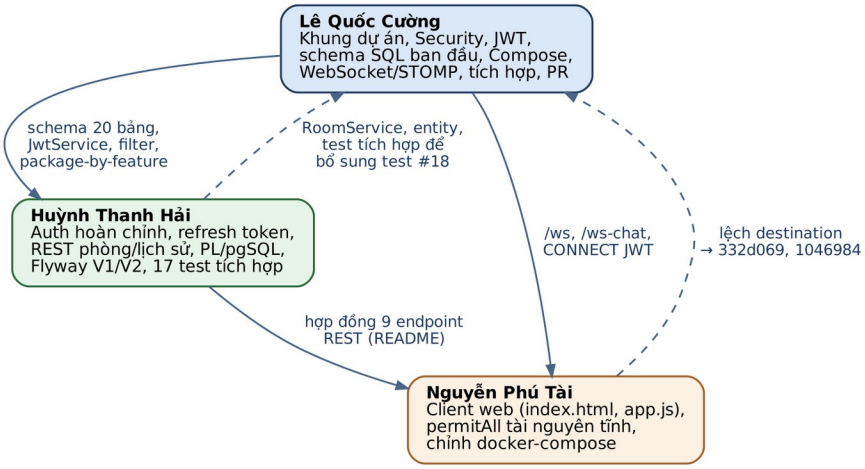
- Hoàn thành: client web dùng đủ 9 endpoint REST và luồng STOMP; công cụ trình diễn giao thức; cấu hình truy cập tài nguyên tĩnh; chỉnh Compose.
- Hạn chế cần sửa: R3 (`groupName`), R4 (giá trị mặc định của Compose), R5 (token trên URL SockJS), R6 (`formatRoomDisplayName` so sánh `p.id`).
- Chưa thực hiện so với phân công: Dockerfile đóng gói ứng dụng, pipeline GitHub Actions, kênh `/topic/presence`, kết nối qua endpoint `/ws` thuần, load test kết nối WebSocket đồng thời, test tự động cho client. Các mục này được đưa vào Chương 12.

10.5.6. Kiến thức và kỹ năng rút ra

Sử dụng SockJS và stomp.js để kết nối một backend Spring; quản lý vòng đời JWT phía client (lưu, gắn, tự làm mới, kết nối lại kênh thời gian thực khi token đổi); đọc và đối chiếu hợp đồng API; làm việc với cấu hình bảo mật và Docker Compose của backend.

10.6. Phối hợp và tích hợp giữa các thành viên

Hình 10.3 tóm tắt các điểm bàn giao giữa ba thành viên. Mỗi mũi tên là một hợp đồng hoặc một sản phẩm mà người này dùng làm nền cho người kia; mũi tên đứt nét là phản hồi khi tích hợp phát hiện vấn đề.



Hình 10.3. *Các điểm bàn giao và phản hồi giữa ba thành viên*

Bảng 10.11. *Các điểm tích hợp giữa thành viên và vấn đề phát sinh*

Điểm tích hợp	Bên giao → bên nhận	Vấn đề và cách xử lý
Lược đồ 20 bảng	Cường → Hải	Hải chuyển thành Flyway V1, thêm hai hàm; không phát sinh lệch
Khung Security/JWT và package	Cường → Hải	Hải mở rộng `JwtService`, đổi đường dẫn mở sang `/api/v1/auth/**`
`RoomService`, entity, bộ test	Hải → Cường	Cường thêm `sendMessage` vào `RoomService` và test #18 vào bộ test của Hải
Hợp đồng REST (README)	Hải → Tài	Khớp, trừ tên trường `groupName` (R3, chưa sửa)
Endpoint và đích STOMP	Cường → Tài	Lệch do client theo tài liệu nháp; Cường đồng bộ trong `332d069`
Danh tính STOMP, origin	Cường → Tài	Client không SUBSCRIBE được, origin 8080 bị từ chối; Cường sửa trong `1046984`
Cấu hình Compose	Tài → cả nhóm	Giá trị mặc định lệch `application.yaml` (R4, chưa sửa)

10.7. Tổng hợp mức độ hoàn thành theo phân công

Bảng 10.12. *Tổng hợp mức độ hoàn thành nhiệm vụ của ba thành viên*

Thành viên	Hoàn thành	Một phần	Chưa thực hiện	Ghi chú
Lê Quốc Cường	8	1	0	Một phần: kênh STOMP còn R1, R2 trên `main`; bản sửa cục bộ chưa push
Huỳnh Thanh Hải	5	0	0	Bao gồm bộ test tích hợp ngoài phân công

Thành viên	Hoàn thành	Một phần	Chưa thực hiện	Ghi chú
Nguyễn Phú Tài	4	4	1	Một phần: Docker, WebSocket (thiếu presence), khung chat (lỗi tạo nhóm), hạ tầng & test; chưa: CI/CD

10.8. Tiểu kết chương

Ba thành viên chia việc theo ba mảng bổ trợ và phối hợp qua Git với tám Pull Request. Lê Quốc Cường dựng khung dự án, bảo mật ban đầu, lược đồ, kênh STOMP và giữ vai trò tích hợp; Huỳnh Thanh Hải hoàn thiện toàn bộ phần REST, xác thực có refresh token, cơ sở dữ liệu với PL/pgSQL và bộ kiểm thử tích hợp làm xương sống kiểm chứng; Nguyễn Phú Tài xây client web trình diễn đầy đủ luồng đầu-cuối và được hợp nhất vào `main` ngày 06/11. Phần DevOps (Dockerfile, CI/CD, load test) chưa hoàn thành so với phân công ban đầu, và một số điểm lệch tích hợp còn tồn đọng được ghi ở Bảng 9.6.

CHƯƠNG 11. ĐÁNH GIÁ VÀ NHÌN LẠI

11.1. So sánh với mục tiêu ban đầu

Bảng 11.1. *Đối chiếu kết quả với các mục tiêu cụ thể*

Mục tiêu	Kết quả	Phần còn lại / giới hạn
1) Lịch sử Java, Spring, Spring Boot và dấu ấn Boot 4 trong mã	Chương 1: các mốc từ 2002 đến 4.1.1; Bảng 1.4 đối chiếu 8 dấu hiệu trong mã nguồn	–
2) Lý do chọn đề tài và công nghệ	Chương 2: 6 lý do chọn đề tài, bộ tiêu chí có trọng số, lý do cho 9 thành phần, ma trận so sánh	Ma trận là đánh giá chủ quan của nhóm
3) Giải thích và vận dụng 9 cơ chế lõi + WebSocket	Chương 4: mỗi cơ chế có vị trí dùng thật và minh chứng; test bean/điều kiện, rollback qua proxy (28/10)	Một số minh chứng chạy trên bản sửa cục bộ, chưa trên `main`
4) Xây dựng backend và client	9 endpoint REST, kênh STOMP, client web đã hợp nhất vào `main`	R1–R6 tồn đọng; chưa có presence, tệp đính kèm, trạng thái đã đọc
5) Kiểm chứng tự động và thí nghiệm	28/28 JUnit, 15/15 smoke (28/10); lab SQL khoá và phân trang; `main` có 23 test	Chưa chạy lại trên `main`; chưa đo tải API, nhiều instance, SockJS
6) Ghi nhận phần việc từng thành viên	Chương 10: đối chiếu từng commit, PR, nhiệm vụ	DevOps (Dockerfile, CI/CD, load test) chưa hoàn thành

Nhìn chung, phần backend và các cơ chế Spring Boot đạt mục tiêu với minh chứng rõ ràng; phần tích hợp đầu-cuối đạt nhưng minh chứng mạnh nhất nằm trên bản sửa cục bộ; phần hạ tầng triển khai chưa đạt so với kế hoạch. Không suy ra ứng dụng sẵn sàng vận hành thực tế từ bộ kiểm thử hiện có.

11.2. Phân tích lỗi (debugging case studies)

Các tình huống dưới đây phân biệt rõ commit nền, bản sửa cục bộ và biến thể thí nghiệm. Nhóm không gán một test thất bại hay một stack trace giả cho lịch sử Git.

11.2.1. Tình huống 1 – Nguy cơ trùng phòng và đối chứng bỏ khoá

Ở commit `a2626e6` và mọi commit sau, V1/V2 đã có khoá; chưa có chứng cứ test #17 từng thất bại ở một commit không khoá. Để kiểm chứng vai trò của khoá,

ngày 28/10 nhóm tạo hai hàm thí nghiệm từ V2, cùng thêm độ trễ 150 ms, chỉ khác nhau ở việc có hay không có khoá. Kết quả (Bảng 5.2): bản bỏ khoá sinh 2 phòng và 4 bản ghi thành viên cho mỗi cặp trong cả ba lượt; bản có khoá giữ 1 phòng, 2 thành viên. Nguyên nhân thiết kế: thao tác kiểm tra và tạo phải được bảo vệ trên cùng cặp người dùng; ràng buộc UNIQUE (`conversation_id`, `user_id`) chỉ ngăn trùng người trong một phòng, không ngăn hai phòng khác nhau cho cùng cặp.

11.2.2. Tình huống 2 – Giữ đúng totalElements khi trang lịch sử rỗng

Kịch bản test #14 có hai tin chưa xoá và yêu cầu một trang ngoài phạm vi. Hàm SQL trả 0 dòng nên không thể lấy `total_count` từ dòng đầu. Nếu service gán tổng bằng 0, phản hồi sai: client tưởng phòng không có tin nào. Mã hiện tại có nhánh đếm dự phòng bằng `countByConversationIdAndDeletedFalse` (mục 6.3). Bài học: một giá trị tổng được "đính kèm" vào từng dòng sẽ biến mất khi không có dòng; mọi API phân trang cần một test cho trang rỗng.

11.2.3. Tình huống 3 – CONNECT thành công nhưng SUBSCRIBE bị từ chối do mất Principal

- Triệu chứng (25/10): mã tại `39b4d75` vượt qua 22 bài JUnit; nhưng khi chạy hai client WebSocket thật, cả hai nhận CONNECTED rồi nhận ERROR ngay khi SUBSCRIBE phòng của mình.
- Giả thuyết ban đầu: người dùng không phải thành viên – bị loại bằng cách kiểm tra cơ sở dữ liệu. Giả thuyết thứ hai: SUBSCRIBE không mang Principal – xác nhận bằng log trong interceptor: `accessor.getUser()` là `null` ở khung SUBSCRIBE.
- Nguyên nhân gốc: interceptor gọi `StompHeaderAccessor.wrap(message)` tạo accessor mới và trả về message mới. Transport STOMP gắn callback ghi nhớ user vào accessor gốc; `setUser` trên accessor mới không kích hoạt callback đó, nên các khung sau của phiên không có user. Unit test không phát hiện được vì test kiểm trực tiếp message trả về từ `preSend`, không đi qua transport.

- Bản sửa cục bộ (25/10): lấy accessor sẵn có bằng `MessageHeaderAccessor.getAccessor(...)`, gọi `setUser` và trả lại message gốc; thêm test hồi quy gắn `userChangeCallback` và khẳng định callback được gọi – test này thất bại trên mã gốc và đạt sau sửa. Kết quả: 25 JUnit, 11 smoke.
- Bản sửa trên `main` (06/11, `1046984`): khôi phục user từ thuộc tính phiên `stompUsername` (mục 4.10.3).
- Bài học: kiểm thử đơn vị với mock không thay được kiểm thử qua transport thật; với kênh WebSocket, cần ít nhất một kịch bản hai client.

11.2.4. Tình huống 4 – Lệch hợp đồng đích STOMP giữa client và server

- Triệu chứng (30/10): client web kết nối được nhưng gửi tin không có phản hồi, không có tin nào được lưu.
- Nguyên nhân: client gửi tới `/app/chat.sendMessage` và đăng ký `/topic/rooms.{roomId}` theo tài liệu nháp; backend ánh xạ `/app/room.{roomId}` và phát tới `/topic/room.{roomId}`. Khung SEND không khớp `@MessageMapping` nào nên bị bỏ qua lặng lẽ; khung SUBSCRIBE tới đích không khớp mẫu phòng thì được chấp nhận nhưng không bao giờ nhận được tin.
- Xử lý: `332d069` thống nhất hai phía sang dạng gạch chéo, bỏ `roomId` khỏi payload vì đích đã chứa id phòng.
- Bài học: hợp đồng giao tiếp cần một nguồn sự thật duy nhất. Lỗi "im lặng" của STOMP khó gỡ hơn lỗi HTTP 404; nhật ký giao thức trong client giúp nhìn thấy khung đã gửi.

11.2.5. Tình huống 5 – Tệp .env bị theo dõi trong Git

- Triệu chứng (02/10): tệp `.env` chứa mật khẩu cơ sở dữ liệu xuất hiện trong kho từ các commit trước.

- Xử lý: Cường mở rộng `.gitignore` và gỡ tệp khỏi chỉ mục (`87fe186`), chuyển hướng dẫn sang `.env.example`.
- Giới hạn: gỡ khỏi chỉ mục không xoá tệp khỏi lịch sử Git; giá trị trong `.env` lúc đó là giá trị phát triển, nhưng với môi trường thật cần đổi mật khẩu đã lộ và cân nhắc viết lại lịch sử.

11.2.6. Tình huống 6 – Kiểm soát đã kiểm chứng nhưng không vào được nhánh chính

- Phát hiện (06/11, khi viết lại báo cáo): đối chiếu `main` với bản sửa cục bộ cho thấy hai kiểm soát – chặn SUBSCRIBE wildcard và chặn SEND vào broker – đã có test và smoke đạt ngày 25/10, 28/10 nhưng không có trên `main` (R1, R2).
- Nguyên nhân: bản sửa cục bộ không được đẩy thành nhánh và PR; khi sửa lỗi Principal theo cách khác trên `main`, hai kiểm soát còn lại không được mang theo.
- Xử lý đề xuất: áp dụng bản vá ở mục 4.10.5 cùng hai bài test và cập nhật kịch bản smoke.
- Bài học quy trình: mọi thay đổi đã kiểm chứng phải đi vào kho; báo cáo phải ghi rõ minh chứng thuộc phiên bản mã nào.

11.3. Bài học kinh nghiệm

Về kỹ thuật

- Hiểu cơ chế quan trọng hơn biết cú pháp: lỗi Principal chỉ tìm ra được khi hiểu vai trò của accessor và callback trong kênh STOMP; quy tắc rollback chỉ thấy rõ khi đọc lại dữ liệu sau ngoại lệ qua proxy thật.
- Một phép thử "đạt" chưa chứng minh cơ chế có tác dụng; cần biến thể đối chứng (bỏ khoá, gỡ ràng buộc) để thấy sự khác biệt.
- Kết quả phụ thuộc phiên bản: tài liệu và ví dụ trên mạng phải được đối chiếu với phiên bản trong `pom.xml`.

Về làm việc nhóm

- Chia việc theo mảng và giao tiếp qua hợp đồng giúp làm song song, nhưng hợp đồng phải được viết ra và cập nhật.
- Commit nhỏ theo hạng mục dễ duyệt và dễ truy vết hơn một commit lớn.
- Tích hợp sớm: client của Tài được đẩy lên ngày 30/10 và phát hiện ba điểm lệch; nếu tích hợp sớm hơn, các điểm này đã được sửa trong tuần 6.

11.4. Khai báo sử dụng công cụ AI

Nhóm khai báo sự hỗ trợ của công cụ AI trong toàn bộ quá trình. AI được dùng để giải thích khái niệm, gợi ý cấu trúc, rà soát hợp đồng, biên tập báo cáo và hỗ trợ sửa lỗi, viết kiểm thử. Công cụ AI không được xem là thành viên nhóm.

Bảng 11.2. *Các công cụ AI đã sử dụng và phạm vi*

Công cụ	Thời điểm	Phạm vi sử dụng
Gemini, ChatGPT, Copilot CLI	Tuần 1–4	Tài liệu ban đầu, giải thích khái niệm, gợi ý cấu trúc báo cáo nháp
GitHub Copilot	25/09, 02/10	Gợi ý mã trong hai commit có dòng `Co-authored-by: Copilot` (`7c56340`, `85eb2a5`)
ChatGPT / Codex	25/10, 28/10	Rà soát hợp đồng, biên tập; hỗ trợ bản sửa Principal và chặn đích; bổ sung test rollback, bean, Mockito, kịch bản E2E và lab SQL
Claude (Anthropic)	30/10 – 06/11	Soạn mục client web từ commit `4699986`, `332d069`; viết lại toàn bộ báo cáo này ngày 06/11 trên cơ sở đọc trực tiếp kho mã tới commit `54ab54e`, báo cáo cũ và tài liệu tham khảo; vẽ lại sơ đồ; phân tích rủi ro R1, R2

Sau mỗi gợi ý của AI, thành viên phụ trách phải tự kiểm ánh xạ lược đồ và DTO, token, chú thích và ví dụ mã bằng mã nguồn, khẳng định trong test hoặc log. Ba ví dụ sai lệch được phát hiện khi rà soát tài liệu:

- Ví dụ 1: tài liệu nháp ghi `/app/chat.sendMessage`, `/topic/rooms.{roomId}`, tham số `access_token`; mã dùng `/app/room.{roomId}`, `/topic/room.{roomId}` (nay là dạng gạch chéo) và header

`access-token`. Hợp đồng được sửa theo mapping và interceptor; client web từng theo bản nháp và phải đồng bộ lại (Tình huống 4).

- Ví dụ 2: tài liệu phụ ghi token băm dạng hex và hàm lịch sử dùng `COUNT(*) OVER()`; mã dùng Base64URL của SHA-256 và `SELECT COUNT(*) INTO v_total`. Sửa theo `RefreshTokenService` và migration.
- Ví dụ 3: bảng phân công trong báo cáo nháp ghi "Hoàn thành" cho load test JMeter, Dockerfile và test coverage; kho mã không có minh chứng cho các mục này. Báo cáo chính thức ghi lại là chưa thực hiện.

Trách nhiệm học thuật: sự hỗ trợ của AI không thay cho việc sinh viên tự chạy lại, tự giải thích mã và kết quả. Mỗi thành viên chịu trách nhiệm đối chiếu và trình bày phần việc của mình khi bảo vệ; nội dung tự đánh giá ở Chương 10 cần được từng thành viên xác nhận.

11.5. Tiểu kết chương

Đồ án đạt các mục tiêu về nghiên cứu cơ chế và xây dựng backend, có minh chứng định lượng; phần hạ tầng triển khai chưa đạt. Sáu tình huống lỗi cho thấy giá trị của kiểm thử qua thành phần thật, của hợp đồng giao tiếp rõ ràng và của việc đưa mọi thay đổi đã kiểm chứng vào kho mã.

CHƯƠNG 12. HƯỚNG PHÁT TRIỂN

12.1. Lộ trình cải tiến sản phẩm

Bảng 12.1. *Lộ trình cải tiến NextChat*

Giai đoạn	Hạng mục	Tiêu chí hoàn thành
Trước buổi bảo vệ	Chạy lại `./mvnw clean verify` trên `main`; vá R1, R2 kèm test; sửa R3 (`groupName`), R5 (token trên URL)	Log 25 bài JUnit đạt; demo tạo nhóm thành công
Ngắn hạn	Cập nhật kịch bản smoke sang đích gạch chéo; `@MessageExceptionHandler`; thống nhất cấu hình Compose (R4); tách `AuthService`, `UserService`	Smoke đạt trên `main`; client nhận lỗi nghiệp vụ STOMP
Trung hạn	Dockerfile, GitHub Actions; trạng thái trực tuyến; phân trang keyset; đánh dấu đã đọc dùng `last_read_message_id`	Pipeline chạy test mỗi PR; đo trước/sau keyset
Dài hạn	Broker relay cho nhiều instance; tệp đính kèm; phiên theo thiết bị và token family; đo tải WebSocket	Chạy 2 instance sau load balancer; số đo p50/p95/p99 có điều kiện rõ

12.2. Báo lỗi nghiệp vụ trên kênh STOMP

Hiện ngoại lệ trong `@MessageMapping` không được báo về client. Spring cho phép khai báo `@MessageExceptionHandler` trong controller và gửi kết quả tới hàng đợi riêng của người dùng nhờ tiền tố `/user` đã cấu hình:

Đoạn mã 12.1. *Đề xuất xử lý lỗi STOMP (chưa có trong mã nguồn)*

```
@MessageExceptionHandler(ResponseStatusException.class)
@SendToUser("/queue/errors")
public Map<String, Object> handleBusinessError(ResponseStatusException ex) {
    return Map.of("success", false,
                  "status", ex.getStatusCode().value(),
                  "message", String.valueOf(ex.getReason()));
}
// Client: stompClient.subscribe('/user/queue/errors', frame =>
showError(JSON.parse(frame.body)));
```

12.3. Trạng thái trực tuyến (presence)

Phân công của Nguyễn Phú Tài có kênh `/topic/presence` nhưng backend chưa phát trạng thái. Thiết kế đề xuất dùng sự kiện phiên của Spring: khi `SessionConnectedEvent` xảy ra, đặt `users.status = 'ONLINE'`; khi `SessionDisconnectEvent` xảy ra và người dùng không còn phiên nào khác, đặt

`OFFLINE` và cập nhật `last_seen_at`; mỗi thay đổi được phát tới `/topic/presence`. Cần đếm số phiên theo người dùng vì một người có thể mở nhiều tab.

Đoạn mã 12.2. *Phác thảo lắng nghe sự kiện phiên (đề xuất)*

```
@Component
public class PresenceListener {
    private final Map<String, AtomicInteger> sessions = new ConcurrentHashMap<>();
    // ... inject UserRepository, SimpMessagingTemplate

    @EventListener
    public void onConnected(SessionConnectedEvent event) {
        String username = event.getUser().getName();
        if (sessions.computeIfAbsent(username, k -> new AtomicInteger()).incrementAndGet() == 1)
{
            updateStatus(username, "ONLINE");
        }
    }

    @EventListener
    public void onDisconnect(SessionDisconnectEvent event) {
        String username = event.getUser() != null ? event.getUser().getName() : null;
        if (username != null && sessions.getOrDefault(username, new
AtomicInteger(1)).decrementAndGet() <= 0) {
            sessions.remove(username);
            updateStatus(username, "OFFLINE");   // kèm last_seen_at
        }
    }
}
```

Với cách khôi phục Principal hiện tại trên `main`, cần kiểm tra `event.getUser()` có giá trị ở các sự kiện này hay phải đọc từ thuộc tính phiên `stompUsername`.

12.4. Mở rộng nhiều instance với broker relay

Simple broker lưu subscription trong bộ nhớ của một tiến trình; khi chạy hai instance, client nối vào instance A sẽ không nhận tin được phát ở instance B. Spring hỗ trợ thay simple broker bằng broker relay tới một broker STOMP ngoài như RabbitMQ (bật plugin STOMP) mà không đổi controller:

Đoạn mã 12.3. *Đề xuất cấu hình broker relay*

```
@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
    registry.setApplicationDestinationPrefixes("/app");
    registry.enableStompBrokerRelay("/topic", "/queue")
            .setRelayHost("rabbitmq").setRelayPort(61613)
            .setClientLogin("nextchat").setClientPasscode(rabbitPassword); // inject bằng @Value
    registry.setUserDestinationPrefix("/user");
}
```

Khi đổi sang broker ngoài, cú pháp đích phụ thuộc broker (RabbitMQ dùng dấu chấm làm phân cách cho `/topic`), nên cần thiết kế lại mẫu đích và cập nhật interceptor cùng client.

12.5. Đóng gói và tích hợp liên tục

Hai hạng mục chưa hoàn thành của phân công DevOps được phác thảo dưới đây để làm điểm xuất phát; chúng chưa có trong kho và chưa được chạy thử.

Đoạn mã 12.4. *Đề xuất Dockerfile nhiều giai đoạn cho nextchat*

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY nextchat/ .
RUN ./mvnw -B -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/nextchat-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Đoạn mã 12.5. *Đề xuất workflow GitHub Actions chạy kiểm thử cho mỗi Pull Request*

```
name: ci
on: [pull_request, push]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - name: Build and test (Testcontainers dùng Docker có sẵn trên runner)
        working-directory: nextchat
        run: ./mvnw -B clean verify
```

12.6. Hiệu năng và đo tải

- Phân trang keyset và chỉ mục (`conversation_id, created_at DESC, id DESC`) như đề xuất ở mục 6.7; đo lại cùng điều kiện lab để so sánh.
- Bật luồng ảo (`spring.threads.virtual.enabled=true`) và đo trước/sau với cùng kịch bản tải.
- Kế hoạch đo tải: kịch bản N người dùng đồng thời giữ kết nối STOMP, mỗi người gửi tin theo chu kỳ; đo độ trễ từ lúc gửi tới lúc

subscriber nhận (p50/p95/p99), thông lượng, tỷ lệ lỗi; ghi rõ cấu hình máy, số instance, kích thước pool kết nối.

- Đếm số câu SQL mỗi request bằng thống kê Hibernate để kiểm chứng việc tránh N+1 ở danh sách phòng.

12.7. Định hướng phát triển của các thành viên

- Lê Quốc Cường: đi sâu vào kiến trúc hệ thống phân tán và nhắn tin (broker, khả năng mở rộng), bảo mật kênh thời gian thực; rèn quy trình đưa mọi thay đổi đã kiểm chứng vào kho qua PR có review.
- Huỳnh Thanh Hải: đi sâu vào thiết kế cơ sở dữ liệu và tối ưu truy vấn (kế hoạch thực thi, chỉ mục, phân trang keyset), thiết kế service layer và kiểm thử đồng thời.
- Nguyễn Phú Tài: hoàn thiện kỹ năng DevOps (Docker, CI/CD, đo tải) và phát triển frontend có cấu trúc (framework component, quản lý trạng thái, kiểm thử tự động giao diện).
- Cả nhóm: tiếp tục rèn kỹ năng đọc tài liệu chính thức, tạo phép thử tái lập và phân biệt kiểm tra chức năng với đo hiệu năng; lưu bằng chứng để kết luận có thể được kiểm tra độc lập.

KẾT LUẬN

Đồ án đã nghiên cứu Spring Boot từ hai góc độ: lịch sử và bản chất cơ chế. Về lịch sử, Spring Framework ra đời năm 2002–2004 từ nhu cầu đơn giản hoá J2EE bằng POJO, IoC/DI và AOP; Spring Boot ra đời năm 2014 từ yêu cầu SPR-9888 nhằm loại bỏ cấu hình lặp lại bằng starter, auto-configuration và máy chủ nhúng; thế hệ Spring Boot 4 (11/2025) mô-đun hoá toàn bộ mã nguồn, và bản 4.1.1 (08/2026) là nền tảng của NextChat với các dấu ấn thấy rõ trong mã của nhóm. Về bản chất, chín cơ chế lõi cùng kênh WebSocket/STOMP được giải thích và đối chiếu với vị trí sử dụng thật, có minh chứng bằng kiểm thử trên ApplicationContext và PostgreSQL thật.

Sản phẩm NextChat gồm 9 endpoint REST có xác thực JWT, refresh token lưu băm và xoay vòng, phòng 1-1 chống tạo trùng bằng khoá advisory, lịch sử phân trang bằng hàm PL/pgSQL, kênh STOMP lưu trước phát sau với danh tính lấy từ Principal, và một client web trình diễn đầu-cuối. Minh chứng định lượng gồm 28/28 bài JUnit và 15/15 kiểm tra đầu-cuối với hai client thật (28/10), thí nghiệm đối chứng khoá và đo phân trang ở ba mức dữ liệu.

Báo cáo cũng ghi nhận trung thực những gì chưa đạt: bản `main` cuối cùng (06/11) cần được chạy lại kiểm thử; hai kiểm soát truy cập trên kênh STOMP cần được vá; phần Dockerfile, CI/CD và đo tải chưa thực hiện. Ba thành viên đã phối hợp qua tám Pull Request với vai trò rõ ràng – Lê Quốc Cường giữ khung kiến trúc và tích hợp, Huỳnh Thanh Hải hoàn thiện REST và cơ sở dữ liệu, Nguyễn Phú Tài xây client – và rút ra bài học về hợp đồng giao tiếp, tích hợp sớm và việc đưa mọi thay đổi đã kiểm chứng vào kho mã. Những bài học đó, cùng nền tảng kỹ thuật đã xây, là điểm xuất phát cho các hướng phát triển ở Chương 12.

TÀI LIỆU THAM KHẢO

[1] Oracle, The Java Language Specification, Java SE 21 Edition. https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[2] Spring, Spring Boot Reference Documentation – Auto-configuration. https://docs.spring.io/spring-boot/reference/using/auto-configuration.html

[3] I. Fette, A. Melnikov, "The WebSocket Protocol", RFC 6455, IETF, 12/2011. https://www.rfc-editor.org/rfc/rfc6455.html

[4] STOMP Protocol Specification, Version 1.2. https://stomp.github.io/stomp-specification-1.2.html

[5] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 05/2015. https://www.rfc-editor.org/rfc/rfc7519.html

[6] PostgreSQL Global Development Group, PostgreSQL 16 Documentation – Explicit Locking: Advisory Locks. https://www.postgresql.org/docs/16/explicit-locking.html#ADVISORY-LOCKS

[7] Spring Framework Reference – Rolling Back a Declarative Transaction. https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/rolling-back.html

[8] Nhóm 24, kho mã springboot-websocket-chat, nhánh `main`, commit `54ab54e28b2faa54bd3d1b8fbab3987d9464c031` (06/11/2026); mốc nền `39b4d75ea8d440cde2fe76075fc856187948f11d` (20/10/2026). https://github.com/QuoocsCuongwf/springboot-websocket-chat

[9] Spring Framework Reference – STOMP: Token Authentication. https://docs.spring.io/spring-framework/reference/web/websocket/stomp/authentication-token-based.html

[10] Nhóm 24, Hồ sơ minh chứng ngày 25/10 và 28/10/2026: log Maven/Surefire, JSON kịch bản đầu-cuối, script và kế hoạch thực thi SQL (thư mục Minh_chung, bản sửa cục bộ dựa trên `39b4d75`).

[11] Spring Framework Reference – Bean Scopes. https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html

[12] PostgreSQL Global Development Group, PostgreSQL 16 Documentation – Using EXPLAIN. https://www.postgresql.org/docs/16/using-explain.html

[13] R. Johnson, Expert One-on-One J2EE Design and Development, Wrox, 2002. ISBN 0-7645-4385-7.

[14] R. Johnson, "Spring Framework: The Origins of a Project and a Name", spring.io blog, 09/11/2006. https://spring.io/blog/2006/11/09/spring-framework-the-origins-of-a-project-and-a-name/

[15] M. Youngstrom, "Improved support for 'containerless' web application architectures [SPR-9888]", spring-projects/spring-framework, issue #14521, 10/2012. https://github.com/spring-projects/spring-framework/issues/14521

[16] P. Webb, "Spring Boot 1.0 GA Released", spring.io blog, 01/04/2014. https://spring.io/blog/2014/04/01/spring-boot-1-0-ga-released

[17] P. Webb, "Spring Boot 2.0 goes GA", spring.io blog, 01/03/2018. https://spring.io/blog/2018/03/01/spring-boot-2-0-goes-ga

[18] Spring, "Spring Boot 3.0 Goes GA", spring.io blog, 24/11/2022. https://spring.io/blog/2022/11/24/spring-boot-3-0-goes-ga

[19] P. Webb, "Spring Boot 4.0.0 available now", spring.io blog, 20/11/2025. https://spring.io/blog/2025/11/20/spring-boot-4-0-0-available-now/

[20] A. Wilkinson, "Spring Boot 4.1.0 available now", spring.io blog, 10/06/2026, https://spring.io/blog/2026/06/10/spring-boot-4/ ; InfoQ, "Spring Boot 4.1", 06/2026, https://infoq.com/news/2026/06/spring-boot-4-1

[21] P. Webb, "Spring Boot 4.1.1 available now", spring.io blog, 20/08/2026. https://spring.io/blog/2026/08/20/spring-boot-4-1-1-available-now/

[22] Spring Framework 4.x Reference – New Features and Enhancements in Spring Framework 4.0. https://docs.spring.io/spring-framework/docs/4.2.x/spring-framework-reference/html/new-in-4.0.html

[23] VMware Tanzu Blog, "WebSocket Architecture in Spring Framework 4.0", 2013. https://blogs.vmware.com/tanzu/websocket-architecture-in-spring-framework-4-0/

[24] J. Hoeller, "Spring Framework 7.0 General Availability", spring.io blog, 13/11/2025.

[25] Spring Framework Reference – STOMP: Flow of Messages. https://docs.spring.io/spring-framework/reference/web/websocket/stomp/message-flow.html

[26] Spring Framework Reference – STOMP: Simple Broker. https://docs.spring.io/spring/reference/web/websocket/stomp/handle-simple-broker.html

[27] Stack Overflow, 2025 Developer Survey – Technology. https://survey.stackoverflow.co/2025/technology/ (số liệu Spring Boot tham khảo qua các bản tổng hợp của cộng đồng).

[28] JetBrains, The State of Developer Ecosystem 2023 – Java. https://www.jetbrains.com/lp/devecosystem-2023/java/

[29] VMware, "Broadcom Completes Acquisition of VMware", 22/11/2023. https://blogs.vmware.com/blog/2023/11/broadcom-completes-acquisition-of-vmware.html

[30] Redgate, Flyway Documentation. https://documentation.red-gate.com/flyway

[31] Testcontainers for Java – PostgreSQL Module. https://java.testcontainers.org/modules/databases/postgres/

[32] Spring Security Reference – Servlet Architecture. https://docs.spring.io/spring-security/reference/servlet/architecture.html

[33] HeroDevs, "Spring Boot Versions, EOL Dates, and Latest Releases (July 2026)", 01/07/2026. https://www.herodevs.com/blog-posts/spring-boot-versions-eol-dates-and-latest-releases-april-2026

[34] Spring Boot Reference – Externalized Configuration. https://docs.spring.io/spring-boot/reference/features/external-config.html

Các nguồn web được truy cập lại ngày 06/11/2026; phiên bản dự án lấy từ `pom.xml`, trang tài liệu web có thể mới hơn phiên bản 4.1.1. Các mốc lịch sử được ưu tiên lấy từ bài công bố phát hành trên `spring.io`; mốc tổ chức (Interface21, SpringSource, Pivotal) là thông tin phổ biến được nhiều nguồn thống nhất.

PHỤ LỤC

Phụ lục 1. Nhật ký công việc theo tuần

Bảng PL.1. *Nhật ký tổng hợp công việc*

Tuần	Công việc	Phụ trách / kết quả
1–2 (11–24/09)	Khảo sát Spring Boot, WebSocket, STOMP, JWT; thống nhất đề tài, phạm vi, phân công; tạo kho mã	Cả nhóm; Cường tạo kho và dự án (`15b9513`, `6ab071a` ngày 22/09)
3 (25/09–01/10)	Security, JWT, seed dữ liệu, schema 20 bảng, Compose, tái cấu trúc package, vai trò; hoàn thiện auth, refresh token, REST phòng/lịch sử, Flyway, test tích hợp	Cường: 8 commit ngày 25/09, PR #1–#3. Hải: `9aeb4aa` (28/09), `a2626e6` (01/10)
4 (02–08/10)	Hợp nhất nhánh API; xử lý `.env`; cấu hình WebSocket và controller STOMP	Cường: PR #4–#6, `deabd07`, `c3acb71`. Hải: `8bb8f4f`
5 (09–15/10)	Đối chiếu JPA, giao dịch, hàm PL/pgSQL; tài liệu nền	Hải và Cường; không có commit
6 (16–22/10)	Interceptor JWT STOMP, lưu tin qua service, test STOMP, đồng bộ tài liệu	Cường: PR #7, `d20e734`, `3eaa91a`, `39b4d75` (20/10)
7 (23–29/10)	Rà soát báo cáo; tái lập lỗi Principal; bản sửa cục bộ; test bổ sung; E2E; lab SQL	Đợt rà soát của nhóm, có AI hỗ trợ. 25/10: 25 JUnit + 11 smoke; 28/10: 28 JUnit + 15 smoke
8 (30/10–05/11)	Client web, cấu hình truy cập, Compose; đồng bộ endpoint STOMP	Tài: `4699986`. Cường: `332d069` (30/10)
9 (06/11)	Sửa giữ Principal và origin; hợp nhất `phu-tai`; viết lại và nộp báo cáo, nhật ký, mã nguồn	Cường: `1046984`, PR #8 (`54ab54e`). Cả nhóm: hồ sơ nộp
10	Chạy lại test trên `main`, vá R1–R3, luyện demo; bảo vệ	Kế hoạch (Phụ lục 8)

Phụ lục 2. Lịch sử commit của kho mã

Bảng PL.2 liệt kê toàn bộ commit trên mọi nhánh của kho tại thời điểm viết báo cáo, sắp xếp theo thời gian (cũ trước). Tên tác giả hiển thị theo cấu hình Git của từng máy: `Le Quoc Cuong`, `lequoccuong` là Lê Quốc Cường (commit hợp nhất PR trên web GitHub mang địa chỉ `noreply` của tài khoản `QuoocsCuongwf`); `ThanhHai151` là Huỳnh Thanh Hải; `Nguyen Phu Tai` là Nguyễn Phú Tài.

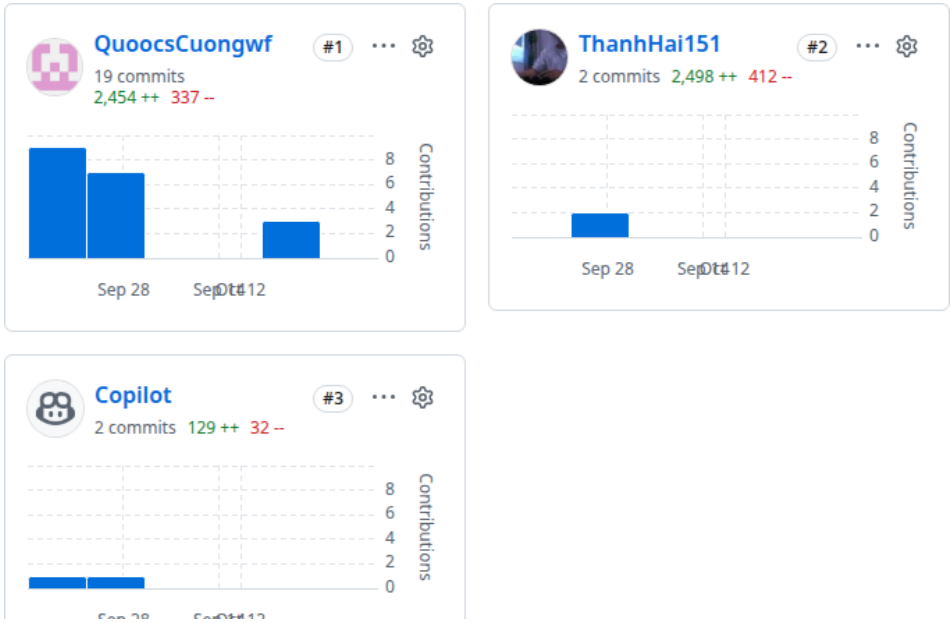
Bảng PL.2. *Toàn bộ commit của kho mã (mọi nhánh, theo thời gian)*

Commit	Thời điểm	Tác giả	Thông điệp commit
`15b9513`	22/09 09:59	lequoccuong	Initial commit
`6ab071a`	22/09 10:12	Le Quoc Cuong	first commit

Commit	Thời điểm	Tác giả	Thông điệp commit
0c94c66	25/09 13:43	Le Quoc Cuong	feat(user): add user entity and jpa repository
993ada7	25/09 13:43	Le Quoc Cuong	feat(security): configure http and websocket security
5dae9ab	25/09 13:52	Le Quoc Cuong	feat(auth): add seed data to test login with profile dev
296e342	25/09 14:01	Le Quoc Cuong	feat(config): add connect database
dfa4a3b	25/09 14:05	Le Quoc Cuong	feat(DevOps): create docker compose and env example
d390323	25/09 14:10	lequoccuong	Merge pull request #1 from QuoocsCuongwf/feat/auth-register-login-jwt
022fe4d	25/09 14:10	lequoccuong	Merge pull request #2 from QuoocsCuongwf/feat/DevOps
67fb244	25/09 15:00	Le Quoc Cuong	refactor(project): refactor project Package by Feature + Common
f7a7e8e	25/09 15:01	lequoccuong	Merge pull request #3 from QuoocsCuongwf/feat/auth-register-login-jwt
7c56340	25/09 15:08	Le Quoc Cuong	feat(auth): add JWT authentication and user roles
9aeb4aa	28/09 12:56	ThanhHai151	Merge branch 'feat/auth-register-login-jwt' into main
a2626e6	01/10 10:31	ThanhHai151	feat(api): complete auth, rooms, and database task
9e2a3d3	02/10 09:23	lequoccuong	Merge pull request #4 from QuoocsCuongwf/feat/auth-register-login-jwt
1a441b5	02/10 09:38	Le Quoc Cuong	feat(application): add path file env
87717e5	02/10 09:41	Le Quoc Cuong	chore: update gitignore to ignore ide and target files
8bb8f4f	02/10 09:42	ThanhHai151	remove task.md
87fe186	02/10 09:44	Le Quoc Cuong	chore: remove tracked .env from resources
acf0844	02/10 09:53	Le Quoc Cuong	chore(application): change file path of file .env
16352f1	02/10 10:12	Le Quoc Cuong	Merge branch 'main' into feat/complete-task-api
a934248	02/10 10:16	lequoccuong	Merge pull request #5 from QuoocsCuongwf/feat/complete-task-api
85eb2a5	02/10 10:58	Le Quoc Cuong	chore: load local environment configuration
5ea2fc2	02/10 11:05	Le Quoc Cuong	Merge main into branch feat/auth-websocket
0772adf	02/10 11:06	lequoccuong	Merge pull request #6 from QuoocsCuongwf/feat/auth-websocket
deabd07	02/10 14:22	Le Quoc Cuong	feat(socket): add web socket config
c3acb71	02/10 14:24	Le Quoc Cuong	feat(socket): add web socket controller with api /room.{roomId}
10115f2	20/10 20:16	lequoccuong	Merge pull request #7 from QuoocsCuongwf/feat/auth-websocket
d20e734	20/10 20:55	lequoccuong	feat(chat): implement JWT-protected STOMP messaging
3eaa91a	20/10 20:56	lequoccuong	test(chat): cover JWT STOMP and realtime messages
39b4d75	20/10 20:56	lequoccuong	docs(report): align report with implemented features
4699986	30/10 14:06	Nguyen Phu Tai	feat: add frontend static rsc and security and docker infrasture configd
332d069	30/10 15:04	lequoccuong	fix(socket): change endpoint
1046984	06/11 08:26	lequoccuong	fix(socket): fix sercurity and change port for socket
54ab54e	06/11 08:27	lequoccuong	Merge pull request #8 from QuoocsCuongwf/phu-tai

Phụ lục 3. Minh chứng làm việc nhóm trên GitHub

Kho mã: `https://github.com/QuoocsCuongwf/springboot-websocket-chat`. Hình PL.1 là ảnh chụp trang Insights/Contributors ngày 28/10/2026.



Hình PL.1. *GitHub Insights/Contributors của kho mã, chụp ngày 28/10/2026*

Ảnh hiển thị tài khoản `QuoocsCuongwf`, `ThanhHai151` và công cụ Copilot theo cách tổng hợp của GitHub. Trang Contributors chỉ thống kê nhánh mặc định và chỉ tính các commit gắn được với tài khoản GitHub; tại thời điểm chụp, commit `4699986` của Nguyễn Phú Tài còn nằm trên nhánh `phu-tai` nên chưa xuất hiện. Sau khi PR #8 được hợp nhất ngày 06/11, commit này đã có trên `main`; việc nó hiển thị dưới tài khoản nào còn phụ thuộc vào địa chỉ email trong commit có được liên kết với một tài khoản GitHub hay không. Tài khoản Copilot xuất hiện do hai commit có dòng `Co-authored-by: Copilot`; Copilot là công cụ, không phải thành viên. Ảnh không thay thế việc đọc diff hay xác nhận phân công; lịch sử Git ở Phụ lục 2 là nguồn đối chiếu chính.

Phụ lục 4. Bảng tự kiểm ba tầng

Bảng PL.3. *Bảng tự kiểm kỹ thuật*

Nhóm nội dung	Đối chiếu trong báo cáo	Giới hạn
Cơ chế Spring Boot	Chương 4 với minh chứng bean/điều kiện, rollback qua proxy; Bảng 9.1 đối chiếu từng cơ chế	Không thay thế rubric chính thức của học phần
Kỹ nghệ backend	Test Mockito cho nhánh lỗi, test Spring qua proxy, log và cấu hình đi kèm; hợp đồng REST, STOMP	Chưa kiểm toàn bộ cấu hình production hay mọi nhánh lỗi
Kỹ thuật chuyên sâu	Lab SQL khoá và phân trang; refresh tuần tự; STOMP hai client và đối chiếu DB	Chưa đo tải API, nhiều instance, refresh đồng thời, SockJS; minh chứng E2E thuộc bản cục bộ
Làm việc nhóm	Chương 10 đối chiếu từng commit, PR, nhiệm vụ	Không quy đổi số commit, số dòng thành phần trăm đóng góp

Phụ lục 5. Hands-on Lab: kiểm chứng phòng 1-1 và luồng gửi tin

Mục tiêu: giải thích vì sao giao dịch một mình chưa ngăn được hai giao dịch cùng đọc thấy "chưa có phòng"; kiểm tra bất biến về số phòng và số thành viên; phân biệt vai trò của PostgreSQL, proxy Spring và kênh STOMP. Bài lab làm lại được từ mã nguồn và hồ sơ minh chứng, không cần giao diện.

Môi trường: Linux, JDK 21, Docker đang chạy, Python 3. Dùng môi trường Python riêng với `requirements-evidence.txt`; cơ sở dữ liệu lab là container tạm riêng và script tự xoá khi kết thúc. Không chạy lệnh `ALTER`/`CHECK` thí nghiệm trên cơ sở dữ liệu phát triển đang dùng.

Đoạn mã PL.1. *Các lệnh chạy lab*

```
cd nextchat && ./mvnw clean verify && cd ..
python3 -m venv .venv-evidence && . .venv-evidence/bin/activate
pip install -r tools/requirements-evidence.txt
python tools/e2e_check.py "$PWD" "$PWD/evidence-e2e"   # cần cập nhật đích sang /room/{id} cho main
python tools/sql_lab.py  "$PWD" "$PWD/evidence-sql"
```

Các bước: (1) chạy `./mvnw clean verify`, ghi tên các lớp test và số bài từng lớp; (2) chạy smoke, đối chiếu hai id tin nhắn với `database-messages.json`; (3) chạy lab SQL, đọc hai hàm biến thể và kết quả của từng worker; (4) mở kế hoạch JSON của 100.000 tin ở trang đầu và trang sâu, tìm "Actual Rows" và buffers.

Đoạn mã PL.2. *Khung mã worker cho bài tập (không phải log thực thi)*

```
gate = threading.Barrier(2)
def worker(pair):
    conn = new_connection()          # kết nối riêng, autocommit=False
    gate.wait()
    room_id = call_lab_function(conn, pair)
    conn.commit()
    return room_id
# Chạy (A,B) và (B,A); đợi cả hai; đếm phòng và thành viên sau commit.
```

Bảng PL.4. *Bài tập và đáp án đối chiếu*

Bài tập	Đáp án / tiêu chí kiểm chứng
1. Đổi chiều (A,B) thành (B,A) có đổi khoá không?	`LEAST`/`GREATEST` tạo cùng chuỗi cặp nên cùng mã khoá. Trong lab có khoá: 1 phòng, 2 thành viên, hai id bằng nhau. Đường chèn bỏ qua hàm vẫn chưa được bảo vệ.
2. Vì sao kiểm thử rollback phải đi qua service/proxy?	`ROLLBACK` bằng SQL chỉ chứng minh cơ sở dữ liệu. Test inject bean, khẳng định proxy, gây lỗi chèn rồi đọc DB sau ngoại lệ: hash cũ còn, không có hash thay thế; gỡ ràng buộc thì xoay vòng thành công. Unit test Mockito không kiểm được giao dịch.
3. Người ngoài SEND tới đúng đích `/app` có được lưu hay phát không?	Service kiểm thành viên và từ chối; smoke khẳng định không có MESSAGE, số tin và tin cuối không đổi; không mặc định phải có khung ERROR nghiệp vụ.
4. Trên `main`, SUBSCRIBE `/topic/room/*` có bị chặn không?	Theo mã nguồn: không (R1). Viết test đơn vị và smoke để xác nhận, sau đó áp dụng bản vá mục 4.10.5 và chạy lại.

Lỗi thường gặp: Testcontainers không chạy – kiểm tra Docker daemon và quyền truy cập socket trước khi kết luận lỗi Spring. JWT thiếu hoặc sai – đặt token ở native header `Authorization` của khung CONNECT, không dùng tham số URL. Đăng ký sai đích – trên `main` dùng `/topic/room/{id}` (gạch chéo), không dùng `rooms` số nhiều hay dạng dấu chấm. Thấy CONNECTED nhưng khung sau thiếu Principal – kiểm tra phiên bản interceptor và thuộc tính phiên `stompUsername`. Không dùng `TRUNCATE messages CASCADE` trên cơ sở dữ liệu có dữ liệu vì quan hệ `last_message_id` có thể kéo theo bảng khác. Lượt lab bỏ khoá dùng `pg_sleep` để tạo chồng lấn; bỏ độ trễ hoặc rào chắn thì việc không tái hiện ở một lượt không chứng minh được không có tranh chấp.

Phụ lục 6. Hướng dẫn chạy demo và biên bản kiểm tra trước buổi bảo vệ

PL6.1. Chuẩn bị máy demo

1) Cài JDK 21, Docker (Docker Desktop hoặc Docker Engine), Git; kiểm tra `java -version`, `docker info`.

2) `git clone https://github.com/QuoocsCuongwf/springboot-websocket-chat` và `git checkout main` (commit `54ab54e` hoặc mới hơn nếu đã vá R1–R3).

3) `cd nextchat && cp .env.example .env`; đặt `POSTGRES_PASSWORD` và một `NEXTCHAT_JWT_SECRET` ngẫu nhiên dài ít nhất 32 byte (ví dụ sinh bằng `openssl rand -base64 48`).

4) `docker compose --env-file .env up -d`; chờ healthcheck báo healthy (`docker compose ps`).

5) `set -a && . ./.env && set +a && ./mvnw spring-boot:run`; chờ log "Started NextchatApplication".

6) Mở `http://localhost:8080` ở một cửa sổ thường và một cửa sổ ẩn danh.

Bảng PL.5. *Lỗi thường gặp khi chạy demo và cách xử lý*

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Ứng dụng dừng với "JWT secret must be at least 32 bytes"	Khoá ngắn	Đặt khoá dài hơn trong `.env`, nạp lại biến môi trường
Ứng dụng dừng với "NEXTCHAT_JWT_SECRET must be configured..."	Chạy profile khác `dev`/`test` với khoá mặc định	Đặt `NEXTCHAT_JWT_SECRET` hoặc chạy profile `dev`
"Connection refused" tới PostgreSQL	Container chưa chạy hoặc chạy không có `.env` (lệch giá trị mặc định, R4)	`docker compose --env-file .env up -d`; kiểm tra `POSTGRES_DB`, `POSTGRES_USER` khớp hai bên
Flyway báo checksum mismatch	Đã sửa tệp migration sau khi áp dụng	Không sửa migration cũ; tạo V3 mới hoặc dựng lại cơ sở dữ liệu demo
Client báo "WS: Disconnected"	Token hết hạn, origin không được phép	Đăng nhập lại; chạy đúng cổng 8080; xem nhật ký giao thức ở cột phải
Tạo nhóm báo lỗi 400	Client gửi `name` thay vì `groupName` (R3)	Áp dụng bản sửa client trước buổi demo

PL6.2. Kịch bản demo đề xuất (khoảng 10 phút)

- Phút 0–2: giới thiệu kiến trúc (Hình 3.1) và hợp đồng (Bảng 8.7, 8.8).
- Phút 2–5: đăng ký, đăng nhập hai tài khoản; chỉ JWT inspector; mở phòng 1-1 từ hai phía, chỉ cùng id phòng (khoá advisory).
- Phút 5–7: gửi tin hai chiều; chỉ khung SEND và MESSAGE trong nhật ký; tải lại trang để thấy lịch sử REST khớp.
- Phút 7–8: xoá access token trong `localStorage`, thao tác tiếp để thấy tự làm mới (xoay vòng).
- Phút 8–10: chạy `./mvnw clean verify` (hoặc mở log đã lưu) và giải thích một bài test tích hợp.

Bảng PL.6. *Biên bản kiểm tra trước buổi bảo vệ (điền khi chạy)*

TT	Hạng mục	Đạt / Chưa	Ghi chú, người kiểm
1	`./mvnw clean verify` trên `main`, lưu log	☐ Đạt ☐ Chưa	…
2	Bản vá R1, R2 và hai test mới	☐ Đạt ☐ Chưa	…
3	Sửa R3 (`groupName`) và R5 (token trên URL)	☐ Đạt ☐ Chưa	…
4	Kịch bản smoke cập nhật đích gạch chéo, chạy lại	☐ Đạt ☐ Chưa	…
5	Kịch bản thủ công Bảng 9.4, bước 1–9	☐ Đạt ☐ Chưa	…
6	Thử SUBSCRIBE `/topic/room/*` bằng tài khoản ngoài phòng (sau vá phải bị từ chối)	☐ Đạt ☐ Chưa	…

Phụ lục 7. Câu hỏi ôn tập cho buổi bảo vệ

Bộ câu hỏi dưới đây được nhóm chuẩn bị theo phần việc của từng thành viên; gợi ý trả lời dẫn về mục tương ứng trong báo cáo để mỗi người tự trình bày bằng lời của mình.

Bảng PL.7. *Câu hỏi ôn tập và gợi ý trả lời*

TT	Người trả lời chính	Câu hỏi	Gợi ý trả lời
1	Cường	Auto-configuration biết tạo `DataSource` như thế nào? Làm sao chứng minh?	Lớp AutoConfiguration với `@ConditionalOnClass`/`@ConditionalOnMissingBean`; `ConditionEvaluationReport`, cờ `--debug` (mục 4.1)
2	Cường	Vì sao CSRF được tắt mà vẫn an toàn?	Xác thực bằng header Bearer, không dùng cookie phiên; trình duyệt không tự gửi token (mục 4.5)
3	Cường	Interceptor làm gì ở CONNECT, SUBSCRIBE, SEND?	Mục 4.10.3; nêu rõ R1, R2 trên `main` và bản vá
4	Cường	Lỗi mất Principal là gì, sửa thế nào? Hai cách sửa khác nhau ra sao?	Tình huống 3 (mục 11.2.3); Bảng 4.9
5	Cường	Vì sao dùng `SimpMessagingTemplate` thay vì `@SendTo`?	Đích động `/topic/room/` + id; cả hai phát sau khi phương thức chạy xong (mục 4.10.4)
6	Cường	Simple broker có chạy được nhiều instance không?	Không; cần broker relay (mục 12.4)
7	Hải	Refresh token được lưu và xoay vòng thế nào? Vì sao băm?	32 byte SecureRandom, SHA-256 Base64URL, xoá rồi phát trong giao dịch (Chương 7)
8	Hải	Điều gì xảy ra nếu hai yêu cầu refresh cùng lúc?	Chưa bảo vệ; đề xuất xoá có điều kiện hoặc khoá hàng (Bảng 7.2)
9	Hải	Vì sao cần khoá advisory khi đã có giao dịch?	READ COMMITTED không thấy dữ liệu chưa commit; UNIQUE hiện có không ngăn hai phòng cho cùng cặp (Chương 5)
10	Hải	`LEAST`/`GREATEST` để làm gì? Va chạm băm có sao không?	Cùng khoá cho hai chiều; va chạm chỉ làm chờ thêm, không sai kết quả (mục 5.3)
11	Hải	Vì sao trang rỗng vẫn trả đúng `totalElements`?	Nhánh đếm dự phòng (mục 6.3, Tình huống 2)
12	Hải	OFFSET chậm ở trang sâu vì sao? Cải tiến?	Duyệt rồi bỏ dòng; keyset và chỉ mục có `id` (mục 6.6, 6.7)
13	Hải	`@EntityGraph` và `open-in-view: false` liên quan gì?	Nạp quan hệ trong giao dịch, dựng DTO trong service (mục 4.6)
14	Hải	Vì sao dùng Testcontainers thay H2?	H2 không có PL/pgSQL, advisory lock (mục 2.4.7)
15	Tài	Client gắn token và tự làm mới khi 401 ra sao? Làm sao tránh lặp vô hạn?	`apiFetch` với `retry=false` ở lần thử lại (mục 9.4.2)
16	Tài	Vì sao phải kết nối lại STOMP sau khi làm mới token?	Interceptor chỉ xác minh JWT ở CONNECT (mục 9.4.2)
17	Tài	Vì sao client không tự vẽ tin vừa gửi?	Máy chủ là nguồn sự thật; cùng id, createdAt (mục 8.5)

TT	Người trả lời chính	Câu hỏi	Gợi ý trả lời
18	Tài	Rủi ro của việc lưu token trong `localStorage`?	XSS; `escapeHtml`; cookie HttpOnly là hướng cải tiến (mục 7.4)
19	Tài	Những nhiệm vụ nào chưa hoàn thành và vì sao?	Dockerfile, CI/CD, load test, presence (Bảng 10.10); kế hoạch ở Chương 12
20	Cả nhóm	Số test đạt có chứng minh hệ thống không lỗi không?	Không; phân biệt chức năng với tải, bản cục bộ với `main` (mục 4.9.3, 9.5)

Phụ lục 8. Lộ trình từ ngày nộp đến buổi bảo vệ

Bảng PL.8. *Việc còn phải thực hiện sau ngày nộp*

Mốc	Việc cần làm	Tiêu chí hoàn thành
06/11 (sau khi nộp)	Chạy `./mvnw clean verify` trên `main`; lưu log vào hồ sơ	Log có tổng số bài, không failure/error
Tuần 10, trước buổi bảo vệ	Vá R1, R2 kèm hai test; sửa R3, R5; cập nhật và chạy lại smoke; điền biên bản Phụ lục 6	Biên bản đủ 6 hạng mục; demo chạy theo kịch bản PL6.2
Buổi bảo vệ	Trình bày theo kịch bản; mỗi thành viên trả lời phần việc của mình	Theo lịch và thủ tục của giảng viên

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN
HƯỚNG DẪN THỰC HÀNH (HANDS-ON LAB)

MÔN HỌC: Các công nghệ lập trình hiện đại (841072)

ĐỀ TÀI: Nghiên cứu Spring Boot – xây dựng hệ thống chat real-time bằng WebSocket

Nhóm: 24 – Lớp: DCT1221 – Nhóm học phần: 01

SINH VIÊN THỰC HIỆN:

HỌ VÀ TÊN	MSSV
Huỳnh Thanh Hải	3123410091
Lê Quốc Cường	3123410040
Nguyễn Phú Tài	3121410435

Giảng viên: Phạm Thi Vương

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC (HANDS-ON LAB)

Xây dựng Hệ thống Nhắn tin Thời gian thực NextChat bằng Spring Boot 4.1.1, WebSocket/STOMP, JWT và PostgreSQL 16

MỤC LỤC

1. Giới thiệu & Mục tiêu bài Hands-on Lab

2. Chuẩn bị Môi trường & Công cụ (Prerequisites)

3. Cách 1: Khởi chạy Nhanh Dự án từ Kho mã Nguồn (Quick Start trong 5 phút)

4. Cách 2: Thực hành Xây dựng Lại Toàn bộ Dự án Từng Bước (Step-by-Step)

- Bước 1: Khởi tạo Dự án Maven, Docker Compose & Cấu hình Ngoài
- Bước 2: Thiết kế Lược đồ CSDL PostgreSQL 16, Flyway & Hàm PL/pgSQL
- Bước 3: Xây dựng Tầng Thực thể JPA (Entities) & Repositories chống N+1
- Bước 4: Xây dựng Khung Phản hồi Chuẩn (`ApiResponse`) & Xử lý Lỗi Tập trung
- Bước 5: Xây dựng Bảo mật Spring Security, Stateless JWT & Refresh Token Rotation
- Bước 6: Xây dựng REST API Người dùng & Phòng Chat (`User`, `Room`)
- Bước 7: Xây dựng Kênh WebSocket / STOMP & Bảo mật Channel Interceptor
- Bước 8: Xây dựng Giao diện Client Web Thử nghiệm (`index.html` & `app.js`)

5. Thực hành Kiểm thử Tự động, Kiểm chứng Thực nghiệm & Demo End-to-End

6. Bảng Xử lý Sự cố Thường gặp (Troubleshooting Guide)

1. Giới thiệu & Mục tiêu bài Hands-on Lab

Tài liệu **Hands-on Lab** này được biên soạn nhằm hướng dẫn người đọc tự tay thiết lập, xây dựng lại từ đầu, vận hành và kiểm chứng toàn bộ hệ thống **NextChat** — ứng dụng nhắn tin thời gian thực sử dụng **Spring Boot 4.1.1 (Java 21)**, **Spring Security (Stateless JWT + Refresh Token Rotation)**, **Spring Data JPA**, **Spring WebSocket/STOMP**, **Flyway** và **PostgreSQL 16**.

Sau khi hoàn thành bài thực hành này, người đọc sẽ nắm vững và tự thực hiện được:

1. Tổ chức dự án Spring Boot theo kiến trúc **Package-by-Feature + Common** (`auth`, `user`, `room`, `common`).

2. Cấu hình PostgreSQL 16 bằng Docker Compose kết hợp **Flyway Migration** (`V1`, `V2`) và chế độ kiểm chứng lược đồ nghiêm ngặt `spring.jpa.hibernate.ddl-auto: validate`.

3. Giải quyết bài toán **Race Condition** khi hai người dùng tạo phòng chat 1-1 đồng thời bằng hàm **PL/pgSQL** `get_or_create_direct_room` và khóa tư vấn mức giao dịch `pg_advisory_xact_lock`.

4. Tối ưu hóa truy vấn danh sách phòng bằng `@EntityGraph` (chống lỗi **N+1 query** khi tắt `open-in-view: false`) và phân trang lịch sử tin nhắn bằng hàm PL/pgSQL `get_room_messages`.

5. Triển khai cơ chế xác thực **JWT không trạng thái (Stateless)** kết hợp **Refresh Token xoay vòng (Rotation)** lưu trữ dưới dạng băm **SHA-256 Base64URL**.

6. Xây dựng kênh truyền thông thời gian thực **WebSocket/STOMP** có kiểm soát truy cập tại tầng `ChannelInterceptor` (`CONNECT`, `SUBSCRIBE`, `SEND`) và cơ chế **lưu CSDL trước — phát broadcast sau**.

7. Tích hợp giao diện **Client Web thử nghiệm** (`index.html`, `app.js`) và chạy bộ kiểm thử tự động

JUnit 5 + Testcontainers cùng các kịch bản kiểm chứng thực nghiệm.

2. Chuẩn bị Môi trường & Công cụ (Prerequisites)

Trước khi bắt đầu, hãy đảm bảo máy tính của bạn (Linux, macOS hoặc Windows với WSL2) đã cài đặt các công cụ sau:

Công cụ	Phiên bản yêu cầu	Lệnh kiểm tra trên Terminal	Ghi chú
JDK (Java)	**Java 21 LTS** (OpenJDK 21)	`java -version`	Bắt buộc tương thích với Spring Boot 4.1.1
Docker & Compose	Docker Engine 24+ & Compose v2	`docker info && docker compose version`	Dùng để chạy PostgreSQL 16 và Testcontainers
Git	Git 2.30+	`git --version`	Quản lý mã nguồn và clone repository
Maven Wrapper	Đi kèm trong dự án (`./mvnw`)	`./mvnw -version`	Không cần cài đặt Maven toàn cục
cURL / Trình duyệt	Chrome / Firefox / Edge + `curl`	`curl --version`	Kiểm thử REST API và giao diện WebSocket
Python 3 *(Tuỳ chọn)*	Python 3.10+	`python3 --version`	Phục vụ chạy kịch bản E2E Smoke & SQL Lab

3. Cách 1: Khởi chạy Nhanh Dự án từ Kho mã Nguồn (Quick Start trong 5 phút)

Nếu bạn muốn tải mã nguồn hoàn chỉnh của Nhóm 24 về máy để chạy thử và quan sát hệ thống ngay lập tức trước khi đi vào chi tiết từng file mã nguồn, hãy thực hiện đúng **6 bước** dưới đây:

Bước 3.1. Clone kho mã nguồn từ GitHub

```
git clone https://github.com/QuoocsCuongwf/springboot-websocket-chat.git
cd springboot-websocket-chat/nextchat
```

Bước 3.2. Tạo file cấu hình biến môi trường `.env`

Sao chép file mẫu `.env.example` sang `.env` và cấu hình khóa bí mật JWT dài tối thiểu 32 bytes:

```
cp .env.example .env
```

Nội dung chuẩn của file `nextchat/.env`:

```
POSTGRES_DB=messenger_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=change-this-database-password
POSTGRES_PORT=5432
NEXTCHAT_JWT_SECRET=nextchat-super-secret-jwt-key-at-least-32-bytes-long-2026
```

Bước 3.3. Khởi động container PostgreSQL 16 bằng Docker Compose

```
docker compose --env-file .env up -d
```

Kiểm tra trạng thái container đến khi cột `STATUS` hiển thị `(healthy)`:

```
docker compose ps
```

Bước 3.4. Nạp biến môi trường và khởi chạy ứng dụng Spring Boot

```
set -a && . ./.env && set +a
./mvnw spring-boot:run
```

Khi ứng dụng khởi động thành công, bạn sẽ thấy trên console:

• Flyway tự động thực thi `V1__messenger_schema.sql` và `V2__lock_direct_room_creation.sql`.

• Hibernate kiểm chứng khớp toàn bộ lược đồ ở chế độ `ddl-auto: validate`.

• `SeedDataConfig` khởi tạo sẵn tài khoản thử nghiệm `testuser` (mật khẩu mặc định ở profile `dev`: `ChangeMe123!`).

• Dòng thông báo: `Started NextchatApplication in ... seconds`.

Bước 3.5. Trải nghiệm ngay trên Giao diện Web

1. Mở trình duyệt ở cửa sổ thường tại địa chỉ: `http://localhost:8080`

2. Mở thêm một cửa sổ ẩn danh (Incognito) tại địa chỉ: `http://localhost:8080`

3. Đăng ký 2 tài khoản (ví dụ `alice` và `bob`), đăng nhập ở 2 cửa sổ, tìm kiếm nhau ở ô **Tìm người dùng**, bấm tạo phòng chat 1-1 và nhắn tin qua lại thời gian thực.

4. Cách 2: Thực hành Xây dựng Lại Toàn bộ Dự án Từng Bước (Step-by-Step)

Phần này hướng dẫn bạn tự tay xây dựng lại toàn bộ dự án NextChat từ cấu trúc thư mục rỗng đến khi hoàn chỉnh tất cả các tầng.

Tổng quan Cấu trúc Thư mục Dự án

```
springboot-websocket-chat/
├── database.yml # Cấu hình HikariCP, JPA validate, tắt OSIV
├── README.md
└── nextchat/
    ├── pom.xml # Khai báo phụ thuộc Spring Boot 4.1.1 & Java 21
    ├── docker-compose.yml # Định nghĩa container postgres:16-alpine
    ├── .env.example # Mẫu biến môi trường
    └── src/
        ├── main/
        │ ├── java/com/quoocscuongwf/nextchat/
        │ │ ├── NextchatApplication.java
        │ │ ├── common/
        │ │ │ ├── config/ # WebSecurityConfig, WebSocketConfig, PasswordConfig,
SeedDataConfig
        │ │ │ ├── dto/ # ApiResponse<T>
        │ │ │ ├── exception/ # GlobalExceptionHandler
        │ │ │ └── security/ # JwtService, JwtAuthenticationFilter,
JwtStompChannelInterceptor, ...
        │ │ ├── auth/ # AuthController, RefreshToken, RefreshTokenRepository,
RefreshTokenService, dto/
        │ │ ├── user/ # User, Role, UserRepository, RoleRepository, UserController, dto/
        │ │ └── room/ # Conversation, Message, Participant, RoomService, RoomController,
RoomWebSocketController, dto/
        │ └── resources/
        │ ├── application.yaml # Cấu hình trung tâm của Spring Boot
        │ ├── db/migration/
        │ │ ├── V1__messenger_schema.sql # Lược đồ 20 bảng + hàm PL/pgSQL
        │ │ └── V2__lock_direct_room_creation.sql
        │ └── static/
        │ ├── index.html # Giao diện SPA kiểm thử REST & STOMP
        │ └── app.js # Logic xử lý REST, Refresh Token & STOMP client
        └── test/
            └── java/com/quoocscuongwf/nextchat/
                ├── api/AuthAndRoomIntegrationTest.java
                └── security/JwtStompChannelInterceptorTest.java
```

Bước 1: Khởi tạo Dự án Maven, Docker Compose & Cấu hình Ngoài

1.1. Thiết lập file `nextchat/pom.xml`

Tạo file `nextchat/pom.xml` sử dụng `spring-boot-starter-parent` phiên bản `4.1.1` và

`java.version` là `21`. Lưu ý ở thế hệ Spring Boot 4, starter cho Web MVC mang tên `spring-boot-starter-webmvc` và starter cho Flyway là `spring-boot-starter-flyway`:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>
    <groupId>com.quoocscuongwf</groupId>
    <artifactId>nextchat</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
        <testcontainers.version>1.21.4</testcontainers.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Core Starters -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jackson</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-websocket</artifactId>
        </dependency>

        <!-- Database & Migration (PostgreSQL 16 + Flyway) -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-database-postgresql</artifactId>
        </dependency>

        <!-- Thư viện JJWT 0.12.6 -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
```

```
            <version>0.12.6</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.12.6</version>
            <scope>runtime</scope>
        </dependency>

        <!-- Test Dependencies (JUnit 5 + Testcontainers) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-websocket-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>postgresql</artifactId>
            <version>${testcontainers.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

1.2. Thiết lập `nextchat/docker-compose.yml`

Tạo file `nextchat/docker-compose.yml` để khởi chạy PostgreSQL 16 kèm `healthcheck`:

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: nextchat-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-nextchat}
      POSTGRES_USER: ${POSTGRES_USER:-nextchat}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-nextchat123}
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-nextchat} -d ${POSTGRES_DB:-
nextchat}"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

1.3. Thiết lập `database.yml` và `nextchat/src/main/resources/application.yaml`

Tạo file `database.yml` ở thư mục gốc của kho mã nhằm quản lý cấu hình HikariCP connection pool, bật `ddl-auto: validate` và tắt `open-in-view: false` (buộc mọi truy vấn lazy phải được giải quyết trong phạm vi `@Transactional` tại tầng Service):

```
# File: database.yml
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:${POSTGRES_PORT:5432}/$
{POSTGRES_DB:messenger_db}}
    username: ${SPRING_DATASOURCE_USERNAME:${POSTGRES_USER:postgres}}
    password: ${SPRING_DATASOURCE_PASSWORD:${POSTGRES_PASSWORD:change-this-database-
password}}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      idle-timeout: 30000
      connection-timeout: 20000
      max-lifetime: 1800000
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
        show_sql: false
```

Tạo file `nextchat/src/main/resources/application.yaml`:

```
# File: nextchat/src/main/resources/application.yaml
spring:
  application:
    name: nextchat
  profiles:
    default: dev
  config:
    import:
      - optional:file:../database.yml
      - optional:file:.env
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:${POSTGRES_PORT:5432}/$
{POSTGRES_DB:messenger_db}}
    username: ${SPRING_DATASOURCE_USERNAME:${POSTGRES_USER:postgres}}
    password: ${SPRING_DATASOURCE_PASSWORD:${POSTGRES_PASSWORD:change-this-database-
password}}
  jpa:
    hibernate:
      ddl-auto: validate

app:
  security:
    jwt:
      secret: ${NEXTCHAT_JWT_SECRET:development-only-secret-change-me-before-
production-12345}
```

```
      expiration: ${NEXTCHAT_JWT_EXPIRATION:PT15M}
      refresh-token-expiration: ${NEXTCHAT_JWT_REFRESH_EXPIRATION:P7D}
  seed:
    password: ${NEXTCHAT_SEED_PASSWORD:ChangeMe123!}
```

Bước 2: Thiết kế Lược đồ CSDL PostgreSQL 16, Flyway & Hàm PL/pgSQL

Flyway tự động quét thư mục `src/main/resources/db/migration`. Chúng ta tạo 2 tệp migration theo đúng thứ tự phiên bản:

2.1. Tạo `V1__messenger_schema.sql`

File `V1__messenger_schema.sql` khởi tạo 20 bảng của hệ thống nhắn tin (trong đó 6 bảng cốt lõi được ánh xạ trực tiếp sang JPA Entity: `users`, `roles`, `user_roles`, `refresh_tokens`, `conversations`, `conversation_participants`, `messages`) cùng với trigger cập nhật `updated_at` và 2 hàm PL/pgSQL quan trọng.

Dưới đây là cấu trúc các bảng cốt lõi và hàm phân trang `get_room_messages` trong `V1__messenger_schema.sql`:

```
-- Trigger tự động cập nhật cột updated_at
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Bảng người dùng, vai trò và refresh_tokens
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    avatar_url VARCHAR(500),
    bio VARCHAR(255),
    phone_number VARCHAR(20),
    status VARCHAR(10) NOT NULL DEFAULT 'OFFLINE'
                        CHECK (status IN ('ONLINE','OFFLINE','AWAY','BUSY')),
    last_seen_at TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE roles (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE user_roles (
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id BIGINT NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

CREATE TABLE refresh_tokens (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token VARCHAR(255) NOT NULL UNIQUE,
    expiry_date TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Bảng hội thoại (PRIVATE / GROUP), thành viên và tin nhắn
```

```
CREATE TABLE conversations (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    type VARCHAR(10) NOT NULL CHECK (type IN ('PRIVATE','GROUP')),
    name VARCHAR(150),
    avatar_url VARCHAR(500),
    created_by BIGINT NOT NULL REFERENCES users(id),
    last_message_id BIGINT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE conversation_participants (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    nickname VARCHAR(100),
    role VARCHAR(10) NOT NULL DEFAULT 'MEMBER'
                                CHECK (role IN ('OWNER','ADMIN','MEMBER')),
    is_muted BOOLEAN NOT NULL DEFAULT FALSE,
    is_pinned BOOLEAN NOT NULL DEFAULT FALSE,
    is_archived BOOLEAN NOT NULL DEFAULT FALSE,
    unread_count INT NOT NULL DEFAULT 0,
    last_read_message_id BIGINT,
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    left_at TIMESTAMP,
    CONSTRAINT uq_conv_participant UNIQUE (conversation_id, user_id)
);

CREATE TABLE messages (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(id),
    message_type VARCHAR(10) NOT NULL DEFAULT 'TEXT'
                                  CHECK (message_type IN
('TEXT','IMAGE','VIDEO','AUDIO','FILE','STICKER','SYSTEM')),
    content TEXT,
    reply_to_message_id BIGINT REFERENCES messages(id) ON DELETE SET NULL,
    forwarded_from_id BIGINT REFERENCES messages(id) ON DELETE SET NULL,
    is_edited BOOLEAN NOT NULL DEFAULT FALSE,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Hàm PL/pgSQL phân trang lịch sử tin nhắn kèm tổng số bản ghi
CREATE OR REPLACE FUNCTION get_room_messages(
    p_room_id BIGINT,
    p_page INT DEFAULT 0,
    p_size INT DEFAULT 20
)
RETURNS TABLE (
    id BIGINT,
    conversation_id BIGINT,
    sender_id BIGINT,
    sender_username VARCHAR(50),
    sender_full_name VARCHAR(150),
    sender_avatar_url VARCHAR(500),
    message_type VARCHAR(10),
    content TEXT,
    reply_to_message_id BIGINT,
    is_edited BOOLEAN,
    is_deleted BOOLEAN,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    total_count BIGINT
) AS $$
DECLARE
    v_offset INT := GREATEST(p_page, 0) * GREATEST(p_size, 1);
    v_total BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_total
```

```
    FROM messages m
    WHERE m.conversation_id = p_room_id
      AND m.is_deleted = FALSE;

    RETURN QUERY
    SELECT
        m.id, m.conversation_id, m.sender_id,
        u.username AS sender_username,
        u.full_name AS sender_full_name,
        u.avatar_url AS sender_avatar_url,
        m.message_type, m.content, m.reply_to_message_id,
        m.is_edited, m.is_deleted, m.created_at, m.updated_at,
        COALESCE(v_total, 0) AS total_count
    FROM messages m
    JOIN users u ON m.sender_id = u.id
    WHERE m.conversation_id = p_room_id
      AND m.is_deleted = FALSE
    ORDER BY m.created_at DESC, m.id DESC
    LIMIT GREATEST(p_size, 1) OFFSET v_offset;
END;
$$ LANGUAGE plpgsql;
```

2.2. Tạo `V2__lock_direct_room_creation.sql` (Chống Race Condition bằng Khóa Advisory)

Khi hai người dùng A và B đồng thời bấm nhắn tin cho nhau ở cùng một mili-giây, mức cô lập mặc định `READ COMMITTED` của PostgreSQL không ngăn được việc cả hai giao dịch cùng kiểm tra thấy "chưa có phòng" và cùng `INSERT` tạo ra 2 phòng chat 1-1 trùng lặp.

Để giải quyết triệt để, tạo file `V2__lock_direct_room_creation.sql` sử dụng khóa tư vấn mức giao dịch `pg_advisory_xact_lock` trên mã băm 64-bit chuẩn hóa thứ tự `LEAST(p_user_a, p_user_b) || ':' || GREATEST(p_user_a, p_user_b)`:

```
CREATE OR REPLACE FUNCTION get_or_create_direct_room(
    p_user_a BIGINT,
    p_user_b BIGINT
)
RETURNS BIGINT AS $$
DECLARE
    v_room_id BIGINT;
BEGIN
    IF p_user_a = p_user_b THEN
        RAISE EXCEPTION 'A private room requires two different users';
    END IF;

    -- Khóa advisory mức giao dịch: cùng một khóa cho cả chiều (A,B) lẫn (B,A)
    PERFORM pg_advisory_xact_lock(
        hashtextextended(
            LEAST(p_user_a, p_user_b)::TEXT || ':' || GREATEST(p_user_a,
p_user_b)::TEXT,
            0
        )
    );

    SELECT cp1.conversation_id INTO v_room_id
    FROM conversation_participants cp1
    JOIN conversation_participants cp2 ON cp1.conversation_id = cp2.conversation_id
    JOIN conversations c ON c.id = cp1.conversation_id
    WHERE c.type = 'PRIVATE'
      AND cp1.user_id = p_user_a
      AND cp2.user_id = p_user_b
      AND cp1.left_at IS NULL
      AND cp2.left_at IS NULL
    ORDER BY c.id
    LIMIT 1;

    IF v_room_id IS NOT NULL THEN
        RETURN v_room_id;
    END IF;
```

```
    INSERT INTO conversations (type, created_by, created_at, updated_at)
    VALUES ('PRIVATE', p_user_a, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
    RETURNING id INTO v_room_id;

    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_a, 'OWNER', CURRENT_TIMESTAMP);

    INSERT INTO conversation_participants (conversation_id, user_id, role, joined_at)
    VALUES (v_room_id, p_user_b, 'MEMBER', CURRENT_TIMESTAMP);

    RETURN v_room_id;
END;
$$ LANGUAGE plpgsql;
```

Bước 3: Xây dựng Tầng Thực thể JPA (Entities) & Repositories chống N+1

1. **Ánh xạ các JPA Entity:** Tạo 6 lớp Entity (`User`, `Role`, `RefreshToken`, `Conversation`, `ConversationParticipant`, `Message`) với `@Id @GeneratedValue(strategy = GenerationType.IDENTITY)` và kiểu thời gian `java.time.Instant` khớp chính xác với các cột trong `V1__messenger_schema.sql`.
2. **Tối ưu truy vấn với `@EntityGraph` trong `ConversationRepository.java`:**

Do chúng ta đã tắt `open-in-view: false`, nếu nạp danh sách phòng rồi duyệt qua `participants` và `lastMessage` sẽ gây ra lỗi `LazyInitializationException` hoặc phát sinh **N+1 câu lệnh SQL**. Chúng ta giải quyết bằng `@EntityGraph`:

```
package com.quoocscuongwf.nextchat.room;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface ConversationRepository extends JpaRepository<Conversation, Long> {

    @EntityGraph(attributePaths = {
        "participants", "participants.user", "createdBy", "lastMessage",
"lastMessage.sender"
    })
    @Query("""
        SELECT DISTINCT c FROM Conversation c
        JOIN c.participants p
        WHERE p.user.id = :userId AND p.leftAt IS NULL
        ORDER BY c.updatedAt DESC
    """)
    List<Conversation> findRoomsByUserId(@Param("userId") Long userId);

    @Query(value = "SELECT get_or_create_direct_room(:userA, :userB)", nativeQuery =
true)
    Long callGetOrCreateDirectRoom(@Param("userA") Long userA, @Param("userB") Long
userB);
}
```

Bước 4: Xây dựng Khung Phản hồi Chuẩn (`ApiResponse`) & Xử lý Lỗi Tập trung

Tất cả các REST API dưới `/api/v1` đều trả về cùng một cấu trúc phong bì JSON thống nhất thông qua Java `record ApiResponse<T>`:

```
package com.quoocscuongwf.nextchat.common.dto;

import java.time.Instant;

public record ApiResponse<T>(
        boolean success,
        String message,
        T data,
```

```
        Instant timestamp
) {
    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(true, message, data, Instant.now());
    }

    public static <T> ApiResponse<T> error(String message, T data) {
        return new ApiResponse<>(false, message, data, Instant.now());
    }
}
```

Kết hợp với `@RestControllerAdvice` trong `GlobalExceptionHandler.java` để bắt mọi ngoại lệ (`MethodArgumentNotValidException`, `ResponseStatusException`, `BadCredentialsException`, `DataIntegrityViolationException`) và trả về đúng mã HTTP kèm phong bì `ApiResponse`.

Bước 5: Xây dựng Bảo mật Spring Security, Stateless JWT & Refresh Token Rotation

5.1. Dịch vụ ký & xác minh JWT (`JwtService.java`)

`JwtService` sử dụng thuật toán HMAC-SHA256 (`Keys.hmacShaKeyFor`). Đặc biệt, hàm khởi tạo kiểm tra độ dài khóa tối thiểu 32 bytes và **tự động từ chối khởi động ứng dụng** nếu phát hiện khóa mặc định được dùng ngoài profile `dev` hoặc `test`:

```
if (secret.getBytes(StandardCharsets.UTF_8).length < 32) {
    throw new IllegalArgumentException("JWT secret must be at least 32 bytes");
}
if (DEVELOPMENT_SECRET.equals(secret)
        && !environment.acceptsProfiles(Profiles.of("dev", "test"))) {
    throw new IllegalStateException("NEXTCHAT_JWT_SECRET must be configured outside dev
and test profiles");
}
```

5.2. Xoay vòng Refresh Token băm SHA-256 (`RefreshTokenService.java`)

- Khi người dùng đăng nhập hoặc làm mới token, hệ thống sinh 32 bytes ngẫu nhiên bằng `SecureRandom` (mã hóa Base64URL không padding) để trả về cho client.
- Cơ sở dữ liệu **không lưu token thô** mà chỉ lưu bản băm `SHA-256` ở cột `refresh_tokens.token`.
- Phương thức `rotateRefreshToken(String rawToken)` chạy trong `@Transactional`: tìm bản băm của token cũ, kiểm tra hạn sử dụng và trạng thái tài khoản, **xóa ngay bản ghi token cũ** rồi phát hành cặp token mới. Nếu kẻ xấu đánh cắp và gửi lại refresh token cũ đã qua sử dụng, hệ thống trả về `401 Unauthorized`.

5.3. Cấu hình chuỗi Filter bảo mật (`WebSecurityConfig.java`)

Thiết lập `SessionCreationPolicy.STATELESS`, tắt CSRF (do dùng header `Authorization: Bearer <token>`), mở quyền truy cập cho tài nguyên tĩnh (`/`, `/index.html`, `/app.js`), các endpoint bắt tay WebSocket (`/ws/**`, `/ws-chat/**`) và `/api/v1/auth/**`, đồng thời yêu cầu xác thực cho mọi đường dẫn `/api/**`.

Bước 6: Xây dựng REST API Người dùng & Phòng Chat (`User`, `Room`)

Triển khai đầy đủ 9 REST API endpoints theo hợp đồng chuẩn:

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng
`POST`	`/api/v1/auth/register`	Đăng ký tài khoản mới (`201 Created`)
`POST`	`/api/v1/auth/login`	Đăng nhập, nhận `accessToken` + `refreshToken` xoay vòng

`POST`	`/api/v1/auth/refresh`	Xoay vòng `refreshToken` nhận cặp token mới
`GET`	`/api/v1/users/me`	Lấy thông tin hồ sơ người dùng hiện tại
`GET`	`/api/v1/users/search?q=...`	Tìm kiếm người dùng theo `username` hoặc `fullName`
`GET`	`/api/v1/rooms`	Lấy danh sách phòng chat của người dùng hiện tại
`POST`	`/api/v1/rooms/direct`	Tạo hoặc lấy phòng chat 1-1 nguyên tử (`targetUserId`)
`POST`	`/api/v1/rooms/group`	Tạo phòng chat nhóm (`groupName`, `memberIds`)
`GET`	`/api/v1/rooms/{roomId}/messages`	Phân trang lịch sử tin nhắn (`page`, `size` từ 1 đến 100)

Lưu ý kỹ thuật quan trọng trong `RoomService.getRoomMessages`:

Khi truy vấn một trang nằm ngoài phạm vi dữ liệu (ví dụ `page=5` khi phòng chỉ có 2 tin nhắn), hàm `get_room_messages` trả về 0 dòng nên không thể đọc cột `total_count` từ dòng kết quả. Do đó, `RoomService` cài đặt nhánh dự phòng gọi `messageRepository.countByConversationIdAndDeletedFalse(roomId)` khi `rows.isEmpty()` để luôn trả về đúng `totalElements` cho client.

Bước 7: Xây dựng Kênh WebSocket / STOMP & Bảo mật Channel Interceptor

7.1. Cấu hình Message Broker (`WebSocketConfig.java`)

- Đăng ký 2 endpoint bắt tay: `/ws` (WebSocket thuần phục vụ kiểm thử tự động) và `/ws-chat` (kích hoạt `.withSockJS()` phục vụ trình duyệt web).
- Tiền tố đích ứng dụng: `/app` (ánh xạ vào `@MessageMapping`).
- Tiền tố đích Simple Broker: `/topic` và `/queue`.
- Gắn `JwtStompChannelInterceptor` vào `configureClientInboundChannel`.

7.2. Kiểm soát truy cập STOMP (`JwtStompChannelInterceptor.java`)

1. **Tại khung `CONNECT`:** Đọc JWT từ native header `Authorization: Bearer <token>`, xác minh chữ ký và hạn dùng qua `JwtService`, nạp `UserDetails`, gán `Authentication` vào accessor và lưu `stompUsername` vào `sessionAttributes`.

2. **Tại các khung tiếp theo (`SUBSCRIBE`, `SEND`):** Khôi phục `Authentication` cho khung truyền từ `sessionAttributes` để `@MessageMapping` luôn nhận được `Principal` hợp lệ.

3. **Tại khung `SUBSCRIBE` tới `/topic/room/{roomId}`:** Kiểm tra người dùng có phải là thành viên đang hoạt động (`left_at IS NULL`) của phòng hay không; nếu là người ngoài phòng thì lập tức ném `IllegalArgumentException` từ chối đăng ký.

4. Tăng cường bảo mật (Bản vá khuyến nghị trước bảo vệ): Trong phương thức `preSend`, kiểm tra chặn các đích `SUBSCRIBE` mang ký tự đại diện (`/topic/room/*`) và chặn khung `SEND` gửi trực tiếp vào `/topic/...` (bắt buộc mọi khung `SEND` của client phải đi qua `/app/room/{roomId}`).

7.3. Xử lý gửi tin nhắn (`RoomWebSocketController.java`)

```
@MessageMapping("/room/{roomId}")
public void sendMessage(
        @DestinationVariable Long roomId,
        @Valid @Payload SendMessageRequest request,
        Principal principal) {
    if (principal == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Authentication
required");
    }
    Long userId = userRepository.findByUsername(principal.getName())
            .filter(user -> user.isActive() && !user.isDeleted())
            .map(User::getId)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED,
"Authentication required"));
    // Lưu vào PostgreSQL và cập nhật last_message_id trong giao dịch trước khi phát
broadcast
    MessageDTO message = roomService.sendMessage(
            userId, roomId, request.content(), request.messageType(),
request.replyToMessageId());
    messagingTemplate.convertAndSend("/topic/room/" + roomId, message);
}
```

Bước 8: Xây dựng Giao diện Client Web Thử nghiệm (`index.html` & `app.js`)

Đặt 2 tệp `index.html` và `app.js` tại `nextchat/src/main/resources/static/` để phục vụ trực tiếp trên cổng `8080`:

- Hàm `apiFetch(path, options, retry = true)` tự động đính kèm `Authorization: Bearer <accessToken>`. Nếu nhận mã `401`, hàm tự động gọi `POST /api/v1/auth/refresh` để xoay vòng token, lưu cặp token mới vào `localStorage`, kết nối lại STOMP và thử lại request ban đầu với `retry = false`.
- Khi người dùng gửi tin nhắn qua `stompClient.send("/app/room/" + activeRoom.id, {}, JSON.stringify(payload))`, giao diện **không tự vẽ tin nhắn cục bộ** mà chờ nhận khung `MESSAGE` từ `/topic/room/{roomId}` do server phát về (kèm `id` và `createdAt` chính thức từ PostgreSQL).

5. Thực hành Kiểm thử Tự động, Kiểm chứng Thực nghiệm & Demo End-to-End

5.1. Chạy Bộ Kiểm thử Tự động JUnit 5 & Testcontainers

Đảm bảo Docker đang chạy, sau đó đứng tại thư mục `nextchat/` và thực thi:

```
cd nextchat
./mvnw clean verify
```

- **Testcontainers** sẽ tự động khởi tạo một container `postgres:16-alpine` cô lập, chạy Flyway migrations (`V1`, `V2`) và thực thi toàn bộ các bài kiểm thử đơn vị (`JwtStompChannelInterceptorTest`) cùng kiểm thử tích hợp (`AuthAndRoomIntegrationTest`), bao gồm cả bài kiểm thử **2 luồng đồng thời tạo phòng 1-1 (`CountDownLatch`)** và kiểm thử gửi/nhận tin nhắn qua STOMP WebSocket thật.

5.2. Kiểm thử Toàn bộ Luồng REST API bằng `curl` trên Terminal

Khi ứng dụng đang chạy ở `http://localhost:8080`, mở một cửa sổ Terminal mới và thực hành tuần tự các lệnh sau:

(1) Đăng ký 2 tài khoản `user_a` và `user_b`

```
curl -s -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"username":"user_a","email":"user_a@example.com","password":"Password123!","fullName":
"Nguyen Van A"}'
```

```
curl -s -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"username":"user_b","email":"user_b@example.com","password":"Password123!","fullName":
"Tran Thi B"}'
```

(2) Đăng nhập tài khoản `user_a` và trích xuất `ACCESS_TOKEN`, `REFRESH_TOKEN`

```
LOGIN_RES=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"user_a","password":"Password123!"}')
echo "$LOGIN_RES"
```

(3) Kiểm chứng Tạo phòng 1-1 Nguyên tử (`POST /api/v1/rooms/direct`)

Thay `<ACCESS_TOKEN>` và `<USER_B_ID>` bằng giá trị nhận được ở trên:

```
curl -s -X POST http://localhost:8080/api/v1/rooms/direct \
  -H "Authorization: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"targetUserId": <USER_B_ID>}'
```

Gọi lại lệnh trên nhiều lần hoặc gọi ngược chiều từ `user_b` sang `user_a`: hệ thống luôn trả về **cùng một `id` phòng duy nhất** nhờ hàm `get_or_create_direct_room` và khóa `pg_advisory_xact_lock`.

(4) Kiểm chứng Xoay vòng Refresh Token (`POST /api/v1/auth/refresh`)

```
curl -s -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refreshToken": "<REFRESH_TOKEN_CU>"}'
```

- Lần gọi thứ nhất trả về `200 OK` kèm cặp `accessToken` và `refreshToken` mới.
- Thử gọi lại đúng lệnh trên một lần nữa với `<REFRESH_TOKEN_CU>`: hệ thống lập tức trả về `401 Unauthorized` (`"Invalid or expired refresh token"`), chứng minh token cũ đã bị thu hồi ngay sau lần sử dụng đầu tiên.

5.3. Kiểm chứng Trực tiếp trong Cơ sở Dữ liệu PostgreSQL

Bạn có thể mở phiên `psql` trực tiếp vào container `nextchat-postgres` để kiểm tra dữ liệu đã lưu:

```
docker exec -it nextchat-postgres psql -U postgres -d messenger_db
```

Thực thi các câu truy vấn kiểm chứng:

```
-- 1. Kiểm tra lịch sử migration Flyway
SELECT installed_rank, version, description, success FROM flyway_schema_history;

-- 2. Kiểm tra Refresh Token chỉ lưu dưới dạng băm SHA-256 (43 ký tự Base64URL)
SELECT id, user_id, token, LENGTH(token) AS hash_len, expiry_date FROM refresh_tokens;

-- 3. Kiểm tra danh sách phòng chat và tin nhắn cuối cùng
SELECT id, type, name, created_by, last_message_id, updated_at FROM conversations;

-- 4. Kiểm tra hàm phân trang PL/pgSQL get_room_messages
SELECT id, sender_username, content, created_at, total_count FROM get_room_messages(1,
0, 10);
```

6. Bảng Xử lý Sự cố Thường gặp (Troubleshooting Guide)

Triệu chứng / Thông báo lỗi	Nguyên nhân gốc rễ	Cách khắc phục triệt để
`IllegalArgumentException: JWT secret must be at least 32 bytes`	Biến `NEXTCHAT_JWT_SECRET` trong `.env` ngắn hơn 32 bytes.	Đặt chuỗi khóa dài tối thiểu 32 ký tự ASCII (hoặc dùng `openssl rand -base64 48`), sau đó chạy lại `set -a && . ./.env && set +a`.
`IllegalStateException: NEXTCHAT_JWT_SECRET must`	Chạy ứng dụng ở profile khác `dev`/`test` nhưng vẫn dùng khóa	Khai báo `NEXTCHAT_JWT_SECRET` riêng biệt trong biến môi trường hệ

`be configured outside dev and test profiles`	secret mặc định.	thống trước khi chạy.
`Connection to localhost:5432 refused` hoặc lỗi xác thực `password authentication failed`	Container PostgreSQL chưa chạy hoặc khởi chạy `docker compose up -d` thiếu cờ `--env-file .env` làm lệch giá trị mặc định.	Chạy `cd nextchat && docker compose down -v && docker compose --env-file .env up -d`, đợi `healthy` rồi nạp lại `.env`.
`FlywayValidateException: Migration checksum mismatch`	Đã chỉnh sửa nội dung file `V1` hoặc `V2` sau khi Flyway đã áp dụng vào database.	Không sửa file migration đã chạy; hãy tạo file `V3__...sql` mới, hoặc xóa volume CSDL dev bằng `docker compose down -v` rồi tạo lại.
Giao diện báo `WS: Disconnected` hoặc nhận khung `ERROR` khi `SUBSCRIBE`	Access Token hết hạn, sai Origin hoặc tài khoản không phải thành viên của phòng chat.	Kiểm tra cột **Protocol Log** bên phải giao diện, đăng nhập lại để lấy token mới và đảm bảo truy cập đúng `http://localhost:8080`.
Testcontainers báo `Could not find a valid Docker environment` khi chạy `./mvnw clean verify`	Docker daemon chưa bật hoặc user hiện tại chưa có quyền truy cập `/var/run/docker.sock`.	Khởi động dịch vụ Docker (`sudo systemctl start docker`) và kiểm tra bằng lệnh `docker ps`.

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN
***

BÁO CÁO ĐỒ ÁN MÔN HỌC
CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

NHẬT KÝ CÔNG VIỆC NHÓM 24

GIẢNG VIÊN HƯỚNG DẪN: PHẠM THI VƯƠNG
NHÓM THỰC HIỆN: NHÓM 24
Lê Quốc Cường — MSSV: 3123410040
Huỳnh Thanh Hải — MSSV: 3123410091
Nguyễn Phú Tài — MSSV: 3121410435

Hồ Chí Minh tháng 11 năm 2026

NHẬT KÝ CÔNG VIỆC NHÓM 24 1
1. Tổng quan Dự án & Hồ sơ Repository Git 3
2. Nhật ký Tiến độ 12 Tuần & Phân công Trách nhiệm Chi tiết 4
3. Phân tích Chi tiết Lịch sử Git Commit theo Các Mốc Kiến trúc 6
3.1. Các Mốc Commit Cốt lõi trên Nhánh Main 6
3.2. Đóng góp trên Nhánh phu-tai (Frontend Client & Infrastructure) 7
3.3. Minh bạch về Tài khoản Contributor trên GitHub 7
4. Ma trận Đồng bộ Kỹ thuật: Git Commits — Code Spring Boot — Báo cáo 8
4.1. Tầng 1 — Các Yêu cầu Bắt buộc Lõi (Spring Boot Core) 8
4.2. Tầng 2 & 3 — Kỹ thuật Nâng cao & Xử lý Đồng thời 8
5. Quy trình Kiểm thử Tự động, Minh chứng Thực thi & Thí nghiệm Độc lập 9
5.1. Kết quả Bộ Kiểm thử Tự động JUnit (./mvnw clean verify) 9
5.2. Kết quả Kiểm thử E2E Smoke qua Mạng Thật (tools/e2e_check.py) 10
5.3. Thí nghiệm SQL Lab Tách biệt (Đo đạc Tầng CSDL) 10
6. Bài học Kinh nghiệm & Nhật ký Sửa lỗi (Debugging Case Studies) 11
Tình huống 1: Lỗi Mất Principal giữa các Khung truyền STOMP 11
Tình huống 2: Giữ đúng totalElements khi Trang Lịch sử Rỗng 11
Tình huống 3: Lỗi Bất đồng Hợp đồng Destination STOMP giữa Client và Backend 12
7. Đánh giá Tổng kết & Định hướng nâng cấp 12
Hướng Phát triển Nâng cấp trong Tương lai 12

Tài liệu được nhóm hệ thống hóa toàn bộ tiến trình phát triển, phân công trách nhiệm và đối chiếu vết mã nguồn của dự án NextChat Backend — hệ thống nhắn tin thời gian thực xây dựng trên nền tảng Spring Boot 4.1.1 (Java 21) và PostgreSQL 16. Tài liệu tích hợp dữ liệu thực tế từ lịch sử Git commit trên repository springboot-websocket-chat, hồ sơ báo cáo đồ án nhóm, báo cáo cá nhân và các biên bản thực thi kiểm thử nhằm cung cấp một bức tranh toàn diện, minh bạch và đạt chuẩn kiểm thử sản xuất (production-ready).

1. Tổng quan Dự án & Hồ sơ Repository Git

Dự án NextChat được phát triển trong khuôn khổ học phần *Các công nghệ lập trình hiện đại* (Mã HP: 841072) tại Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, dưới sự hướng dẫn của giảng viên Phạm Thi Vương.

1. Tên ứng dụng: NextChat Real-time Messaging Backend
2. Repository GitHub chính thức: https://github.com/QuoocsCuongwf/springboot-websocket-chat
3. Nhánh phát triển chính (Main Baseline): Snapshot niêm phong 39b4d75 (docs(report): align report with implemented features)
4. Nhánh phát triển giao diện client: Nhánh phu-tai (chứa commit 4699986 và 332d069)
5. Thành viên Nhóm 24:
    5.1. Lê Quốc Cường (MSSV: 3123410040) — Trưởng nhóm / Architect & Lead Developer (Tài khoản GitHub: QuoocsCuongwf)
    5.2. Huỳnh Thanh Hải (MSSV: 3123410091) — Core Backend & Database Engineer (Tài khoản GitHub: ThanhHai151)
    5.3. Nguyễn Phú Tài (MSSV: 3121410435) — Frontend Client & Infrastructure Developer (Tác giả commit 4699986 trên nhánh phu-tai)

2. Nhật ký Tiến độ 12 Tuần & Phân công Trách nhiệm Chi tiết

Tiến độ đồ án được thực hiện nghiêm túc qua 12 tuần theo lịch học phần. Bảng dưới đây đối chiếu chi tiết giữa kế hoạch phân công, người phụ trách chính, mốc thời gian thực tế và kết quả mã nguồn đạt được trên Git:

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
Tuần 1–2 (11/09–24/09)	Khảo sát bài toán chat real-time; chốt công nghệ Java 21, Spring Boot 4.1.1, WebSocket/STOMP,	Lê Quốc Cường (Phụ trách),	Hoàn thành. Khởi tạo repository springboot-websocket-c

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
	PostgreSQL 16, JWT; khởi tạo kho mã.	Huỳnh Thanh Hải, Nguyễn Phú Tài	hat; commit đầu tiên 15b9513 (Initial commit) / 6ab071a ngày 22/09/2026.
Tuần 3 (25/09–01/10)	Thiết lập cấu hình Spring Boot, tổ chức package theo chức năng (package-by-feature), cấu hình Docker Compose cho PostgreSQL 16-alpine (kèm healthcheck pg_isready), viết migration Flyway V1__messenger_schema.sql (20 bảng).	Huỳnh Thanh Hải (Database/Auth), Lê Quốc Cường (Core/Config)	Hoàn thành. Commits 67fb244 (tổ chức package), dfa4a3b (Docker Compose & .env.example), a2626e6 (hoàn thiện Auth, Room REST API & database schema). Ánh xạ 52 thuộc tính cột của 6 JPA Entity.
Tuần 4 (02/10–08/10)	Cấu hình WebSocket Message Broker (WebSocketConfig) với endpoint /ws và /ws-chat, xây dựng RoomWebSocketController và hàm PL/pgSQL phân trang tin nhắn get_room_messages.	Lê Quốc Cường (WebSocket), Huỳnh Thanh Hải (PL/pgSQL)	Hoàn thành. Commits deabd07 (cấu hình WebSocket), c3acb71 (thêm STOMP controller /room.{roomId}), 5ea2fc2 (merge main into branch feat/auth-websocket).
Tuần 5 (09/10–15/10)	Tối ưu hóa truy cập dữ liệu JPA, áp dụng @EntityGraph cho ConversationRepository chống lỗi N+1 query, xây dựng giao dịch nguyên tử @Transactional và hàm PL/pgSQL get_or_create_direct_room.	Huỳnh Thanh Hải (REST/JPA), Lê Quốc Cường (Transaction)	Hoàn thành. Rà soát hiệu năng nạp dữ liệu danh sách phòng (getUserRooms), xây dựng cơ chế bọc DTO trong phạm vi giao dịch với open-in-view: false.

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
Tuần 6 (16/10–22/10)	Hoàn thiện luồng Refresh Token Rotation băm SHA-256 (RefreshTokenService), cài đặt bộ lọc JwtStompChannelInterceptor bảo mật WebSocket, bổ sung bộ kiểm thử JUnit.	Lê Quốc Cường (Security/STOMP), Huỳnh Thanh Hải (Auth)	Hoàn thành. Commits d20e734 (hoàn thiện JWT STOMP security), 3eaa91a (test suite cho chat STOMP), 39b4d75 (Docs align report - niêm phong nhánh main ngày 20/10).
Tuần 7 (23/10–29/10)	Rà soát chuyên sâu báo cáo, phát hiện và sửa lỗi mất Principal trong STOMP interceptor (MessageHeaderAccessor.getAccessor), mở rộng test lên 28 JUnit PASS và 15 E2E smoke PASS, thực hiện thí nghiệm SQL Lab.	Lê Quốc Cường (Lead/QA), Huỳnh Thanh Hải (Testing)	Hoàn thành đợt rà soát. Bổ sung các bản sửa cục bộ giữ Principal, test rollback Spring (SpringMechanismIntegrationTest), unit test Mockito (RoomServiceUnitTest) và thí nghiệm SQL tools/sql_lab.py.
Tuần 8 (30/10–05/11)	Phát triển ứng dụng Client Web thử nghiệm (HTML/JS đơn trang), cấu hình tài nguyên tĩnh, đồng nhất endpoint STOMP destination (/app/room/{roomId} và /topic/room/{roomId}).	Nguyễn Phú Tài (Client UI), Lê Quốc Cường (STOMP Sync)	Hoàn thành trên nhánh phu-tai. Nguyễn Phú Tài commit 4699986 (Client web, static config, Compose tweak); Lê Quốc Cường commit 332d069 (đồng bộ endpoint STOMP).
Tuần 9 (Từ 06/11)	Đóng gói hồ sơ báo cáo đồ án nhóm, báo cáo cá nhân, nhật ký công việc, biên bản làm việc nhóm và mã nguồn nộp giảng viên.	Cả nhóm (Lê Quốc Cường tổng hợp)	Hoàn thành. Xuất bản hồ sơ chính thức Thứ 6_Nhóm 24_Final.

3. Phân tích Chi tiết Lịch sử Git Commit theo Các Mốc Kiến trúc

Lịch sử Git commit của kho lưu trữ springboot-websocket-chat phản ánh chính xác quá trình tiến hóa của mã nguồn từ nền tảng đến các tính năng nâng cao.

3.1. Các Mốc Commit Cốt lõi trên Nhánh Main

- Commit 15b9513 / 6ab071a (Initial commit): Khởi tạo repository, thiết lập file LICENSE (Apache-2.0) và .gitignore.
- Commit 67fb244 (refactor: organize package structure by feature): Tái cấu trúc thư mục mã nguồn Java theo mô hình *package-by-feature + common*. Tách biệt thành các gói com.quoocscuongwf.nextchat.auth, user, room và common.
- Commit dfa4a3b (chore: add docker-compose and environment configuration): Xây dựng file docker-compose.yml khởi chạy container PostgreSQL 16-alpine với cấu hình volume lưu trữ bền vững và healthcheck pg_isready. Tạo file mẫu .env.example phục vụ nạp biến môi trường ngoài Git.
- Commit a2626e6 (feat(api): complete auth, rooms, and database task): Commit diện rộng của Huỳnh Thanh Hải và Lê Quốc Cường: hoàn thiện các mô-đun REST API xác thực (/api/v1/auth), quản lý hồ sơ người dùng (/api/v1/users), phòng chat (/api/v1/rooms), nạp file migration Flyway V1__messenger_schema.sql (20 bảng) và file cấu hình database.yml.
- Commit deabd07 & c3acb71 (feat(websocket): configure STOMP message broker and room controller): Đăng ký WebSocket broker với endpoint /ws và /ws-chat, xây dựng RoomWebSocketController tiếp nhận tin nhắn qua @MessageMapping("/room.{roomId}") và phát broadcast qua @SendTo("/topic/room.{roomId}").
- Commit 5ea2fc2 (Merge main into branch feat/auth-websocket): Hợp nhất nhánh tính năng WebSocket và xác thực JWT vào nhánh phát triển chính.
- Commit d20e734 (feat(security): complete JWT STOMP channel interceptor): Cài đặt JwtStompChannelInterceptor trích xuất và xác thực token JWT tại khung truyền CONNECT, đồng thời kiểm tra quyền thành viên phòng chat khi client gửi lệnh SUBSCRIBE.
- Commit 3eaa91a (test(chat): cover JWT STOMP and realtime messages): Bổ sung bộ kiểm thử đơn vị cho JwtStompChannelInterceptorTest và bài test tích hợp luồng lưu trữ tin nhắn real-time.
- Commit 39b4d75 (docs(report): align report with implemented features): Commit niêm phong mã nguồn nhánh main trước đợt rà soát cuối, đồng bộ tài liệu báo cáo khớp 100% với các tính năng thực tế.

3.2. Đóng góp trên Nhánh phu-tai (Frontend Client & Infrastructure)

- Commit 4699986 (Nguyễn Phú Tài, ngày 30/10/2026):
    - Xây dựng ứng dụng Client Web thử nghiệm một trang (index.html và app.js tại src/main/resources/static) kết nối đủ 9 REST API endpoints và SockJS/STOMP.
    - Cấu hình mở đường dẫn công khai cho tài nguyên tĩnh trong WebSecurityConfig.
    - Tinh chỉnh tham số mặc định CSDL trong docker-compose.yml.
- Commit 332d069 (Lê Quốc Cường, ngày 30/10/2026):
- Đồng bộ hóa cú pháp destination STOMP giữa Client và Backend sang dạng đường dẫn chuẩn /app/room/{roomId} và /topic/room/{roomId}, tích hợp SimpMessagingTemplate trong controller để phát tin linh hoạt.

3.3. Minh bạch về Tài khoản Contributor trên GitHub

Trên giao diện GitHub Insights/Contributors của repository springboot-websocket-chat, hệ thống ghi nhận 4 tài khoản chính thức :

- QuoocsCuongwf (Lê Quốc Cường)
- ThanhHai151 (Huỳnh Thanh Hải)
- Copilot (Công cụ AI hỗ trợ)
- NguyenPhuTai254 (Nguyễn Phú Tài)

4. Ma trận Đồng bộ Kỹ thuật: Git Commits — Code Spring Boot — Báo cáo

Mã nguồn trong repository và nội dung báo cáo tuân thủ nghiêm ngặt các tiêu chuẩn kỹ thuật backend được quy định trong tài liệu hướng dẫn môn học.

4.1. Tầng 1 — Các Yêu cầu Bắt buộc Lõi (Spring Boot Core)

- Auto-Configuration & Starters : Khai báo các starter spring-boot-starter-web, data-jpa, security, websocket, flyway. Bài test SpringMechanismIntegrationTest.realBeansAreSingletonsAndAutoConfigurationMatches xác nhận bean DataSource đạt singleton và log ConditionEvaluationReport khớp đúng HikariDataSource.
- IoC Container & Constructor Injection : Toàn bộ các dependency trong Controller và Service được tiêm qua hàm khởi tạo (Constructor Injection). Ứng dụng chỉ định nghĩa 5 Bean cấu hình bảo mật/dữ liệu (WebSecurityConfig, PasswordConfig, SeedDataConfig, WebSocketConfig), hoàn toàn không tạo DataSource hay EntityManagerFactory thủ công.

- Cấu hình ngoài & Profile : Quản lý thuộc tính qua application.yaml, database.yml và .env. JwtService có cơ chế tự ngắt ứng dụng ngay khi khởi động nếu phát hiện khóa JWT secret mặc định bị sử dụng ngoài môi trường dev và test.
- Spring Security & Stateless JWT : Bộ lọc JwtAuthenticationFilter tra cứu trực tiếp CSDL qua DatabaseUserDetailsService (chỉ chấp nhận isActive && !isDeleted). Nếu tài khoản bị khóa, mọi access token còn hạn lập tức bị từ chối ở request kế tiếp.
- Spring Data JPA & Chống lỗi N+1 Query : Tắt open-in-view: false trong database.yml. Sử dụng @EntityGraph trong ConversationRepository.findRoomsByUserId để nạp danh sách phòng kèm thành viên và tin nhắn cuối trong đúng 1–2 câu lệnh SQL.
- Quản lý CSDL bằng Flyway & ddl-auto: validate : Flyway thực thi hai file migration (V1__messenger_schema.sql và V2__lock_direct_room_creation.sql). Hibernate chạy chế độ validate đối chiếu khớp 52/52 thuộc tính cột của 6 JPA Entity chính (User, Role, RefreshToken, Conversation, ConversationParticipant, Message).

4.2. Tầng 2 & 3 — Kỹ thuật Nâng cao & Xử lý Đồng thời

- Triệt tiêu Race Condition bằng Khóa Advisory CSDL:
  - *Bài toán*: Hai người dùng A và B bấm nhắn tin cho nhau tại cùng một mili-giây, dẫn đến nguy cơ tạo 2 phòng chat 1-1 trùng lặp.
- *Giải pháp*: Xây dựng hàm PL/pgSQL get_or_create_direct_room tích hợp khóa tư vấn mức giao dịch pg_advisory_xact_lock trên mã băm chuẩn hóa thứ tự ID:
-

```SQL
PERFORM pg_advisory_xact_lock(
   hashtextextended(
      LEAST(p_user1_id, p_user2_id)::text || ':' || GREATEST(p_user1_id,
p_user2_id)::text,
      0
   )
);
```

- Refresh Token Rotation băm SHA-256:
  - RefreshTokenService.rotateRefreshToken thực thi trong giao dịch @Transactional. Token thô 32 bytes (Base64URL) gửi cho client, nhưng chỉ lưu bản băm SHA-256 (43 ký tự Base64URL) trong CSDL.

    - Khi client gọi /api/v1/auth/refresh, token cũ bị xóa ngay lập tức và token mới được phát hành. Nếu token cũ bị kẻ xấu dùng lại, hệ thống từ chối với lỗi 401 Unauthorized do bản băm cũ không còn tồn tại.
- WebSocket / STOMP Interceptor Security:
    - JwtStompChannelInterceptor xác thực JWT tại frame CONNECT.
    - Tại frame SUBSCRIBE tới /topic/room.{roomId}, interceptor tra cứu CSDL xác nhận người dùng đang là thành viên active (left_at IS NULL) của phòng.
    - Tại frame SEND, interceptor ép buộc destination phải khớp mẫu /app/room.{roomId}, ngăn chặn hoàn toàn việc client gửi tin trực tiếp vào broker bypass qua tầng nghiệp vụ.

5. Quy trình Kiểm thử Tự động, Minh chứng Thực thi & Thí nghiệm Độc lập

Hệ thống được bảo chứng bằng 3 cấp độ kiểm thử minh bạch:

5.1. Kết quả Bộ Kiểm thử Tự động JUnit (./mvnw clean verify)

Lần thực thi ngày 28/10/2026 ghi nhận 28/28 bài test PASS (0 Failures, 0 Errors, 0 Skipped):

- AuthAndRoomIntegrationTest (18 bài test tích hợp) : Vận hành trên CSDL postgres:16-alpine thật được khởi tạo tự động qua Testcontainers . Bao phủ toàn bộ 9 endpoint REST API, luồng đăng ký, đăng nhập, xoay vòng Refresh Token (Test #8), phân trang lịch sử tin nhắn (Test #14), chặn 403 người ngoài phòng (Test #16), kiểm thử Race Condition 2 luồng đồng thời tạo phòng 1-1 (Test #17) và lưu trữ tin nhắn real-time (Test #18).
- JwtStompChannelInterceptorTest (7 bài test unit) : Kiểm thử độc lập interceptor xác thực JWT khi CONNECT, chặn token hết hạn, kiểm soát quyền khi SUBSCRIBE và chặn các destination vi phạm.
- SpringMechanismIntegrationTest (2 bài test) : Kiểm chứng bean DataSource singleton, kiểm tra ConditionEvaluationReport HikariCP và xác nhận cơ chế rollback giao dịch Spring qua proxy khi câu lệnh INSERT token thay thế bị lỗi.
- RoomServiceUnitTest (1 bài test) : Kiểm thử đơn vị bằng Mockito cho RoomService.sendMessage, xác nhận người ngoài phòng bị chặn 403 Forbidden trước khi thực hiện bất kỳ lệnh ghi nào vào CSDL.

5.2. Kết quả Kiểm thử E2E Smoke qua Mạng Thật (tools/e2e_check.py)

Kịch bản tools/e2e_check.py khởi chạy ứng dụng trên cổng tạm kết nối PostgreSQL 16 thật và điều khiển 2 client WebSocket native trao đổi tin nhắn. Kết quả đạt 15/15 checks PASS :

1. Đăng ký/đăng nhập 2 tài khoản thử nghiệm A và B.
2. Mở 2 kết nối WebSocket /ws kèm JWT header.
3. Client A gửi tin nhắn tới /app/room.7, Client B gửi tin phản hồi.
4. Cả 2 client cùng nhận đúng MessageDTO chứa ID do server cấp.
5. Đối chiếu CSDL PostgreSQL xác nhận lưu trữ đủ 2 bản ghi tin nhắn và cập nhật last_message_id.
6. Người dùng ngoài C cố tình SUBSCRIBE hoặc SEND vào phòng bị chặn lỗi 403 / ERROR frame, CSDL giữ nguyên không bị sai lệch.

5.3. Thí nghiệm SQL Lab Tách biệt (Đo đạc Tầng CSDL)

Để phân tích hiệu năng CSDL mà không làm kéo dài thời gian build test Spring Boot, nhóm xây dựng script độc lập tools/sql_lab.py thực thi trực tiếp trên PostgreSQL 16.15 (giới hạn 1 vCPU, 512 MiB RAM):

- Thí nghiệm Khảo sát Phân trang Lịch sử (get_room_messages) :
  - *Mức 1.000 tin*: Offset 0 đạt 0.715 ms; Offset 900 đạt 1.393 ms.
  - *Mức 10.000 tin*: Offset 0 đạt 1.974 ms; Offset 9.000 đạt 10.864 ms.
  - *Mức 100.000 tin*: Offset 0 đạt 18.029 ms; Offset 90.000 đạt 131.194 ms.
- *Kết luận*: Kết quả đo đạc giải thích rõ chi phí duyệt bản ghi OFFSET và câu lệnh COUNT(*) khi tập dữ liệu phình to, làm cơ sở đề xuất hướng phát triển Keyset / Cursor Pagination.
- Thí nghiệm Đối chứng Khóa Advisory (pg_advisory_xact_lock) :
  - Chạy 3 lượt thử nghiệm với 2 worker gọi hàm đồng thời (có chèn delay nhân tạo pg_sleep(0.15) để mở cửa sổ tranh chấp).
  - *Biến thể có khóa (locked)*: 3/3 lượt đều chỉ sinh ra đúng 1 phòng chat và 2 participant (2 worker nhận cùng roomId).
  - *Biến thể bỏ khóa (unlocked)*: 3/3 lượt đều bị sinh ra 2 phòng chat trùng lặp và 4 participant.
  - *Kết luận*: Minh chứng thực nghiệm khẳng định vai trò bắt buộc của khóa tư vấn trong việc bảo đảm tính nguyên tử (idempotent) của thao tác tạo phòng 1-1.

6. Bài học Kinh nghiệm & Nhật ký Sửa lỗi (Debugging Case Studies)

Trong quá trình phát triển và rà soát hệ thống, nhóm đã ghi nhận 3 tình huống kỹ thuật tiêu biểu:

Tình huống 1: Lỗi Mất Principal giữa các Khung truyền STOMP

- Hiện tượng : Client gọi CONNECT thành công (nhận CONNECTED), nhưng khi gửi frame SUBSCRIBE lập tức bị server ngắt kết nối với lỗi ERROR.
- Nguyên nhân : Trong mã ban đầu, interceptor khởi tạo accessor mới bằng StompHeaderAccessor.wrap(message). Việc này làm đứt gãy callback gán bởi STOMP transport, dẫn đến thông tin Principal bị thất lạc ở các khung truyền tiếp theo.
- Giải pháp : Cập nhật interceptor trích xuất accessor sẵn có qua MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class), thực hiện setUser(authentication) trực tiếp trên accessor gốc và trả về thông điệp ban đầu.

Tình huống 2: Giữ đúng totalElements khi Trang Lịch sử Rỗng

- Hiện tượng : Khi client truy vấn lịch sử tin nhắn ở một trang ngoài phạm vi (ví dụ page=5, size=10 trên phòng chỉ có 2 tin), phản hồi bị trả về totalElements = 0.
- Nguyên nhân : Hàm PL/pgSQL get_room_messages lặp giá trị total_count trên từng dòng kết quả. Khi danh sách dòng trả về rỗng, service không đọc được total_count từ mảng kết quả.
- Giải pháp : Xây dựng nhánh xử lý dự phòng trong RoomService.getRoomMessages: nếu danh sách dòng rỗng, service tự động gọi messageRepository.countByConversationIdAndDeletedFalse(roomId) để đảm bảo giữ nguyên giá trị tổng số bản ghi chính xác cho client.

Tình huống 3: Lỗi Bất đồng Hợp đồng Destination STOMP giữa Client và Backend

- Hiện tượng : Client web thử nghiệm gửi tin nhắn tới /app/chat.sendMessage nhưng backend không tiếp nhận.
- Nguyên nhân : Sự sai lệch giữa tài liệu thiết kế nháp ban đầu (/app/chat.sendMessage) và annotation mã nguồn thực tế trong RoomWebSocketController (/room.{roomId}).
- Giải pháp : Nhóm thực hiện đồng bộ dứt điểm tại commit 332d069 trên nhánh phu-tai: chuẩn hóa destination gửi thành /app/room/{roomId} và destination đăng ký nhận thành /topic/room/{roomId}, đồng thời cập nhật đồng bộ trong interceptor, controller và client app.js.

7. Đánh giá Tổng kết & Định hướng nâng cấp

Nhìn lại toàn bộ lộ trình 12 tuần, dự án NextChat Backend đã hoàn tất trọn vẹn các mục tiêu nghiên cứu cốt lõi:

1. Xây dựng thành công hạ tầng Backend REST API & WebSocket/STOMP an toàn, tin cậy trên nền Spring Boot 4.1.1 và Java 21.
2. Quản lý dữ liệu quan hệ chặt chẽ bằng PostgreSQL 16, Flyway migration và các hàm PL/pgSQL tối ưu hóa.
3. Giải quyết dứt điểm các bài toán phức tạp về concurrency (Race Condition tạo phòng 1-1) và security (Refresh Token Rotation băm SHA-256, STOMP Interceptor authentication).
4. Chứng minh tính đúng đắn bằng bộ test tự động 28 JUnit PASS và 15 E2E Smoke PASS.

Hướng Phát triển Nâng cấp trong Tương lai

- Chuyển đổi Cursor Pagination : Tái cấu trúc API lịch sử tin nhắn sang cơ chế Keyset Pagination (dựa trên created_at và id) để đạt thời gian phản hồi $O(1)$ cố định cho các màn hình cuộn vô hạn với hàng triệu bản ghi.
- Hạ tầng Redis Broker Relay : Thay thế Simple Broker trong bộ nhớ bằng Redis Pub/Sub hoặc RabbitMQ STOMP Relay, cho phép mở rộng hệ thống backend theo mô hình đa tiến trình/đa instance (Horizontal Scaling).
- Hoàn thiện Dockerfile & CI/CD : Đóng gói ứng dụng Spring Boot bằng Dockerfile multi-stage build và thiết lập pipeline GitHub Actions tự động thực thi ./mvnw clean verify cho mọi Pull Request.